

EdgeJS QuickJS WASIX

Sonia Sadhbh Kolasinska in collaboration with Christoph Herzog, Wasmer

May 16, 2026

Abstract

This book consolidates the EdgeJS QuickJS WASIX implementation notes, runtime design records, troubleshooting registries, framework-adapter findings, and deployment/package work. The node-test chapter has been refreshed to remove stale historical baseline material and retain the current QuickJS macOS CI failure grouping and active issue registry.

Contents

1 Editorial Note	16
2 Program Definition	16
3 QuickJS WASM Implementation Plan	18
3.1 Goal	18
3.2 Current Status Note	18
3.3 Research Summary	18
3.4 Chosen Direction	18
3.5 Implementation Phases	19
3.5.1 1. Establish the Build Scaffold	19
3.5.2 2. Build a Minimal QuickJS Provider	19
3.5.3 3. Run the Smallest Edge Bootstrap	20
3.5.4 4. Expand N-API by Runtime Surface	20
3.5.5 5. Package as quickjs-wasm	21
3.5.6 6. Bring Up Runtime Features Incrementally	21
3.5.7 7. Optimize Only After Correctness	22
3.6 Testing Strategy	22
3.7 Main Risks	22
3.8 Success Criteria	23
4 QuickJS N-API Class Reference	23
5 N-API QuickJS Classes	24
5.1 Status Summary	24
5.2 Core Environment And Values	24
5.3 Functions And Externals	24
5.4 References And Finalizers	24
5.5 Async, Promises, And Cleanup	24
5.6 Unofficial Node/V8 Compatibility Helpers	25
5.7 Diagnostics	25
6 napi_allocator__	26
7 napi_callback_info__	27
8 quickjs::detail::napi_callsite__	28

9	<code>quickjs::detail::napi_contextify__</code>	29
10	<code>napi_deferred__</code>	30
11	<code>napi_env_cleanup_hook__</code>	31
12	<code>napi_env__</code>	32
13	<code>napi_external_backing_store_hint__</code>	33
14	<code>napi_external__</code>	34
15	<code>napi_function__</code>	35
16	<code>quickjs::detail::napi_lifetime_tracker__</code>	36
17	<code>quickjs::detail::napi_module_wrap__</code>	37
18	<code>napi_promises__</code>	38
19	<code>napi_ref__</code>	39
20	<code>napi_scope__</code>	40
21	<code>quickjs::detail::napi_serdes__</code>	41
22	<code>napi_value__</code>	42
23	Chronological Development Narrative	42
24	Edge QuickJS Development Index	43
24.1	Canonical Issue Registries	43
24.2	Chronological Notes	43
24.3	Development Task Notes	44
24.4	Status Icons	44
25	QuickJS N-API EdgeJS merge analysis	45
25.1	Scope	45
25.2	Executive Summary	45
25.3	Theirs EdgeJS Integration	45
25.3.1	CMake Provider Plumbing	45
25.3.2	Header Export Behavior	46
25.3.3	Edge Runtime Hooks	46
25.3.4	Duplicate Symbol Avoidance	46
25.3.5	WASIX Package Scaffold	46
25.3.6	Runtime/Library Workarounds Outside N-API	47
25.3.7	Docs/Plans	47
25.4	Theirs <code>napi/quickjs</code>	47
25.4.1	Build Model	48
25.4.2	QuickJS-NG Patch	48
25.4.3	Runtime Shape	48
25.4.4	API Surface	49
25.5	Our <code>napi/quickjs</code>	49
25.5.1	Strengths	49
25.5.2	Gaps For EdgeJS Integration	49
25.6	Recommended Development Plan	50
25.6.1	1. Port The Provider Selection Plumbing	50
25.6.2	2. Add Embedded WASM Export Semantics	50
25.6.3	3. Make Our Provider Embeddable By EdgeJS CMake	50

25.6.4 4. Decide N-API Symbol Ownership	50
25.6.5 5. Port Runtime Hooks And Low-Risk Fixes	51
25.6.6 6. Add The WASIX Package Scaffold	51
25.6.7 7. Verification Sequence	51
25.7 Open Questions	51
25.8 Proposed First Implementation Slice	51
26 QuickJS N-API EdgeJS execution and bootstrap troubleshooting	53
26.1 Scope	53
26.2 Initial Symptom	53
26.3 Instrumentation Added	53
26.3.1 Edge Builtin/Binding Tracing	53
26.3.2 QuickJS Contextify Compile Diagnostics	54
26.4 Bootstrap Failure 1: internal/vm	54
26.4.1 Root Cause	55
26.4.2 Current Status	55
26.5 Runtime Failure 2: Missing Dispose Symbols	55
26.5.1 Root Cause	55
26.5.2 Current Status	55
26.6 Verification	56
26.7 Current State	56
26.8 Things To Clean Up Or Decide	57
26.8.1 Debug Tracing	57
26.8.2 QuickJS Teardown Assertion	57
26.8.3 N-API Test Coverage	57
26.9 Commands That Were Useful	57
27 Edge QuickJS REPL TTY troubleshooting	59
27.1 Context	59
27.2 Short version	59
27.3 LLDB investigation	59
27.4 Native Edge callback result	60
27.5 JavaScript stream trace	61
27.6 Why stdin was paused	61
27.7 File handle close trace	61
27.8 VSCode LLDB trace confirmation	62
27.9 Quick confirmation workaround	63
27.10Current hypothesis	63
27.11Temporary probe: ignoring readStop	64
27.12What to do next	64
27.13Useful commands	64
27.14Bottom line	65
28 Edge QuickJS REPL promise hooks and microtasks	66
28.1 Context	66
28.2 What the colleague branch showed	66
28.3 Root cause	67
28.4 Why this currently requires a QuickJS source change	67
28.5 Current Status	67
28.5.1 QuickJS runtime	67
28.5.2 QuickJS N-API backend	68
28.6 Verification	68
28.7 Remaining separate issue	69
29 Edge QuickJS WASIX Wasmer run enablement	70
29.1 Context	70
29.2 Build shape	70

29.3 Wasmer compatibility notes	70
29.4 Bootstrap failure: Atomics under WASIX	71
29.5 HTTP server hang	71
29.6 Root cause of the stream pointer mismatch	72
29.7 Stream fix	72
29.8 Verification	73
29.9 Current status	73
30 Edge QuickJS web app enablement for Astro, Vite, and Next.js	75
30.1 Current Status Note	75
30.2 Context	75
30.3 Edge QuickJS package shape	75
30.4 Runtime changes that made framework adapters possible	76
30.4.1 ContextifyScript must be a class	76
30.4.2 QuickJS promise hooks and microtasks	76
30.4.3 Atomics and SharedArrayBuffer under WASIX	77
30.4.4 HTTP stream listener attachment	77
30.4.5 Network tracing	77
30.4.6 QuickJS teardown and lifetime status	78
30.5 Running static sites through Edge QuickJS	78
30.6 Astro: ~/src/astro-app	78
30.7 Vite: ~/src/vite-app	79
30.8 Next.js: ~/src/next-app	80
30.8.1 Package shape	80
30.8.2 Runtime router	81
30.8.3 Dynamic shell generation	81
30.8.4 Table route emission bug	82
30.8.5 .dist staging	82
30.8.6 Next.js verification commands	83
30.9 What this does and does not prove	83
30.9.1 Proven	83
30.9.2 Not proven	83
30.10 Practical app authoring rules	84
30.11 Remaining work	84
31 Edge QuickJS framework standalone builds	85
31.1 Current Status Note	85
31.2 Context	85
31.3 Astro standalone	85
31.4 Vite standalone	86
31.4.1 Native CLI static-root caveat	86
31.5 Next.js standalone	87
31.5.1 Native CLI main-entry caveat	87
31.6 Next.js runtime findings	88
31.6.1 Edge V8 inspector limitation	88
31.6.2 Edge WASIX builtin failure formatting	88
31.6.3 Edge QuickJS v8 / serdes blocker	89
31.7 Builtin error formatting	89
31.8 Verification	90
31.9 Current standalone status	90
31.10 Follow-up	90
32 Edge QuickJS runtime change containment rollback	91
32.1 Goal	91
32.2 Rollback Surface	91
32.3 Compatibility Strategy	92
32.4 Current Result	92

32.5 Working Rule	93
33 Node compatibility test failure analysis	94
33.1 2026-05-16 QuickJS macOS CI Failure Log	94
34 Cleanup And Containment Subtasks	95
35 Shared runtime rollback	96
35.1 Scope	96
35.2 Status	96
35.3 Verification	96
35.4 Notes	96
36 Native inspector fallback	97
36.1 Scope	97
36.2 Dependencies	97
36.3 Current Implementation	97
36.4 Verification	97
36.5 Status Notes	98
36.6 Ownership	98
37 Intl fallback module	99
37.1 Scope	99
37.2 Current Implementation	99
37.3 Intended Behavior	99
37.4 Verification Status	99
37.5 Ownership	99
37.6 Status Notes	100
38 Compile-time trace diagnostics	101
38.1 Scope	101
38.2 Current Implementation	101
38.3 Trace Families	101
38.4 Verification Status	101
38.5 Ownership	101
38.6 Status Notes 2026-05-07	101
39 Verification and remaining cleanup	103
39.1 Scope	103
39.2 Current Build State	103
39.3 Required Smoke Tests	103
39.4 Known Unrelated Dirty State	103
40 Internal N-API Promises Refactor	104
40.1 Scope	104
40.2 Current State	104
40.3 Verification	104
41 Internal N-API Serdes Refactor	106
41.1 Scope	106
41.2 Verification	106
41.3 Current State	106
41.4 Verification Result	106
42 Internal N-API Contextify Refactor	107
42.1 Scope	107
42.2 Known Limitation	107
42.3 Current State	107

42.4 Verification	107
43 Internal N-API Set Property Refactor	108
43.1 Scope	108
43.2 Verification	108
43.3 Current State	108
44 Subtask 001: QuickJS Module API and Record	109
44.1 Scope	109
44.2 Write Ownership	109
44.3 Dependencies	109
44.4 Required Behavior	109
44.5 QuickJS API Patch	110
44.6 Verification Expectations	110
44.7 Implementation Result	110
45 Subtask 002: Node Loader Interop and Synthetic Modules	111
45.1 Scope	111
45.2 Write Ownership	111
45.3 Dependencies	111
45.4 Required Behavior	111
45.5 Important Call Sites	111
45.6 Verification Expectations	112
45.7 Implementation Result	112
46 Subtask 003: Dynamic Import, Import Meta, and Required Facade	113
46.1 Scope	113
46.2 Write Ownership	113
46.3 Dependencies	113
46.4 Required Behavior	113
46.5 Verification Expectations	113
46.6 Implementation Result	114
47 Subtask 004: Verification	115
47.1 Scope	115
47.2 Write Ownership	115
47.3 Dependencies	115
47.4 Target Test Matrix	115
47.5 Required Commands	116
47.6 Verification Run	116
47.7 Symbol Dispose Regression	117
48 Dev 003: QuickJS Module Loading	118
48.1 Goal	118
48.2 Architecture Summary	118
48.3 Subtasks	118
48.4 Dependency Order	119
48.5 Parallelization Notes	119
49 V8 N-API lifetime refactor: current state and gap analysis	120
49.1 Scope	120
49.2 Current Edge V8 Shape	120
49.3 QuickJS Lessons	120
49.4 Node V8 Reference Point	121
49.5 Main Gaps	121
49.6 Desired End State	121
49.7 Implementation Result	122

49.8 Reference Rule	122
50 V8 N-API lifetime refactor: subtask plan	123
50.1 001 Baseline and Parity Harness	123
50.2 002 Direct napi_value Representation	123
50.3 003 Real Handle Scopes	124
50.4 004 Callback and Accessor Trampolines	124
50.5 005 Node-Shaped References and Finalizers	125
50.6 006 Wraps, Externals, Type Tags, and Buffers	125
50.7 007 Cleanup and Unofficial N-API Audit	125
51 V8 N-API lifetime refactor: risk and rollout	127
51.1 Rollout Strategy	127
51.2 Key Risks	127
51.3 Design Constraints	127
51.4 Verification Checklist	127
51.5 Verification Run	128
52 V8 N-API lifetime refactor: unofficial N-API audit	129
52.1 Scope	129
52.2 Summary	129
52.3 2026-05-16 V8 Node-Compat Follow-Up	129
52.4 Findings	130
52.5 Intentional long-lived globals	132
52.6 Stage 007 recommended changes	132
53 Troubleshooting Registry	133
54 Edge QuickJS Troubleshooting Registry	134
54.1 Status Icons	134
54.2 Node Compatibility	134
54.3 Node Test	135
54.4 Astro SSR	135
54.5 Vite App	136
54.6 Next App	136
54.7 Wasmer Deploy	136
55 Node Test Compatibility Current State	137
56 Node Test: QuickJS Compatibility Failures	138
56.1 Source	138
56.2 Issues	138
57 Node Test: Buffer limits and deprecation parity	140
57.1 What Is The Issue	140
57.2 2026-05-15 Update	140
57.3 2026-05-15 QuickJS Limit And UTF-8 Update	141
57.4 How Should We Fix It	141
58 Node Test: console inspect and stack formatting	142
58.1 What Is The Issue	142
58.2 How Should We Fix It	142
58.3 2026-05-15 Update	142
59 Node Test: node:test public API exports	144
59.1 What Is The Issue	144
59.2 Current Status	144

60 Node Test: diagnostics channel module loader events	145
60.1 What Is The Issue	145
60.2 How Should We Fix It	145
61 Node Test: diagnostics channel async context	146
61.1 What Is The Issue	146
61.2 2026-05-15 Update	146
61.3 How Should We Fix It	146
62 Node Test: EventEmitterAsyncResource private-field errors	148
62.1 What Is The Issue	148
62.2 2026-05-15 Update	148
62.3 How Should We Fix It	148
63 Node Test: fetch Response body and HTTP proxy env	149
63.1 What Is The Issue	149
63.2 How Should We Fix It	149
64 Node Test: HTTPS proxy tunnel errors	151
64.1 What Is The Issue	151
64.2 2026-05-15 Native Error Stack Update	151
64.3 Remaining Work	152
65 Node Test: HTTP timers and header limits	153
65.1 What Is The Issue	153
65.2 How Should We Fix It	153
66 Node Test: OS constants and userInfo errors	154
66.1 What Is The Issue	154
66.2 How Should We Fix It	154
67 Node Test: FastUtf8Stream synchronous wait	155
67.1 What Is The Issue	155
67.2 How Should We Fix It	155
68 Node Test: explicit resource management syntax	156
68.1 What Is The Issue	156
68.2 How Should We Fix It	156
69 Node Test: stream missing builtins and async iterators	157
69.1 What Is The Issue	157
69.2 How Should We Fix It	157
70 Node Test: StringDecoder UTF-8 boundaries	158
70.1 What Is The Issue	158
70.2 2026-05-15 Native Decoder Update	158
70.3 Implementation Notes	158
71 Node Test: URL and data URL validation	159
71.1 What Is The Issue	159
71.2 How Should We Fix It	159
72 Node Test: WHATWG URL inspect and search params	160
72.1 What Is The Issue	160
72.2 2026-05-15 QuickJS Error Text And UTF-8 Update	160
72.3 2026-05-15 URL Inspect And Brand Update	161
72.4 Implementation Notes	161

73 Node Test: HTTP/2 native lifecycle crashes	162
73.1 Symptoms	162
73.2 Root Cause	162
73.3 Fix	162
73.4 Verification	162
73.5 V8 JSStream Destroy Follow-up	163
73.6 2026-05-16 QuickJS macOS CI Follow-up	163
74 Node Test: TLS SecureContext, SNI, and setKeyCert lifetime	164
74.1 Current Reproduction	164
74.2 Root Cause: inherited wrap metadata	164
74.3 Root Cause: switched SecureContext lifetime	165
74.4 Fix	165
74.5 Verification	165
75 Astro SSR	166
76 Astro SSR: es-module-lexer WebAssembly Import	167
76.1 Issue	167
76.2 Diagnosis	167
76.3 Status Notes	167
76.4 Constraints	168
76.5 Validation	168
77 Astro SSR: depd CallSite Method Compatibility	169
77.1 Issue	169
77.2 Diagnosis	169
77.3 Current Status	170
77.4 Constraints	170
77.5 Validation	170
78 Astro SSR: CommonJS Re-Export Named Exports	172
78.1 Issue	172
78.2 Diagnosis	172
78.3 Why this compatibility layer exists	172
78.4 Current Status	173
78.5 Constraints	173
78.6 Validation	173
79 Astro SSR: Missing Intl Global	175
79.1 Issue	175
79.2 Diagnosis	175
79.3 Current Status	175
79.4 Constraints	176
79.5 Validation	176
80 Astro SSR: Listen EPERM On Localhost	177
80.1 Issue	177
80.2 Diagnosis	177
80.3 Status Notes	178
80.4 Constraints	178
80.5 Validation	178
81 Astro SSR: Floating UI Utils DOM Subpath	179
81.1 Issue	179
81.2 Diagnosis	179
81.3 Current Status	180

81.4 Constraints	180
81.5 Validation	180
82 Astro SSR: React Remove Scroll Bar Constants Subpath	181
82.1 Issue	181
82.2 Diagnosis	181
82.3 Current Status	181
82.4 Status Notes	182
82.5 Constraints	182
82.6 Validation	182
83 Astro SSR: Zustand Ind Create Export	183
83.1 Issue	183
83.2 Diagnosis	183
83.3 Current Status	184
83.4 Status Notes	184
83.5 Constraints	184
83.6 Validation	184
84 Astro SSR: Zustand ESM Default Export	186
84.1 Issue	186
84.2 Diagnosis	186
84.3 Current Status	187
84.4 Status Notes	187
84.5 Constraints	187
84.6 Validation	187
85 Astro SSR: Use Gesture Controller Export	188
85.1 Issue	188
85.2 Diagnosis	188
85.3 Current Status	189
85.4 Status Notes	189
85.5 Constraints	189
85.6 Validation	189
86 Astro SSR: Route Stack Overflow	190
86.1 Issue	190
86.2 Diagnosis	190
86.3 Status Notes	190
86.4 Constraints	191
86.5 Validation	191
87 Astro SSR: WASIX pnpm Symlink Resolution	192
87.1 Issue	192
87.2 Diagnosis	192
87.3 Status Notes	192
87.4 Validation	193
88 Astro SSR: Lucide React ChevronDown Export	194
88.1 Issue	194
88.2 Diagnosis	194
88.3 Status Notes	194
88.4 Validation	195
89 Astro SSR: pnpm Deploy Externalized Runtime Links	196
89.1 Issue	196
89.2 Diagnosis	196

89.3 Status Notes	197
89.4 Current Status	197
89.5 Validation	197
90 Vite App	198
91 Vite App Standalone Build Findings	199
91.1 Query	199
91.2 Summary	199
91.3 Options For Next Work	200
91.4 Conclusion	200
92 Next.js	200
93 Next App Standalone: require("v8") / Serdes Findings	201
93.1 Context	201
93.2 Reduction	201
93.3 LLDB Findings	202
93.4 Code Evidence	202
93.5 Impact On Standalone Next	202
93.6 Likely Runtime Fix	202
93.7 Current Conclusion	203
93.8 Current Status	203
94 Next App Standalone: require("inspector") Stub	204
94.1 Context	204
94.2 Reduction	204
94.3 Historical Action Plan	204
94.4 Current Status	205
95 Next App Standalone: Route Request Stack Exhaustion	206
95.1 Context	206
95.2 Impact	206
95.3 Action Plan	206
95.4 Native Versus Wasmer Samples	207
95.5 Stack Size Probe	207
95.6 Updated Investigation Plan	207
96 Next config SWC options Buffer rejected by N-API	209
96.1 Symptom	209
96.2 Current Finding	209
96.3 Fix	210
96.4 Plan	210
96.5 Risks	211
96.6 Verification	211
97 Next App: entryCSSFiles work store async context	212
97.1 Symptom	212
97.2 Initial Finding	212
97.3 Root Cause	212
97.4 Implemented Fix	213
97.5 Verification	213
97.6 Debug Teardown Follow-Up	213
97.7 Original Action Plan	214
98 Node Compatibility	214
99 Node Compatibility Troubleshooting	215

99.1 Areas	215
99.2 Current Status	215
100Known Issue: NAPI_EXTERN= WASIX linkage rule	216
100.1Current State	216
100.2Source Notes	216
100.3Known Incompatibility	216
100.4Risk	216
100.5Current Status	216
101Known Issue: Framework static and ad hoc server adapters	217
101.1Current State	217
101.2Source Notes	217
101.3Known Incompatibility	217
101.4Risk	217
101.5Current Status	217
102Known Issue: pnpm deploy graph materialization	218
102.1Current State	218
102.2Source Notes	218
102.3Known Incompatibility	218
102.4Risk	218
102.5Current Status	218
103Deploy And Packaging Node Compatibility	219
104Known Issue: Minimal Intl.DateTimeFormat fallback	220
104.1Current State	220
104.2Source Notes	220
104.3Known Incompatibility	220
104.4Risk	220
104.5Current Status	220
105Known Issue: Native unavailable inspector stub	221
105.1Current State	221
105.2Source Notes	221
105.3Known Incompatibility	221
105.4Risk	221
105.5Current Status	221
106Known Issue: Stream wrapper-specific unwrap fallback	222
106.1Current State	222
106.2Source Notes	222
106.3Historical Incompatibility	222
106.4Risk	222
106.5Current Status	222
107url.parse() deprecation gating and node_modules callers	223
107.1Failure	223
107.2Diagnosis	223
107.3Fix	223
107.4Verification	224
107.5QuickJS Follow-up	224
108V8 CTest Runtime Fixture Environment Attachment	225
108.1Observation	225
108.2Diagnosis	225
108.3Action Plan	225

108.4	Result	225
109	EdgeJS Runtime Node Compatibility	226
110	Known Issue: Buffer	227
110.1	Current State	227
110.2	Known Incompatibility	227
110.3	Current Status	227
111	Known Issue: Console bootstrap binding	228
111.1	Current State	228
111.2	Known Incompatibility	228
111.3	Current Status	228
112	Known Issue: Contextify diagnostics	229
112.1	Current State	229
112.2	Known Incompatibility	229
112.3	Current Status	229
113	Known Issue: Environment lifecycle	230
113.1	Current State	230
113.2	Known Incompatibility	230
113.3	Current Status	230
114	Known Issue: Global shims	231
114.1	Current State	231
114.2	Known Incompatibility	231
114.3	Current Status	231
115	Known Issue: Promise hooks and microtasks	232
115.1	Current State	232
115.2	Known Incompatibility	232
115.3	Current Status	232
116	Known Issue: Module loading	233
116.1	Current State	233
116.2	Known Incompatibility	233
116.3	Design Decision	234
116.4	Target Architecture	234
116.5	Implementation Phases	234
116.5.11.	QuickJS Module Introspection and Split Lifecycle	234
116.5.22.	Source Text ModuleWrap	235
116.5.33.	Synthetic Modules for CJS, Builtins, JSON, and WASM Facades	235
116.5.44.	require(esm) and Required Module Facades	235
116.5.55.	Dynamic import() and import.meta	235
116.5.66.	Contextify and Format Detection Tightening	236
116.5.77.	Verification	236
116.6	Development Task Notes	236
116.7	Non-Goals	236
117	Known Issue: Property setting semantics	238
117.1	Current State	238
117.2	Known Incompatibility	238
117.3	Current Status	238
118	Known Issue: QuickJS utility ownership	239
118.1	Current State	239
118.2	Known Incompatibility	239

118.3	Current Status	239
119	Known Issue: Serialization and deserialization	240
119.1	Current State	240
119.2	Known Incompatibility	240
119.3	Current Status	240
120	Known Issue: QuickJS WASIX atomics guard patch	241
120.1	Current State	241
120.2	Source Notes	241
120.3	Known Incompatibility	241
120.4	Risk	241
120.5	Current Status	241
121	Known Issue: V8-shaped CallSite methods	242
121.1	Current State	242
121.2	Source Notes	242
121.3	Known Incompatibility	242
121.4	Risk	242
121.5	Current Status	242
122	Known Issue: QuickJS N-API lifetime tracing	243
122.1	Current State	243
122.2	Action Plan	243
122.3	Initial Investigation Notes	244
122.4	Instrumentation Added	244
122.5	Lifetime Map	244
122.5.1	Scope and ownership picture	245
122.5.2	External data flow	246
122.5.3	Function and class metadata flow	247
122.5.4	QuickJS WeakRef intrinsic and N-API weak refs	248
122.5.5	2026-05-13 external data split	249
122.6	EdgeJS Embedder Findings	250
122.7	Verification	250
122.8	Current Leak Suspects	251
122.9	Vector-Backed Handle Plan	251
122.10	Historical Vector-Backed Handle Implementation	251
122.11	2026-05-12 Fixed-Block Allocator Plan	252
122.12	2026-05-12 Extending Fixed-Block Allocation To Other N-API Helpers	254
122.13	2026-05-12 Object Prototype Lifetime Buckets	255
122.14	2026-05-11 Periodic Allocator Stats	255
122.15	2026-05-11 Server Stats Growth Investigation	256
122.16	Verification After Vector-Backed Handles	257
122.17	2026-05-15 allocator used/free block refactor	257
122.18	2026-05-13 Async-Wrap Destroy Hook Scope Fix	258
122.18.1	2026-05-13 Internal Binding Async Scope Audit	259
122.18.2	2026-05-13 napi_wrap(...) result references must be weak	261
122.19	2026-05-13 Allocator Handle Decoding Checks	261
122.20	2026-05-13 Allocator Constructor/Destructor Payloads	262
122.21	2026-05-13 External Backing Store Allocator	263
122.22	2026-05-12 Compact Detailed Lifetime Tables	264
122.23	2026-05-12 HTTP Load Detailed Snapshot	264
122.24	2026-05-12 Allocator Hook Lifetime Refactor	267
122.25	2026-05-11 Root test_6 Debug Check	267
122.26	2026-05-11 Reentrancy Audit	268
122.27	2026-05-11 String And Symbol Value Dump Plan	269
122.28	2026-05-11 Tracker-Owned Value Extraction Correction	270

122.2	2026-05-11 Native Callback Handle-Scope Investigation	271
122.3	2026-05-11 Handle Representation And Trampoline RAII Update	272
122.3	2026-05-11 String And Symbol Value Dump Plan	274
122.3	2026-05-11 QuickJS Tag Breakdown Plan	274
122.3	2026-05-11 Standalone N-API Segfault Regression Plan	275
122.3	14 DB Breakpoints And Calls	276
122.3	Node/V8 Lifetime Comparison	276
122.3	QuickJS Scope Model	277
122.3	2026-05-11 Server Stats Growth Findings	278
122.3	2026-05-12 Native Event Scope Plan	279
	122.38 Node/V8 native event scope comparison	279
	122.38 Native event scope implementation	283
	122.38 Periodic lifetime stats verbosity split	284
122.3	2026-05-12 Env-Owned Reference Allocator Plan	284
122.4	2026-05-16 Platform Task And Contextify Teardown Pass	284
122.4	2026-05-13 Env-Owned Ref Teardown Assertion	285
122.4	2026-05-15 Reentrant Allocator Destroy Assertion	286
122.4	2026-05-13 Root-Scope Function Value Growth	287
122.4	2026-05-11 Scope Handle Allocator Plan	288
123	V8 structured clone handle scope	290
123.1	Symptom	290
123.2	Diagnosis	290
123.3	Action Plan	290
123.4	Fix	291
123.5	Verification	291
123.6	Residual Issues	291
124	QuickJS Contextify Module Syntax Detection	292
124.1	Failure	292
124.2	Diagnosis	292
124.3	Correctness Bar	293
124.4	Upstream-Quality Design	293
124.5	Edge Integration Plan	294
124.6	Non-Goals	294
124.7	Open Questions	294
124.8	Implementation Result	295
125	V8 finalizer GC microtask reentry	296
125.1	Failure	296
125.2	Diagnosis	296
125.3	Fix	296
125.4	Verification	296
126	N-API Node Compatibility Known Issues	298
127	Wasmer Deploy And WASIX Packaging	298
128	Wasmer Deploy: pnpm Directory Symlinks In WEBC Package	299
128.1	Issue	299
128.2	Source Findings	299
128.3	Diagnosis	300
128.4	Action Plan	300
128.5	Current Status	301
128.6	Validation	301
128.7	Open Questions	302

129	Wasmer Deploy: QuickJS WASIX N-API Import Module Mismatch	303
129.1	Issue	303
129.2	Diagnosis	303
129.3	Current Status	303
129.4	Verification	303
129.5	Follow-Up Rule	304
130	Wasmer Deploy: CI safe-mode missing QuickJS artifact	305
130.1	Issue	305
130.2	Action Plan	305
130.3	Current Status	305
130.4	Verification	306
130.5	Follow-Up: Native Linux Job	306
130.5.1	May 7, 2026 Native QuickJS N-API Release Build Follow-Up	306
130.5.2	May 7, 2026 QuickJS Workflow Split Follow-Up	307
131	Wasmer Deploy: WASIX safe-mode HTTPS exits before callbacks	308
131.1	Issue	308
131.2	Action Plan	308
131.3	Current Status	308
131.4	Verification	309

1 Editorial Note

This PDF was generated from `plans/quickjs-wasm` and its subdirectories. Source paths are shown before each included note for traceability. Node-test material has been curated for this edition: older historical baseline sections are omitted, while the current registry, issue pages, and May 16, 2026 QuickJS macOS CI grouping are retained.

2 Program Definition

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/task.md

this repo contains edgejs, a node.js alternative, driven through napi bindings, and wasmer / webassembly integration.

the current setup mainly targets using v8 as the engine, compiling edgejs itself to webassembly, and then using host napi bindings through WASM imports.

We want an alternative way to run edgejs in webassembly with the quickjs JS engine, but with quickjs compiled directly to webassembly instead of going through the napi host bridge.

do some research, and come up with a plan how to implement this. notes: * it might make sense to look for existing forks of quickjs that can already compile to webassembly. * use `nix run github:wasix-org/wasix#wasixcc` for a working wasixcc compiler * the end goal is to build a `quickjs-wasm/wasmer.toml` package definition that contains a wasm binary for edgejs, that uses quickjs directly from wasm and can be used to run code

be token efficient in your research and internal reasoning

come up with a high quality step by step plan for how to implement this, and write it to `./plan.md`

Source: `/Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/plan.md`

3 QuickJS WASM Implementation Plan

3.1 Goal

Build a `quickjs-wasm/wasmer.toml` package that runs Edge.js as a WASIX WebAssembly binary with QuickJS compiled into the same guest module. This should avoid the current host-provided N-API bridge used by `EDGE_NAPI_PROVIDER=imports` and instead provide N-API from inside the wasm binary.

The important design point is that Edge.js is already organized around an engine boundary: `src/` uses N-API and must not depend on V8 directly. The QuickJS path should preserve that boundary by adding a new N-API provider rather than rewriting the runtime kernel.

3.2 Current Status Note

The early plan below predates the later cleanup decision around module loading. The QuickJS C++ CommonJS facade/module-loader hack has now been removed: `unofficial_module_loader.{h,cc}` and `quickjs_cjs_exports.{h,cc}` are gone from `napi/quickjs/src`.

The missing engine-side module primitive has since moved into vendored QuickJS itself. Commit `577fb31caf2e973b111431b0` adds APIs for enumerating module requests, binding host-linked modules, linking/evaluating modules, querying module state, initializing import meta, and handling dynamic imports. The current `napi/quickjs/src/internal/napi_module` code uses those APIs to implement the V8-shaped `unofficial_napi_module_wrap_*` surface.

That does not restore the removed C++ package resolver or CommonJS export scanner. The current design direction is to keep package resolution, CommonJS wrapping, package conditions, and builtin policy in Node's JavaScript loaders/translators or another proper EdgeJS-owned runtime path.

3.3 Research Summary

- The current native path uses `EDGE_NAPI_PROVIDER=bundled-v8` and links `napi/v8`.
- The current WASIX path uses `EDGE_NAPI_PROVIDER=imports`, builds `build-wasix/edgejs.wasm`, and leaves N-API symbols unresolved for the Wasmer host to provide.
- `RunWithFreshEnv()` creates an environment through `unofficial_napi_create_env()`, then `RunScriptWithGlobals()` installs process, timers, module loading, primordials, builtins, and the event loop.
- Therefore the hard part is not compiling QuickJS to wasm. The hard part is implementing enough Node-API plus `unofficial_napi` over QuickJS for the existing Edge bootstrap and internal bindings.
- Existing QuickJS WASI projects are useful references, especially QuickJS-NG/WASI and `vercel-labs/quickjs-wasi`, but they are not drop-in Node-compatible runtimes.
- `quickjs-emsripten` is mostly a JS-host embedding library, not a good WASIX executable backend.
- Javy is useful as a QuickJS/WASI compilation reference, but it compiles applications to wasm rather than providing a reusable Edge.js runtime.
- StarlingMonkey is better aligned with WinterCG/Web APIs than this QuickJS requirement and would bypass the existing N-API architecture.

3.4 Chosen Direction

Add a new provider:

`EDGE_NAPI_PROVIDER=quickjs`

with implementation under:

`napi/quickjs/`

This provider should:

- compile QuickJS or QuickJS-NG as C/C++ sources into the Edge WASIX binary;
- implement the public `node_api.h` functions used by Edge;
- implement the private `unofficial_napi.h` hooks used by Edge bootstrap, modules, workers, diagnostics, and testing;
- expose a normal edge executable target with no imported N-API symbols;
- produce `quickjs-wasm/wasmer.toml` pointing at the resulting wasm binary.

Prefer QuickJS-NG for the first prototype because it has active WASI work, CMake-based integration, and references for reactor/event-loop APIs. Keep the provider boundary narrow enough that switching to upstream QuickJS later remains possible.

3.5 Implementation Phases

3.5.1 1. Establish the Build Scaffold

1. Add `quickjs-wasm/` with:
 - `wasmer.toml`;
 - a build script, likely `quickjs-wasm/build.sh`;
 - a short README with the supported smoke commands.
2. Mirror the existing WASIX build flow from `wasix/build-wasix.sh`, but use a separate build directory such as `build-quickjs-wasix`.
3. Run the build with `wasixcc` available from:


```
nix run github:wasix-org/wasix#wasixcc
```

 or an equivalent shell where `wasixcc`, `wasixcc++`, `wasixar`, and `wasixranlib` are on PATH.
4. Extend `CMakeLists.txt`:
 - allow `EDGE_NAPI_PROVIDER=quickjs`;
 - add `napi/quickjs` as a subdirectory for that provider;
 - link `edge_node_api` and `edge_runtime` against `napi_quickjs`;
 - make the same lifecycle hook paths currently gated by `EDGE_BUNDLED_NAPI_V8` available to QuickJS, preferably through a neutral compile definition such as `EDGE_EMBEDDED_NAPI_PROVIDER`;
 - do not enable `EDGE_ALLOW_UNDEFINED_IMPORTS` for this provider except for unrelated WASIX system imports.
5. Ensure `napi/quickjs` CMake source lists only checked-in files before the first configure/build attempt.
6. Add a link check that fails if the final QuickJS wasm still imports N-API symbols.

Deliverable: a wasm binary that links QuickJS into the guest and starts, even if it only prints a provider initialization error.

3.5.2 2. Build a Minimal QuickJS Provider

1. Add `napi/quickjs` with:
 - `CMakeLists.txt`;
 - a CMake `FetchContent` import for QuickJS-NG from a pinned release archive with `URL_HASH SHA256`;
 - `napi_quickjs_env.*`;
 - `napi_quickjs_values.*`;
 - `napi_quickjs_unofficial.*`.
2. Implement environment creation and teardown:
 - `unofficial_napi_create_env`;
 - `unofficial_napi_create_env_with_options`;
 - `unofficial_napi_release_env`;
 - `unofficial_napi_release_env_with_loop`;
 - cleanup and destroy callbacks.

3. Map `napi_env`, `napi_value`, and `napi_ref` to QuickJS runtime/context, `JSValue`, and explicit reference records.
 - Placeholder C++ value graphs are acceptable only for link bring-up; replace them with QuickJS `JSValue` ownership before relying on GC or runtime correctness.
4. Implement the primitive value APIs needed by bootstrap:
 - undefined/null/boolean/number/string creation;
 - type checks and conversions;
 - object creation;
 - property get/set/has/delete;
 - function creation and calls;
 - error creation and pending exception state.
5. Add a tiny provider-only C++ test that creates an env, evaluates `1 + 1`, and tears down without Edge bootstrap.

Deliverable: the provider can create a QuickJS context and evaluate simple JS through N-API-like calls.

3.5.3 3. Run the Smallest Edge Bootstrap

1. Start with `EdgeRunScriptSource()` rather than full CLI script loading.
2. Support enough N-API for:
 - console;
 - process object installation;
 - `internalBinding`;
 - builtin execution through `EdgeExecuteBuiltin`;
 - basic script execution without reintroducing the removed C++ CJS loader hack.
3. Implement `unofficial_napi_process_microtasks` with QuickJS job draining.
4. Stub non-critical V8-specific diagnostics with conservative values where Edge can continue:
 - heap statistics;
 - CPU/heap profiler hooks;
 - V8 version-like metadata.
5. Do not implement workers, ESM, snapshots, inspector, or profiling in this phase unless bootstrap requires a narrow stub.

Deliverable:

```
edge -e "console.log('hello from quickjs')"
```

runs inside the QuickJS-backed WASIX binary.

3.5.4 4. Expand N-API by Runtime Surface

Implement APIs in the order Edge is likely to hit them:

1. Core values:
 - arrays;
 - symbols;
 - property descriptors;
 - strict equality;
 - instance checks;
 - private data / wrap / unwrap.
2. Functions and callbacks:
 - callback info;
 - `this`;
 - constructor calls;
 - thrown exceptions;
 - fatal error hooks.
3. References and lifecycle:
 - strong and weak references;

- finalizers;
 - env cleanup hooks;
 - external values.
4. Buffers and binary data:
 - ArrayBuffer;
 - TypedArray;
 - DataView;
 - Buffer compatibility;
 - external backing-store ownership rules.
 5. Promises and async:
 - promise creation;
 - promise details;
 - handled markers;
 - async work hooks needed by timers and libuv.
 6. Module support:
 - no C++ CommonJS facade/module-loader hack in the QuickJS backend;
 - ESM/source text module support once enough ModuleWrap behavior exists;
 - Node-compatible CommonJS/ESM behavior only through Node's JavaScript loaders/translators or another proper EdgeJS-owned runtime layer;
 - synthetic modules and cached data can initially be unsupported unless tests or bootstrap require them.

Use compile errors, missing-symbol reports, and failing smoke tests to maintain a tracked N-API gap list. Avoid implementing broad APIs before a caller needs them.

Deliverable: a checked-in compatibility matrix for implemented, stubbed, and unsupported N-API / unofficial_napi calls.

3.5.5 5. Package as quickjs-wasm

1. Create quickjs-wasm/wasmer.toml similar to the root wasmer.toml, but point at:

```
../build-quickjs-wasix/edgejs.wasm
```

or copy the artifact into quickjs-wasm/edgejs.wasm during packaging.

2. Mount certificates as the current package does:

```
[fs]
"/usr/local/ssl" = "../ssl-certs"
```

3. Decide whether JS builtins are:

- compiled into builtin_catalog.cc only; or
- mounted from lib/ during development.

Prefer compiled builtins for distribution and optional mounts for debugging.

4. Add package smoke commands:

```
wasmer run quickjs-wasm/wasmer.toml -- -e "console.log(1 + 1)"
wasmer run quickjs-wasm/wasmer.toml -- examples/hello.js
```

Deliverable: quickjs-wasm/wasmer.toml can run the QuickJS-backed Edge binary through Wasmer.

3.5.6 6. Bring Up Runtime Features Incrementally

After basic CLI execution works, add features in this order:

1. Filesystem-backed require() and path resolution.
2. Timers and event-loop checkpoints.
3. Buffer and encoding paths.
4. fs, path, os, and process APIs.

5. HTTP parser and minimal server/client networking.
6. Crypto and TLS.
7. Workers.
8. ESM and top-level await.
9. Diagnostics, profiling, heap snapshots, and inspector-like APIs.

Each feature should land with one end-to-end WASIX smoke test and the smallest provider API expansion needed to support it.

3.5.7 7. Optimize Only After Correctness

Do not start with snapshots or bytecode caching. First make source execution correct and observable.

Then evaluate:

- QuickJS bytecode for builtins;
- precompiled builtin catalog generation;
- startup snapshots based on QuickJS-NG/WASI references;
- binary size reductions;
- `wasm-opt --emit-exnref` and the same exception mode used by the current WASIX build.

Snapshots are a later optimization because they interact with native callbacks, libuv state, file descriptors, promises, and embedder data.

3.6 Testing Strategy

Use a staged gate:

1. Provider unit tests:
 - values;
 - functions;
 - exceptions;
 - references;
 - ArrayBuffer/Buffer;
 - promises and microtasks.
2. Native Edge runner tests that already call `EdgeRunScriptSource()`.
3. WASIX smoke tests using the QuickJS package.
4. Selected Node compatibility tests for the feature being brought up.
5. Benchmark only after the API surface is stable enough to compare.

Useful early smoke cases:

```
console.log("ok")
console.log(process.argv.length)
setTimeout(() => console.log("timer"), 0)
const fs = require("fs"); console.log(typeof fs.readFileSync)
```

3.7 Main Risks

- N-API compatibility depth is the dominant risk.
- QuickJS has different GC, job queue, stack, exception, and module semantics from V8.
- Some current unofficial_napi APIs are V8-shaped and should be stubbed, redesigned, or isolated for QuickJS instead of copied literally.
- Workers and structured clone are likely to be expensive because they require cross-context value transfer semantics.
- Full ESM support requires a deeper QuickJS module-loader integration than simple CommonJS execution. QuickJS itself does not lack CommonJS more than V8 does; Node implements CJS/ESM around the engine. The QuickJS risk is implementing enough `ModuleWrap`, package resolution, dynamic import, and CJS facade behavior so Node's JS loaders can drive the same semantics.

- WASIX threading, atomics, and exceptions must stay aligned with the existing `wasix/build-wasix.sh` flags.

3.8 Success Criteria

The first complete version is successful when:

- `quickjs-wasm/wasmer.toml` exists and points at a QuickJS-backed wasm binary;
- the binary has no imported N-API symbols;
- `edgejs -e "console.log('hello')"` works through Wasmer;
- simple file execution works;
- the implemented N-API surface is documented;
- unsupported runtime features fail clearly rather than crashing.

4 QuickJS N-API Class Reference

5 N-API QuickJS Classes

Status: Current as of 2026-05-15.

This directory documents the small internal classes and opaque structs used by the QuickJS-backed N-API implementation.

The core design rule is explicit lifetime ownership. QuickJS values that cross the public N-API surface live in allocator-backed records, rather than in V8 local handles. Scope-local values are owned by `napi_scope__`; persistent refs and env helpers are owned by `napi_env__`; the shared `napi_allocator__` keeps the public handles stable and pointer-shaped.

5.1 Status Summary

Area	Status	Notes
Environment, scopes, values	Current	Pointer-shaped handles backed by fixed-slab allocators.
Refs and finalizers	Current	Env-owned refs survive local scopes and are teardown-safe under reentrancy.
Externals and backing stores	Current	Finalizer metadata is centralized in env-owned hints.
Unofficial V8/Node helpers	Current	Implemented where QuickJS supports the behavior; stable fallbacks otherwise.
Diagnostics	Current	Compile-time gated lifetime tracking, no effect when disabled.

5.2 Core Environment And Values

- [napi_env](#)
- [napi_scope](#)
- [napi_value](#)
- [napi_allocator](#)
- [napi_callback_info](#)

5.3 Functions And Externals

- [napi_function](#)
- [napi_external](#)
- [napi_external_backing_store_hint](#)

5.4 References And Finalizers

- [napi_ref](#)

5.5 Async, Promises, And Cleanup

- [napi_deferred](#)
- [napi_env_cleanup_hook](#)
- [napi_promises](#)

5.6 Unofficial Node/V8 Compatibility Helpers

- [napi_contextify](#)
- [napi_module_wrap](#)
- [napi_serdes](#)
- [napi_callsite](#)

5.7 Diagnostics

- [napi_lifetime_tracker](#)

6 napi_allocator__

Status: Current as of 2026-05-15.

napi_allocator__ is the shared allocator used by the QuickJS N-API backend for small pointer-shaped helper objects.

It lives in napi/lib/src/napi_allocator.h, with the intrusive link primitive implemented header-only in napi/lib/src/napi_intrinsic_link.h. The allocator is a blazing-fast free list implemented as linked slabs, perfect for small JavaScript N-API wrapper objects that fly around. Each slab stores a fixed number of slots, and allocation/deallocation are O(1) operations in Release mode. The hot allocate(...) and destroy(T *) paths do not walk lists.

The allocator exposes only T * payload pointers. It has no public Handle type. Ownership is recovered from a payload pointer by deriving the containing slot and aligned block, then reading the block owner through unsafe_owner(T *). owns(T *) compares that owner with the allocator owner.

The allocator keeps three intrusive circular lists:

- first_free_: blocks where all slots are free.
- first_partial_: blocks with at least one free slot and at least one live slot.
- first_used_: blocks with all slots used.

Each block has exactly one allocator-list link_, and that link is present in exactly one of those lists, except while destroy(T *) has put the block in vacuum before running the payload destructor. Each block also has first_free_slot_ and first_used_slot_ sentinels for its slot-local free and used chains. Each slot has free_link_ and used_link_ intrusive links.

allocate(...) tries first_partial_first, then first_free_, creating a new fixed-size block only when both are empty. destroy(T *) remembers whether the block was linked when the call began, unlinks the slot from the used chain, and unlinks a linked block before running the payload destructor. It then records release, runs the destructor, and links the slot back to the block free chain. If the call entered with the block already in vacuum, it leaves the block in vacuum; the outer destroy that put the block there owns the final relink. If this call put a linked block in vacuum, it relinks the block to first_free_, first_partial_, or first_used_ based on the final slot state.

take_used() removes one live slot from first_used_ or first_partial_ and returns the still-constructed payload pointer to the caller. Env teardown uses take_next_used() for ref cleanup so finalizers can delete refs without invalidating an iterator.

begin() / end() iterate active payloads by walking full blocks and partial blocks, then following each block's first_used_slot_ chain through slot used_link_ links. Aggregate queries such as count_active() and storage_slot_count() intentionally walk lists; the hot allocate/destroy path stays constant time.

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/classes/napi_callback_info.md

7 napi_callback_info__

Status: Current as of 2026-05-15.

napi_callback_info__ stores one native callback invocation frame.

It lives in napi/quickjs/src/internal/napi_callback_info.h and .cc. The record stores the env, this value, optional new.target, argument count, argument array, and addon callback data pointer.

Instances are stack-allocated by napi_function__::trampoline(...) for the duration of a JS-to-native call. Public APIs such as napi_get_cb_info(...) and napi_get_new_target(...) reinterpret the public napi_callback_info back to this stack record and read from it.

The class does not own QuickJS values. It only references the JSValueConst arguments supplied by QuickJS for the active call. Returned values are wrapped through the current callback handle scope, not stored in napi_callback_info__.

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/classes/napi_callsite.md

8 quickjs::detail::napi_callsite__

Status: Current as of 2026-05-15.

napi_callsite__ adapts QuickJS stack-frame data to V8-shaped callsite helper APIs.

It lives in napi/quickjs/src/internal/napi_callsite.h and .cc under the quickjs::detail namespace. The class is static-only.

get_call_sites(...) and get_current_stack_trace(...) call JS_GetCurrentStackTrace(...) with different skip counts and wrap the resulting QuickJS array into the current N-API scope. The implementation caps frame count to a small fixed maximum to avoid accidental unbounded stack capture.

get_caller_location(...) captures one caller frame, extracts line, column, and script name/source URL when available, and returns a three-element JavaScript array. Missing or incomplete stack data is reported as a successful nullptr result rather than throwing.

Source: */Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/classes/napi_contextify.md*

9 quickjs::detail::napi_contextify__

Status: Current as of 2026-05-15.

`napi_contextify__` implements the QuickJS-backed subset of Node/V8 `contextify` and `script-compilation` helpers.

It lives in `napi/quickjs/src/internal/napi_contextify.h` and `.cc` under the `quickjs::detail` namespace. `napi_env__` owns one instance for the lifetime of the env.

The class handles source-map toggles, source-map error-source callbacks, preserved error formatting, context creation/disposal, script execution, function compilation, cached-data helpers, and module-syntax checks. Where QuickJS cannot expose a V8-equivalent detail, methods return stable fallback values rather than pretending to provide exact V8 internals.

Compilation paths annotate QuickJS exceptions with internal properties that carry resource name, offsets, builtin ID, and mapped line information. Trace output is controlled by QuickJS `contextify/builtin` trace flags and can add a compact compile summary to `stderr` and to the error stack.

`teardown()` frees stored callback state. Most helpers accept public `napi_value` handles, unwrap them to `JS-ValueConst`, perform QuickJS work, and return results through normal env scope wrapping.

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/classes/napi_deferred.md

10 napi_deferred__

Status: Current as of 2026-05-15.

`napi_deferred__` is the QuickJS N-API promise resolver record.

It lives in `napi/quickjs/src/internal/napi_deferred.h` and `.cc`. The record stores the env plus the QuickJS resolve and reject functions returned by promise creation.

`napi_create_promise(...)` creates a promise and stores its resolving functions in an env-owned deferred slot. Later, `napi_resolve_deferred(...)` or `napi_reject_deferred(...)` calls the stored function with the supplied resolution/rejection value and then destroys the deferred record.

The destructor frees both stored QuickJS functions. That makes the deferred record the persistent owner of the resolver pair, while the promise returned to JavaScript remains an ordinary scoped `napi_value`.

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/classes/napi_env_cleanup_hook.md

11 napi_env_cleanup_hook__

Status: Current as of 2026-05-15.

napi_env_cleanup_hook__ stores one env cleanup hook registration.

It lives in napi/quickjs/src/internal/napi_env_cleanup_hook.h and .cc. The record stores the owning env, cleanup callback, and callback argument.

Creation delegates to napi_env__::create_cleanup_hook(...), which allocates from the env-owned cleanup-hook allocator and also records the hook pointer in registration order. matches(...) is used by remove paths to find the exact hook/arg pair.

run() calls the registered hook with its argument. During env teardown, napi_env__::prepare_teardown() drains cleanup hooks while the QuickJS context and env-owned helper allocators are still valid.

12 napi_env__

Status: Current as of 2026-05-15.

napi_env__ is the QuickJS N-API environment. It binds one JSContext to the public N-API surface, owns env-wide helper storage, and controls teardown order.

The class lives in napi/quickjs/src/internal/napi_env.h and .cc. It stores the QuickJS context, module API version, last N-API error state, pending exception state, instance data, cleanup-hook registrations, the root/current handle-scope handles, external memory accounting, and env-owned subsystems for promises, contextify, and module-wrap compatibility.

QuickJS needs explicit handle storage, so the env owns allocator pools for napi_scope__, napi_ref__, napi_env_cleanup_hook__, napi_deferred__, and napi_external_backing_store_hint__. The pools use the shared fixed-slab napi_allocator__, which gives stable pointer-shaped handles while feeding lifetime diagnostics when tracking is enabled.

The root scope is created during env construction and remains the fallback current scope. Public N-API functions that produce values call wrap_value_in_current_scope(...), which stores a JSValue in the active napi_scope__. Refs are env-owned instead of scope-owned, because N-API refs outlive local handle scopes and are released by explicit ref APIs or env teardown.

Teardown is deliberately ordered. prepare_teardown() runs cleanup hooks, closes deferreds, clears refs with take_next_used() while the QuickJS context is still alive, closes the root scope, tears down promise/contextify/module-wrap subsystems, and marks the env torn down. The destructor then finalizes instance data; allocator members close naturally after the destructor body.

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/classes/napi_external_backing_store_hint.m

13 napi_external_backing_store_hint__

Status: Current as of 2026-05-15.

`napi_external_backing_store_hint__` stores native payload and finalizer metadata for QuickJS externals, wrapped objects, and external `ArrayBuffers`.

It lives in `napi/quickjs/src/internal/napi_external_backing_store_hint.h` and `.cc`. The struct stores the env, runtime pointer, native data pointer, optional finalizer callback, optional finalizer hint, finalizer-invoked state, a detach flag, and an optional weak target used when a finalizer should report a different JavaScript target.

Creation delegates to `napi_env__::create_external_backing_store(...)`, so hints are allocated from the env-owned `external_backing_stores_` allocator. Destroy paths go back through the stored env with `napi_env__::destroy_external_backing_store(...)`.

`invoke_finalizer()` is idempotent. It calls the stored finalizer at most once with the env, external data pointer, and finalizer hint. `begin_detach()` and `end_detach()` coordinate explicit `ArrayBuffer` detach with later QuickJS backing-store callbacks.

The runtime pointer is stored because QuickJS finalizers and `ArrayBuffer` deleters may be called from runtime-level callbacks. Destroy paths prefer env-owned storage while the env is alive and fall back to runtime-level cleanup when only the QuickJS runtime is available.

Source: `/Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/classes/napi_external.md`

14 napi_external__

Status: Current as of 2026-05-15.

`napi_external__` is the QuickJS external class and helper surface for native opaque payloads and internal object-wrap records.

It lives in `napi/quickjs/src/internal/napi_external.h` and `.cc`. The class registers two QuickJS class IDs:

- `NapiExternal` is the public `napi_create_external(...)` representation. Values of this class are the only values that `napi_typeof(...)` reports as `napi_external`, and the only values accepted by `napi_get_value_external(...)`.
- `NapiExternalRecord` stores internal wrap/finalizer metadata without making the owning JavaScript object look like a public external value.

Public external values store a `napi_external_backing_store_hint__` as their opaque pointer. `get_value(...)` returns the hint's native data pointer.

The same helper owns object-wrap metadata conventions. Wrapped objects can carry an external record either as the object's own opaque value or through the `__napi_wrap__` property. Those records use `NapiExternalRecord`, not `NapiExternal`, so constructed N-API class instances stay ordinary prototype-backed objects. Type tags, buffer markers, and finalizer metadata are stored under internal property names so public N-API code does not need to know QuickJS class details.

For Buffer-like objects, `mark_buffer(...)`, `is_buffer(...)`, and `get_buffer_info(...)` use a marker property plus QuickJS typed-array APIs to recover backing data and byte length.

Finalization is delegated to `napi_external_backing_store_hint__`. The QuickJS class finalizer invokes the N-API finalizer once and then asks the env-owned hint allocator to destroy the hint. External `ArrayBuffer` deleters use the same hint path, with a detach guard so the finalizer is not run twice during explicit detach flows.

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/classes/napi_function.md

15 napi_function__

Status: Current as of 2026-05-15.

`napi_function__` builds QuickJS functions that call native N-API callbacks.

It lives in `napi/quickjs/src/internal/napi_function.h` and `.cc`. The public creation path stores the native callback pointer and user data as raw QuickJS externals in `JS_NewCFunctionData(...)`, then wraps the resulting function into the current N-API scope.

Calls enter through `napi_function__::trampoline(...)`. The trampoline recovers the env from the QuickJS context opaque pointer, opens a callback-local handle scope, builds a stack `napi_callback_info__`, invokes the native callback, and duplicates the callback result before closing the temporary scope.

Constructor calls are handled specially. The trampoline creates an object with the function prototype, exposes it as `this`, sets `new.target`, and returns either the explicit callback result or the constructed object. `make_constructible(...)` sets QuickJS constructor metadata and prototype linkage.

The function data externals are intentionally raw QuickJS values, not public `napi_value` slots. That avoids leaking callback metadata into handle scopes.

Source: */Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/classes/napi_lifetime_tracker.md*

16 quickjs::detail::napi_lifetime_tracker__

Status: Current as of 2026-05-15.

`napi_lifetime_tracker__` is the diagnostic lifetime tracker for the QuickJS backend.

It lives in `napi/quickjs/src/internal/napi_lifetime_tracker.h` and `.cc` under the `quickjs::detail` namespace. It records allocator create/release events for `napi_value__`, `napi_ref__`, `napi_env_cleanup_hook__`, `napi_deferred__`, and `napi_external_backing_store_hint__`, plus semantic scope escapes.

The tracker is compile-time gated by `NAPI_ENABLE_LIFETIME_TRACKER`. When the flag is enabled, `napi_allocator_lifetime__` specializations route allocator events into per-type tracking. When disabled, the same call sites compile to no-ops.

Dumping is controlled at runtime with `EDGE_TRACE_NAPI_LIFETIME=1`, with additional periodic and string/tag/object detail controlled by the generic lifetime tracker flags. `napi_quickjs_lifetime_dump(...)` exists as an extern entry point so LLDB or diagnostics can request a snapshot.

The tracker is intentionally observational. It does not own handles or alter runtime lifetime; it reports the shape of env-owned refs/helpers and scope-owned value slots so leaks and root-scope retention can be investigated.

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/classes/napi_module_wrap.md

17 quickjs::detail::napi_module_wrap__

Status: Current as of 2026-05-15.

napi_module_wrap__ is the QuickJS-backed implementation of the unofficial module-wrap surface.

It lives in napi/quickjs/src/internal/napi_module_wrap.h and .cc under the quickjs::detail namespace. napi_env__ owns one instance and exposes it through env->module_wrap().

The class installs QuickJS runtime callbacks for import-meta initialization and dynamic import. It stores optional JavaScript callbacks for both host hooks and keeps vectors of module records plus script-referrer metadata.

Each internal record tracks one source or synthetic module: the JSModuleDef, module value, wrapper value, host-defined option ID, synthetic eval function, source object, import requests, linked records, and duplicated linked module values. The public handle returned by create calls is an opaque pointer to one of these records.

Source-text modules cache their import requests from QuickJS module metadata. Synthetic modules call back into JavaScript during initialization and support setting exports. Evaluation, namespace lookup, status/error reporting, top-level-await checks, cached-data creation, and CommonJS facade creation are adapted to the parts QuickJS can provide.

teardown() unregisters QuickJS host hooks, frees every active record, releases script-referrer metadata, and frees stored callback values.

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/classes/napi_promises.md

18 napi_promises__

Status: Current as of 2026-05-15.

napi_promises__ stores QuickJS promise integration state for unofficial Node/V8-shaped promise APIs.

It lives in napi/quickjs/src/internal/napi_promises.h and .cc. The class is an env-owned subsystem and stores the env, QuickJS context, optional promise reject callback, four lifecycle hooks, continuation-preserved embedder data, per-promise async context frames, and a stack used while entering/leaving promise callbacks.

The constructor does not install global hooks by itself; the public/unofficial N-API entry points route callbacks into this subsystem. Stored callback values are duplicated QuickJS values and freed during teardown().

set_reject_callback(...) and set_hooks(...) validate optional function values before storing them. promise_hook(...) is the QuickJS hook bridge for init/before/after/resolve events. It captures or restores continuation data and calls the corresponding JavaScript hook when one is registered.

rejection_tracker(...) adapts QuickJS promise rejection notifications to the V8-shaped N-API callback surface. microtask_job(...) is a tiny QuickJS job wrapper that calls a queued JavaScript function.

19 napi_ref__

Status: Current as of 2026-05-15.

`napi_ref__` is the persistent N-API reference object for the QuickJS backend.

It lives in `napi/quickjs/src/internal/napi_ref.h` and `.cc`. The ref stores its owning env, a JSValue, a QuickJS native weak-ref link, and the logical N-API reference count returned by `napi_reference_ref(...)` and `napi_reference_unref(...)`.

A positive refcount owns a strong duplicate of the target value. A zero refcount tries to install a QuickJS native weak reference with `JS_AddNativeWeakRefLink`. If weak linking is unavailable for the target but the value is ref-counted, the ref falls back to holding a strong duplicate so the stored JSValue remains valid.

`add_ref()` promotes a weak ref back to strong ownership. `rem_ref()` decrements the logical count and, when it reaches zero, attempts to make the target weak and releases the strong duplicate if that weak transition succeeds.

Weak target finalization clears the stored value and resets the logical count. Env teardown takes refs out of the allocator one at a time and calls `clear_for_teardown()` while the QuickJS context is still valid. Deleting a ref that has already been cleared is a no-op, which keeps finalizer reentrancy safe.

Refs are allocated from `napi_env__::refs_`, so public `napi_ref` handles are stable pointers to env-owned allocator payloads.

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/classes/napi_scope.md

20 napi_scope__

Status: Current as of 2026-05-15.

`napi_scope__` is the QuickJS handle-scope record.

It lives in `napi/quickjs/src/internal/napi_scope.h` and `.cc`. Each scope stores its owning env, lexical scope level, parent handle, an allocator of `napi_value__` slots, and lifecycle flags for closed/escaped state.

QuickJS does not provide V8-style local handle blocks, so `napi_scope__` is the backend's local-value owner. `wrap_value(...)` allocates a `napi_value__` in the scope's `values_` allocator. Closing the scope closes that allocator, which destroys every active `napi_value__` and frees the wrapped QuickJS values.

Escapable scopes are represented by the same internal type. `escape_value(...)` checks that the value is owned by this scope, duplicates the underlying `JValueConst` into the parent scope with `owned=false`, and records the escape for diagnostics. `mark_escaped()` prevents public escapable-scope APIs from escaping more than once.

Scopes are themselves env-owned allocator payloads. Public `napi_handle_scope` handles are typed pointers to `napi_scope__` slots, and `napi_env__` is responsible for translating public scope handles back to internal scope records.

Source: */Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/classes/napi_serdes.md*

21 quickjs::detail::napi_serdes__

Status: Current as of 2026-05-15.

`napi_serdes__` implements QuickJS-backed serialization/deserialization helpers for the unofficial V8-shaped serializer API.

It lives in `napi/quickjs/src/internal/napi_serdes.h` and `.cc` under the `quickjs::detail` namespace. The class is static-only; serializer/deserializer instances are JavaScript objects that wrap native serializer or deserializer structs with N-API finalizers.

The lightweight serializer stores a byte vector. The deserializer stores a byte vector plus a read offset. Primitive methods append or read little-endian integers, doubles, and raw bytes. `ArrayBuffer`-like inputs are read from QuickJS `ArrayBuffer`, `TypedArray`, or `DataView` values.

Structured clone helpers use QuickJS object serialization where supported: `serialize_value(...)` returns an opaque payload buffer and `deserialize_value(...)` reconstructs a scoped N-API value from that payload. `release_serialized_value(...)` frees payload buffers created by the serializer.

The compatibility layer is intentionally narrow. It keeps Node-facing serdes behavior out of public N-API entry points while preserving QuickJS exception state or reporting N-API status on failure.

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/classes/napi_value.md

22 napi_value__

Status: Current as of 2026-05-15.

napi_value__ is the QuickJS-backed local N-API value slot.

It lives in napi/quickjs/src/internal/napi_value.h and .cc. The struct stores the owning env and one JSValue. If constructed with owned=true, it takes the incoming JSValue as-is. If constructed with owned=false, it duplicates the incoming value with JS_DupValue(...).

The destructor frees the stored JSValue with JS_FreeValue(...) while the env and context are still valid. This makes napi_value__ the central RAII wrapper for QuickJS values exposed through public N-API handles.

Unlike the V8 backend, napi_value is not a direct engine local handle. It is a pointer to a napi_value__ payload owned by the current napi_scope__. Public N-API code reaches the underlying QuickJS value through napi_quickjs_value_inner(...), which validates the env/scope relationship in debug builds and returns JSValueConst.

Persistence belongs to napi_ref__, not napi_value__. A value slot should be treated as local to its owning handle scope.

23 Chronological Development Narrative

24 Edge QuickJS Development Index

		Remarks
Status	[open]	Canonical entry point for QuickJS WASIX development notes and issue registries.
Severity	Low	Documentation structure only; runtime impact is tracked in the linked issue pages.

This directory has two kinds of documentation:

- chronological notes that preserve development history;
- issue pages that are the canonical home for current status, severity, and known limitations.

Avoid restating issue details in index pages. Link to the issue page instead.

24.1 Canonical Issue Registries

- [Troubleshooting registry](#)
- [Node compatibility known issues](#)
- [Node test known issues](#)
- [Astro SSR issues](#)
- [Vite app issues](#)
- [Next app issues](#)
- [Wasmer deploy issues](#)

24.2 Chronological Notes

These files are historical context. If a current problem is described here and also has a troubleshooting page, treat the troubleshooting page as canonical.

Note	Status	Scope
001_merge_analysis.md	[done]	Initial merge analysis and integration boundaries.
002_native_bootstrap_contextify.md	[done]	Native QuickJS bootstrap and contextify investigation history.
003_repl_tty_readline.md	[done]	REPL TTY/readline investigation history.
004_promise_hooks_microtasks.md	[done]	Promise hooks and microtask investigation history.
005_wasix_wasmer_http.md	[done]	WASIX/Wasmer HTTP bring-up history.
006_framework_app_adapters.md	[caveated]	Framework adapter exploration history; current issues live under troubleshooting.
007_framework_standalone_builds.md	[caveated]	Standalone framework build findings; current issues live under troubleshooting.
008_runtime_change_containment_rollback.md	[done]	Runtime containment and rollback history.

Note	Status	Scope
009_node_test_failures_analysis.md	[open]	Node test failure clustering; per-problem pages live under <code>troubleshooting/node-test</code> .

24.3 Development Task Notes

Development task directories preserve task-scoped context. They are not the canonical home for known incompatibilities once a troubleshooting page exists.

Directory	Status	Scope
dev_001_pr_cleanup_containment	[done]	Runtime cleanup containment history.
dev_002_napi_promises_refactor	[done]	QuickJS N-API internal refactor history.
dev_003_quickjs_module_loading	[open]	QuickJS ModuleWrap and Node JS loader interop plan for CommonJS/ESM parity.
dev_004_v8_napi_lifetime_refactor	[done]	V8 N-API lifetime, handle-scope, reference, wrap, and finalizer refactor implemented and verified against shared V8/QuickJS N-API suites.

24.4 Status Icons

- [open]: open or active.
- [done]: resolved or stable.
- [caveated]: accepted with caveats, partial compatibility, or retained limitation.
- [blocked]: unresolved blocker.

25 QuickJS N-API EdgeJS merge analysis

		Remarks
Status	[done]	Historical merge analysis. Current runtime issues are tracked in troubleshooting pages.
Severity	Low	

Date: 2026-05-04

25.1 Scope

This note compares:

- Theirs EdgeJS tree: ~/src/wasmer-io/edgejs
- Theirs N-API submodule: ~/src/wasmer-io/edgejs/napi
- Our EdgeJS tree: ~/src/edgejs
- Our N-API submodule: ~/src/edgejs/napi

The theirs EdgeJS checkout is on branch quickjs at c2e04af1 update napi. Its napi submodule points to wasmerio/napi@1d818e52 (origin/quickjs, more async fixes).

Our current tree is on napi/quickjs-integration and its napi submodule is at 06207826 (Unofficial NAPI impl), with the refactored QuickJS provider and the standalone QuickJS N-API test harness.

25.2 Executive Summary

The theirs branch has the EdgeJS-side integration we need:

- EDGE_NAPI_PROVIDER=quickjs in top-level CMake.
- embedded-provider compile definitions for Edge lifecycle hooks.
- WASIX build/package scaffolding under quickjs-wasm/.
- a wasm link check that rejects imported napi_*, node_api_*, and unofficial_napi_* symbols.
- small runtime workarounds needed to get the QuickJS-backed Edge binary further through startup and smoke tests.

The theirs napi/quickjs implementation is much more prototype-shaped than ours:

- one big env header plus three large source files;
- fetches QuickJS-NG v0.14.0 with FetchContent;
- patches QuickJS-NG quickjs.c at CMake time for promise hook behavior;
- has no local napi/quickjs/tests harness;
- contains several broad stubs/defaults to satisfy Edge/WASIX linkage.

Our QuickJS provider is cleaner and better tested, but it is not yet integrated as an EdgeJS provider. The right integration direction is to port the EdgeJS integration and packaging ideas from the theirs branch, while keeping our refactored provider architecture and filling any missing Edge-required N-API symbols deliberately.

25.3 Theirs EdgeJS Integration

25.3.1 CMake Provider Plumbing

In ~/src/wasmer-io/edgejs/CMakeLists.txt, the theirs branch:

- Extends EDGE_NAPI_PROVIDER from bundled-v8|imports to bundled-v8|imports|quickjs.
- Adds:

- `add_subdirectory("${PROJECT_ROOT}/napi/quickjs" ...)`
- `target_link_libraries(edge_node_api PUBLIC napi_quickjs)`
- `target_link_libraries(edge_runtime PUBLIC napi_quickjs)`
- Changes undefined import policy:
 - WASIX + imports keeps `EDGE_ALLOW_UNDEFINED_IMPORTS=ON`.
 - `quickjs` forces `EDGE_ALLOW_UNDEFINED_IMPORTS=OFF`.
- Generalizes V8-only embedded behavior to `EDGE_EMBEDDED_NAPI_PROVIDER`.
- Keeps `EDGE_BUNDLED_NAPI_V8` for the V8 provider where V8-specific code still needs it.
- Adds `EDGE_QUICKJS_OWNS_NAPI_SYMBOLS` to `edge_node_api` when provider is QuickJS.

That last point matters: theirs QuickJS provider owns many public N-API symbols directly, so `src/node_api.cc` is partially compiled out to avoid duplicate definitions.

25.3.2 Header Export Behavior

In `~/src/wasmer-io/edgejs/napi/include/js_native_api.h`, the theirs branch changes wasm symbol attributes:

- normal WASIX/imports mode still marks N-API declarations as wasm imports;
- `defined(__wasm__)` && `defined(EDGE_EMBEDDED_NAPI_PROVIDER)` marks them as exported/default-visible symbols instead.

This is necessary for an embedded QuickJS provider, because the final wasm binary should define N-API symbols itself rather than import them from Wasmer.

Our current `~/src/edgejs/napi/include/js_native_api.h` does not have this embedded-provider wasm export branch yet.

25.3.3 Edge Runtime Hooks

Two Edge runtime files switch from `EDGE_BUNDLED_NAPI_V8` to the provider-neutral `EDGE_EMBEDDED_NAPI_PROVIDER`:

- `~/src/wasmer-io/edgejs/src/edge_environment.cc`
- `~/src/wasmer-io/edgejs/src/edge_task_queue.cc`

This lets QuickJS receive the same environment attachment, cleanup, context token, and promise rejection hooks that the bundled V8 path already used.

25.3.4 Duplicate Symbol Avoidance

`~/src/wasmer-io/edgejs/src/node_api.cc` wraps many fallback N-API functions in:

```
#if !defined(EDGE_QUICKJS_OWNS_NAPI_SYMBOLS)
...
#endif
```

This avoids duplicate definitions when `napi_quickjs` provides those symbols. For our integration, this is a design choice:

- either keep this model and make our provider own all required symbols;
- or keep more of `edge_node_api` active as provider-neutral fallback glue.

I prefer the second path initially unless linkage proves it impractical. It reduces the first integration blast radius.

25.3.5 WASIX Package Scaffold

The theirs branch adds `~/src/wasmer-io/edgejs/quickjs-wasm/`:

- `build.sh`
- `README.md`
- `wasmer.toml`

- `echo-server.js`

`quickjs-wasm/build.sh` does useful work we should reuse:

- sanitizes host include/link environment variables before cross-build;
- calls `wasix/setup-wasix-deps.sh`;
- builds static WASIX OpenSSL if missing;
- configures CMake with:
 - `-DCMAKE_TOOLCHAIN_FILE=wasix/wasix-toolchain.cmake`
 - `-DEDGE_NAPI_PROVIDER=quickjs`
 - `-DEDGE_BUILD_CLI=ON`
 - `-DBUILD_TESTING=OFF`
- runs `wasm-opt --emit-exnref` when available;
- emits `edge.wasm` and `edgejs.wasm`;
- parses the `wasm` import section and fails if any N-API symbols remain imports.

That final link check is especially valuable and should come across almost unchanged.

25.3.6 Runtime/Library Workarounds Outside N-API

The theirs branch also changes these areas:

- `lib/internal/modules/esm/utils.js`
 - Uses `SafeMap` instead of `SafeWeakMap` when `process.versions.v8` is `0.0.0-node.0`.
 - This is a QuickJS `WeakRef`/GC workaround. It is useful to know about, but we should only port it with a focused repro.
- `src/edge_module_loader.cc`
 - Adds better last-N-API-error reporting when `unofficial_napi_contextify_compile_function` fails.
 - This is low-risk and worth porting.
- `src/edge_http_parser.cc`
 - Reads numeric callback return values and handles `HPE_PAUSED`.
 - Likely a correctness improvement independent of QuickJS; worth testing against HTTP parser behavior before porting.
- `src/edge_stream_base.cc`
 - Replaces encoding-aware string conversion with UTF-8 text to buffer copy and ignores `encoding_name`.
 - This is suspicious as a general change. Do not port blindly; it may have been a workaround for a missing QuickJS Buffer/encoding path.
- `wasix/src/wasix_compat.cc`
 - Adds WASIX stubs for `uv_get_free_memory`, `uv_get_total_memory`, `uv_resident_set_memory`, `uv_cpu_info`, `uv_interface_addresses`, `uv_free_interface_addresses`, and `OSL_set_max_threads`.
 - These are useful for WASIX link/runtime stability.

25.3.7 Docs/Plans

The theirs branch adds:

- `~/src/wasmer-io/edgejs/docs/quickjs.md`
- `~/src/wasmer-io/edgejs/plans/quickjs-wasm/plan.md`
- `~/src/wasmer-io/edgejs/plans/quickjs-wasm/task.md`

These capture the intended architecture: QuickJS compiled into the guest `wasm` binary as an embedded N-API provider, rather than using the host-provided N-API imports bridge.

25.4 Theirs `napi/quickjs`

The theirs provider lives at:

~/src/wasmer-io/edgejs/napi/quickjs

It contains only six files:

- CMakeLists.txt
- patch_quickjs_ng.cmake
- internal/napi_quickjs_env.h
- napi_quickjs_env.cc
- napi_quickjs_values.cc
- napi_quickjs_unofficial.cc

25.4.1 Build Model

The theirs provider fetches QuickJS-NG v0.14.0:

```
FetchContent_Declare(quickjs_ng_sources
  URL "https://github.com/quickjs-ng/quickjs/archive/refs/tags/v0.14.0.tar.gz"
  URL_HASH SHA256=928e9406add99eb8623348f2cfcd916eade9a263c60d42be79bc7aee4ee8453
)
```

It builds quickjs_ng from dtoa.c, libregexp.c, libunicode.c, and quickjs.c, then links napi_quickjs against it.

This is different from our current standalone provider, which expects a QuickJS library already built from ~/src/edgejs/quickjs into build-quickjs-napi/quickjs/libqjs.a.

25.4.2 QuickJS-NG Patch

patch_quickjs_ng.cmake mutates fetched quickjs.c at configure time to add a helper that recovers the promise from QuickJS promise resolving functions and to fire promise hooks around promise_reaction_job.

This likely made Edge async hooks or promise context behavior work sooner, but it is fragile:

- it depends on private QuickJS internals;
- it is tied to a specific QuickJS-NG source shape;
- it rewrites third-party source in the CMake build directory.

For our provider, we should treat this as a behavioral clue, not as code to copy directly. If promise hook parity requires QuickJS internals, prefer an explicit patch file or a small local QuickJS fork/change rather than CMake string replacement.

25.4.3 Runtime Shape

The theirs provider stores nearly all state in public structs:

- napi_value__
- napi_ref__
- napi_handle_scope__
- napi_escapable_handle_scope__
- napi_callback_info__
- napi_deferred__
- napi_async_work__
- napi_env__

It tracks JSRuntime*, JSContext*, last error, pending exception, promise hooks, cleanup hooks, refs, values, native callbacks, Edge environment hooks, and context-token callbacks directly on napi_env__.

This is straightforward for a prototype, but it is much less maintainable than our internal class split.

25.4.4 API Surface

The theirs provider implements or stubs a broad set of symbols:

- core N-API value creation, conversion, properties, references, exceptions, arrays, buffers, typed arrays, promises, dates, BigInts;
- env cleanup hooks;
- callback scopes;
- async work;
- threadsafe-function stubs/defaults;
- `napi_get_uv_event_loop` as unsupported;
- many unofficial `napi_*` hooks used by Edge;
- contextify run/compile/cached-data helpers;
- a small set of module-wrap status/default helpers;
- stable empty/default responses for V8-only profiling/heap APIs.

This broad symbol ownership explains why the EdgeJS branch disabled parts of `src/node_api.cc` under `EDGE_QUICKJS_OWN_NAPI_SYMBOLS`.

25.5 Our napi/quickjs

Our provider lives at:

`~/src/edgejs/napi/quickjs`

It is at 06207826 and has the newer refactor:

- `src/js_native_api_quickjs.cc`
- `src/unofficial_napi.cc`
- small internal classes under `src/internal/`
- standalone test runners under `tests/runners/`
- `napi/quickjs/tests/CMakeLists.txt`

25.5.1 Strengths

Compared with the theirs provider, ours has:

- better internal ownership boundaries;
- RAII-ish wrappers for values, refs, scopes, callbacks, cleanup hooks, externals, and functions;
- a dedicated QuickJS N-API test suite;
- recent passing baseline of the QuickJS N-API suite;
- more complete unofficial N-API behavior in several areas:
 - env creation/release from an existing QuickJS context for tests;
 - contextify make/run/dispose/compile/cache-data;
 - source-map/error formatting helpers;
 - structured clone and serialize/deserialize via QuickJS object read/write;
 - heap/memory/hash approximations;
 - module-wrap APIs with explicit stable fallbacks.

25.5.2 Gaps For EdgeJS Integration

Our EdgeJS root does not yet have the theirs top-level integration:

- `~/src/edgejs/CMakeLists.txt` only accepts `bundled-v8|imports`.
- `EDGE_EMBEDDED_NAPI_PROVIDER` is not used by Edge runtime hooks yet.
- `napi/include/js_native_api.h` still marks wasm N-API symbols as imports even when an embedded provider is desired.
- `quickjs-wasm/` packaging does not exist in our root.

Our provider may also need additional public N-API symbols if we choose the theirs `EDGE_QUICKJS_OWN_NAPI_SYMBOLS` model. Examples visible in the theirs provider but not obviously present in our current `js_native_api` surface include:

- async work APIs;
- callback scope APIs;
- threadsafe-function APIs;
- `napi_get_uv_event_loop`;
- some `node_api_*` helpers;
- finalizer/post-finalizer style glue currently supplied by Edge's `src/node_api.cc`.

We should decide symbol ownership before porting the `src/node_api.cc` guard.

25.6 Recommended Development Plan

25.6.1 1. Port The Provider Selection Plumbing

Bring the top-level CMake pieces into `~/src/edgejs`:

- add `quickjs` to `EDGE_NAPI_PROVIDER`;
- add the `napi/quickjs` subdirectory for that provider;
- link `edge_node_api` and `edge_runtime` to `napi_quickjs`;
- set `EDGE_ALLOW_UNDEFINED_IMPORTS=OFF` for `quickjs`;
- introduce `EDGE_EMBEDDED_NAPI_PROVIDER` for embedded providers.

Do this without immediately copying `EDGE_QUICKJS_OWN_NAPI_SYMBOLS` behavior unless the link requires it.

25.6.2 2. Add Embedded WASM Export Semantics

Port the `js_native_api.h` export/import distinction:

- `wasm + imports` provider: keep N-API declarations as imports;
- `wasm + embedded` provider: make N-API declarations default-visible exports.

This is required for a final QuickJS `wasm` that has no imported N-API symbols.

25.6.3 3. Make Our Provider Embeddable By EdgeJS CMake

Adapt our `~/src/edgejs/napi/quickjs/CMakeLists.txt` so it can work both standalone and as an EdgeJS subdirectory:

- honor parent-provided `NAPI_INCLUDE_ROOT`;
- allow `NAPI_QUICKJS_BUILD_TESTS=OFF` for Edge builds;
- avoid hardcoding `build-quickjs-napi/quickjs/libqjs.a` in the Edge provider path;
- decide whether Edge/WASIX should use our checked-in `/quickjs` build or a vendored/fetched QuickJS-NG source.

Conservative recommendation: first use the existing `/quickjs` build approach already passing our tests, then revisit QuickJS-NG only if WASIX build support requires it.

25.6.4 4. Decide N-API Symbol Ownership

Before disabling Edge's fallback `src/node_api.cc` symbols, run a link attempt and list missing/duplicate symbols.

Preferred first pass:

- keep `src/node_api.cc` fallbacks where they are provider-neutral;
- let `napi_quickjs` own core engine-specific symbols;
- only add `EDGE_QUICKJS_OWN_NAPI_SYMBOLS` after our provider implements the same symbol breadth as the theirs provider.

This should avoid forcing async/threadsafe/uv-loop stubs into our provider before we know Edge actually needs them for smoke tests.

25.6.5 5. Port Runtime Hooks And Low-Risk Fixes

Port these early:

- `EDGE_EMBEDDED_NAPI_PROVIDER` in `edge_environment.cc`;
- `EDGE_EMBEDDED_NAPI_PROVIDER` in `edge_task_queue.cc`;
- better contextify compile error reporting in `edge_module_loader.cc`;
- WASIX compatibility stubs in `wasix/src/wasix_compat.cc`.

Hold back or gate these until reproduced:

- `SafeWeakMap` to `SafeMap` workaround in ESM utils;
- `edge_stream_base.cc` string-write encoding change;
- HTTP parser callback return-value change, unless tests show it is required.

25.6.6 6. Add The WASIX Package Scaffold

Port `quickjs-wasm/` into our root after native provider linking works.

Keep the theirs build script's `wasm` import-section check. That check should be part of the acceptance criteria for QuickJS/WASIX.

25.6.7 7. Verification Sequence

Suggested order:

1. `make build-napi-quickjs`
2. `make test-napi-quickjs-only`
3. Configure native Edge with `-DEDGE_NAPI_PROVIDER=quickjs -DEDGE_BUILD_CLI=OFF` and verify `napi_quickjs`, `edge_node_api`, and `edge_runtime` link.
4. Configure native Edge CLI if feasible and run:
 - `edge --version`
 - `edge -e "console.log('hello from quickjs')"`
5. Configure WASIX with `EDGE_NAPI_PROVIDER=quickjs`.
6. Verify final `wasm` has no imported N-API symbols.
7. Run Wasmer smoke tests:
 - `wasmer run quickjs-wasm/ -- --version`
 - `wasmer run quickjs-wasm/ -- -e "console.log(1 + 1)"`
 - builtin require smoke tests for `buffer`, `events`, `path`, and `http`.

25.7 Open Questions

- Should the embedded provider use our existing QuickJS source tree or switch to QuickJS-NG for WASIX?
- Do we need promise hooks deep enough to require QuickJS source changes?
- Should `edge_node_api` remain a provider-neutral fallback library, or should each embedded provider own every N-API symbol?
- Which theirs runtime workarounds are true QuickJS requirements versus temporary gaps in the prototype provider?
- What is the minimum EdgeJS smoke-test matrix for declaring the first development phase done?

25.8 Proposed First Implementation Slice

Start with a narrow integration branch in `~/src/edgejs`:

1. Add `EDGE_NAPI_PROVIDER=quickjs` to top-level CMake.
2. Add `EDGE_EMBEDDED_NAPI_PROVIDER` macro handling.

3. Update `napi/include/js_native_api.h` for embedded wasm exports.
4. Make `napi/quickjs` build as a subdirectory with tests disabled.
5. Attempt native link and resolve only the symbol issues that appear.
6. Keep all runtime workaround ports separate and test-driven.

This should let us preserve our cleaner provider while borrowing the theirs branch's proven EdgeJS integration shape.

26 QuickJS N-API EdgeJS execution and bootstrap troubleshooting

		Remarks
Status	[done]	Historical bootstrap and contextify investigation.
Severity	Low	Current contextify status is tracked in troubleshooting/node-compat/napi/003_contextify.md.

Date: 2026-05-04

26.1 Scope

This note records the execution and troubleshooting work done after 001_merge_analysis.md.

Goal for this pass:

```
./build-edge-quickjs-cli/edge ./quickjs-wasm/echo-server.js
```

should bootstrap EdgeJS with the QuickJS N-API provider, run the user script, bind the HTTP server, and answer a request.

All our code changes were made under:

- ~/src/edgejs
- ~/src/edgejs/napi

26.2 Initial Symptom

Running the script appeared to fail at QuickJS runtime teardown:

Assertion failed: (list_empty(&rt->gc_obj_list)), function JS_FreeRuntime, file quickjs.c, line 2

The important finding was that this assertion was not the first/root failure. It was cleanup noise after EdgeJS had already failed during bootstrap.

We used LLDB to break at JS_FreeRuntime and inspect error_out before the teardown assertion terminated the process. The original error was:

Failed to execute internal/bootstrap/node: undefined[Error: Failed to execute builtin 'internal/

So the user script was not running yet. EdgeJS was failing while executing internal/bootstrap/node.

26.3 Instrumentation Added

26.3.1 Edge Builtin/Binding Tracing

In ~/src/edgejs/src/edge_module_loader.cc:

- Added gated builtin compile tracing:

```
EDGE_TRACE_BUILTINS=1
```

This prints lines such as:

```
EDGE_TRACE_BUILTINS compile internal/vm
```

- Added gated internalBinding() tracing:

`EDGE_TRACE_INTERNAL_BINDING=1`

This prints requests, cache hits, resolved value type, and contextify resolver state.

- Added `DescribeAndClearPendingException()` and used it when `ExecuteBuiltinFromNative()` fails in `napi_call_function()`.

This preserves the thrown JS stack in the higher-level native error instead of returning a generic builtin execution failure.

26.3.2 QuickJS Contextify Compile Diagnostics

In `~/src/edgejs/napi/quickjs/src/unofficial_napi.cc`:

- Added contextify compile exception annotations:
 - `node:quickjsContextifyCompile`
 - `node:quickjsCompileResourceName`
 - `node:quickjsCompileBuiltinId`
 - `node:quickjsCompileLineOffset`
 - `node:quickjsCompileColumnOffset`
 - `node:quickjsCompileQuickJSLine`
 - `node:quickjsCompileMappedLine`
- Added optional compile tracing:

`EDGE_TRACE_QUICKJS_CONTEXTIFY=1`

or:

`EDGE_TRACE_BUILTINS=1`

When enabled, compile exceptions get a one-line summary prepended to the stack.

- Preserved the caught QuickJS exception in N-API's pending/last-exception slot before returning `napi_pending_exception`.
- Added `sourceURL` to Function-constructor compiled source so the result object can record the intended source resource name.

26.4 Bootstrap Failure 1: internal/vm

The first real JavaScript failure was:

`TypeError: Cannot convert undefined or null to object`

The chain was:

```
• /lib/internal/bootstrap/node.js:303
} = require('internal/process/execution');
• /lib/internal/process/execution.js:41
} = require('internal/vm');
• /lib/internal/vm.js:13
runInContext,
```

The failing code in `internal/vm.js` is:

```
const {
  runInContext,
} = ContextifyScript.prototype;
```

`internalBinding('contextify')` did resolve, but `ContextifyScript.prototype` was missing.

26.4.1 Root Cause

ResolveContextifyBinding() created ContextifyScript with napi_create_function(), then tried to read and modify its prototype.

Our QuickJS napi_create_function() creates a callable QuickJS C function, but it does not install a JS constructor prototype. Therefore:

ContextifyScript.prototype

was undefined, and destructuring from it triggered QuickJS JS_ToObject() on undefined.

26.4.2 Current Status

Changed ResolveContextifyBinding() to create ContextifyScript with napi_define_class() and define prototype methods there:

- runInContext
- createCachedData

This gives Node's internal/vm the constructor shape it expects.

After this change, bootstrap moved past internal/vm.

26.5 Runtime Failure 2: Missing Dispose Symbols

After fixing ContextifyScript, EdgeJS reached the user script and started loading http, but failed while evaluating _http_server.

The new chain reached:

- /lib/_http_server.js:617

```
Server.prototype[SymbolAsyncDispose] = assignFunctionName(SymbolAsyncDispose, async function() {
```

The stack included:

```
at get description (native)
at call (native)
at assignFunctionName (<input>:943:61)
```

The relevant JS was:

- /lib/internal/util.js:941

```
const symbolDescription = SymbolPrototypeGetDescription(name);
```

26.5.1 Root Cause

The embedded QuickJS runtime did not provide Node's expected well-known symbols:

- Symbol.dispose
- Symbol.asyncDispose

Node's primordials therefore captured SymbolAsyncDispose as undefined. Later, _http_server passed that undefined into assignFunctionName(), which attempted to read Symbol.prototype.description from a non-symbol.

26.5.2 Current Status

Added a QuickJS env initialization shim in /napi/quickjs/src/unofficial_napi.cc:

- EnsureNodeWellKnownSymbols(ctx)
- EnsureSymbolProperty(ctx, symbol_ctor, "dispose", "Symbol.dispose")
- EnsureSymbolProperty(ctx, symbol_ctor, "asyncDispose", "Symbol.asyncDispose")

May 10, 2026 update: this regression reappeared after later QuickJS N-API refactors. The shim was restored in `unofficial_napi_create_env_from_context()`, `require('http')` now loads, `examples/http-server.js` starts with the native QuickJS CLI, and `napi/tests/js-native-api/13_symbol/test.js` now asserts both well-known symbols and descriptions.

This runs immediately after `JS_NewContext(rt)` in `unofficial_napi_create_env_with_options()`, before EdgeJS executes `internal/per_context/primordials`.

After this change, the http path got through `_http_server`.

26.6 Verification

Rebuilt the native QuickJS Edge CLI:

```
cmake --build build-edge-quickjs-cli --target edge -j4
```

Verified the target command:

```
./build-edge-quickjs-cli/edge ./quickjs-wasm/echo-server.js
```

Expected output:

```
quickjs edge echo listening on 3000
```

Verified a request:

```
curl -sS http://127.0.0.1:3000/
```

Response:

```
quickjs edge echo: GET /
```

Important testing note: direct server runs and local `curl` from Codex's sandbox can be misleading because the sandbox may block bind/connect operations. The successful verification was done with the command allowed to run outside the sandbox, matching the behavior expected from a normal user shell.

26.7 Current State

The echo server works when run normally outside the sandbox:

```
./build-edge-quickjs-cli/edge ./quickjs-wasm/echo-server.js
```

and responds to HTTP requests on port 3000.

The main EdgeJS-side functional fix is:

- `ContextifyScript` now uses `napi_define_class()` so `ContextifyScript.prototype.runInContext` exists for `internal/vm.js`.

The main QuickJS-side functional fix is:

- QuickJS env creation now installs Node-required `Symbol.dispose` and `Symbol.asyncDispose` when missing.

The main diagnostic improvements are:

- builtin compile tracing via `EDGE_TRACE_BUILTINS`
- internal binding tracing via `EDGE_TRACE_INTERNAL_BINDING`
- contextify compile exception annotations via `EDGE_TRACE_QUICKJS_CONTEXTIFY`
- better propagated JS stacks when native builtin execution fails

26.8 Things To Clean Up Or Decide

26.8.1 Debug Tracing

The tracing is gated behind environment variables, so it is not noisy by default. Still, before a final cleanup pass we should decide whether to keep all of these:

- `EDGE_TRACE_BUILTINS`
- `EDGE_TRACE_INTERNAL_BINDING`
- `EDGE_TRACE_QUICKJS_CONTEXTIFY`

`EDGE_TRACE_BUILTINS` and `contextify` exception annotations are useful. The broader `EDGE_TRACE_INTERNAL_BINDING` trace may be more temporary.

26.8.2 QuickJS Teardown Assertion

The original `JS_FreeRuntime` assertion is not the current blocker for `echo-server.js` working. It can still appear if the process exits during an earlier failure path or when testing inside the sandbox.

We explicitly deferred that cleanup/GC assertion while chasing bootstrap and script execution. It should be handled separately with a focused QuickJS lifetime/handle-scope audit, not mixed into bootstrap fixes.

26.8.3 N-API Test Coverage

Existing targeted QuickJS N-API tests were added/kept around:

- `napi_create_arraybuffer()` accepting `data == nullptr`
- private symbol creation accepting `NAPI_AUTO_LENGTH`
- `contextify` compile/cached-data behavior

Before considering this development slice done, rerun:

```
CMAKE_BUILD_TYPE=Debug make build-napi-quickjs
make test-napi-quickjs-only
```

and then rebuild the Edge CLI again.

26.9 Commands That Were Useful

Capture pre-teardown failure state:

```
lldb --batch \  
-o "breakpoint set -n JS_FreeRuntime" \  
-o run \  
-o "frame select 3" \  
-o "frame variable error_out" \  
-- ./build-edge-quickjs-cli/edge ./quickjs-wasm/echo-server.js
```

Trace builtin compilation:

```
EDGE_TRACE_BUILTINS=1 \  
./build-edge-quickjs-cli/edge ./quickjs-wasm/echo-server.js
```

Trace internal binding resolution:

```
EDGE_TRACE_INTERNAL_BINDING=1 \  
./build-edge-quickjs-cli/edge ./quickjs-wasm/echo-server.js
```

Trace QuickJS `contextify` compile failures:

```
EDGE_TRACE_QUICKJS_CONTEXTIFY=1 \  
./build-edge-quickjs-cli/edge ./quickjs-wasm/echo-server.js
```

Verify the working server:

```
./build-edge-quickjs-cli/edge ./quickjs-wasm/echo-server.js  
curl -sS http://127.0.0.1:3000/
```

27 Edge QuickJS REPL TTY troubleshooting

		Remarks
Status	[done]	Historical REPL TTY/readline investigation.
Severity	Low	Current promise/microtask status is tracked in node-compat issue pages.

27.1 Context

This note captures the debugging session for the QuickJS-backed Edge REPL hang. The user-visible symptom was:

```
./build-edge-quickjs-cli/edge
```

The REPL banner and prompt appeared, but typed input was not echoed, evaluated, or otherwise handled.

The goal of this investigation was to follow the TTY read path from libuv's kqueue polling through Edge's native TTY wrapper and into the JavaScript REPL stack, then identify where input stopped moving.

27.2 Short version

The TTY is reading correctly.

The typed bytes wake `kevent()`, libuv dispatches the read watcher, Edge's native TTY `OnRead()` receives the bytes, `EdgeStreamBaseOnUvRead()` forwards them to JavaScript, and `lib/internal/stream_base_commons.js:onStream` receives a valid `FastBuffer`.

The bytes do not reach `readline` because the REPL input stream has already been paused by REPL history initialization. History initialization pauses `readline` while it opens and prepares `/Users/sadhbh/.node_repl_history`.

The immediate hang is inside the async history initialization flow:

```
const hnd = await fs.promises.open(this[kHistoryPath], 'a+', 0o0600);
await hnd.close();
```

Native file handle close completes and resolves its N-API promise, but the JavaScript continuation after `await hnd.close()` does not run. The current suspect is QuickJS promise/job behavior around the `FileHandle.close()` wrapper, especially:

```
SafePromisePrototypeFinally(
  this[kHandle].close(),
  () => { this[kClosePromise] = undefined; },
)
```

So the REPL is not blocked because kqueue or TTY read is broken. It is blocked because REPL history paused `stdin` and an `fs` promise/finally chain never resumes the history initialization.

27.3 LLDB investigation

The requested breakpoint was placed at:

```
~/src/edgejs/deps/uv/src/unix/kqueue.c:293
```

When typing into the hung REPL, `kevent()` returned a real readable event:

```

nfds = 1
timeout = -1
events[0].ident = 12
events[0].filter = -1    // EVFILT_READ
events[0].flags = 1
events[0].data = 1

```

Stepping forward in `uv__io_poll()` showed:

```

fd = 12
loop->watchers[fd] = 0x...
w->fd = 12
w->pevents = 1
w->events = 1
w->cb = uv__stream_io at deps/uv/src/unix/stream.c:1189

```

This means the kernel event was real, libuv had an active watcher for the fd, and the event was dispatched into libuv's stream read machinery.

The native stack from the read looked like:

```

uv__io_poll
uv__stream_io
uv__read
edge_tty_wrap.cc::OnRead
EdgeStreamBaseOnUvRead
EdgeStreamEmitRead
DefaultOnRead
CallJsOnRead
EdgeAsyncWrapMakeCallback

```

Inside `edge_tty_wrap.cc::OnRead`, LLDB showed:

```

nread = 4
buf->base starts with "2+3\n"
buf->len = 65536

```

The important detail is `nread = 4`; the backing allocation contained stale bytes after the first four bytes, but the stream contract is to consume only `nread` bytes. This was not the source of the hang.

27.4 Native Edge callback result

The read callback reached `CallJsOnRead()` in:

```
~/src/edgejs/src/edge_stream_base.cc
```

The stream state was set:

```

SetStreamState(base->env, kEdgeReadBytesOnError, static_cast<int32_t>(nread));
SetStreamState(base->env, kEdgeArrayBufferOffset, static_cast<int32_t>(offset));

```

The JS callback existed and was callable. `EdgeAsyncWrapMakeCallback()` returned `napi_ok`, produced a result value, and there was no pending exception immediately after the callback.

That ruled out:

- kqueue not waking
- libuv watcher not registered
- native TTY `OnRead()` not firing
- missing onread callback
- immediate JS exception from `onStreamRead()`

27.5 JavaScript stream trace

Temporary tracing was added behind `EDGE_TRACE_TTY` in:

```
~/src/edgejs/src/edge_tty_wrap.cc
~/src/edgejs/src/edge_stream_base.cc
~/src/edgejs/lib/internal/stream_base_commons.js
~/src/edgejs/lib/internal/readline/emitKeypressEvents.js
~/src/edgejs/lib/internal/readline/interface.js
~/src/edgejs/lib/internal/streams/readable.js
~/src/edgejs/lib/internal/repl/history.js
~/src/edgejs/src/internal_binding/binding_fs.cc
```

After typing `2+3\n`, the JavaScript stream trace showed:

```
EDGE_TRACE_TTY onread fd=0 nread=4 len=65536
EDGE_TRACE_TTY js stream_base onStreamRead nread= 4 destroyed= false readableLength= 0 flowing= 1
EDGE_TRACE_TTY js stream_base push-before offset= 0 buf.length= 4 buf= 2+3
EDGE_TRACE_TTY js stream_base push-after result= false readableLength= 4 flowing= false
EDGE_TRACE_TTY js stream_base push-backpressure-readStop
```

This proves:

- `onStreamRead()` receives the input.
- The `FastBuffer` is correct and has length 4.
- The stream is not flowing.
- `stream.push(buf)` buffers the data instead of emitting it to readline.
- Because the high-water mark is zero and the stream is paused, `push()` returns false and `stream_base` calls `handle.readStop()`.

So the REPL looked locked because `stdin` was paused at the JS stream layer.

27.6 Why stdin was paused

Tracing in `lib/internal/streams/readable.js`, `lib/internal/readline/interface.js`, and `lib/internal/repl/history.js` showed the sequence:

```
Welcome to Edge.js 0.0.0-f287153 (Node.js v24.13.2).
Type ".help" for more information.
> EDGE_TRACE_TTY js repl_history initialize pause /Users/sadhbh/.node_repl_history
EDGE_TRACE_TTY js readline pause
EDGE_TRACE_TTY js repl_history open a+ begin
EDGE_TRACE_TTY js readable resume_ before flowing= false data= 1 readable= 0
EDGE_TRACE_TTY readStart fd=0 rc=0 has_ref=1 loop_alive=1
EDGE_TRACE_TTY js readable resume_ after flowing= false data= 1 readable= 0
EDGE_TRACE_TTY readStop ignored fd=0 raw_mode=true
EDGE_TRACE_TTY js repl_history open a+ done
```

The pause comes from `ReplHistory.initialize()`:

```
this[kContext].pause();
this[kInitializeHistory](onReadyCallback).catch(...);
```

That is normal Node behavior. REPL history pauses input while it initializes persistent history, and it resumes the context once history is ready.

In our QuickJS run, history initialization starts but never reaches the resume point.

27.7 File handle close trace

History initialization reached:

```
const hnd = await fs.promises.open(this[kHistoryPath], 'a+', 0o0600);
await hnd.close();
```

The trace printed:

```
EDGE_TRACE_TTY js repl_history open a+ begin
EDGE_TRACE_TTY js repl_history open a+ done
```

It did not print:

```
EDGE_TRACE_TTY js repl_history close a+ done
```

Native tracing around FileHandleClose showed:

```
EDGE_TRACE_TTY fs FileHandleClose start fd=13 rc=0 loop=0x...
EDGE_TRACE_TTY fs FileHandleClose after fd=13 result=0
EDGE_TRACE_TTY fs FileHandleClose finish fd=13 result=0 deferred=0x...
```

This proves:

- `fs.promises.open()` completes.
- `FileHandle.close()` enters native code.
- `uv_fs_close()` is submitted successfully.
- libuv calls the close completion callback.
- native resolves the N-API deferred promise.
- `EdgeRunCallbackScopeCheckpoint(env)` is called after resolving.

Despite that, the JS async function waiting on `await hnd.close()` does not continue.

27.8 VSCode LLDB trace confirmation

Running the same build from VSCode LLDB with:

```
"env": {
  "EDGE_TRACE_TTY": "1"
}
```

produced the same sequence:

```
Welcome to Edge.js 0.0.0-f287153 (Node.js v24.13.2).
Type ".help" for more information.
EDGE_TRACE_TTY readStop fd=0 rc=0 has_ref=1 loop_alive=0
EDGE_TRACE_TTY js readable resume before flowing= null data= 1 readable= 0
EDGE_TRACE_TTY js readable resume after flowing= true data= 1 readable= 0
EDGE_TRACE_TTY setRawMode fd=0 flag=true rc=0
EDGE_TRACE_TTY js readable resume before flowing= true data= 1 readable= 0
EDGE_TRACE_TTY js readable resume after flowing= true data= 1 readable= 0
> EDGE_TRACE_TTY js repl_history initialize pause /Users/sadhbh/.node_repl_history
EDGE_TRACE_TTY js readline pause
EDGE_TRACE_TTY js repl_history open a+ begin
EDGE_TRACE_TTY js readable resume_ before flowing= false data= 1 readable= 0
EDGE_TRACE_TTY readStart fd=0 rc=0 has_ref=1 loop_alive=1
EDGE_TRACE_TTY js readable resume_ after flowing= false data= 1 readable= 0
EDGE_TRACE_TTY readStop ignored fd=0 raw_mode=true
EDGE_TRACE_TTY js repl_history open a+ done
EDGE_TRACE_TTY fs FileHandleClose start fd=13 rc=0 loop=0x...
EDGE_TRACE_TTY fs FileHandleClose after fd=13 result=0
EDGE_TRACE_TTY fs FileHandleClose finish fd=13 result=0 deferred=0x...
```

The important missing line is still:

```
EDGE_TRACE_TTY js repl_history close a+ done
```

When a key was pressed after that point, the trace showed:

```
EDGE_TRACE_TTY onread fd=0 nread=1 len=65536
EDGE_TRACE_TTY js stream_base onStreamRead nread= 1 destroyed= false readableLength= 0 flowing= 1
EDGE_TRACE_TTY js stream_base push-before offset= 0 buf.length= 1 buf= v
EDGE_TRACE_TTY js stream_base push-after result= false readableLength= 1 flowing= false
EDGE_TRACE_TTY js stream_base push-backpressure-readStop
EDGE_TRACE_TTY readStop ignored fd=0 raw_mode=true
```

This confirms the same conclusion from inside VSCode:

- TTY input still reaches native code.
- The typed byte is delivered to `onStreamRead()`.
- The stream is paused: `isPaused= true, flowing= false`.
- The byte is buffered and readline does not receive it.
- The pause comes from REPL history initialization, not from the debugger or kqueue.

27.9 Quick confirmation workaround

To confirm the diagnosis without fixing promise handling yet, disable persistent REPL history in the VSCode launch configuration:

```
{
  "type": "lldb",
  "request": "launch",
  "name": "Debug Edge QuickJs",
  "program": "${workspaceFolder}/build-edge-quickjs-cli/edge",
  "args": [],
  "cwd": "${workspaceFolder}",
  "env": {
    "EDGE_TRACE_TTY": "1",
    "NODE_REPL_HISTORY": ""
  }
}
```

`NODE_REPL_HISTORY=""` makes `ReplHistory.initialize()` take the disabled-history path, which avoids the `fs.promises.open() / FileHandle.close()` setup that currently wedges the REPL. If the REPL becomes interactive with that setting, it is another confirmation that the core issue is the file-handle close promise continuation, not TTY reading.

27.10 Current hypothesis

The current best hypothesis is a QuickJS promise/job issue, not a TTY issue.

The wrapper in `lib/internal/fs/promises.js` returns:

```
this[kClosePromise] = SafePromisePrototypeFinally(
  this[kHandle].close(),
  () => { this[kClosePromise] = undefined; },
);
```

`this[kHandle].close()` is the native N-API promise. Native resolves that promise, but the promise chain created by `SafePromisePrototypeFinally()` does not appear to settle in a way that wakes the awaiting async function.

Relevant implementation areas:

```
~/src/edgejs/napi/quickjs/src/js_native_api_quickjs.cc
~/src/edgejs/napi/quickjs/src/unofficial_napi.cc
~/src/edgejs/src/edge_runtime.cc
~/src/edgejs/src/internal_binding/binding_fs.cc
```

```
~/src/edgejs/lib/internal/per_context/primordials.js
~/src/edgejs/lib/internal/fs/promises.js
```

In particular:

- `napi_resolve_deferred()` in the QuickJS backend calls the stored promise resolve function.
- `unofficial_napi_process_microtasks()` calls `JS_ExecutePendingJob()` until the QuickJS job queue is empty.
- `EdgeRunCallbackScopeCheckpoint()` invokes `unofficial_napi_process_microtasks()` when appropriate.
- Native file close calls `EdgeRunCallbackScopeCheckpoint()` after resolving the deferred.

The missing piece is why the promise/finally/async-await continuation does not run even after the native deferred has been resolved and a callback checkpoint is requested.

27.11 Temporary probe: ignoring readStop

One temporary probe changed `edge_tty_wrap.cc` so `readStop()` was ignored for fd 0 while raw mode was enabled.

That was not a real fix. It was useful because it proved native TTY reads could continue and that `OnRead()` saw typed bytes. With that probe, input still did not reach readline because JS remained paused.

This probe should not be treated as the final solution.

27.12 What to do next

1. Build a small reproduction around `fs.promises.open(...).then(h => h.close())` and `await h.close()` under the QuickJS Edge CLI.
2. Add targeted tracing or tests for `SafePromisePrototypeFinally()` with a native N-API promise:

```
const p = nativePromise();
await SafePromisePrototypeFinally(p, () => {});
```
3. Inspect whether `napi_resolve_deferred()` enqueues the expected QuickJS jobs and whether all jobs are executed by `unofficial_napi_process_microtasks()`.
4. Check whether QuickJS's `Promise.prototype.finally` behavior with `SafePromise` subclassing matches what Node's `primordials` expect.
5. Once promise continuation works, remove the temporary TTY/readline/history/fs instrumentation and remove the `readStop()` ignore probe.
6. Re-test:

```
./build-edge-quickjs-cli/edge
```

Expected behavior: the REPL prompt accepts input, echoes/evaluates commands, and remains interactive.

27.13 Useful commands

Build:

```
cmake --build build-edge-quickjs-cli --target edge -j4
```

Run traced REPL:

```
EDGE_TRACE_TTY=1 ./build-edge-quickjs-cli/edge
```

Run traced REPL with persistent history disabled:

```
EDGE_TRACE_TTY=1 NODE_REPL_HISTORY="" ./build-edge-quickjs-cli/edge
```

LLDB entry point:


```
lldb ./build-edge-quickjs-cli/edge
```

Breakpoint requested for the kqueue investigation:

```
breakpoint set --file ~/src/edgejs/deps/uv/src/unix/kqueue.c --line 293
```

Useful native breakpoints:

```
breakpoint set --file src/edge_tty_wrap.cc --name OnRead
```

```
breakpoint set --file src/edge_stream_base.cc --name EdgeStreamBaseOnUvRead
```

```
breakpoint set --file src/edge_stream_base.cc --name CallJsOnRead
```

27.14 Bottom line

The REPL is not failing because QuickJS Edge cannot read from TTY. It can.

The REPL is failing because persistent history pauses readline and then gets stuck awaiting `FileHandle.close()`. Native close resolves the promise, but the JavaScript promise/finally/await continuation does not run. The next fix should focus on QuickJS N-API promise resolution and microtask/job draining, especially with `SafePromisePrototypeFinally()`.

28 Edge QuickJS REPL promise hooks and microtasks

		Remarks
Status	[done]	Historical promise hook and microtask investigation.
Severity	Low	Current status is tracked in troubleshooting/node-compat/napi/006_microtasks.md.

28.1 Context

The QuickJS-backed Edge CLI could enter REPL mode only when persistent REPL history was disabled:

```
NODE_REPL_HISTORY="" ./build-edge-quickjs-cli/edge
```

With normal history enabled, the prompt appeared but input was stuck. TTY tracing showed that native TTY reads were working and bytes reached the JS stream layer, but readline stayed paused. The pause came from lib/internal/repl/history.js, which pauses the REPL while it initializes persistent history.

The last observed history trace before the lock was:

```
open a+ done
FileHandleClose finish ...
```

The expected next trace was:

```
close a+ done
```

So the stalled point was the async continuation after:

```
await hnd.close();
```

This made the problem look like a promise/async continuation issue rather than a TTY or libuv read issue.

28.2 What the colleague branch showed

The colleague branch was useful as implementation evidence, but not as a native-CLI proof. Its quickjs-wasm/build.sh only builds the WASIX target with:

```
-DCMAKE_TOOLCHAIN_FILE=wasix/wasix-toolchain.cmake
-DEDGE_NAPI_PROVIDER=quickjs
-DEDGE_BUILD_CLI=ON
```

It does not demonstrate that a native QuickJS Edge REPL works.

Still, two relevant ideas were present in the colleague N-API implementation:

1. Register a QuickJS runtime promise hook with JS_SetPromiseHook(...).
2. Preserve and restore continuation_preserved_embedder_data around promise jobs.

Their QuickJS-NG CMake patch also injected JS_PROMISE_HOOK_BEFORE and JS_PROMISE_HOOK_AFTER around promise_reaction_job(). Our local QuickJS had promise hook types and JS_SetPromiseHook(...), but ordinary promise reaction jobs were not emitting before/after hooks.

28.3 Root cause

Our N-API QuickJS backend stored promise hook callbacks from `unofficial_napi_set_promise_hooks()`, but it did not register a QuickJS runtime hook and did not preserve async context frames across promise jobs.

Separately, our local QuickJS did not emit `JS_PROMISE_H00K_BEFORE` and `JS_PROMISE_H00K_AFTER` around normal `promise_reaction_job()` execution. That means even a registered N-API hook would not see the core `.then / await` continuations that Node's async context and REPL history initialization depend on.

One extra wrinkle: QuickJS represents `async/await` continuations with `JS_CLASS_ASYNC_FUNCTION_RESOLVE / JS_CLASS_ASYNC_FUNCTION_REJECT`, not only with normal promise resolve/reject function objects. A direct copy of the colleague helper would recover promise identity for normal promise reactions, but could still miss `async-function` continuations.

28.4 Why this currently requires a QuickJS source change

For the current vendored QuickJS source, the fix cannot live purely in the N-API layer. The missing transition happens inside QuickJS's internal `promise_reaction_job()`. By the time an expression such as:

```
await hnd.close();
```

resumes, QuickJS is running an engine-owned queued promise job, not calling back through N-API. If `promise_reaction_job()` does not emit `JS_PROMISE_H00K_BEFORE` and `JS_PROMISE_H00K_AFTER`, the N-API backend has no reliable point where it can restore and then unwind `continuation_preserved_embedder_data`.

So there are three realistic paths:

1. Keep the small vendored QuickJS patch. This is what we did, and it is the direct fix for our current source tree.
2. Upgrade or switch to a QuickJS/QuickJS-NG version that already emits promise hooks around normal reaction jobs. To qualify, its `promise_reaction_job()` must call the runtime promise hook before and after invoking the reaction handler.
3. Monkey-patch Promise behavior in JS. This is theoretically possible but not recommended: it is fragile, changes global Promise behavior, and is likely to miss `async/await` or other internal promise paths.

In short: we do not inherently need a permanent fork of QuickJS, but we do need an engine that exposes before/after promise reaction hooks. Our current vendored QuickJS does not, so this repo currently needs the source patch unless we move to an upstream version with equivalent behavior.

28.5 Current Status

28.5.1 QuickJS runtime

File:

```
~/src/edgejs/quickjs/quickjs.c
```

Added helpers near the promise reaction data definitions:

- `js_promise_function_promise(...)`
- `js_promise_reaction_promise(...)`

The first helper recovers the promise associated with normal QuickJS promise resolve/reject functions.

The second helper also handles `async-function` resolve/reject objects by walking from the `async-function` resolver back to the `async-function`'s public promise. This is the important difference from the colleague patch for the REPL-history case, because `await hnd.close()` resumes through the `async-function` machinery.

Then `promise_reaction_job()` now:

1. Recovers the promise identity from the fulfillment or rejection resolving function.
2. Emits `JS_PROMISE_H00K_BEFORE` before invoking the reaction handler.
3. Emits `JS_PROMISE_H00K_AFTER` after invoking the reaction handler.

28.5.2 QuickJS N-API backend

File:

~/src/edgejs/napi/quickjs/src/unofficial_napi.cc

The original implementation added per-env storage for promise context frames:

```
std::unordered_map<void *, JSValue> promise_context_frames;
std::vector<JSValue> promise_context_frame_stack;
```

The current refactored implementation keeps that behavior in:

```
~/src/edgejs/napi/quickjs/src/internal/napi_promises.h
~/src/edgejs/napi/quickjs/src/internal/napi_promises.cc
```

unofficial_napi.cc now registers and delegates to the napi_promises__ subsystem instead of carrying this state directly.

Added a QuickJS promise hook implementation:

- On JS_PROMISE_HOOK_INIT, capture the current continuation_preserved_embedder_data for the newly-created promise.
- On JS_PROMISE_HOOK_BEFORE, push the current frame and restore the frame captured for that promise.
- On JS_PROMISE_HOOK_AFTER, restore the previous frame and remove the completed promise frame entry.
- Forward registered JS promise hooks (init, before, after, settled) when they are present.

Registered the hook during env creation:

```
JS_SetPromiseHook(rt, QuickjsPromiseHook, env);
```

Also updated:

- unofficial_napi_set_promise_hooks(...) to install the QuickJS hook after storing callbacks.
- unofficial_napi_set_promise_reject_callback(...) to register JS_SetHostPromiseRejectionTracker(...)
- unofficial_napi_enqueue_microtask(...) to enqueue a real QuickJS job with JS_EnqueueJob(...) instead of synthesizing a Promise.resolve().then(...).

28.6 Verification

Rebuilt the QuickJS Edge CLI:

```
cmake --build build-edge-quickjs-cli --target edge -j4
```

Verified the REPL works with persistent history enabled by pointing HOME at a temporary directory:

```
env HOME=/private/tmp/edge-qjs-home ./build-edge-quickjs-cli/edge
```

Inside the REPL:

```
> 10+32
42
>
```

Verified async context survives promise continuations:

```
./build-edge-quickjs-cli/edge --async-context-frame -e \
  "const { AsyncLocalStorage } = require('async_hooks'); const als = new AsyncLocalStorage(); als
```

Observed:

```
als 123
```

The N-API test suite now also covers the same behavior directly with an ordinary .then(...) reaction:

```
napi_quickjs_test_35_promise.Test35Promise.PromiseReactionRestoresContinuationPreservedEmbedderData
```

That test captures `continuation_preserved_embedder_data` when the promise is created, changes the current frame before draining microtasks, and verifies the captured frame is restored inside the reaction callback before the outside frame is restored afterward.

Rebuilt and ran the QuickJS N-API tests:

```
env CMAKE_BUILD_TYPE=Debug make build-napi-quickjs
make test-napi-quickjs-only
```

Result:

100% tests passed, 0 tests failed out of 43

28.7 Remaining separate issue

Exiting the QuickJS CLI still hits the existing teardown assertion:

Assertion failed: (`list_empty(&rt->gc_obj_list)`), function `JS_FreeRuntime`

That is separate from the REPL lock. The REPL now becomes ready, resumes after history initialization, accepts input, and evaluates commands with persistent history enabled.

29 Edge QuickJS WASIX Wasmer run enablement

		Remarks
Status	[done]	Historical WASIX/Wasmer HTTP bring-up note.
Severity	Low	Current deploy and stream issues are tracked in troubleshooting pages.

29.1 Context

The goal for this phase was to make the QuickJS-backed Edge WASIX build work under Wasmer, including both the default package entrypoint and running a JS script through the package:

```
wasmer run .  
wasmer run --volume ./:/app . -- /app/echo-server.js
```

The desired end state was not just “the binary starts”; the HTTP echo server also needed to accept a connection, dispatch the request into JS, and return a response.

29.2 Build shape

The relevant local Edge build output is:

```
~/src/edgejs/build-quickjs-wasix/edge
```

For local package testing, the built executable is also copied to the package names Wasmer expects:

```
~/src/edgejs/build-quickjs-wasix/edge.wasm  
~/src/edgejs/build-quickjs-wasix/edgejs.wasm
```

The direct build commands used during debugging were:

```
cmake --build build-quickjs-wasix --target edge -j4  
cmake --build build-edge-quickjs-cli --target edge -j4
```

The Makefile now has helper targets for these:

```
make build-quickjs-wasix-edge JOBS=4  
make build-edge-quickjs-cli-edge JOBS=4  
make build-quickjs-edge-targets JOBS=4  
make clean-quickjs-wasix  
make clean-edge-quickjs-cli  
make clean-quickjs-edge-targets  
make test-only-quickjs
```

29.3 Wasmer compatibility notes

The Wasmer binary that worked for local testing was built from ~/src/wasmer with the CLI features that allow artifact loading and LLVM execution:

```
cargo build --release \  
  --manifest-path lib/cli/Cargo.toml \  
  --features llvm,napi-v8,wasmer-artifact-create,static-artifact-create,wasmer-artifact-load,static-artifact-load \  
  --bin wasmer \  
  --locked
```

Earlier, a Wasmer invocation failed with:

```
compile error: Validate("exception refs not supported without the exception handling feature ...")
```

That was a Wasmer/runtime feature mismatch for this WASIX artifact. Running with a compatible locally-built Wasmer, and during investigation using:

```
--llvm --enable-exceptions --disable-cache
```

got us past validation. Later, after the artifact and runtime path were in a good state, the simpler package commands worked too:

```
wasmer run .  
wasmer run --volume ./:/app . -- /app/echo-server.js
```

For network tests, Wasmer needs networking enabled:

```
wasmer run --net --volume ./:/app . -- /app/echo-server.js
```

29.4 Bootstrap failure: Atomics under WASIX

After temporarily getting past the QuickJS teardown assertion so that startup errors were visible, Wasmer reported:

```
Failed to execute internal/per_context/primordials: undefinedError  
    at ownKeys (native)  
    at copyPropsRenamed (<input>:78:36)
```

The failure happened very early in Node bootstrap, while executing `internal/per_context/primordials`.

The underlying problem was that QuickJS's Atomics/thread support was disabled for all `__wasi__` builds. Edge's Node bootstrap expects Atomics and SharedArrayBuffer to exist for this WASIX target when `wasm` atomics are available.

We changed the QuickJS guards so WASIX can use atomics when `__wasm_atomics__` is defined:

```
~/src/edgejs/quickjs/quickjs.c  
~/src/edgejs/quickjs/cutils.h
```

The important condition became:

```
(!defined(__wasi__) || defined(__wasm_atomics__))
```

instead of excluding `__wasi__` unconditionally.

After that, this WASIX check succeeded:

```
wasmer run . -- -e "console.log(typeof Atomics, typeof SharedArrayBuffer)"
```

Expected output:

```
object function
```

That fixed the bootstrap failure in `internal/per_context/primordials`.

29.5 HTTP server hang

Once bootstrap worked, the echo server could start:

```
quickjs edge echo listening on 3000
```

But `curl http://127.0.0.1:3000/` connected and then hung. Adding:

```
console.log(`quickjs edge echo: ${req.method} ${req.url}`)
```

inside `echo-server.js` showed that the HTTP request callback fired in the native QuickJS CLI but did not fire in the WASIX run.

We added gated network tracing with `EDGE_TRACE_NET=1` across:

```
~/src/edgejs/src/edge_tcp_wrap.cc
~/src/edgejs/src/edge_stream_base.cc
~/src/edgejs/src/edge_stream_listener.cc
~/src/edgejs/src/edge_http_parser.cc
~/src/edgejs/lib/_http_server.js
~/src/edgejs/lib/internal/stream_base_commons.js
```

The trace showed that libuv networking was working:

```
tcp bind6 ... rc=0(OK)
tcp listen ... rc=0(OK)
tcp onconnection ... status=0(OK)
tcp accept ... rc=0(OK)
tcp read_start ... rc=0(OK)
stream read ... nread=77
```

But the request bytes were delivered to the wrong stream listener. The key trace shape was:

```
http_parser consume ... stream=0x...dd0
stream uv_read base=0x...dd8 ...
stream missing_onread ...
```

The parser listener was attached at 0x...dd0, while libuv reads arrived on 0x...dd8.

29.6 Root cause of the stream pointer mismatch

TcpWrap is laid out with the N-API env first and the stream base second:

```
struct TcpWrap {
    napi_env env = nullptr;
    EdgeStreamBase base{};
    uv_tcp_t handle{};
    int socket_type = kTcpSocket;
};
```

So:

```
0x...dd0 == TcpWrap*
0x...dd8 == &TcpWrap::base
```

parser.consume(socket._handle) calls EdgeStreamBaseFromValue(...).

On QuickJS, socket._handle was reported as napi_external, because our QuickJS N-API constructor path creates class instances with the same QuickJS class id used for N-API externals. The conversion code therefore treated the wrapped TCP object as a raw external pointer and returned the wrapper address instead of unwrapping it and returning &wrap->base.

That attached the HTTP parser listener to the wrong EdgeStreamBase. The real libuv read path continued using the correct &wrap->base, so the parser never saw the request bytes.

29.7 Stream fix

We added wrapper-specific stream-base accessors and made the generic stream conversion prefer them before falling back to raw external data:

```
~/src/edgejs/src/edge_tcp_wrap.h
~/src/edgejs/src/edge_tcp_wrap.cc
~/src/edgejs/src/edge_pipe_wrap.h
~/src/edgejs/src/edge_pipe_wrap.cc
~/src/edgejs/src/edge_tty_wrap.h
~/src/edgejs/src/edge_tty_wrap.cc
~/src/edgejs/src/edge_stream_base.cc
```


The TCP/Pipe/TTY helpers now:

1. Accept `napi_object`, `napi_function`, and QuickJS's current `napi_external`-reported class instances.
2. Call `napi_unwrap(...)`.
3. Validate the native wrapper by checking `handle->data == wrap`.
4. Validate the libuv handle type, for example `UV_TCP` for TCP.
5. Return `&wrap->base`.

After the fix, the trace changed to the correct shape:

```
tcp get_stream_base unwrap_status=0 wrap=0x...dd0
tcp get_stream_base base=0x...dd8 handle=0x...e70 data=0x...dd0 type=12 expected=12
stream from_value tcp base=0x...dd8
http_parser consume ... stream=0x...dd8
stream uv_read base=0x...dd8 current=<http parser listener>
http_parser consumed_read ... nread=77
js http_server parserOnIncoming method= GET url= /
quickjs edge echo: GET /
```

curl then received a normal response:

```
HTTP/1.1 200 OK
content-type: text/plain
```

```
quickjs edge echo: GET /
```

29.8 Verification

The traced verification command used during debugging was:

```
~/src/wasmer/target/release/wasmer run \
  --llvm --enable-exceptions --disable-cache --net \
  --env EDGE_TRACE_NET=1 \
  --env PORT=3003 \
  --volume ~/src/edgejs/quickjs-wasm:/app \
  . -- /app/echo-server.js
```

Then:

```
curl -v --max-time 8 http://127.0.0.1:3003/
```

This returned:

```
quickjs edge echo: GET /
```

The clean non-traced run also worked and stayed alive after the keep-alive close window.

The final user-confirmed commands also work:

```
wasmer run --volume ./:/app . -- /app/echo-server.js
wasmer run .
```

29.9 Current status

The QuickJS Edge WASIX package can now bootstrap under Wasmer and run the echo server path correctly.

The important implementation lessons are:

1. WASIX QuickJS needs `Atomics` enabled when the target has `wasmm atomics`.
2. The original HTTP parser stream bug was caused by QuickJS N-API class instances being represented with the same QuickJS class ID as public `napi_create_external(...)` values. That ambiguity is now fixed in the QuickJS N-API layer: constructed class instances are ordinary prototype-backed objects, and internal wrap records use a separate `NapiExternalRecord` class.

3. The HTTP parser consume path depends on the listener being attached to the exact `EdgeStreamBase` used by the libuv read callbacks.
4. Embedded QuickJS WASIX targets that include N-API headers must compile with the embedded-provider declaration mode (`NAPI_EXTERN=`). If one static library sees default `__wasm__` N-API imports from module `napi` while another sees the same unresolved calls through `env`, the final `wasm-ld` link fails with an import module mismatch. The concrete fixed case was `edge_environment_core`.

The stream wrapper-specific fallback remains useful as defensive native conversion code, but the root type-reporting problem now belongs to the historical record rather than current QuickJS N-API behavior.

30 Edge QuickJS web app enablement for Astro, Vite, and Next.js

		Remarks
Status	[caveated]	Historical framework adapter exploration.
Severity	Low	Current app-specific issues are tracked under troubleshooting/.

30.1 Current Status Note

This is a historical note from the period when framework validation used small CJS adapters. That path depended on QuickJS C++ CommonJS facade/module-loader support. The C++ CJS/module-loader hack has since been removed from napi/quickjs/src, so the CJS adapter examples below are preserved as history, not as the current QuickJS N-API support model.

30.2 Context

001_merge_analysis.md through 005_wasix_wasmer_http.md cover the provider integration, bootstrap debugging, REPL microtasks, WASIX atomics, and the HTTP stream listener bug that prevented a simple echo server from handling requests under Wasmer.

This note captures the next layer of work: using the published Edge QuickJS Wasmer package to run real framework outputs:

- an Astro static site at ~/src/astro-app;
- a Vite/React SPA at ~/src/vite-app;
- a Next.js app at ~/src/next-app.

The important distinction is that Edge QuickJS now runs the HTTP/static adapter inside WASIX. Framework build systems still run outside the WASIX runtime during the build step.

30.3 Edge QuickJS package shape

The root package definition is:

~/src/edgejs/wasmer.toml

Current package identity:

```
[package]
entrypoint = "edgejs-quickjs"
name = "sadhbh-c0d3/edgejs-quickjs"
description = "Edge.js with an embedded QuickJS N-API provider"
version = "0.0.1"
```

The command exports the built WASIX Edge binary as the edge command:

```
[[module]]
name = "edge"
source = "../build-quickjs-wasix/edgejs.wasm"

[[command]]
name = "edge"
module = "edge"
```

```
runner = "https://webc.org/runner/wasi"
```

```
[command.annotations.wasi]
```

```
atom = "edge"
```

The package also mounts SSL certificates:

```
[fs]
```

```
"/usr/local/ssl" = "./ssl-certs"
```

Applications consume it with:

```
[dependencies]
```

```
"sadhbh-c0d3/edgejs-quickjs" = "0.0.1"
```

or, where semver range is wanted:

```
[dependencies]
```

```
"sadhbh-c0d3/edgejs-quickjs" = "^0.0.1"
```

and point their command at:

```
module = "sadhbh-c0d3/edgejs-quickjs:edge"
```

```
runner = "https://webc.org/runner/wasi"
```

The app-specific main-args then name the JavaScript adapter file to execute.

30.4 Runtime changes that made framework adapters possible

These are the Edge QuickJS fixes that matter for Astro, Vite, and Next.js.

30.4.1 ContextifyScript must be a class

Node's `lib/internal/vm.js` constructs `ContextifyScript` with `new ContextifyScript(...)`.

The original QuickJS path registered `ContextifyScript` with `napi_create_function()`. That produced a callable function, but not a N-API class shape compatible with Node's new `ContextifyScript(...)` path.

The fix was in:

```
~/src/edgejs/src/edge_module_loader.cc
```

`ResolveContextifyBinding()` now defines `ContextifyScript` with `napi_define_class(...)`.

This was necessary before `internal/vm`, CommonJS compilation, and user script execution could work reliably.

30.4.2 QuickJS promise hooks and microtasks

The REPL investigation showed that bytes reached the JS stream layer, but some promise continuations did not run. The missing piece was QuickJS promise hook integration plus proper microtask/job draining.

The fix is described in `004_promise_hooks_microtasks.md` and involved:

```
~/src/edgejs/quickjs/quickjs.c
```

```
~/src/edgejs/napi/quickjs/src/unofficial_napi.cc
```

The relevant behavior:

- `unofficial_napi_set_promise_hooks(...)` stores Node's hook callbacks and registers a QuickJS runtime promise hook.
- `unofficial_napi_enqueue_microtask(...)` enqueues a real QuickJS job instead of relying on a fake callback path.
- `unofficial_napi_process_microtasks(...)` drains QuickJS pending jobs with `JS_ExecutePendingJob(...)`.
- The vendored QuickJS source emits before/after promise hook events around the internal promise reaction job.

This matters for web app adapters because framework code often relies on Promise, async callbacks, stream scheduling, file reads, and server startup continuations even when the final adapter looks like a simple static server.

30.4.3 Atomics and SharedArrayBuffer under WASIX

Wasmer startup initially failed in `internal/per_context/primordials` because QuickJS disabled atomics for every `__wasi__` target.

The fix is described in `005_wasix_wasmer_http.md` and lives in the QuickJS submodule:

```
~/src/edgejs/quickjs/quickjs.c
~/src/edgejs/quickjs/cutils.h
```

The important condition changed from excluding `__wasi__` unconditionally to allowing atomics when the WASIX build defines `wasml_atomics`:

```
(!defined(__wasi__) || defined(__wasml_atomics__))
```

Without this, Node bootstrap can fail before any app adapter gets to run.

30.4.4 HTTP stream listener attachment

The echo-server hang under Wasmer was caused by the HTTP parser listener being attached to the wrapper address instead of the wrapper's `EdgeStreamBase`.

The fix is described in `005_wasix_wasmer_http.md` and touched:

```
~/src/edgejs/src/edge_tcp_wrap.h
~/src/edgejs/src/edge_tcp_wrap.cc
~/src/edgejs/src/edge_pipe_wrap.h
~/src/edgejs/src/edge_pipe_wrap.cc
~/src/edgejs/src/edge_tty_wrap.h
~/src/edgejs/src/edge_tty_wrap.cc
~/src/edgejs/src/edge_stream_base.cc
```

The generic stream conversion now tries wrapper-specific unwrapping first:

- TCP;
- Pipe;
- TTY.

Those helpers validate the unwrapped native handle and return the exact `EdgeStreamBase` used by libuv reads. Only after those checks does the generic path fall back to raw external data.

This is the core networking fix that allowed:

```
http.createServer(...)
```

to accept a connection, parse the request, call the JS request callback, and write a response under Wasmer.

30.4.5 Network tracing

The debug flag added during the HTTP work remains useful:

```
EDGE_TRACE_NET=1
```

It traces through native and JS layers:

```
~/src/edgejs/src/edge_tcp_wrap.cc
~/src/edgejs/src/edge_stream_base.cc
~/src/edgejs/src/edge_stream_listener.cc
~/src/edgejs/src/edge_http_parser.cc
```

```
~/src/edgejs/lib/_http_server.js
~/src/edgejs/lib/internal/stream_base_commons.js
```

The useful success shape is:

```
tcp bind/listen/accept/read_start ... OK
http_parser consume ... stream=<same EdgeStreamBase as uv_read>
http_parser consumed_read ... nread=<request bytes>
js http_server parserOnIncoming method= GET url= /
```

The old broken shape was stream missing_onread after bytes were read.

30.4.6 QuickJS teardown and lifetime status

Historical note: during this adapter bring-up, the N-API QuickJS submodule had `JS_FreeRuntime(...)` commented out in the env release path:

```
~/src/edgejs/napi/quickjs/src/unofficial_napi.cc
```

The submodule commit is:

```
a200c14 Disabling JS_FreeRuntime until gc leaks are solved
```

That workaround prevented the known teardown assertion from aborting successful Wasmer app runs:

```
Assertion failed: list_empty(&rt->gc_obj_list)
```

This was intentionally not a real GC/leak fix. The current QuickJS N-API source has `JS_FreeRuntime(...)` enabled again. Lifetime diagnostics now use allocator-backed handles and `napi_lifetime_tracker__`; the remaining server growth work is tracked in `troubleshooting/node-compat/napi/014_lifetime_tracing.md`, especially native event paths that allocate request-local N-API values into the env root scope.

30.5 Running static sites through Edge QuickJS

The common pattern for Astro and Vite is:

1. Build the framework output with its normal Node-based build tool.
2. Mount the generated static directory into WASIX.
3. Run a small CommonJS http + fs adapter under Edge QuickJS.
4. Pass `--net` to Wasmer when serving HTTP.

The adapter should be conservative:

- use CommonJS (`.cjs`) so Edge can execute it through the current CJS path;
- use Node builtins we know are working: `fs`, `path`, `http`, `url` where needed;
- avoid depending on the framework's dev server or native Node add-ons at runtime;
- normalize request paths and reject traversal;
- support GET and HEAD where possible;
- send explicit content types for JS, CSS, JSON, images, WASM, HTML, and RSC payloads.

30.6 Astro: ~/src/astro-app

Astro's build output can be served as static files. The adapter added there is:

```
~/src/astro-app/server-edgejs.cjs
```

It is a minimal static server:

- root is `/app/dist`;
- port defaults to `process.env.PORT || 3000`;
- `/` maps to `index.html`;
- directories map to `index.html`;
- files are served with a small content type table;

- errors are logged and returned as 500.

The working shape is:

```
cd ~/src/astro-app
npm run build
wasmer run \
  --net \
  --env PORT=3000 \
  --volume ./:/app \
  sadhbh-c0d3/edgejs-quickjs@0.0.1 \
  -- /app/server-edgejs.cjs
```

This uses the published Edge QuickJS package directly and mounts the project at /app, so the adapter can read /app/dist.

This is static serving, not Astro SSR. It proves the Edge QuickJS WASIX package can run a normal Node-style static HTTP server around Astro's generated output.

30.7 Vite: ~/src/vite-app

The Vite version was changed from a PHP/WinterJS package shape to an Edge QuickJS package shape.

Important files:

```
~/src/vite-app/wasmer.toml
~/src/vite-app/server/router.cjs
```

The package depends on the published runtime:

```
[dependencies]
"sadhbh-c0d3/edgejs-quickjs" = "^0.0.1"
```

The package mounts only app artifacts:

```
[fs]
"/dist" = "./dist"
"/server" = "./server"
```

The command executes the CJS router:

```
[[command]]
name = "run"
module = "sadhbh-c0d3/edgejs-quickjs:edge"
runner = "https://webc.org/runner/wasi"
```

```
[command.annotations.wasi]
main-args = ["/server/router.cjs"]
```

server/router.cjs is a Vite SPA static adapter:

- root is /dist;
- port defaults to process.env.PORT || 8080;
- supports GET and HEAD;
- rejects path traversal;
- serves existing files and directories;
- falls back to /dist/index.html for SPA routes;
- sends immutable cache headers for /assets/;
- has content types for .html, .js, .mjs, .css, .json, .wasm, common images, and .xcf.

The expected run flow is:

```
cd ~/src/vite-app
npm run build
wasmer run --net .
```

The Vite case mainly validated the published package consumption story:

- no local ~/src/edgejs/quickjs-wasm path is needed;
- no ssl-certs package dependency is needed in the app;
- wasmer.toml should reference sadhbh-c0d3/edgejs-quickjs@0.0.1 as a dependency and execute its edge atom.

30.8 Next.js: ~/src/next-app

Next.js needed a more specialized adapter than Astro/Vite because the app uses App Router output, RSC payloads, static _next assets, public files, and a few dynamic routes.

Important files:

```
~/src/next-app/package.json
~/src/next-app/wasmer.toml
~/src/next-app/server/router.cjs
~/src/next-app/server/generate-next-dynamic-shells.cjs
~/src/next-app/server/build-edge-dist.cjs
```

30.8.1 Package shape

The app package depends on Edge QuickJS:

```
[dependencies]
"sadbh-c0d3/edgejs-quickjs" = "0.0.1"
```

The Wasmer package now mounts staged runtime files, not the full .next directory:

```
[fs]
"/web" = ".dist/web"
"/public" = ".dist/public"
"/server" = ".dist/server"

[[command]]
name = "run"
module = "sadbh-c0d3/edgejs-quickjs:edge"
runner = "https://webc.org/runner/wasi"
```

```
[command.annotations.wasi]
main-args = ["/server/router.cjs"]
```

The package scripts are:

```
"edge:build": "next build && node server/generate-next-dynamic-shells.cjs && node server/build-edge-dist.cjs",
"edge:stage": "node server/build-edge-dist.cjs",
"edge:preview": "wasmer run --net ."
```

The correct build command is:

```
npm run edge:build
```

not:

```
npm run build edge:build
```


30.8.2 Runtime router

server/router.cjs is the CJS adapter executed by Edge QuickJS.

It serves:

- /_next/static/* from /web/static;
- /favicon.ico from /web/server/app/favicon.ico.body;
- public files from /public;
- static App Router HTML/RSC files from /web/server/app;
- generated dynamic shells from /server/generated;
- _not-found.html as the 404 fallback.

It detects RSC requests with:

```
req.headers.rsc === "1" || url.searchParams.has("_rsc")
```

and serves text/x-component for .rsc responses.

The dynamic routes currently handled are:

```
/lobby/:id  
/tables/:id?lobbyId=:lobbyId
```

Those routes use generated files with placeholders:

```
/server/generated/lobby.html  
/server/generated/lobby.rsc  
/server/generated/table.html  
/server/generated/table.rsc
```

At request time the router replaces:

```
__EDGE_LOBBY_ID__  
__EDGE_TABLE_ID__
```

with URL-safe route values.

/local-rpc is intentionally not implemented in the WASI static adapter and returns 502.

30.8.3 Dynamic shell generation

server/generate-next-dynamic-shells.cjs runs after next build on the host Node runtime, not inside Edge QuickJS.

It:

1. imports the emitted Next route module, for example .next/server/app/(site)/lobby/[id]/page.js;
2. starts a temporary localhost Node HTTP server;
3. calls the Next route module handler;
4. fetches both HTML and RSC variants;
5. writes placeholder-based generated shells into server/generated.

Routes currently generated:

```
{  
  modulePath: ".next/server/app/(site)/lobby/[id]/page.js",  
  url: "/lobby/__EDGE_LOBBY_ID__",  
  html: "lobby.html",  
  rsc: "lobby.rsc",  
}  
  
{  
  modulePath: ".next/server/app/tables/[id]/page.js",  
  url: "/tables/__EDGE_TABLE_ID__?lobbyId=__EDGE_LOBBY_ID__",  
  html: "table.html",  
}
```

```
  rsc: "table.rsc",
}
```

The generator also installs AsyncLocalStorage on globalThis for the Next route module:

```
globalThis.AsyncLocalStorage = globalThis.AsyncLocalStorage || AsyncLocalStorage;
```

30.8.4 Table route emission bug

The table dynamic shell was initially missing:

Skipping /tables/__EDGE_TABLE_ID__?lobbyId=__EDGE_LOBBY_ID__:
.next/server/app/tables/[id]/page.js was not emitted by next build

and clicking Join opened:

Next dynamic route shell is missing. Run `npm run edge:build`.

Root cause:

```
~/src/next-app/app/tables/[id]/layout.js
```

had:

```
export const runtime = 'edge';
```

Next emitted:

```
{
  "/tables/[id]/page": "app-edge-has-no-entrypoint"
}
```

instead of .next/server/app/tables/[id]/page.js. The generator can only render the Node server page module, so it skipped the table route.

Fix:

- remove the forced runtime = 'edge' from the table route layout;
- rebuild with npm run edge:build.

After that, Next reports:

```
f /tables/[id]
```

and emits:

```
.next/server/app/tables/[id]/page.js
```

The lobby Join/Return URL was also changed to carry the current lobby:

```
href={` /tables/${id}?lobbyId=${encodeURIComponent(lobbyId)}`}
```

so the generated table shell receives the right searchParams.lobbyId.

30.8.5 .dist staging

Mounting the whole .next directory worked but was unnecessarily large:

```
37M .next
```

The build now stages only the runtime files used by server/router.cjs:

```
~/src/next-app/server/build-edge-dist.cjs
```

It copies:

- .next/static to .dist/web/static;
- selected .next/server/app files to .dist/web/server/app;
- public to .dist/public;

- server/router.cjs to .dist/server/router.cjs;
- server/generated to .dist/server/generated.

Only these Next app artifact extensions are kept:

```
.body
.html
.rsc
```

The staging step excludes .segments directories because the current router does not serve those paths.

Observed size after staging:

```
37M .next
8.6M .dist
6.9M .dist/web/static
256K .dist/web/server/app
1.3M .dist/public
68K .dist/server
```

This is the directory Wasmer packages now mount.

30.8.6 Next.js verification commands

Build:

```
cd ~/src/next-app
npm run edge:build
```

Package check:

```
wasmer package build --check .
```

Preview:

```
wasmer run --net --env PORT=3022 .
```

Useful probes:

```
curl -I http://127.0.0.1:3022/
curl -I http://127.0.0.1:3022/main-lobby
curl -i 'http://127.0.0.1:3022/tables/123?lobbyId=1'
curl -i -H 'rsc: 1' 'http://127.0.0.1:3022/lobby/1'
curl -i http://127.0.0.1:3022/_next/static/chunks/49dced3c3039a42d.css
```

The verified result was 200 OK for the HTML, RSC, and CSS paths.

30.9 What this does and does not prove

30.9.1 Proven

- The published Edge QuickJS Wasmer package can run CommonJS HTTP server adapters.
- WASIX networking works for HTTP server workloads when run with --net.
- Static file serving with fs.readFileSync, fs.statSync, path, Buffer, content types, and headers works.
- Vite/React and Astro static outputs can be served directly.
- Next App Router static HTML/RSC output can be served by a custom adapter.
- Selected Next dynamic routes can be served by host-generated HTML/RSC shells plus placeholder replacement.

30.9.2 Not proven

- Full Next.js server runtime execution inside Edge QuickJS WASIX.

- Next route handlers, middleware, API routes, streaming SSR, or per-request server component execution inside Edge QuickJS.
- Astro SSR inside Edge QuickJS.
- Vite dev server or HMR inside Edge QuickJS.
- Native Node add-ons inside the WASIX app package.
- The QuickJS runtime teardown leak/assertion fix.

For now the strategy is:

framework build/SSR shell generation on host Node
small CJS HTTP adapter at runtime under Edge QuickJS WASIX

That is good enough for static and mostly-static sites, and it is a useful pressure test for Edge QuickJS's Node compatibility surface.

30.10 Practical app authoring rules

For a new static-ish app:

1. Build with the framework's normal command.
2. Add a small `.cjs` router using only stable Node builtins.
3. Mount generated artifacts explicitly in `wasmer.toml`.
4. Use `sadhbh-c0d3/edgejs-quickjs@0.0.1` as a dependency.
5. Run with `wasmer run --net ..`

For Next.js:

1. Avoid `export const runtime = 'edge'` on routes that the shell generator must import from `.next/server/app/.../page.js`.
2. Keep `server/generate-next-dynamic-shells.cjs` route list in sync with every dynamic route the adapter should serve.
3. Keep `server/router.cjs` route matching and placeholder replacement in sync with generated shells.
4. Run `npm run edge:build`, not plain `npm run build`, before `wasmer run`.
5. Package `.dist`, not the full `.next`, unless the router starts needing more Next internals.

30.11 Remaining work

1. Continue the native-event handle-scope work from `troubleshooting/node-compat/napi/014_lifetime_tracing`, so server request churn does not retain temporary N-API values in the env root scope.
2. Decide whether `EDGE_TRACE_NET` and `EDGE_TRACE_TTY` should stay in-tree as supported diagnostics or be reduced before merging.
3. Add a tiny framework smoke-test matrix for:
 - `echo-server.js`;
 - Astro static;
 - Vite static SPA fallback;
 - Next static route;
 - Next generated dynamic HTML route;
 - Next generated dynamic RSC route.
4. Consider a reusable adapter template package once the static-server pattern repeats one more time.
5. For real SSR, investigate whether framework server bundles can be made QuickJS-compatible, or whether build-time shell generation remains the intended model for this runtime.

31 Edge QuickJS framework standalone builds

		Remarks
Status	[caveated]	Historical standalone build findings. Current standalone issues are tracked under app-specific troubleshooting pages.
Severity	Low	

31.1 Current Status Note

This note records the earlier standalone-build investigation. Any CJS execution paths described here should be read as historical: the QuickJS C++ CommonJS facade/module-loader hack has now been removed.

We later added the missing module mechanics to the vendored QuickJS source in 577fb31caf2e973b111431b6cb009f7595c and the QuickJS N-API napi_module_wrap__ layer now adapts Node/V8-shaped module_wrap calls onto those engine APIs. Package resolution, CommonJS wrapping, package conditions, and builtin policy still belong in Node's JavaScript loaders/translators or another proper EdgeJS-owned runtime path.

31.2 Context

006_framework_app_adapters.md captured the first working framework-app path for Edge QuickJS: small app-owned adapters for Astro, Vite, and Next.js outputs.

The next goal was to remove as much app-specific server/router glue as possible and prefer framework-standard standalone build outputs:

- Astro standalone Node adapter output;
- Vite SSR standalone packaging via vite-plugin-standalone;
- Next.js official output: "standalone" build.

The anonymized app paths used in notes are:

```
~/src/astro-app
~/src/vite-app
~/src/next-app
```

31.3 Astro standalone

Astro already has the cleanest standalone story. It can emit a server build with the Node adapter in standalone mode:

```
export default defineConfig({
  output: 'server',
  adapter: node({
    mode: 'standalone',
  }),
});
```

That build produces a server entry under:

```
dist/server/entry.mjs
```

For Edge QuickJS compatibility, the local follow-up was to bundle that emitted server entry into a CJS artifact that the current Edge CLI can execute:

```
dist/server/entry.cjs
```

The `.mjs` entry is still not the working EdgeJS execution path; Astro standalone currently works through the bundled CJS artifact.

The important point is that Astro owns the server semantics. The app does not need to keep a bespoke HTTP router once the build emits the standalone server entry.

One observed Astro app caveat was `astro-blog`, which failed because the standalone dependency graph pulled in `sharp`. That failure appears tied to `sharp`'s native / WebAssembly dependency shape rather than Astro standalone itself.

31.4 Vite standalone

Plain Vite SPA builds do not emit an HTTP server entry. A normal Vite build only produces static client files under: `dist/`

So the earlier `vite-app` path used an app-owned static router to serve files, handle GET / HEAD, prevent path traversal, set content types, and fall back to `index.html` for client-side routes.

The standalone experiment found that `vite-plugin-standalone` is a useful fit when the app provides a server entry for the plugin to package. The plugin uses the SSR build path, `esbuild`, and `@vercel/nft` to produce a self-contained server artifact.

The resulting `vite-app` direction is:

- use `vite-plugin-standalone`;
- keep the minimal server semantics as a build-owned entry;
- drop the old manually staged `server/router.cjs` / `server/router.php` style;
- execute the standalone output from the Wasmer package.

This is not the same as Astro, because Vite still does not invent HTTP server semantics for a plain SPA. But it does let the project keep the standalone version as the maintained path, with dependency tracing and packaging handled by the Vite plugin instead of custom Edge staging scripts.

More detail is captured in:

`plans/quickjs-wasm/development/troubleshooting/vite-app/001_standalone_build.md`

31.4.1 Native CLI static-root caveat

The Vite standalone entry tested in the app used:

```
var root = process.env.STATIC_ROOT || "/dist";
```

That default works under:

```
wasmer run --net .
```

because `/dist` is resolved inside the WASIX / Wasmer package filesystem, where the packaged app assets are available.

It does not work the same way when running the host native QuickJS CLI directly:

```
~/src/dev/edgejs/build-edge-quickjs-cli/edge ./dist/server/standalone-entry.js
```

In that mode, `/dist` means the host machine's absolute `/dist`, not the app's local `dist/` directory. The server can start, but the first HTTP request falls through to `fs.readFileSync("/dist/index.html")`, which fails in `fs.openSync(...)` because the host path does not exist.

Two useful native-CLI fixes are:

```
STATIC_ROOT="$PWD/dist" ~/src/dev/edgejs/build-edge-quickjs-cli/edge ./dist/server/standalone-entry.js
```

or make the entry default relative to the emitted server file:

```
var root = process.env.STATIC_ROOT || path.resolve(__dirname, "..");
```

Since the entry is emitted under `dist/server/`, that resolves to the app's `dist/` directory for native CLI runs and should still resolve to `/dist` in the Wasmer package shape.

The accompanying DEP0176 warning about `fs.F_OK` is separate from the failed read. It is triggered by the fs shim's deprecated `fs.F_OK` getter path, likely via `fs.existsSync(...)`.

31.5 Next.js standalone

Next.js also has an official standalone build mode:

```
const nextConfig = {
  output: "standalone",
};
```

The app-level target is:

```
.next/standalone/server.js
```

The old custom `server/` directory is not part of the intended path. The standalone output works under stock Node:

```
node ./app/server.js
```

When copied into a test folder, the Edge QuickJS repro shape was:

```
cp -rf ./next/standalone ./testing/app/
cd ./testing
~/src/dev/edgejs/build-edge-quickjs-cli/edge ./app/server.js
```

That failed before the server could start.

31.5.1 Native CLI main-entry caveat

Unlike the Vite standalone static server, the Next standalone server does not use `STATIC_ROOT` as its primary asset root. Its generated entry starts with:

```
const dir = path.join(__dirname)

process.env.NODE_ENV = 'production'
process.chdir(__dirname)
```

and then loads Next:

```
require('next')
const { startServer } = require('next/dist/server/lib/start-server')
```

So this command is not fixed by pointing `STATIC_ROOT` at `.testing`:

```
STATIC_ROOT="$PWD/.testing" ~/src/dev/edgejs/build-edge-quickjs-cli/edge ./next/standalone/server.js
```

The observed native QuickJS CLI failure was:

```
Failed to execute builtin 'internal/main/run_main_module':
undefined    at normalizeRequirableId (<input>:302:35)
              at defaultResolveImpl (<input>:1061:36)
              at resolveForCJSWithHooks (<input>:1066:41)
              at wrapModuleLoad (<input>:247:34)
```

This is an earlier main-module / CJS resolution failure. A minimal native CLI repro showed that direct script execution was not reliably placing the script path in `process.argv[1]`; then `internal/main/run_main_module` reaches:

```
require('internal/modules/cjs/loader').Module.runMain(mainEntry);
```

with `mainEntry` undefined. That eventually calls the CJS resolver with an undefined request and fails in `Built-inModule.normalizeRequirableId(...)`.

This should be tracked separately from static asset root handling. The runtime fix is likely in the native QuickJS CLI argument / main-entry bootstrap path, before continuing to the already-known Next blockers such as QuickJS serdes support for `require("v8")`.

31.6 Next.js runtime findings

The Next standalone output is valid, but it exposed two separate Edge runtime gaps.

31.6.1 Edge V8 inspector limitation

Edge V8 fails on the same standalone server because Next's startup path eventually requires:

```
require("inspector");
```

The runtime reports:

Error: Inspector is not available

This is not a QuickJS branch regression. Comparing main with `napi/quickjs-integration-int` showed the relevant inspector files are unchanged:

```
src/internal_binding/binding_config.cc
lib/inspector.js
src/builtin_catalog.cc
```

Both main and the QuickJS integration branch currently set:

```
const bool has_inspector = false;
```

in `internalBinding("config")`, so `lib/inspector.js` throws as soon as it is required.

The concrete native V8 standalone repro was:

```
~/src/dev/edgejs/build-edge/edge .next/standalone/server.js
```

with:

```
Failed to execute builtin 'internal/main/run_main_module':
node:inspector:25
  throw new ERR_INSPECTOR_NOT_AVAILABLE();
  ^
```

Error: Inspector is not available
code: 'ERR_INSPECTOR_NOT_AVAILABLE'

31.6.2 Edge WASIX builtin failure formatting

The WASIX package path fails with a less specific wrapped builtin error:

```
wasmer run --net .
```

reported:

```
undefined[Error: Failed to execute builtin 'internal/main/run_main_module':      at anonymous (<in
  at compileForInternalLoader (<input>:400:10)
  at compileForPublicLoader (<input>:339:10)
  at loadBuiltinModule (<input>:127:7)
  at loadBuiltinWithHooks (<input>:1197:33)
  at <anonymous> (<input>:1287:69)
  at traceSync (<input>:330:27)
  at wrapModuleLoad (<input>:247:34)
```



```
    at <anonymous> (<input>:1550:27)
]
```

This appears to be the WASIX runtime surfacing only the wrapped builtin failure instead of the more specific underlying module/runtime error.

31.6.3 Edge QuickJS v8 / serdes blocker

Edge QuickJS fails earlier. The standalone failure reduces to:

```
require("v8");
```

LLDB showed the original exception before Edge wrapped it:

```
TypeError: cannot read property 'prototype' of undefined
```

The failing source is in lib/v8.js:

```
Serializer.prototype._getDataCloneError = Error;
```

The V8 backend exports real `Serializer` and `Deserializer` constructors from `internalBinding("serdes")`. The QuickJS backend currently returns an empty object in:

```
napi/quickjs/src/unofficial_napi.cc
```

So `require("v8")` fails because `internalBinding("serdes").Serializer` is undefined.

The immediate QuickJS runtime fix is to implement a minimal serdes binding that exports stable `Serializer` and `Deserializer` constructors. Full `v8.serialize()` / `v8.deserialize()` behavior can use the existing QuickJS structured clone helpers based on `JS_WriteObject(...)` and `JS_ReadObject(...)`.

More detail is captured in:

```
plans/quickjs-wasm/development/troubleshooting/next-app/001_standalone_v8_serdes.md
```

31.7 Builtin error formatting

While debugging the Next standalone failure, native builtin execution failures in:

```
src/edge_module_loader.cc
```

were found to have regressed from Node-like formatting. The QuickJS debugging path had changed builtin failures to clear the pending JS exception and throw a new wrapper error:

```
Failed to execute builtin 'internal/main/run_main_module': <original error>
```

That exposed hidden QuickJS builtin failures, but it made Edge V8 less Node-like. The original Edge V8 output preserved Node's source frame and error object formatting:

```
node:inspector:25
  throw new ERR_INSPECTOR_NOT_AVAILABLE();
  ^
```

Error: Inspector is not available

The current fix preserves the pending JS exception for builtin call failures and only prints one extra context line:

```
Failed to execute builtin 'internal/main/run_main_module':
node:inspector:25
  throw new ERR_INSPECTOR_NOT_AVAILABLE();
  ^
```

Error: Inspector is not available

The rule is:

- add diagnostic context;
- do not clear and replace the original JS exception;
- let the normal fatal-exception path format the original error.

Synthetic Failed to execute builtin ... errors are still appropriate for setup or compile failures where there is no pending JS exception to preserve.

31.8 Verification

Both V8 and QuickJS binaries were rebuilt after the formatting change:

```
cmake --build build-edge --target edge -j4
cmake --build build-edge-quickjs-cli --target edge -j4
```

The V8 inspector repro now preserves original formatting with the added context:

```
build-edge/edge -e "require('inspector')"
```

The Next standalone repro under Edge V8 does the same:

```
build-edge/edge ./app/server.js
```

The QuickJS standalone repro still fails on the known v8 / serdes path, but the builtin failure path no longer replaces the original pending exception.

31.9 Current standalone status

- Astro has a framework-owned standalone server output and works with EdgeJS when the emitted .mjs server is bundled to CJS. The .mjs entry itself is not the working path yet. The native QuickJS runtime now includes a minimal Intl.DateTimeFormat fallback for framework bootstrap code that only needs simple time formatting, such as Astro's SSR logger. This is deliberately not a full ECMA-402 Intl implementation. Astro apps that pull in sharp can still fail on the sharp native / WebAssembly dependency shape.
- Vite can keep a standalone build path with vite-plugin-standalone, provided the project supplies the server semantics the plugin packages. This path is believed to work for the tested Vite app shape.
- Next.js emits the correct official standalone output, but its EdgeJS status depends on which Edge runtime is used:
 - WASIX currently reports only a wrapped Failed to execute builtin 'internal/main/run_main_module' error from wasmer run --net ..
 - native V8 reaches the known inspector limitation because require("inspector") throws ERR_INSPECTOR_NOT_AVAILABLE.
 - native QuickJS has the native CLI main-entry issue above, then still needs QuickJS serdes support for require("v8").

31.10 Follow-up

The next runtime tasks for full Next standalone under Edge QuickJS are to fix the native CLI main-entry bootstrap path and then implement the QuickJS serdes surface needed by require("v8").

32 Edge QuickJS runtime change containment rollback

		Remarks
Status	[done]	Historical runtime containment and rollback note.
Severity	Low	Current known issues are tracked in the troubleshooting registry.

32.1 Goal

Rollback the shared EdgeJS runtime directories so they match the upstream wasmer-io/edgejs tree again, and keep all QuickJS-specific compatibility work contained in:

- napi/quickjs/
- optionally src/

The directories to restore from ~/src/dev/wasmer-io/edgejs are:

- lib/
- napi/v8/
- napi/src/
- napi/include/

This keeps the QuickJS backend from carrying hidden changes in shared JavaScript library files, the V8 provider, or the common N-API headers/sources.

32.2 Rollback Surface

Initial comparison showed:

- napi/src/ already matches the donor tree.
- napi/include/ already matches the donor tree.
- napi/v8/ only has an extra .DS_Store file in the QuickJS worktree.
- lib/ differs in eight files:
 - _http_server.js
 - inspector.js
 - internal/modules/esm/utils.js
 - internal/readline/emitKeypressEvents.js
 - internal/readline/interface.js
 - internal/repl/history.js
 - internal/stream_base_commons.js
 - internal/streams/readable.js

Most lib/ differences are diagnostics for EDGE_TRACE_NET or EDGE_TRACE_TTY. Those should not remain in shared runtime files. If equivalent diagnostics are still needed, move them to QuickJS-native stream, TTY, or HTTP parser code under napi/quickjs/ or to neutral native runtime tracing under src/.

The functional risk is lib/inspector.js. The current QuickJS tree contains a JavaScript fallback that makes require("inspector") return a stable unavailable stub when the build has no inspector. After rollback, the shared library returns to the upstream behavior of throwing ERR_INSPECTOR_NOT_AVAILABLE when internal-Binding("config").hasInspector is false. If framework code still needs require("inspector") to be linkable under QuickJS, the fix should move below the JavaScript library layer.

32.3 Compatibility Strategy

1. Restore the requested directories from the donor tree exactly, including removing files that exist only in the QuickJS worktree under those paths.
2. Rebuild the native QuickJS Edge CLI and run focused smoke tests:
 - `./build-edge-quickjs-cli/edge --version`
 - `./build-edge-quickjs-cli/edge -e "console.log('hello from quickjs')"`
 - `./build-edge-quickjs-cli/edge ./quickjs-wasm/echo-server.js`
3. If `require("inspector")` is still needed by Next.js or another framework, keep `lib/inspector.js` upstream-clean and make the runtime expose a narrow native compatibility path in C/C++ instead of JavaScript. The chosen fallback is to report inspector module availability through `internalBinding("config")` and provide `internalBinding("inspector")` with unavailable/no-op behavior compatible with upstream `lib/inspector.js`. This allows imports to link while active debugger/session operations still fail explicitly.
4. If stream, REPL, or HTTP behavior regresses after losing JS tracing, diagnose from QuickJS-owned code and keep fixes in wrapper/native code rather than reintroducing shared `lib/` edits.
5. Rebuild WASIX after QuickJS-impacting fixes under `napi/quickjs/` or `src/` using:

```
cd ~/src/dev/edgejs/quickjs-wasm/ && ./build.sh
```

The QuickJS WASIX target links the embedded QuickJS N-API implementation into the final wasm. Edge targets that include N-API headers before linking `napi_quickjs`, such as `edge_environment_core`, must still define `NAPI_EXTERN=` for the QuickJS provider. Otherwise wasm objects can disagree on the import module for the same `napi_*` symbols (`napi` versus `env`) and fail at the final `wasm-ld` link.

32.4 Current Result

The structured-clone API split required Edge messaging call sites to use the three-argument unofficial `napi_structured_clone(...)` for no-transfer clones and `unofficial_napi_structured_clone_with_transfer` when a transfer list is present. After that source fix, the native root build and native QuickJS CLI build passed.

The QuickJS WASIX build then failed at the final link because `edge_environment_core` saw default WASM N-API imports from the common header. Adding `NAPI_EXTERN=` for that target under `EDGE_NAPI_PROVIDER=quickjs` resolved the mismatch. Verified:

```
make build
make build-edge-quickjs-cli
cd ~/src/dev/edgejs/quickjs-wasm && ./build.sh
```

`quickjs-wasm/build.sh` produced `build-quickjs-wasix/edge.wasm` and `build-quickjs-wasix/edgejs.wasm`, and its final no-N-API-imports check passed.

The QuickJS source submodule now lives under the N-API project at `napi/quickjs/deps/quickjs`, not at the EdgeJS repository root. The root `EDGE_NAPI_PROVIDER=quickjs` CMake path should add that nested QuickJS CMake project and force `NAPI_QUICKJS_INCLUDE_DIR` to the same directory so old build caches do not keep pointing at the removed root `quickjs/` path.

The standalone N-API Cargo workflow also needs to stay on the standalone release family instead of following the vendored path manifest blindly. The vendored `napi/Cargo.toml` tracks local `0.702 / 7.2` path crates, but the `crates.io 7.2.0-alpha.2` graph requires Rust 1.92. With the default Rust 1.91 toolchain, `~/src/dev/edgejs/napi/cargo-standalone.sh test --lib -- --nocapture` failed before tests started. Pinning `napi/Cargo.standalone.toml` to Wasmer 7.1.0 and WASIX/virtual-fs 0.701.0 restores the standalone `crates.io` graph and the library tests pass.

32.5 Working Rule

From this rollback onward, changes needed only for Edge QuickJS should not be made in `lib/`, `napi/v8/`, `napi/src/`, or `napi/include/`. Shared runtime changes should happen only when they are correct for all providers and can be justified independently of QuickJS.

33 Node compatibility test failure analysis

		Remarks
Status	[open]	Historical failure clustering; individual failures have node-test issue pages.
Severity	High	Node compatibility failures remain significant until the linked issue pages close.

Date: 2026-05-07 Last updated: 2026-05-15

This note is chronological. The May 7 sections describe the original V8/default-run failure clustering. Current QuickJS issue status lives in the later dated sections and the canonical [troubleshooting/node-test](#) registry. EdgeJS root verification now focuses on Edge runtime tests; native N-API test suites are owned by the N-API repository and should not be treated as EdgeJS CI or root Makefile gates.

Command under investigation:

```
make test-only TEST_JOBS=4
NODE_TEST_RUNNER=build-edge/edge ./test/nodejs_test_harness --category=node:buffer,node:console,node:process,node:worker
--skip-tests=known_issues/test-stdin-is-always-net.socket.js,parallel/test-dns-perf_hooks.js,pa
```

The pasted CI run ends with 92 failures out of 1685 tests. A local rerun from this workspace also exits with code 2, but the local failure count is larger because the current workspace build/environment exposes the same broken surfaces across more tests. The root-cause grouping below is based on the pasted CI signatures and source inspection.

Earlier node-test baseline and root-cause narrative sections were intentionally omitted from this PDF refresh. The current registry and the May 16, 2026 QuickJS macOS CI grouping below are the retained node-test source of truth for this edition.

33.1 2026-05-16 QuickJS macOS CI Failure Log

The test and build / quickjs-macos log captured in test-failures-qjs-2.log still matches the expected QuickJS compatibility baseline shape: the CI job built and packaged the QuickJS macOS CLI, then make test-quickjs-only TEST_JOBS=4 finished with 1742 passing tests and 37 failures.

Grouped by likely source-level cause:

- Buffer deprecation filtering: test-buffer-constructor-node-modules-paths.js emitted the Buffer() deprecation warning from a node_modules fixture path.
- Console/inspect formatting: test-console-issue-43095.js still throws on a revoked proxy during inspection; pseudo-tty/console_colors.js still differs in stack/color formatting.
- DNS/c-ares lifecycle: test-dns-multi-channel.js produced EDESTRUCTION; test-dns-channel-timeout.js, test-dns-setserver-when-querying.js, and test-dns-resolver-max-timeout.js crashed with signal 6.
- Fetch/proxy/Undici parser bootstrap: test-http-proxy-fetch.mjs, test-use-env-proxy-cli-http.mjs, test-fetch.mjs, test-https-proxy-fetch.mjs, and test-use-env-proxy-cli-https.mjs failed because Undici tried to initialize its WebAssembly llhttp path while QuickJS has no global WebAssembly.
- HTTP proxy URL validation/timer: test-http-proxy-request-invalid-char-in-url.mjs timed out.
- Diagnostics/workers: test-diagnostics-channel-worker-threads.js timed out.

- Sync waiting on the main thread: `test-http-keep-alive-timeout-race-condition.js`, `test-fastutf8stream-flush-sync.js`, and `test-fastutf8stream-retry.js` failed with `TypeError: cannot block in this thread`.
- Explicit resource management syntax: `test-stream-duplex-destroy.js`, `test-stream-readable-dispose.js`, `test-stream-transform-destroy.js`, and `test-stream-writable-destroy.js` failed while parsing using.
- Stream semantics: `test-stream-readable-async-iterators.js` hit a destroy assertion and `test-stream-pipeline.js` called a non-function iterator helper.
- Crypto/WebCrypto/worker transfer: `test-crypto-dh-modp2-views.js`, `test-crypto-key-objects-messageport.js`, `test-crypto-subtle-cross-realm.js`, `test-webcrypto-digest.js`, `test-crypto-prime.js`, `test-crypto-worker-thread.js`, and `test-webcrypto-cryptokey-workers.js` still fail by a mix of bad DH view handling, MessagePort/key clone semantics, crashes, and worker timeouts.
- Domain/promise context: `test-domain-vm-promise-isolation.js` still observes the active domain where Node expects undefined; `test-domain-multiple-errors.js` timed out.
- HTTP/2 follow-up: `test-http2-client-set-priority.js` and `test-http2-compatible-serverresponse-writehead.js` crashed with signal 11, `test-http2-response-splitting.js` failed a response-splitting assertion, and `test-http2-reset-flood.js` timed out. This reopens `troubleshooting/node-test/017_http2_native_lifecycle_crashes.md` for the new QuickJS macOS crash cluster.

Local reproduction was blocked in this checkout because `napi/quickjs/deps/quickjs` is empty, so `make build-edge-quickjs-cli JOBS=4` failed during CMake configure before producing `build-edge-quickjs-cli/edge`. Repopulate the QuickJS dependency/submodule before rerunning the targeted tests and the crash cases under LLDB.

34 Cleanup And Containment Subtasks

35 Shared runtime rollback

		Remarks
Status	[done]	Historical containment task. Current issues are tracked in troubleshooting pages.
Severity	Low	

35.1 Scope

Restore these directories from ~/src/dev/wasmer-io/edgejs:

- lib/
- napi/v8/
- napi/src/
- napi/include/

This records the containment policy that QuickJS-only changes should not live in shared EdgeJS library code, V8 provider code, or common N-API code.

35.2 Status

Local status: completed.

35.3 Verification

These comparisons passed after rollback:

```
diff -qr ~/src/dev/wasmer-io/edgejs/lib ~/src/dev/edgejs/lib
diff -qr ~/src/dev/wasmer-io/edgejs/napi/v8 ~/src/dev/edgejs/napi/v8
diff -qr ~/src/dev/wasmer-io/edgejs/napi/src ~/src/dev/edgejs/napi/src
diff -qr ~/src/dev/wasmer-io/edgejs/napi/include ~/src/dev/edgejs/napi/include
```

35.4 Notes

The remaining git status modifications under lib/ are expected because the local branch differs from its own git base; the acceptance check for this task is the donor-tree diff -qr, not a clean git diff.

36 Native inspector fallback

		Remarks
Status	[caveated]	Native unavailable-inspector fallback exists with known shape limits.
Severity	Medium	Public inspector import behavior can affect framework startup and metadata expectations.

36.1 Scope

Keep `lib/inspector.js` donor-clean. Provide the unavailable inspector behavior from C/C++ instead of patching the shared JavaScript module.

36.2 Dependencies

Depends on `001_shared_runtime_rollback.md`, because `lib/inspector.js` must stay donor-clean.

36.3 Current Implementation

- `src/internal_binding/binding_inspector.cc` implements `internalBinding("inspector")` as a native fallback object.
- `src/internal_binding/dispatch.cc` only routes the inspector binding name to that resolver.
- `src/edge_module_loader.cc` special-cases the inspector builtin at the native builtins compile hook, so `public require("inspector") / require("node:inspector")` return the native fallback without executing donor-clean `lib/inspector.js`, which still throws when compiled normally with `internalBinding("config").hasInspector === false`.
- The C++ `require-builtin` bridge also returns the native fallback for direct native requests for builtin id `inspector`.
- Keeping `hasInspector` false is important: setting it true activates broad bootstrap inspector paths such as console wrapping and async inspector hooks.

36.4 Verification

This passed:

```
./build-edge-quickjs-cli/edge -e "const inspector=require('inspector'); console.log(typeof inspector)"
```

Expected output shape:

```
function undefined
ERR_INSPECTOR_NOT_AVAILABLE
ERR_INSPECTOR_NOT_AVAILABLE
```

Also passed:

```
./build-edge-quickjs-cli/edge -e "console.log(process.features.inspector, internalBinding('config').hasInspector)"
```

Expected:

```
false false
```

36.5 Status Notes

- `require("inspector")` and `require("node:inspector")` return the same native fallback object, and `internalBinding("config").hasInspector / process.features.inspector` remain false.
- The native public fallback is intentionally smaller than upstream `lib/inspector.js`: passive code can import the module, call `url()`, and construct `Session`; active APIs such as `open()` and `Session.connect()` report `ERR_INSPECTOR_NOT_AVAILABLE`.
- `Session` exposes a small `EventEmitter`-shaped method surface so passive listener setup and `inspector/promises` module initialization work.
- Builtin metadata now reports `inspector` as publicly requireable: `internalBinding("builtins").builtinCategories` includes `inspector`, and `cannotBeRequired` does not.

Lightweight probes:

```
./build-edge-quickjs-cli/edge -e "const a=require('inspector'); const b=require('inspector'); console.log(a===b)"
./build-edge-quickjs-cli/edge -e "const p=require('inspector/promises'); console.log(typeof p.Session)"
./build-edge-quickjs-cli/edge -e "const inspector=require('inspector'); const s=new inspector.Session(); console.log(s.open())"
./build-edge-quickjs-cli/edge -e "const cats=internalBinding('builtins').builtinCategories; console.log(cats.inspector)"
```

May 14, 2026 rerun on `/Users/syrusakbary/Development/edgejs` passed these native QuickJS probes after rebuilding `build-edge-quickjs-cli/edge`. The `private-poker` app command:

```
PORT=3100 /Users/syrusakbary/Development/edgejs/build-edge-quickjs-cli/edge npm run start
```

advanced past `ERR_INSPECTOR_NOT_AVAILABLE` and reached a later `next.config.ts` transform failure:

```
Error: Failed to get Buffer pointer and length
    at transform (null) {
      code: 'InvalidArg'
    }
}
```

36.6 Ownership

Code ownership for this subtask is `src/internal_binding/binding_inspector.cc` and the `inspector` special-cases in `src/edge_module_loader.cc`.

37 Intl fallback module

		Remarks
Status	[caveated]	Minimal Intl fallback exists with known ECMA-402 limits.
Severity	Medium	Framework bootstrap can depend on Intl.DateTimeFormat, but the fallback is partial.

37.1 Scope

Keep the minimal Intl.DateTimeFormat fallback isolated in edge_intl_fallback.cc, implemented with N-API instead of raw JavaScript evaluation.

37.2 Current Implementation

- Added src/edge_intl_fallback.h.
- Added src/edge_intl_fallback.cc.
- Added the source file to edge_runtime in CMakeLists.txt.
- Replaced the old InstallMinimalIntlFallback(...) string-eval block in src/edge_runtime.cc with EdgeInstallMinimalIntlFallback(...).

37.3 Intended Behavior

If a real globalThis.Intl.DateTimeFormat function exists, leave it untouched. Otherwise install a deliberately minimal fallback that supports:

- new Intl.DateTimeFormat(locales, options)
- .format(value)
- .resolvedOptions()
- Intl.DateTimeFormat.supportedLocalesOf()

This fallback is only meant to unblock framework bootstrap formatting, not to claim full ECMA-402 compatibility.

37.4 Verification Status

After rebuild, run:

```
./build-edge-quickjs-cli/edge -e "const f = new Intl.DateTimeFormat([], { hour: '2-digit', minute: '2-digit' })"
```

Expected shape:

```
string 2-digit
```

37.5 Ownership

Code ownership for this subtask is src/edge_intl_fallback.{h,cc}, src/edge_runtime.cc, and the edge_runtime source list in CMakeLists.txt.

37.6 Status Notes

2026-05-07 source inspection, with no source edits:

- The fallback has been split into `src/edge_intl_fallback.{h,cc}` and wired into the `edge_runtime` target.
- `src/edge_runtime.cc` now delegates to `EdgeInstallMinimalIntlFallback(...)` instead of keeping a local string-eval fallback block.
- The fallback implementation uses N-API calls to inspect/install `globalThis.Intl.DateTimeFormat`; no raw JavaScript evaluation appears in `src/edge_intl_fallback.cc`.
- The implemented minimal behavior covers the current subtask's documented new `Intl.DateTimeFormat(locales, options)` path, `.format(value)`, `.resolvedOptions()`, and static `supportedLocalesOf()`.
- Caveat: the earlier Astro troubleshooting note described the fallback as “callable/constructible”, but this subtask's intended behavior only names the new `Intl.DateTimeFormat(...)` constructor path. If call-without-new compatibility is part of the review ask, it should be verified separately because the current N-API class constructor path is not explicitly shaped for that behavior.

38 Compile-time trace diagnostics

		Remarks
Status	[done]	Historical trace macro cleanup task. Diagnostic naming does not block runtime behavior.
Severity	Low	

38.1 Scope

Preserve the native C/C++ tracing that helped diagnose QuickJS TTY, HTTP, stream, module, and bootstrap issues, but make it compile-time disableable so production builds can compile the trace checks out.

38.2 Current Implementation

- Added `src/edge_trace.h` for Edge runtime code.
- Added `napi/quickjs/src/internal/quickjs_trace.h` for QuickJS N-API code.
- Added `EDGE_ENABLE_TRACE_DIAGNOSTICS=$<IF:$<CONFIG:Debug>,1,0>` to `edge_runtime` and `napi_quickjs` compile definitions.
- Converted `std::getenv("EDGE_TRACE_*") != nullptr` trace checks to `EDGE_TRACE_ENABLED("EDGE_TRACE_*")`.

38.3 Trace Families

Known trace switches currently covered:

- `EDGE_TRACE_BOOTSTRAP`
- `EDGE_TRACE_BUILTINS`
- `EDGE_TRACE_INTERNAL_BINDING`
- `EDGE_TRACE_NET`
- `EDGE_TRACE_QUICKJS_CONTEXTIFY`
- `EDGE_TRACE_QUICKJS_MODULES`
- `EDGE_TRACE_TTY`

38.4 Verification Status

After rebuild, check that there are no raw `std::getenv("EDGE_TRACE_...")` checks left in `src/` or `napi/quickjs/src/`:

```
rg -n 'std::getenv\("EDGE_TRACE|EDGE_TRACE_[A-Z_]+\)" != nullptr' src napi/quickjs/src
```

Also confirm a Release-style build compiles with `EDGE_ENABLE_TRACE_DIAGNOSTICS` set to `0` and the QuickJS smoke tests still pass.

38.5 Ownership

Code ownership for this subtask spans trace call sites in `src/` and `napi/quickjs/src/`, plus the new trace headers and CMake compile definitions.

38.6 Status Notes 2026-05-07

Lightweight read-only review checked:

```
rg -n "EDGE_ENABLE_TRACE_DIAGNOSTICS|EDGE_TRACE_ENABLED|EDGE_TRACE_[A-Z_]+|std::getenv\(|getenv\(|  
rg -n "getenv\\(\\s*\"EDGE_TRACE|std::getenv\\(\\s*\"EDGE_TRACE" . --glob '!plans/**' --glob '!build*  
rg -n "#include .*edge_trace|#include .*quickjs_trace|EDGE_TRACE_ENABLED" src napi/quickjs/src
```

Findings:

- No raw `std::getenv("EDGE_TRACE_*") / getenv("EDGE_TRACE_*")` call sites were found outside the trace helper headers.
- Current `EDGE_TRACE_ENABLED(...)` callers include `edge_trace.h` or `internal/quickjs_trace.h` directly in the same translation unit.
- The reviewed CMake files define `EDGE_ENABLE_TRACE_DIAGNOSTICS` for `edge_runtime` and `napi_quickjs`; `edge_cli.cc` uses the header fallback rather than a target-specific compile definition.
- Full Release-style compile verification was not run in this review because this pass was intentionally limited to lightweight read-only commands.

39 Verification and remaining cleanup

		Remarks
Status	[done]	Historical verification note. Current verification status is captured by the latest issue pages and test runs.
Severity	Low	

39.1 Scope

Tie together the rollback, native inspector fallback, Intl fallback split, and compile-time trace diagnostics.

39.2 Current Build State

The focused rebuilds have passed after the structured-clone API split and the QuickJS WASIX NAPI_EXTERN= target fix:

```
make build
make build-edge-quickjs-cli
cd ~/src/dev/edgejs/quickjs-wasm && ./build.sh
```

The WASIX build produced build-quickjs-wasix/edge.wasm and edgejs.wasm, and the script's final no-N-API-imports check passed.

39.3 Required Smoke Tests

Run after rebuild:

```
./build-edge-quickjs-cli/edge --version
./build-edge-quickjs-cli/edge -e "console.log('hello from quickjs')"
./build-edge-quickjs-cli/edge -e "const inspector=require('inspector'); console.log(typeof inspector)"
./build-edge-quickjs-cli/edge -e "const f = new Intl.DateTimeFormat([], { hour: '2-digit', minute: '2-digit' }); console.log(f.format(new Date(2020, 0, 1, 12, 12, 12)))"
```

Also rerun the donor-tree comparisons from 001_shared_runtime_rollback.md.

When touching targets that include N-API headers and link into the embedded QuickJS WASIX binary, verify the target gets NAPI_EXTERN= for EDGE_NAPI_PROVIDER=quickjs; otherwise the final wasm link can fail with napi_i versus env import module mismatches.

39.4 Known Unrelated Dirty State

wasmer.toml was already modified before this cleanup thread. Do not revert it unless explicitly asked.

40 Internal N-API Promises Refactor

		Remarks
Status	[done]	Promise and microtask logic lives in napi_promises__; native QuickJS and V8 suites pass. Promise hooks and rejection tracking must behave consistently for native QuickJS N-API.
Severity	High	

40.1 Scope

Move QuickJS N-API promise hook, promise rejection tracker, microtask job, and continuation-preserved embedder data state into:

```
napi/quickjs/src/internal/napi_promises.h
napi/quickjs/src/internal/napi_promises.cc
```

The class must be named `napi_promises__`, and `napi_env__` should own a direct object of that class rather than storing individual promise fields.

40.2 Current State

- `napi_promises.{h,cc}` contains class `napi_promises__`.
- `napi_env__` owns a direct `napi_promises__` `promises__` object and no longer stores individual promise hook, rejection callback, microtask, or continuation embedder data fields.
- `unofficial_napi.cc` calls `env->promises()` and delegates to `napi_promises__::promise_hook`, `napi_promises__::rejection_tracker`, and `napi_promises__::microtask_job`.
- Runtime promise hook and rejection tracker wiring is now owned by `napi_promises__`; `unofficial_napi.cc` delegates `attach/update/clear` calls instead of calling `JS_SetPromiseHook(...)` and `JS_SetHostPromiseRejectionTracker(...)` directly.
- Continuation-preserved embedder data now covers both ordinary promise reactions and async-function `await` resumptions. The vendored QuickJS `promise_reaction_job(...)` recovers hook identity from resolving functions first, then from the async-function reaction handler when the throwaway capability slots are undefined.
- Captured frames are released on promise fulfilment or rejected-promise tracking, not immediately on `AFTER`, so a multi-`await` async function keeps the same request frame across every resume.
- `quickjs/CMakeLists.txt` builds `src/internal/napi_promises.cc`.
- The earlier separate microtask files have been removed after all references were eliminated.
- The rejection handled callback keeps the V8-shaped event payload: event 1 passes undefined as the reason.
- `PromiseHooksObserveLifecycleEvents` uses a thenable because stock QuickJS emits before/after hooks for thenable resolution jobs, not for ordinary already-resolved promise reactions.
- `PromiseReactionRestoresContinuationPreservedEmbedderData` and `AsyncAwaitRestoresContinuationPreservedEmbedderData` cover request-frame restoration across promise jobs and consecutive `await` resumptions.

40.3 Verification

Run from `/Users/sadhbh/src/dev/edgejs/napi`:

```
make test-native-quickjs
make test-native-v8
```


Completed on May 16, 2026:

```
cd /Users/sadhbh/src/dev/edgejs/napi && make test-native-quickjs
```

Result: 67/67 tests passed, including the promise hook, rejection callback, promise reaction, and async/await continuation coverage.

Also run because the promise test is shared:

```
cd /Users/sadhbh/src/dev/edgejs/napi && make test-native-v8
```

Result: 45/45 tests passed.

41 Internal N-API Serdes Refactor

		Remarks
Status	[done]	Serdes state is owned by napi_serdes__; native QuickJS tests pass. Serialization behavior is observable through unofficial N-API and framework imports.
Severity	Medium	

41.1 Scope

Serdes state lives in:

```
napi/quickjs/src/internal/napi_serdes.h
napi/quickjs/src/internal/napi_serdes.cc
```

napi_serdes__ owns serializer/deserializer state and keeps unofficial_napi.cc focused on public entry points.

41.2 Verification

Run from /Users/sadhbh/src/dev/edgejs/napi:

```
make test-native-quickjs
```

Run V8 too if the shared tests or headers are touched in a way that may affect both engines.

41.3 Current State

- napi_serdes__ owns the serializer, deserializer, and serialized payload native state.
- unofficial_napi.cc delegates serialize/deserialize payload handling to napi_serdes__ and uses the internal serdes callback methods for internalBinding("serdes").
- quickjs/CMakeLists.txt builds src/internal/napi_serdes.cc.

41.4 Verification Result

Run from /Users/sadhbh/src/dev/edgejs/napi:

```
make test-native-quickjs
```

Result: passed, 45/45 QuickJS N-API tests passing.

42 Internal N-API Contextify Refactor

		Remarks
Status	[done]	Contextify helpers live in napi_contextify__; native QuickJS tests pass.
Severity	Medium	Contextify diagnostics affect script compilation and source-map reporting.

42.1 Scope

Contextify state lives in:

```
napi/quickjs/src/internal/napi_contextify.h
napi/quickjs/src/internal/napi_contextify.cc
```

napi_contextify__ wraps contextify helpers/state, and napi_env__ owns an instance directly.

42.2 Known Limitation

These V8-shaped APIs remain limited for QuickJS:

```
unofficial_napi_preserve_error_source_message
unofficial_napi_set_source_maps_enabled
unofficial_napi_set_get_source_map_error_source_callback
```

The reason is documented near the implementation and tests.

42.3 Current State

- Added napi/quickjs/src/internal/napi_contextify.{h,cc} with class napi_contextify__.
- Moved contextify make/run/dispose/compile/cache-data/module-syntax handling, compile exception annotation, source-map toggles, and caught-error formatting fallbacks behind the class.
- napi_env__ owns a direct quickjs::detail::napi_contextify__ member and exposes contextify() accessors.
- napi/quickjs/src/unofficial_napi.cc now delegates the contextify and source-map/error-formatting unofficial APIs through env->contextify().
- napi/quickjs/CMakeLists.txt builds src/internal/napi_contextify.cc. The source-map APIs remain intentionally limited for QuickJS. The class stores the enabled flag and validates/stores the optional callback, but preserve_error_source_message() remains a stable no-op because QuickJS does not expose a V8-style message object for arbitrary caught Error values.

42.4 Verification

Run from /Users/sadhbh/src/dev/edgejs/napi:

```
make test-native-quickjs
```

Result on 2026-05-09:

100% tests passed, 0 tests failed out of 45

43 Internal N-API Set Property Refactor

		Remarks
Status	[done]	Property setting logic lives in napi_set_property; native QuickJS tests pass.
Severity	Medium	Property assignment differences can surface as observable N-API behavior.

43.1 Scope

Property setting logic lives in:

```
napi/quickjs/src/internal/napi_set_property.h
napi/quickjs/src/internal/napi_set_property.cc
js_native_api_quickjs.cc delegates to this focused internal helper.
```

43.2 Verification

Run from /Users/sadhbh/src/dev/edgejs/napi:

```
make test-native-quickjs
```

43.3 Current State

Updated napi/quickjs/src/js_native_api_quickjs.cc to include the new internal header and updated napi/quickjs/CMakeLists.txt to compile src/internal/napi_set_property.cc.

A source search under napi/quickjs/src and napi/quickjs/CMakeLists.txt only finds the new internal helper path and the napi_set_property public entry point.

Updated the fallback mechanism to avoid matching QuickJS exception text. After JS_SetProperty fails, set_property_with_node_compat now uses QuickJS descriptor APIs to inspect the target and its prototype chain:

- if the receiver already has the property, keep the original QuickJS failure;
- if a prototype accessor has a setter, keep the original QuickJS failure;
- if the inherited descriptor has no setter or is readonly, define a Node-like own property on the receiver with JS_DefinePropertyValue(..., JS_PROP_C_W_E).

Verification attempted:

```
cd /Users/sadhbh/src/dev/edgejs/napi
make test-native-quickjs
```

A later full QuickJS N-API run passed after the concurrent contextify and exception/source-map test changes settled.

Additional local check:

```
git -C napi diff --check -- quickjs/src/internal/napi_set_property.h quickjs/src/internal/napi_set_property.h
```

This completed without whitespace errors.

44 Subtask 001: QuickJS Module API and Record

		Remarks
Status	[done]	Implemented in vendored QuickJS plus napi_module_wrap__; focused source-text module checks pass.
Severity	High	Without a real source-text ModuleWrap, Node's JS loaders cannot link or evaluate module graphs.

44.1 Scope

Implement the QuickJS backend's core source-text module record and add the minimal vendored QuickJS APIs needed to mirror V8's ModuleWrap lifecycle.

44.2 Write Ownership

Primary files:

- napi/quickjs/src/quickjs/quickjs.h
- napi/quickjs/src/quickjs/quickjs.c
- napi/quickjs/src/internal/napi_module_wrap.h
- napi/quickjs/src/internal/napi_module_wrap.cc
- napi/quickjs/src/unofficial_napi.cc
- napi/quickjs/CMakeLists.txt

Do not edit Node JS loader files in this subtask except for small diagnostics needed to prove integration points.

44.3 Dependencies

None. This subtask should begin by comparing the V8 implementation in:

napi/v8/src/unofficial_napi_contextify.cc

Focus on behavior, not V8-specific object lifetime assumptions.

44.4 Required Behavior

- Compile source text modules with QuickJS compile-only module evaluation.
- Store a stable JSModuleDef* and owning JSValue.
- Expose module requests with specifier, attributes, and evaluation phase.
- Implement link(...) as a dependency-record table populated by JS loader output.
- Implement instantiate() as QuickJS link-only behavior, not evaluation.
- Implement evaluate() and evaluateSync() through evaluate-only QuickJS behavior.
- Map QuickJS module status to internalBinding('module_wrap') constants.
- Preserve and expose module evaluation errors through getError().
- Report top-level await and async graph state.

44.5 QuickJS API Patch

Add the smallest public C API needed by the N-API backend:

- request count and request metadata;
- module status;
- top-level await and async graph checks;
- link-only;
- evaluate-only;
- module evaluation error retrieval.

Keep the API names clearly embedder-oriented and avoid leaking large internal struct definitions into N-API C++ code.

44.6 Verification Expectations

Add focused tests before broad framework runs:

```
make build-napi-quickjs
make test-napi-quickjs-only
```

At this stage, passing tests should include source-text module creation, request enumeration, link/instantiate/evaluate status transitions, and namespace access for a dependency-free module.

44.7 Implementation Result

The vendored QuickJS API now exposes V8-shaped module introspection and split lifecycle operations: request enumeration, explicit host linking, link-only, evaluate-only, status, evaluation error, top-level-await, async-graph, `import.meta`, and dynamic import hooks.

`napi/quickjs/src/internal/napi_module_wrap.{h,cc}` owns the QuickJS-backed module records, with `unofficial_napi.cc` reduced to forwarding glue for the module-wrap symbols. Source text modules compile in QuickJS compile-only mode and expose request objects with specifier, attributes, and phase.

45 Subtask 002: Node Loader Interop and Synthetic Modules

		Remarks
Status	[done]	Implemented through explicit host-linked requests and QuickJS C modules for synthetic records. CommonJS parity needs the JS ESM translators' synthetic modules to work.
Severity	High	

45.1 Scope

Connect QuickJS source-text module records to Node's JavaScript loader-selected dependency graph, then implement synthetic modules used by CJS, builtin, JSON, and facade translators.

45.2 Write Ownership

Primary files:

- napi/quickjs/src/internal/napi_module_wrap.h
- napi/quickjs/src/internal/napi_module_wrap.cc
- napi/quickjs/src/unofficial_napi.cc

Possible supporting files:

- napi/quickjs/src/internal/napi_util.*
- napi/quickjs/tests/
- napi/tests/js-native-api/

Do not add a package resolver under napi/quickjs/src.

45.3 Dependencies

Depends on subtask 001_quickjs_module_api_and_record.md.

45.4 Required Behavior

- During `ModuleWrap.link(...)`, store the linked records by request index.
- During QuickJS module linking/evaluation, install a normalizer/loader that maps a parent module URL plus parsed request data to the already linked child record.
- Validate duplicate request keys so the same request links to the same module, matching the V8 path's `ERR_MODULE_LINK_MISMATCH` behavior.
- Implement `create_synthetic(...)` with `JS_NewCModule(...)` and declared exports.
- Run the JS synthetic evaluation callback with `this` bound to the JS wrapper.
- Implement `setExport(...)` so translator callbacks can populate QuickJS module exports.
- Ensure CJS translator synthetic wrappers expose named exports, default, and `module.exports`.

45.5 Important Call Sites

Synthetic modules are created by:

`lib/internal/modules/esm/translators.js`

The most important function to satisfy is `createCJSModuleWrap(...)`, which builds a `ModuleWrap` around CommonJS execution and calls `this.setExport(...)` during evaluation.

45.6 Verification Expectations

Targeted checks:

```
make build-napi-quickjs
make test-napi-quickjs-only
build-edge-quickjs-cli/edge -e "console.log(require('module') !== undefined)"
```

Add tests for:

- ESM importing a local CJS file with named and default exports;
- ESM importing `node:path`;
- JSON import;
- CJS cache sharing with the ESM loader.

45.7 Implementation Result

`ModuleWrap.link(...)` now validates the linked module count, detects duplicate request key mismatches, and installs JS-loader-selected dependencies with `JS_SetModuleRequestModule(...)`. QuickJS does not resolve packages in this path; Node's JavaScript loaders still own resolution and translator policy.

Synthetic modules use `JS_NewCModule(...)`, declared exports, and `JS_SetModuleExport(...)`. The C init callback invokes the JavaScript synthetic evaluation callback with the wrapper as `this`, enabling translator code to populate exports through `setExport(...)`.

46 Subtask 003: Dynamic Import, Import Meta, and Required Facade

		Remarks
Status	[done]	Implemented; dynamic import, import-meta, top-level-await, async-graph, and module-request focused checks pass. Dynamic import and require(esm) are required for V8-like loader parity.
Severity	High	

46.1 Scope

Complete the module-wrap behavior that depends on runtime callbacks: `import()`, `import.meta`, `evaluateSync(...)`, async graph detection, and `createRequiredModuleFacade(...)`.

46.2 Write Ownership

Primary files:

- `napi/quickjs/src/quickjs/quickjs.h`
- `napi/quickjs/src/quickjs/quickjs.c`
- `napi/quickjs/src/internal/napi_module_wrap.h`
- `napi/quickjs/src/internal/napi_module_wrap.cc`
- `napi/quickjs/src/unofficial_napi.cc`

Possible supporting files:

- `lib/internal/modules/esm/utils.js` only if QuickJS needs a very narrow compatibility adjustment after the native callback is present.

46.3 Dependencies

Depends on subtasks `001_quickjs_module_api_and_record.md` and `002_node_loader_interop_and_synthetic_mo`

46.4 Required Behavior

- Store the JS dynamic import callback registered by `setImportModuleDynamicallyCallback(...)`.
- Patch or wrap QuickJS dynamic import so it calls the registered JS callback instead of resolving through a native package loader.
- Preserve import attribute objects and phase values expected by `lib/internal/modules/esm/utils.js`.
- Store the JS `import.meta` initializer registered by `setInitializeImportMetaObjectCallback(...)`.
- Initialize the QuickJS import-meta object once per module with the owning wrapper.
- Implement `evaluateSync(...)` for `require(esm)` and reject async graphs with `ERR_REQUIRE_ASYNC_MODULE`.
- Implement `createRequiredModuleFacade(...)` using a source-text facade that re-exports the original module and adds `__esModule`.

46.5 Verification Expectations

Add targeted tests for:

- `await import('./esm.mjs');`

- dynamic import from inside CJS-compiled code;
- `import.meta.url`;
- CJS `require()` of a synchronous ESM file;
- CJS `require()` of an ESM graph with TLA throwing the expected error;
- `__esModule` facade behavior for `require(esm)`.

Run:

```
make build-edge-quickjs-cli JOBS=4
cmake --build build-edge-quickjs-cli --target edge -j4
make test-napi-quickjs
```

46.6 Implementation Result

QuickJS dynamic import and import-meta hooks now call the registered JavaScript callbacks from module-wrap state. Source text modules and contextified scripts preserve host-defined option symbols so dynamic import from `vm.Script` can route back through Node's loader callback.

`evaluateSync(...)` evaluates the QuickJS module promise synchronously when it is already fulfilled and throws the Node-compatible `async-module` error for pending graphs. `createRequiredModuleFacade(...)` compiles the standard facade source, links its `original` request to the existing module, evaluates it, and returns the facade namespace.

The API carries source-phase import metadata. QuickJS source-phase module value semantics are still not implemented beyond parsing/introspection because this task keeps current QuickJS static imports in evaluation phase.

47 Subtask 004: Verification

		Remarks
Status	[caveated]	Native build, N-API, Edge CLI, and focused module-wrap checks pass; WASIX is deferred and the two broader VM failures are excluded because they also fail under Edge's V8 backend. Module loading regressions affect bootstrap, framework apps, and WASIX packaging.
Severity	High	

47.1 Scope

Build the focused and end-to-end verification suite for QuickJS module loading parity.

47.2 Write Ownership

Primary files:

- napi/quickjs/tests/
- napi/tests/js-native-api/
- plans/quickjs-wasm/development/troubleshooting/node-compat/napi/007_module_loading.md

Potential smoke scripts should live near existing framework or WASIX smoke infrastructure rather than inside app output directories.

47.3 Dependencies

Can start as test design immediately. Executable test updates depend on the implementation subtasks.

47.4 Target Test Matrix

- Classic CJS:
 - local `require('./file.cjs')`;
 - directory `index.js`;
 - package `main`;
 - package `exports`;
 - `node:` and bare builtins.
- ESM:
 - static local import;
 - package import through JS resolver;
 - import attributes for JSON;
 - missing export diagnostics.
- Interop:
 - ESM imports CJS named/default/module.exports;
 - CJS requires synchronous ESM;
 - CJS requires ESM with TLA and receives `ERR_REQUIRE_ASYNC_MODULE`;
 - CJS and ESM cache sharing for JSON and CJS.
- Runtime callbacks:
 - `dynamic import()`;

- `import.meta.url`;
- required module facade and `__esModule`.
- Packaging:
 - pnpm symlink/materialized layouts;
 - WASIX QuickJS rebuild and smoke commands as a separate, deferred gate;
 - Astro/Vite/Next app smoke checks once core native tests pass.

47.5 Required Commands

Use the focused checks first:

```
make build-napi-quickjs
make test-napi-quickjs-only
```

Then run the complete local gates:

```
make test-napi-quickjs
make build-edge-quickjs-cli JOBS=4
cmake --build build-edge-quickjs-cli --target edge -j4
```

For the current native module-loading verification pass, WASIX is explicitly excluded. When WASIX is re-enabled, changes under `src/`, `lib/`, or `napi/quickjs/` should finish with:

```
cd /Users/syrusakbary/Development/edgejs/quickjs-wasm/ && ./build.sh
```

Smoke commands after a successful WASIX build:

```
wasmer run . -- --version
wasmer run . -- -e "console.log('hello from quickjs')"
wasmer run --net --volume ./quickjs-wasm:/app . -- /app/echo-server.js
```

47.6 Verification Run

Passing native checks:

- `make build-napi-quickjs`
- `make test-napi-quickjs` (45/45 CTest tests passing)
- `make build-edge-quickjs-cli JOBS=4`
- `test/parallel/test-internal-module-wrap.js` with the QuickJS Edge CLI
- `test/parallel/test-vm-module-synthetic.js`
- `test/parallel/test-vm-module-import-meta.js`
- `test/parallel/test-vm-module-dynamic-import.js`
- `test/parallel/test-vm-module-hastoplevelawait.js`
- `test/parallel/test-vm-module-hasasynctrace.js`
- `test/parallel/test-vm-module-link.js`
- `test/parallel/test-vm-module-modulerequests.js`
- manual CommonJS `require(esm)` smoke returning default, named export, and `__esModule`

Deferred gates:

- `cd /Users/syrusakbary/Development/edgejs/quickjs-wasm/ && ./build.sh`
- WASIX smoke commands under `wasmer run`
- Astro/Vite/Next framework smoke checks

Excluded Edge-wide VM baseline gaps:

- `test/parallel/test-vm-module-basic.js` reaches module execution, then fails on QuickJS context inspection shape. The same test is not a useful QuickJS module-loading gate yet because `build-edge/edge` with V8 hangs on the `SourceTextModule.evaluate({ timeout })` case.
- `test/parallel/test-vm-module-linkmodulerequests.js` reaches module assertions under QuickJS, but should not gate QuickJS parity yet. Rebuilt `build-edge/edge` with V8 fails the same file:

import source Foo from "foo" is still a syntax error and the instantiate diagnostic is only Module is not linked.

V8 baseline check:

```
cmake --build build-edge --target edge -j4
build-edge/edge -p "process.versions.v8"
NODE_SKIP_FLAG_CHECK=1 build-edge/edge --experimental-vm-modules test/parallel/test-vm-module-ba
NODE_SKIP_FLAG_CHECK=1 build-edge/edge --experimental-vm-modules --js-source-phase-imports test/p
```

Observed V8 version: 13.6.233.17-node.0. The basic test timed out after 15s; the link-module-requests test passed 2/4 and failed the source-phase syntax and diagnostic cases. These tests become valid QuickJS gates only after the Edge V8 backend passes them or the repository carries an Edge-specific expectation.

47.7 Symbol Dispose Regression

The old `_http_server` bootstrap failure:

```
at get description (native)
at assignFunctionName
```

was reproduced as missing `Symbol.dispose` / `Symbol.asyncDispose` in QuickJS. The shim was restored in `napi/quickjs/src/unofficial_napi.cc` and covered by `napi/tests/js-native-api/13_symbol/test.js`.

48 Dev 003: QuickJS Module Loading

		Remarks
Status	[caveated]	Core QuickJS ModuleWrap bridge is implemented and native focused gates pass; the remaining broader VM failures are V8-baseline gaps in Edge.js and are not QuickJS module-loading blockers. Full CommonJS parity depends on ESM/CJS interop, synthetic facades, and require(esm).
Severity	High	

48.1 Goal

Make the QuickJS backend support module loading through the same JavaScript loader and translator stack used by the V8 backend. The native work should provide the engine-level ModuleWrap behavior that Node's JS loaders expect, not a second package resolver or CJS implementation in C++.

Canonical issue page:

plans/quickjs-wasm/development/troubleshooting/node-compat/napi/007_module_loading.md

48.2 Architecture Summary

The existing `src/internal_binding/binding_module_wrap.cc` already exposes the N-API-facing `internalBinding('module_wrap')` class. It forwards to `unofficial_napi_module_wrap_*`. The V8 backend implements those calls in `napi/v8/src/unofficial_napi_contextify.cc`; the QuickJS backend currently stubs them in `napi/quickjs/src/unofficial_napi.cc`.

The implementation should add a QuickJS internal module record class and route the existing unofficial API calls to it. Node's JavaScript files under `lib/internal/modules/` remain responsible for package semantics.

48.3 Subtasks

Note	Status	Scope
001_quickjs_module_api_and_record.md	[done]	QuickJS module API patch and core <code>napi_module_wrap__</code> record.
002_node_loader_interop_and_synthetic_modules.md	[done]	Prelinked loader lookup and synthetic modules for CJS, builtins, JSON, and facades.
003_dynamic_import_import_meta_required_facade.md	[caveated]	Dynamic import, <code>import.meta</code> , <code>require(esm)</code> , and required module facade parity.
004_verification.md	[caveated]	Native N-API tests, Node loader tests, Edge CLI checks, and deferred WASIX/framework smoke checks.

48.4 Dependency Order

1. 001 must land first. It provides the QuickJS lifecycle split that all other module behavior depends on.
2. 002 depends on 001 and unlocks JS loader interop plus CommonJS facades.
3. 003 depends on 001 and 002; it completes dynamic and synchronous interop paths.
4. 004 can start with test design immediately, but the executable tests depend on the implementation subtasks.

48.5 Parallelization Notes

The implementation subtasks mostly touch the same QuickJS module-wrap files, so they should not be assigned to parallel workers with overlapping write sets. Parallel work is useful for read-only V8 comparison, test planning, and framework smoke scripts after the core API shape is decided.

49 V8 N-API lifetime refactor: current state and gap analysis

		Remarks
Status	[done]	Implemented.
Severity	Medium	The V8 backend is functional, but its handle, reference, wrap, and finalizer lifetimes are less Node-like than both Node's V8 N-API and the newer QuickJS backend.

49.1 Scope

Compare:

- QuickJS reference implementation: `napi/quickjs/src/`
- Edge V8 implementation: `napi/v8/src/`
- Node V8 implementation reference: `node/src/js_native_api_v8.*`

The goal is not to port QuickJS mechanics directly into V8. The goal is to bring the Edge V8 implementation back toward Node's V8 ownership model, while reusing the cleanup discipline and diagnostics lessons learned in the QuickJS backend.

49.2 Current Edge V8 Shape

The Edge V8 provider currently defines `napi_value__` as an owning wrapper around `v8::Global<v8::Value>` in `napi/v8/src/internal/napi_v8_env.h`. Every call to `napi_v8_wrap_value(...)` allocates a heap object and promotes a local V8 handle to a persistent/global handle. Handle scopes in `napi/v8/src/js_native_api_v8.cc` are lightweight structs with only env and escaped state; they do not contain real V8 `HandleScope` / `EscapableHandleScope` objects and do not bound the lifetime of returned `napi_value` handles.

References, wraps, buffers, finalizers, and type tags are mostly independent records held through `std::vector<void*>`, `v8::Global`, private properties, and weak callbacks. This works for many paths, but it is unlike Node's implementation and makes local handles look persistent by default.

49.3 QuickJS Lessons

The QuickJS backend solved the same class of bugs through explicit ownership:

- `napi_value__` owns a `JSValue`, and values are allocated into an active `napi_scope__`.
- `napi_scope__` owns local value slots and implements close/escape behavior.
- `napi_ref__` is separate env-owned storage for persistent references.
- `external/wrap/finalizer` state lives in dedicated external backing-store records and is finalized by engine-owned weak/finalizer paths.
- `napi_function__::trampoline(...)` opens a callback-local handle scope and duplicates the return value before closing it.
- `napi_lifetime_tracker__` can report value, reference, scope, external, and callback object counts without changing normal runtime behavior.

For V8, the key lesson is not “copy QuickJS slots”. It is “make local handles local again, keep persistent ownership explicit, and make finalizer ownership trackable”.

49.4 Node V8 Reference Point

Node's `node/src/js_native_api_v8.h` has the desired `napi_value` shape:

```
static_assert(sizeof(v8::Local<v8::Value>) == sizeof(napi_value),
              "Cannot convert between v8::Local<v8::Value> and napi_value");

inline napi_value JsValueFromV8LocalValue(v8::Local<v8::Value> local) {
    return reinterpret_cast<napi_value>(*local);
}

inline v8::Local<v8::Value> V8LocalValueFromJsValue(napi_value v) {
    v8::Local<v8::Value> local;
    memcpy(&static_cast<void*>(&local), &v, sizeof(v));
    return local;
}
```

Node then uses real `v8::HandleScope` and `v8::EscapableHandleScope` wrappers for N-API scopes, while `napi_ref` is a tracked `v8impl::Reference` around `v8::Global`. Wraps and finalizers are also represented by tracked references with explicit runtime or userland ownership.

49.5 Main Gaps

1. `napi_value` representation is too heavy and too persistent. Edge V8 currently turns locals into heap-allocated global handles. This bypasses V8's native local-handle scope model and can retain values far longer than intended.
2. handle scopes are not real handle scopes. `napi_open_handle_scope(...)` allocates a plain struct; closing it does not release local V8 handles. `napi_escape_handle(...)` returns the same handle instead of using `EscapableHandleScope::Escape(...)`.
3. callback info owns heap-wrapped arguments. Function/accessor trampolines allocate `napi_value__` wrappers for this, args, return values, and `new_target`, then delete some of them manually. Node passes local handles through the callback wrapper.
4. references are close in spirit but not Node-like enough. `napi_ref__` has a `v8::Global`, weak callback, and refcount, but it is not a `RefTracker`, is not linked into env lists, and has a known "do not delete weak ref during GC" workaround that intentionally leaks the bookkeeping object.
5. wraps/finalizers are split across private properties and vectors. Node stores a tracked reference behind the wrapper private key. Edge V8 stores raw native data, an optional raw ref pointer, and a separate finalizer record. This makes ownership rules harder to reason about and diverges from Node.
6. type tags and externals diverge from Node behavior. Edge V8 uses an env vector of `v8::Global` entries for object type tags and raw `v8::External` values for externals. Node uses object/private state for objects and an `ExternalWrapper` for external values, including type tags.
7. cleanup is not unified. Env teardown manually walks several vectors. Node's env carries tracked lists for references/finalizers and has explicit finalizer enqueue/dequeue behavior.

49.6 Desired End State

- `napi_value` in `napi/v8/src` is a direct V8 local-handle slot cast, following Node's `JsValueFromV8LocalValue(...)` / `V8LocalValueFromJsValue(...)`.
- N-API handle scopes allocate real V8 handle-scope wrappers and enforce mismatch checks.
- `napi_ref`, wraps, externals, and finalizers use Node-shaped tracked reference classes adapted to Edge's smaller env.
- Local handles are not promoted to `v8::Global` unless the API explicitly asks for persistence: references, env state, cached constructors, context records, module records, buffers, or finalizers.

- A V8 lifetime tracker, inspired by the QuickJS tracker, can report live references/finalizers/wrap records and open-scope counts in diagnostic builds.

49.7 Implementation Result

Implemented across the Stage 001-006 task sequence:

- `napi_value` now follows Node-style direct local-handle conversion.
- N-API handle scopes use real V8 `HandleScope` / `EscapableHandleScope` wrappers with LIFO close validation and real escape behavior.
- Callback info is stack-backed and reads directly from V8 callback state.
- `napi_ref__`, `napi_ref_with_data__`, `napi_ref_with_finalizer__`, `napi_ref_tracker__`, `napi_external_wrapper__`, handle-scope wrappers, and callback-info wrappers now live as `napi_***__` internal classes under `napi/v8/src/internal/`, with utility callbacks in `napi_util.*`.
- `napi_ref__` is env-tracked; weak refs reset/unlink through finalization, and the previous weak-delete leak workaround was removed. Finalizer references use a Node-shaped finalizing list and env-mediated drain queue.
- Cleanup hooks now run in Node-style LIFO passes and tolerate removal by later hooks.
- Wraps, finalizers, externals, and type tags were moved onto tracked, Node-shaped ownership.
- V8 now has QuickJS-style lifetime tracker plumbing under `napi/v8/src/internal/napi_lifetime_tracker.*`, enabled with `NAPI_V8_ENABLE_LIFETIME_TRACKER=ON` and dumped with `EDGE_TRACE_NAPI_LIFETIME=1`.
- The V8 tracker uses the same output markers as QuickJS (`[napi-lifetime-stats]`, `[napi-lifetime-slots]`, `[napi-lifetime-types]`, `[napi-lifetime-tags]`, and related tables), while keeping V8 semantics thin: public `napi_value` handles are direct current scope V8 locals, and the tracker records their creation/release for diagnostics without making them persistent.
- Simple V8 value constructors now follow Node's direct local-handle result pattern where Edge's standalone env boundary permits it. This includes undefined/null/global/boolean, scalar number/bigint creators, object/array, array-with-length, symbols, and dates.
- Root `make test-native-v8` and `make test-native-quickjs` now forward to the native N-API backend test targets under `napi/`.

49.8 Reference Rule

For continuing V8 N-API work, treat `napi/quickjs/src` as the current Edge behavioral baseline and `node/src/js_native_api_v8.cc` as the V8-native reference. The `napi/v8` implementation should follow Node's V8 shape unless there is a concrete Edge embedding constraint, such as Node-private Environment state, Node's split `node_api.cc` ownership, libuv/runtime hooks, or Edge-specific env/contextify/module-wrap integration.

Verification after implementation:

```
make -C napi test-napi TEST_JOBS=4
make -C napi test-napi-quickjs TEST_JOBS=4
make test-native-v8
make test-native-quickjs
```

The shared native V8 and QuickJS suites now pass 48/48. The tracker-enabled V8 build also passed 48/48, and a focused `EDGE_TRACE_NAPI_LIFETIME=1 Test16Reference.PortedCoreFlow` run emitted balanced `napi_ref` lifetime stats at env teardown.

50 V8 N-API lifetime refactor: subtask plan

		Remarks
Status	[done]	Implemented and verified.
Severity	Medium	Refactor should be staged carefully because it changes public N-API handle semantics throughout the V8 provider.

50.1 001 Baseline and Parity Harness

Owner: tests and diagnostics only.

Status: done.

Dependencies: none.

Write scope:

- napi/tests/js-native-api/
- napi/v8/tests/ if present or newly added test glue
- optional diagnostic-only files under napi/v8/src/internal/

Tasks:

- Run the current V8 N-API suite and record baseline failures.
- Add focused tests for handle-scope close, escapable scope escape-once, callback argument lifetime, weak reference collection, wrap finalizer timing, napi_remove_wrap(...), external type tags, and cleanup-hook ordering.
- Add a V8 diagnostic tracker or counters behind a compile-time/runtime flag.

Verification:

- Current V8 tests still pass before the refactor starts.
- New tests should initially expose the known semantic gaps where appropriate.

50.2 002 Direct napi_value Representation

Owner: core V8 handle conversion.

Status: done.

Dependencies: 001.

Write scope:

- napi/v8/src/internal/napi_v8_env.h
- napi/v8/src/js_native_api_v8.cc
- call sites in napi/v8/src/unofficial_napi*.cc that assume napi_value__ heap ownership

Tasks:

- Replace struct napi_value__ heap/global wrapper with Node-style conversion helpers:
 - JsValueFromV8LocalValue(...)
 - V8LocalValueFromJsValue(...)
- Make napi_v8_wrap_value(...) return a direct local-handle cast, not a new heap allocation.
- Make napi_v8_unwrap_value(...) reconstruct a v8::Local<v8::Value>.
- Remove manual delete paths for local napi_value objects.

- Audit every place that stores `napi_value` beyond the active call. Convert those locations to `v8::Global`, `napi_ref`, or a dedicated record.

Verification:

- Build succeeds with no use of `new napi_value__ / delete napi_value`.
- Existing V8 N-API tests pass at least through primitive/object creation, function calls, property access, and script execution.

50.3 003 Real Handle Scopes

Owner: V8 scope semantics.

Status: done.

Dependencies: 002.

Write scope:

- `napi/v8/src/js_native_api_v8.cc`
- `napi/v8/src/internal/napi_v8_env.h`

Tasks:

- Add Node-style `HandleScopeWrapper` and `EscapableHandleScopeWrapper`.
- Track `env->open_handle_scopes`.
- Implement `napi_open_handle_scope(...)` / `napi_close_handle_scope(...)` using real `v8::HandleScope`.
- Implement `napi_open_escapable_handle_scope(...)`, `napi_close_escapable_handle_scope(...)`, and `napi_escape_handle(...)` using real `v8::EscapableHandleScope::Escape(...)`.
- Return `napi_handle_scope_mismatch` when close order/count is wrong.

Verification:

- Scope and escape tests match Node behavior.
- Callback return values survive because they are either escaped or still valid in the caller's active V8 scope.

50.4 004 Callback and Accessor Trampolines

Owner: JS-to-native callback entry.

Status: done.

Dependencies: 002 and 003.

Write scope:

- `napi/v8/src/js_native_api_v8.cc`

Tasks:

- Replace heap-owned `napi_callback_info__::args` with a Node-style callback wrapper that reads directly from `v8::FunctionCallbackInfo`.
- Return `this`, `args`, and `new_target` through direct local casts.
- Remove `CallbackInfoOwnsValue(...)` and callback-local delete logic.
- Ensure native callback exceptions flow through the `env` last-exception logic.

Verification:

- Constructor, function, getter, setter, and thrown-exception tests pass.
- No callback path allocates persistent handles for temporary args.

50.5 005 Node-Shaped References and Finalizers

Owner: persistent references, wraps, finalizers.

Status: done.

Dependencies: 002.

Write scope:

- `new napi/v8/src/internal/napi_ref.*` or equivalent
- `napi/v8/src/internal/napi_v8_env.h`
- `napi/v8/src/js_native_api_v8.cc`

Tasks:

- Port/adapt Node's `RefTracker`, `Reference`, `ReferenceWithData`, and `ReferenceWithFinalizer` model.
- Link references/finalizers into env-owned lists rather than detached raw pointers.
- Make weak callbacks reset the `v8::Global` and dispatch finalizers through the env finalizer path.
- Remove the current `napi_delete_reference(...)` workaround that leaves weak ref bookkeeping alive.
- Preserve Edge-specific env cleanup and destroy callbacks.

Verification:

- Weak-ref GC tests pass without use-after-free and without intentionally leaked ref records.
- Env teardown finalizes all runtime-owned references exactly once.

50.6 006 Wraps, Externals, Type Tags, and Buffers

Owner: object-attached native state.

Status: done.

Dependencies: 005.

Write scope:

- `napi/v8/src/js_native_api_v8.cc`
- possible new internal files for external/wrap helpers

Tasks:

- Rework `napi_wrap(...)` to store a Node-shaped tracked reference in the wrapper private key.
- Make `napi_unwrap(...)` and `napi_remove_wrap(...)` operate on that reference with keep/remove modes.
- Require a finalizer when returning a wrap reference, matching Node's ownership contract.
- Add/adapt `ExternalWrapper` so external values can carry native data and type tags without env-side identity vectors.
- Store object type tags in V8 private data, not `env->type_tag_entries`.
- Audit buffer records and external backing stores so finalizers are reachable from engine-owned objects and env teardown.

Verification:

- `napi_wrap`, `napi_unwrap`, `napi_remove_wrap`, `napi_add_finalizer`, `napi_create_external`, and type-tag tests match Node behavior.

50.7 007 Cleanup and Unofficial N-API Audit

Owner: integration cleanup.

Status: audit done; implementation-sensitive findings are captured in `004_unofficial_napi_audit.md`.

Dependencies: 002-006.

Write scope:

- napi/v8/src/unofficial_napi*.cc
- napi/v8/src/internal/
- env teardown in napi/v8/src/js_native_api_v8.cc

Tasks:

- Audit all unofficial N-API paths for stale assumptions about heap napi_value__ wrappers.
- Keep persistent v8::Global storage only for real long-lived records: contexts, modules, promises, source-map callbacks, serializer/deserializer state, and platform state.
- Make env teardown deterministic: cleanup hooks, pending finalizers, refs, buffers, async/threadsafe stubs, instance data, destroy callbacks.
- Add tracker output comparable to the QuickJS lifetime tracker for V8 references, finalizers, wrappers, buffers, and open scopes.

Verification:

- Full V8 N-API test suite.
- Edge V8 CLI smoke tests.
- Focused app smoke tests that previously exercised wraps, streams, buffers, and contextify/module-wrap paths.

51 V8 N-API lifetime refactor: risk and rollout

		Remarks
Status	[done]	Rollout completed for the N-API shared suite scope.
Severity	Medium	The highest risk is breaking existing Edge V8 behavior while making handles more Node-like.

51.1 Rollout Strategy

1. Land tests/diagnostics first.
2. Convert napi_value and handle scopes together, but keep the patch small enough to review by direct comparison with Node's js_native_api_v8.*.
3. Convert callback trampolines next, because the direct-handle model changes how callback args and return values must be handled.
4. Convert references/finalizers/wraps after direct locals are stable.
5. Convert externals/type tags/buffers after the tracked-reference model exists.
6. Audit unofficial N-API last, because it has many legitimate long-lived v8::Global records that should not be removed blindly.

51.2 Key Risks

- Existing Edge code may rely on the current accidental persistence of napi_value__. The test suite should expose those sites before the refactor is merged.
- V8 weak callbacks and finalizers have strict reset/deletion rules. Follow Node's Reference and RefTracker design closely rather than inventing a third ownership model.
- napi_wrap(...) behavior should follow Node's contract: if a wrap reference is returned, a finalizer is required and that reference is normally deleted in response to the finalizer.
- unofficial_napi_contextify.cc and module-wrap code intentionally keep contexts/modules in v8::Global; those records are long-lived by design and should survive the local-handle cleanup.

51.3 Design Constraints

- Direct napi_value handles are current-scope locals. This is intentional design, not a defect to fix.
- Public napi_value handles must not be made persistent. Persistence belongs to napi_ref or internal v8::Global records only where the API contract explicitly owns longer lifetime.

51.4 Verification Checklist

- make build-napi-v8 or the repo's current V8 build target.
- V8 N-API test suite.
- Focused tests for:
 - handle scope close and mismatch;
 - escapable handle escape-once;
 - callback args, return values, constructor new_target;
 - weak ref collection and napi_get_reference_value(...) after GC;
 - wrap finalizer and napi_remove_wrap(...);
 - external value data and type tags;
 - cleanup hook and instance-data finalizer order.
- Edge V8 runtime smoke tests for:

- `-e "console.log('hello from v8')"`
- simple HTTP server;
- contextify/script execution;
- module-wrap/dynamic import path if enabled.

51.5 Verification Run

Completed:

```
make -C napi test-napi TEST_JOBS=4
make -C napi test-napi-quickjs TEST_JOBS=4
make test-native-quickjs
make test-native-v8
make -C napi test-native-v8 EXTRA_CMAKE_ARGS=-DNAPI_V8_ENABLE_LIFETIME_TRACKER=ON
NAPI_V8_DIST_ROOT=/Users/sadhbh/src/dev/edgejs/napi/target/debug/build/wasmer-napi-dc5f75328e8ed
NAPI_V8_DIST_ROOT=/Users/sadhbh/src/dev/edgejs/napi/target/debug/build/wasmer-napi-dc5f75328e8ed
EDGE_TRACE_NAPI_LIFETIME=1 ./build-edge/edge -e "console.log('edge v8 napi ok')"
EDGE_TRACE_NAPI_LIFETIME=1 /Users/sadhbh/src/dev/edgejs/build-napi-v8/v8/tests/napi_v8_test_16_re
git diff --check
```

Results:

- V8 shared N-API suite: 48/48 passing.
- QuickJS shared N-API suite: 48/48 passing.
- V8 lifetime tracker enabled build: 48/48 passing.
- V8 full lifetime diagnostics build (tracker, periodic stats, tag stats, and string/symbol dump): 48/48 passing.
- Edge V8 CLI build with full lifetime diagnostics: passing.
- Edge V8 CLI smoke with lifetime trace: printed `edge v8 napi ok` and emitted QuickJS-shaped `[napi-lifetime-*`] teardown tables.
- V8 lifetime trace smoke: balanced `napi_ref` created/released counts and zero live refs at env teardown for `Test16Reference.PortedCoreFlow`; output uses the same `[napi-lifetime-*`] marker names as QuickJS.
- Whitespace check: passing.

The Edge V8 runtime smoke tests remain a useful follow-up if this refactor is merged into a broader Edge runtime validation pass.

52 V8 N-API lifetime refactor: unofficial N-API audit

		Remarks
Status	[done]	Stage 007 audit complete; 2026-05-16 escaped-local follow-up implemented. Several unofficial N-API result paths rely on the current heap-owned napi_value__ wrapper instead of the intended current-scope local-handle design.
Severity	High	

52.1 Scope

Audited source areas:

- napi/v8/src/unofficial_napi.cc
- napi/v8/src/unofficial_napi_contextify.cc
- napi/v8/src/unofficial_napi_error_utils.cc
- napi/v8/src/unofficial_napi_error_utils.h
- napi/v8/src/internal/napi_v8_env.h
- napi/v8/src/internal/unofficial_napi_bridge.h
- napi/v8/src/internal/node_v8_default_flags.h

This pass looked for stale assumptions that napi_value is a heap object backed by v8::Global, and for any attempt to persist public napi_value handles outside the current N-API/V8 handle scope.

52.2 Summary

The unofficial N-API files mostly avoid storing raw napi_value in durable records. Contextify and module-wrap state correctly use napi_ref or v8::Global for durable values. The largest Stage 007 risk is result-producing helpers that create a nested v8::HandleScope, call napi_v8_wrap_value(...), and return the napi_value after that nested scope closes. That works today because napi_value__ promotes every local to a v8::Global; it will not work after napi_value becomes a direct local-handle cast.

The internal header still exposes the pre-refactor lifetime model directly: napi_value__ owns v8::Global<v8::Value> and napi_callback_info__ stores napi_value fields and vectors (napi/v8/src/internal/napi_v8_env.h:15, napi/v8/src/internal/napi_v8_env.h:25). Stage 007 should treat any remaining source that depends on public napi_value persistence as a blocker after Stages 002-004, because current-scope local handles are the intended design.

52.3 2026-05-16 V8 Node-Compat Follow-Up

The v8-macos CI log in test-failures-v8.log exposed the predicted escaped-local failure mode after the direct-local napi_value refactor. Several Node compatibility tests crashed or misbehaved with V8 heap dumps showing <NativeContext[302]> in places where contextify-returned values were expected, including cross-realm ArrayBuffer, String, and Uint8Array cases exercised by Buffer, querystring, crypto subtle, and VM/domain tests.

The fix updated result-producing paths in:

- napi/v8/src/unofficial_napi_contextify.cc
- napi/v8/src/unofficial_napi_error_utils.cc

Single-result paths now use `v8::EscapableHandleScope` and `Escape(...)` before returning through `napi_v8_wrap_value(...)`. Error helpers with multiple `napi_value*` outputs no longer open a nested `v8::HandleScope`, so their returned locals are allocated in the caller/current N-API scope instead of a closing temporary scope. Public `napi_value` handles remain direct current-scope locals; persistence still belongs to `napi_ref` or explicit `v8::Global` records.

The same follow-up also fixed contextify exception propagation: `V8 TryCatch::ReThrow()` alone did not set the N-API pending-exception state that the Edge runtime expects when `unofficial_napi_contextify_run_script(...)` returns. Contextify now wraps the caught V8 exception as a current-scope `napi_value` and throws it through N-API before returning `napi_pending_exception`. This restored top-level `-e` failures and getter exceptions that flow through `vm.runInThisContext(...)`.

Verification run for the patch:

```
git -C napi diff --check -- v8/src/unofficial_napi_contextify.cc v8/src/unofficial_napi_error_ut.  
cmake --build build-edge --target edge -j4  
build-edge/edge -e "throw new Error('xyz')"  
build-edge/edge test/parallel/test-os-userinfo-handles-getter-errors.js
```

The direct throw command is expected to exit nonzero and print the thrown stack.

52.4 Findings

Risk	Area	Evidence	Recommendation
High	Internal value and callback representation	napi_value__ is still an owning v8::Global wrapper (napi/v8/src/internal/napi_value.h:15) and callback info still stores this_arg, new_target, and std::vector<napi_value> (napi/v8/src/internal/napi_value.h:25).	Stage 007 should verify Stages 002 and 004 have removed these fields before merging. Public napi_value handles should remain current-scope locals; callback code should read from v8::FunctionCallbackInfo and only escape a return value into the caller/current parent scope when required by V8 scope nesting.
High	Result values returned from nested handle scopes	Error helpers open v8::HandleScope and return wrapped locals at napi/v8/src/unofficial_napi_value_utils.cc:405, napi/v8/src/unofficial_napi_value_utils.cc:426, napi/v8/src/unofficial_napi_value_utils.cc:471, napi/v8/src/unofficial_napi_value_utils.cc:489, napi/v8/src/unofficial_napi_value_utils.cc:512, and napi/v8/src/unofficial_napi_error_utils.cc:528. Similar patterns exist in call-site helpers (napi/v8/src/unofficial_napi.cc:2463, napi/v8/src/unofficial_napi.cc:2514) and private symbol creation (napi/v8/src/unofficial_napi.cc:2612, napi/v8/src/unofficial_napi.cc:2637).	Convert result-producing nested scopes to v8::EscapableHandleScope and escape the V8 locals before converting to napi_value, or otherwise ensure the nested scopes are only active on the caller's active N-API/V8 handle scope.

Risk	Area	Evidence	Recommendation
High	Contextify compile/run result APIs	unofficial_napi_contextify, compile function, CJS loader compile, module request output, module evaluation, namespace, error, cached-data, and facade paths all open v8::HandleScope and return wrapped locals (napi/v8/src/unofficial_napi_contextify.cc:1598, napi/v8/src/unofficial_napi_contextify.cc:1711, napi/v8/src/unofficial_napi_contextify.cc:1768, napi/v8/src/unofficial_napi_contextify.cc:1912, napi/v8/src/unofficial_napi_contextify.cc:1927, napi/v8/src/unofficial_napi_contextify.cc:1994, napi/v8/src/unofficial_napi_contextify.cc:2320, napi/v8/src/unofficial_napi_contextify.cc:2353, napi/v8/src/unofficial_napi_contextify.cc:2420, napi/v8/src/unofficial_napi_contextify.cc:2447, napi/v8/src/unofficial_napi_contextify.cc:2524, napi/v8/src/unofficial_napi_contextify.cc:2532, napi/v8/src/unofficial_napi_contextify.cc:2552, napi/v8/src/unofficial_napi_contextify.cc:2553, napi/v8/src/unofficial_napi_contextify.cc:2667, napi/v8/src/unofficial_napi_contextify.cc:2679, napi/v8/src/unofficial_napi_contextify.cc:2746, napi/v8/src/unofficial_napi_contextify.cc:2780).	Audit every in_context(...), napi_value* result_out path after direct handles land. Use escaped locals for returned values and keep purely internal temporaries inside ordinary HandleScope. Add focused tests to verify that contextify doesn't return values, complete cache buffers, module namespace, and retrieval and napi_value and napi_value dec
Medium	V8 callback entry points that synthesize N-API values	Dynamic import and import-meta callbacks create temporary napi_value arrays from V8 locals inside V8 host callbacks (napi/v8/src/unofficial_napi_contextify.cc:1194, napi/v8/src/unofficial_napi_contextify.cc:1215, napi/v8/src/unofficial_napi_contextify.cc:1272).	Keep these values temporary, but ensure the V8 callback has an explicit local scope compatible with the direct-handle model and that any results returned to V8 are escaped as v8::Local (napi/v8/src/unofficial_napi_contextify.cc:1272).
Medium	Serializer/deserializer native contexts	SerializerContext and DeserializerContext keep weak v8::Global<v8::Object> wrapper records and raw napi_env pointers (napi/v8/src/unofficial_napi_contextify.cc:1130, napi/v8/src/unofficial_napi.cc:1136, napi/v8/src/unofficial_napi.cc:1344).	The wrapper globals are intentional. Stage 007 should add env-tracked cleanup or diagnostics for these records so env teardown cannot leave raw napi_env users only governed by 915 weak callbacks

Risk	Area	Evidence	Recommendation
Medium	Teardown coverage for global registries	Env destroy resets promise callbacks/hooks, prepare-stack callbacks, error formatting state, profiler state, fatal/near-heap callbacks, and platform targets (napi/v8/src/unofficial_napi_refs.cc:1740, napi/v8/src/unofficial_napi_context.cc:1737, napi/v8/src/unofficial_napi_contexts.cc:1732, napi/v8/src/unofficial_napi_prepare_trace_hooks.cc:1739, napi/v8/src/unofficial_napi.cc:1744). Context and module-wrap cleanup hooks reset their records (napi/v8/src/unofficial_napi_contextify.cc:775, napi/v8/src/unofficial_napi_contextify.cc:802).	Keep this cleanup ordering, but add tracker output for all surviving global maps and weak wrapper contexts so Stage 007 can prove teardown reaches zero

52.5 Intentional long-lived globals

These `v8::Global` records should remain long-lived, or be converted only into equivalent tracked long-lived records:

- Env-owned context and private keys: `context_ref`, `last_exception`, `last_exception_message`, `wrap/buffer` private keys in `napi/v8/src/internal/napi_v8_env.h:57` and `napi/v8/src/internal/napi_v8_env.h:46`. Stage 006 may move these into object/private state, but Stage 007 should not remove them without that replacement.
- Source-map and preserved error formatting state in `napi/v8/src/unofficial_napi_error_utils.cc:35`, reset through `napi/v8/src/unofficial_napi_error_utils.cc:447`.
- Contextify records: context globals plus persistent key refs in `napi/v8/src/unofficial_napi_contextify.cc:42`, stored through `napi/v8/src/unofficial_napi_contextify.cc:1297`.
- Module-wrap records: `wrapper/callback/source/host-option` `napi_refs` and `context/module` globals in `napi/v8/src/unofficial_napi_contextify.cc:68`, destroyed through `napi/v8/src/unofficial_napi_contextify.cc:1297`.
- Promise reject callbacks and promise hooks in `napi/v8/src/unofficial_napi.cc:87`, set at `napi/v8/src/unofficial_napi.cc:2224` and `napi/v8/src/unofficial_napi.cc:2240`.
- Prepare-stack-trace callbacks in `napi/v8/src/unofficial_napi.cc:72`, reset by `napi/v8/src/unofficial_napi.cc:1744`.
- Serializer/deserializer wrapper globals in `napi/v8/src/unofficial_napi.cc:1132` and `napi/v8/src/unofficial_napi.cc:1133`.
- The temporary `UnofficialEnvScope` context global in `napi/v8/src/unofficial_napi.cc:50`, which keeps the bootstrap context alive until the env is created and then reset during scope release.

52.6 Stage 007 recommended changes

1. After Stages 002-004, run `rg -n "napi_value__|std::vector<napi_value>|delete .*napi_value|CallbackInfo::napi_value__" napi/v8/src` and treat any remaining local wrapper ownership as a blocker.
2. Convert all unofficial APIs that both open a nested `v8::HandleScope` and set a `napi_value*` output to escape the returned V8 local or allocate it in the caller's active scope.
3. Keep `module/context/promise/source-map/serializer` globals, but register them in a V8 lifetime tracker so teardown diagnostics can distinguish intentional long-lived records from leaks.
4. Add focused smoke tests for the result-returning unofficial APIs after direct handles land: `contextify` `run/compile`, `CJS loader compile`, `module namespace/error/cached-data/facade`, `error source-line/throw-at helpers`, `call-site helpers`, `private symbol creation`, `structured clone`, `serialize/deserialize`, and `serdes binding construction`.
5. Verify `unofficial_napi_destroy_env_instance(...)` still clears every global map before isolate disposal, then compare tracker counts before and after `unofficial_napi_release_env(...)`.

53 Troubleshooting Registry

54 Edge QuickJS Troubleshooting Registry

		Remarks
Status	[open]	Canonical registry for QuickJS WASIX troubleshooting issue pages.
Severity	Low	Documentation registry only; individual issue pages carry runtime severity.

Each problem should have one issue page. Keep diagnosis, status, severity, and known limitations on that page. This registry should stay compact and avoid duplicating issue details.

54.1 Status Icons

- [open]: open or active.
- [done]: resolved or stable.
- [caveated]: accepted with caveats, partial compatibility, or retained limitation.
- [blocked]: unresolved blocker.

54.2 Node Compatibility

Status	Severity	Issue	Topic
[caveated]	Medium	001_buffer.md	Buffer
[caveated]	Medium	002_console.md	Console bootstrap binding
[done]	High	003_contextify.md	Contextify diagnostics
[caveated]	High	004_environment.md	Environment lifecycle
[caveated]	Medium	005_global_shims.md	Global shims
[done]	High	006_microtasks.md	Promise hooks and microtasks
[caveated]	High	007_module_loading.md	Module loading
[done]	Medium	008_properties.md	Property setting semantics
[done]	Low	009_quickjs_utilities.md	QuickJS utility ownership
[done]	Medium	010_serdes.md	Serialization and deserialization
[open]	Medium	011_quickjs_wasix_atomics.md	QuickJS WASIX atomics guard patch
[caveated]	Medium	013_v8_shaped_callsite_methods.md	V8 shaped CallSite methods
[caveated]	Medium	014_lifetime_tracing.md	QuickJS N-API lifetime tracing
[caveated]	Medium	003_minimal_intl_fallback.md	Minimal Intl.DateTimeFormat fallback
[caveated]	Medium	004_native_inspector_stub.md	Native unavailable inspector stub

Status	Severity	Issue	Topic
[caveated]	High	010_stream_wrapper_unwrap_fallback	Stream wrapper unwrap fallback
[caveated]	Medium	012_v8_cctest_environment_attachment	V8 CTest runtime fixture environment attachment
[caveated]	Medium	016_napi_extn_wasix_linkage_rule	NAPI_EXTN= WASIX linkage rule
[caveated]	Low	017_framework_static_server_adapters	Framework static and ad hoc server adapters
[caveated]	Medium	018_pnpm_deploy_graph_materialization	pnpm deploy graph materialization

54.3 Node Test

Status	Severity	Issue	Topic
[open]	Medium	001_buffer_limits_and_deprecation_parity	Buffer limits and deprecation parity
[open]	Medium	002_console_inspect_and_stack_formatting	Console inspect and stack formatting
[open]	High	003_node_test_public_api_exports	node test public API exports
[open]	Medium	004_diagnostics_channel_module_loader_events	Diagnostics channel module loader events
[open]	Medium	005_diagnostics_channel_async_context	Diagnostics channel async context
[done]	Low	006_eventemitter_asyncresource_private_field_errors	EventEmitter AsyncResource private-field errors
[open]	High	007_fetch_response_body_and_http_proxy_env	Fetch Response body and HTTP proxy env
[caveated]	High	008_https_proxy_tunnel_errors	HTTPS proxy tunnel errors
[open]	Medium	009_http_timers_and_header_limits	HTTP timers and header limits
[open]	Low	010_os_constants_and_userinfo_errors	OS constants and userInfo errors
[open]	Medium	011_fastutf8stream_sync_wait	FastUtf8Stream synchronous wait
[open]	High	012_explicit_resource_management_syntax	Explicit resource management syntax
[open]	Medium	013_stream_missing_builtins_and_async_iterators	Stream missing builtins and async iterators
[done]	Medium	014_string_decoder_utf8_boundaries	StringDecoder UTF-8 boundaries
[open]	Medium	015_url_and_data_url_validation	URL and data URL validation
[done]	Medium	016_whatwg_url_inspect_and_search_params	WHATWG URL inspect and search params
[open]	High	017_http2_native_lifecycle_crashes	HTTP/2 native lifecycle crashes
[done]	High	018_tls_securecontext_sni_and_setkeycert_lifetime	TLS SecureContext, SNI, and setKeyCert lifetime

54.4 Astro SSR

Status	Severity	Issue	Topic
[open]	High	001_es_module_lexer_webassembly	es module lexer WebAssembly import
[done]	High	002_depd_callsite_methods.depd	CallSite method compatibility
[caveated]	High	003_cjs_reexport_named_exports	CommonJS re-export named exports
[caveated]	Medium	004_missing_intl.md	Missing Intl global
[caveated]	Low	005_listen_eperm.md	Listen EPERM on localhost
[done]	High	006_floating_ui_utils_dom.md	Floating UI utils DOM subpath
[done]	High	007_react_remove_scroll_bar	React Remove Scroll Bar constants subpath
[done]	High	008_zustand_ind_create_exports	Zustand ind create export
[done]	High	009_zustand_esm_default_exports	Zustand ESM default export
[done]	High	010_use_gesture_controller	Use Gesture Controller export
[done]	High	011_route_stack_overflow.md	Route stack overflow
[done]	High	012_wasix_pnpm_symlink_resolution	WASI pnpm symlink resolution
[done]	High	013_lucide_react_chevron-down	Lucide React ChevronDown export
[done]	Medium	014_pnpm_deploy_externalized_runtime_links	pnpm deploy externalized runtime links

54.5 Vite App

Status	Severity	Issue	Topic
[caveated]	Low	001_standalone_build.md	Standalone build findings

54.6 Next App

Status	Severity	Issue	Topic
[done]	High	001_standalone_v8_serdes.md	require("v8") / serdes findings
[done]	High	002_standalone_inspector_stub	require("inspector") stub
[open]	High	003_route_stack_exhausted	Route request stack exhaustion
[done]	High	004_next_config_swc_options	SWC options Buffer rejected by N-API
[done]	High	005_entry_css_work_store_async	css work store store async context

54.7 Wasmer Deploy

Status	Severity	Issue	Topic
[caveated]	High	001_pnpm_directory_symlinks	pnpm directory symlinks in WEBC package
[done]	High	002_quickjs_wasix_napi_import_module_mismatch	QuickJS WASIX NAPI import module mismatch
[caveated]	High	003_ci_safe_mode_missing_quickjs_artifact	CI safe-mode missing QuickJS artifact
[done]	High	004_wasix_safe_mode_https_exits_before_callbacks	WASIX safe-mode HTTPS exits before callbacks

55 Node Test Compatibility Current State

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/development/troubleshooting/node-test/README.md

56 Node Test: QuickJS Compatibility Failures

		Remarks
Status	[open]	Registry for native QuickJS Node compatibility test issue pages. Node compatibility failures remain significant until the linked issue pages close.
Severity	High	

The issue pages are canonical. Keep failure details, diagnosis, and known limitations on the individual page for each problem.

56.1 Source

/Users/sadhbh/src/dev/edgejs/test-only.log

56.2 Issues

Status	Severity	Issue	Topic
[open]	Medium	001_buffer_limits_and_deprecations.md	Buffer limits and deprecation parity
[open]	Medium	002_console_inspect_and_stack_formatting.md	Console inspect and stack formatting
[open]	High	003_node_test_public_api_exports.md	node:test public API exports
[open]	Medium	004_diagnostics_channel_module_loader.md	Diagnostics channel module loader events
[open]	Medium	005_diagnostics_channel_async_context.md	Diagnostics channel async context
[done]	Low	006_eventemitter_async_resource_private_fields.md	EventEmitterAsyncResource private-field errors
[open]	High	007_fetch_response_body_and_proxy_env.md	Fetch Response body and HTTP proxy env
[caveated]	High	008_https_proxy_tunnel_errors.md	HTTPS proxy tunnel errors
[open]	Medium	009_http_timers_and_header_limits.md	HTTP timers and header limits
[open]	Low	010_os_constants_and_userinfo.md	OS constants and userinfo errors
[open]	Medium	011_fastutf8stream_sync_wait.md	FastUtf8Stream synchronous wait
[open]	High	012_explicit_resource_management_syntax.md	Explicit resource management syntax
[open]	Medium	013_stream_missing_builtins_and_async_iterators.md	Stream missing builtins and async iterators

Status	Severity	Issue	Topic
[done]	Medium	014_string_decoder_utf8_boundaries.md	StringDecoder UTF-8 boundaries
[open]	Medium	015_url_and_data_url_validation.md	URL and data URL validation
[done]	Medium	016_whatwg_url_inspect_and_searchparams.md	WHATWG URL inspect and search params
[open]	High	017_http2_native_lifecycle_crashes.md	HTTP/2 native lifecycle crashes
[done]	High	018_tls_securecontext_sni_lifetime.md	TLS SecureContext, SNI, and setKeyCert lifetime

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/development/troubleshooting/node-test/001_buffer_limits_and_deprecations.md

57 Node Test: Buffer limits and deprecation parity

		Remarks
Status	[open]	Buffer length fixes landed; Buffer() deprecation filtering remains open.
Severity	Medium	Blocks multiple Buffer compatibility tests but does not explain broader runtime failures.

Source log:

/Users/sadhbh/src/dev/edgejs/test-only.log

Affected tests:

- parallel/test-buffer-alloc
- parallel/test-buffer-constants
- parallel/test-buffer-constructor-node-modules
- parallel/test-buffer-constructor-node-modules-paths
- parallel/test-buffer-tostring-4gb

57.1 What Is The Issue

The Buffer failures split into two related areas:

- Oversized allocations expose QuickJS native typed-array errors such as RangeError: invalid array index or RangeError: invalid array buffer length instead of Node's Invalid typed array length: 9007199254740992 / Invalid string length behavior.
- Buffer() deprecation warning suppression for code under node_modules differs from Node. The current run emits [DEP0005] in paths where the test expects no stderr.

test-buffer-tostring-4gb also reaches raw typed-array construction before the test can exercise Node's large-buffer string conversion behavior.

57.2 2026-05-15 Update

The direct typed-array constructor message from parallel/test-buffer-alloc is fixed in the vendored QuickJS source by giving typed-array length conversion a Node-compatible RangeError: Invalid typed array length: <value> path before QuickJS falls back to its generic invalid array index text.

Targeted verification:

```
cmake --build build-edge-quickjs-cli --target edge -j4
build-edge-quickjs-cli/edge -e "const { kMaxLength } = require('buffer'); try { new Uint8Array(k
```

Observed output:

RangeError: Invalid typed array length: 9007199254740992

The full parallel/test-buffer-alloc file then got past that assertion but still failed later on unmatched-surrogate UTF-8 replacement bytes at line 857 (237 !== 239). That was treated as a separate Buffer/string encoding issue.

57.3 2026-05-15 QuickJS Limit And UTF-8 Update

The vendored QuickJS source now replaces unmatched UTF-16 surrogate code units with U+FFFD when exporting normal UTF-8 through JS_ToCStringLen2. This fixes the `Buffer.from('ab\\ud800cd', 'utf8')` byte sequence in `parallel/test-buffer-alloc`.

QuickJS string and allocation limit behavior was also normalized:

- QuickJS `RangeError` text for string overflow is now `Invalid string length`.
- `internalBinding('buffer').kStringMaxLength` now reflects QuickJS's actual string ceiling, `0x3fffffff`, through a QuickJS-only runtime compile definition. The same shared Buffer binding still reports V8's native limit for the V8 backend.
- `ArrayBuffer` construction beyond QuickJS's hard `INT32_MAX` backing-store cap now reports `Array buffer allocation failed`, letting 4GB allocation tests skip through their accepted OOM path.

Targeted verification passed:

```
build-edge-quickjs-cli/edge test/parallel/test-buffer-alloc.js
build-edge-quickjs-cli/edge test/parallel/test-buffer-constants.js
build-edge-quickjs-cli/edge test/parallel/test-buffer-tostring-4gb.js
build-edge-quickjs-cli/edge test/sequential/test-buffer-creation-regression.js
```

The 4GB tests intentionally skipped with accepted allocation-failure messages.

57.4 How Should We Fix It

Keep the direct typed-array constructor, UTF-8 surrogate replacement, string limit, and `ArrayBuffer` allocation-cap fixes in vendored QuickJS/native binding code. Do not move these into `lib/`.

Remaining work is deprecation filtering. Inspect the `Buffer()` warning gate in `lib/buffer.js` and the native `internalBinding('util').isInsideNodeModules()` implementation. Reproduce the two failing tests with `--trace-deprecation` and compare the structured stack frames with the V8 path. Fix the `node-modules` classification so synthetic and real fixture paths are treated the same way Node treats them.

Targeted verification:

```
build-edge-quickjs-cli/edge test/parallel/test-buffer-alloc.js
build-edge-quickjs-cli/edge test/parallel/test-buffer-constants.js
build-edge-quickjs-cli/edge test/parallel/test-buffer-constructor-node-modules.js
build-edge-quickjs-cli/edge test/parallel/test-buffer-constructor-node-modules-paths.js
build-edge-quickjs-cli/edge test/parallel/test-buffer-tostring-4gb.js
```

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/development/troubleshooting/node-test/002_console_inspect_and_stack_formatting.md

58 Node Test: console inspect and stack formatting

		Remarks
Status	[open]	Console table fixed; revoked proxy and pseudo-TTY stack formatting remain open.
Severity	Medium	Breaks console output compatibility and pseudo-TTY formatting checks.

Affected tests:

- parallel/test-console-issue-43095
- parallel/test-console-table
- pseudo-tty/console_colors

58.1 What Is The Issue

`console.dir()` / `util.inspect()` does not tolerate revoked proxies the way Node does. The log shows `TypeError: revoked proxy escaping from getOwnPropertyDescriptor()` during inspection.

`console.table()` rendered the table header/footer for a Map iterator but omitted the expected rows. The JS table code calls `internalBinding('util').previewEntries()` for Map/Set objects and iterators; the QuickJS N-API implementation returned an empty array for every input, so the table formatter had no rows to render.

The pseudo-TTY color test output has the right ANSI color markers in early lines, but stack frames are QuickJS bootstrap frames like `<input>:1806:27`. The expected output wants Node-style file/resource names and internal frame coloring.

58.2 How Should We Fix It

Keep fixes native-only; lib/ files are off-limits.

- Wrap revoked-proxy descriptor/property probes in the same defensive paths Node uses in `internal/util/inspect`, returning a stable placeholder instead of throwing, but implement the compatibility hook on the native side.
- Implement QuickJS-backed `unofficial_napi_preview_entries()` so Map/Set objects and iterators return V8-shaped preview arrays without advancing iterators. This is now implemented by `JS_PreviewEntries()` in vendored QuickJS and fixes `parallel/test-console-table`.
- Reuse the QuickJS source-map/error formatting helpers already present for other stack fixes so `console.trace()` and printed Error stacks retain the original script filename, CommonJS wrapper name, and `node:internal/...` formatting.

58.3 2026-05-15 Update

Implemented a native QuickJS preview helper:

- `quickjs.h` exposes `JS_PreviewEntries(ctx, value, &is_key_value)`.
- `quickjs.c` snapshots Map, Set, Map Iterator, and Set Iterator records directly from QuickJS collection storage without mutating iterator state.
- `quickjs/src/unofficial_napi.cc` now routes `unofficial_napi_preview_entries()` through that helper instead of returning an empty array.

Verified:

```
cmake --build build-edge-quickjs-cli --target edge -j4  
build-edge-quickjs-cli/edge --expose-internal -e "<previewEntries Map/Set smoke>"  
build-edge-quickjs-cli/edge test/parallel/test-console-table.js
```

Targeted verification:

```
build-edge-quickjs-cli/edge test/parallel/test-console-issue-43095.js  
build-edge-quickjs-cli/edge test/parallel/test-console-table.js  
/opt/homebrew/opt/python@3.14/bin/python3.14 test/tools/pseudo-tty.py build-edge-quickjs-cli/edge
```

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/development/troubleshooting/node-test/003_node_test_public_api_exports.md

59 Node Test: node:test public API exports

		Remarks
Status	[open]	Planned investigation. Any ESM test importing the full public node:test API can fail at link time.
Severity	High	

Affected tests:

- parallel/test-dgram-async-dispose
- parallel/test-events-add-abort-listener

59.1 What Is The Issue

Both tests fail during ESM module linking:

SyntaxError: Could not find export 'describe' in module 'node:test'

lib/test.js exports describe: suite for CommonJS consumers, but the QuickJS ESM facade for the builtin does not declare that named export before module linking. QuickJS requires named exports to be declared before evaluation, unlike CommonJS property access after require().

59.2 Current Status

The earlier idea of extending QuickJS C++ CommonJS facade generation is no longer current. The C++ CJS/module-loader hack has been removed. If this Node test issue is addressed, the named-export behavior should come from Node's JavaScript loaders/translators or another proper EdgeJS-owned runtime path, not from new QuickJS C++ source-text heuristics.

At minimum, node:test must expose test, it, describe, suite, hooks, run, mock, snapshot, and assert consistently with lib/test.js if this compatibility gap is reopened.

Targeted verification:

```
build-edge-quickjs-cli/edge --test-reporter=test/common/test-error-reporter.js --test-reporter-d
build-edge-quickjs-cli/edge --test-reporter=test/common/test-error-reporter.js --test-reporter-d
```


Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/development/troubleshooting/node-test/004_diagnostics_channel_module_loader.md

60 Node Test: diagnostics channel module loader events

		Remarks
Status	[open]	Planned investigation. Diagnostics are missing for ESM module loading, which hides loader lifecycle events from observers.
Severity	Medium	

Affected tests:

- parallel/test-diagnostics-channel-module-import
- parallel/test-diagnostics-channel-module-import-error

60.1 What Is The Issue

Both tests subscribe to module import diagnostics and receive an empty event array. The failure cases expect start, end, error, asyncStart, and asyncEnd records, including the requested URL and parent module URL.

The QuickJS ESM loader currently reports module resolution/evaluation errors to the caller, but it does not publish Node's diagnostics-channel lifecycle events around the import.

60.2 How Should We Fix It

Find the QuickJS ESM load/translate/evaluate path and add the same JS-side diagnostic publications that Node's loader emits. The event payload must include at least:

- url
- parentURL
- name
- error for failed imports

Make sure failed resolution publishes both synchronous and async diagnostic events before rejecting the import promise. The fix should live near the module loader bridge, not inside `diagnostics_channel` itself, because normal pub/sub tests already pass in this run.

Targeted verification:

```
build-edge-quickjs-cli/edge test/parallel/test-diagnostics-channel-module-import.js
build-edge-quickjs-cli/edge test/parallel/test-diagnostics-channel-module-import-error.js
```

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/development/troubleshooting/node-test/005_diagnostics_channel_async_context.md

61 Node Test: diagnostics channel async context

		Remarks
Status	[open]	Planned investigation. Tracing channels lose context across promises and a worker-thread diagnostic test hangs.
Severity	Medium	

Affected tests:

- parallel/test-diagnostics-channel-tracing-channel-args-types
- parallel/test-diagnostics-channel-tracing-channel-promise-run-stores
- parallel/test-diagnostics-channel-worker-threads

61.1 What Is The Issue

The tracing-channel argument validation failure is mostly message parity: QuickJS reports `TypeError: not an object`, while the test expects a message matching `Cannot convert undefined or null to object`.

The promise-run-stores test is a behavioral failure. The expected async context store `{ foo: 'bar' }` is undefined after a promise boundary, so the QuickJS promise/job integration is not preserving the async resource context required by diagnostics tracing.

The worker-thread diagnostics test times out, which may be a separate missing worker capability or a hang caused by diagnostics subscriptions crossing worker startup/shutdown.

61.2 2026-05-15 Update

The parallel/test-diagnostics-channel-tracing-channel-args-types message failure is fixed in the vendored QuickJS source. `Object.getPrototypeOf()` and `Reflect.getPrototypeOf()` now throw `TypeError: Cannot convert undefined or null to object` for null/undefined, matching the V8/Node text that `Channel[Symbol.hasInstance]` exposes during diagnostics-channel validation.

Targeted verification:

```
cmake --build build-edge-quickjs-cli --target edge -j4
build-edge-quickjs-cli/edge test/parallel/test-diagnostics-channel-tracing-channel-args-types.js
```

The broader async-context and worker-thread diagnostics failures remain tracked by this note.

61.3 How Should We Fix It

Treat this as two passes:

1. Keep the argument-validation message parity in vendored QuickJS, where `Object.getPrototypeOf()` and `Reflect.getPrototypeOf()` produce their observable `TypeError` text.
2. Trace async context propagation through QuickJS promise hooks, microtask draining, and `AsyncLocalStorage`. The existing promise hook/microtask work should be extended so diagnostics-channel run-stores sees the active store after promise continuations.

For the timeout, rerun the worker test alone with worker/diagnostics traces after the async-context fix. If it still hangs, create a follow-up worker-thread issue rather than mixing it into diagnostics-channel store propagation.

Targeted verification:

```
build-edge-quickjs-cli/edge test/parallel/test-diagnostics-channel-tracing-channel-args-types.js  
build-edge-quickjs-cli/edge test/parallel/test-diagnostics-channel-tracing-channel-promise-run-s  
build-edge-quickjs-cli/edge test/parallel/test-diagnostics-channel-worker-threads.js
```

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/development/troubleshooting/node-test/006_eventemitter_asyncresource_private_fields.md

62 Node Test: EventEmitterAsyncResource private-field errors

		Remarks
Status	[done]	Fixed in the vendored QuickJS private-field TypeError path.
Severity	Low	The behavior is likely correct, but the observable TypeError text differs from Node/V8.

Affected test:

- parallel/test-eventemitter-asyncresource

62.1 What Is The Issue

The test intentionally calls an EventEmitterAsyncResource method with an invalid receiver. QuickJS throws: `TypeError: private class field '#asyncResource' does not exist`

The Node test expects a TypeError whose message matches:

```
/Cannot read private member/
```

This is a cross-engine message compatibility gap, not necessarily a semantic failure.

62.2 2026-05-15 Update

Fixed in napi/quickjs/deps/quickjs/quickjs.c by changing the missing private field error text from QuickJS's `private class field ... does not exist` wording to the Node/V8-compatible `Cannot read private member ... from an object whose class did not declare it`.

Targeted verification:

```
cmake --build build-edge-quickjs-cli --target edge -j4
build-edge-quickjs-cli/edge test/parallel/test-eventemitter-asyncresource.js
```

62.3 How Should We Fix It

Keep this as a vendored QuickJS compatibility patch. The failing Node API reaches the engine private-field brand check directly, so the stable fix is to make the missing-private-field TypeError text match the V8/Node wording at that throw site.

Targeted verification:

```
build-edge-quickjs-cli/edge test/parallel/test-eventemitter-asyncresource.js
```

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/development/troubleshooting/node-test/007_fetch_response_body_and_proxy_env.md

63 Node Test: fetch Response body and HTTP proxy env

		Remarks
Status	[open]	Planned investigation. Breaks global <code>fetch()</code> and HTTP proxy environment tests.
Severity	High	

Affected tests:

- `client-proxy/test-http-proxy-fetch`
- `client-proxy/test-use-env-proxy-cli-http`
- `parallel/test-fetch`

63.1 What Is The Issue

Current `make test-quickjs-only` runs fail earlier in Undici's HTTP parser loader:

```
TypeError: fetch failed
[cause]: ReferenceError: WebAssembly is not defined
```

Undici's lazy `llhttp` path expects `globalThis.WebAssembly`. Until QuickJS Edge provides enough of that API or routes Undici to a non-wasm parser path, `fetch` and `fetch-through-proxy` tests cannot reach the older Response body assertions.

Earlier runs exposed the response construction/body issue below.

The HTTP proxy fetch fixtures fail with:

```
TypeError: cannot read property 'text' of undefined
```

The fixture calls `await fetch(...).then((res) => res.text())`, so the failure means the fulfilled value is undefined or the response body bridge is incorrectly detached. `parallel/test-fetch` also fails an assertion at the early fetch smoke check.

The HTTP proxy environment test shows that plain HTTP proxying reaches the server and can print status/header information, but the child fixture still exits with code 1 because the `fetch()` response path is broken.

63.2 How Should We Fix It

Start with a minimal native QuickJS CLI repro:

```
build-edge-quickjs-cli/edge -e "fetch('http://127.0.0.1:<port>').then(async r => console.log(r &
```

First resolve the missing `WebAssembly` dependency in Undici's lazy HTTP parser path. Then inspect the JS `fetch` implementation and the native HTTP/client bridge:

- ensure the promise resolves to a real `Response` instance;
- ensure `Response.prototype.text()` is installed and bound to the response body;
- verify that proxy environment handling does not replace the fetch result with an internal completion value;
- rerun with `EDGE_TRACE_NET=1` if the body stream is present but not delivered.

Do not special-case the tests. Fix the global `fetch()` response construction so plain fetch, HTTP proxy fetch, and environment proxy fetch share the same path.

Targeted verification:

```
build-edge-quickjs-cli/edge test/parallel/test-fetch.mjs  
build-edge-quickjs-cli/edge test/client-proxy/test-http-proxy-fetch.mjs  
build-edge-quickjs-cli/edge test/client-proxy/test-use-env-proxy-cli-http.mjs
```

64 Node Test: HTTPS proxy tunnel errors

		Remarks
Status	[caveated]	Request tunnel error formatting fixed; fetch proxy remains WebAssembly-blocked. HTTPS proxy request failures need stable Node-compatible error output.
Severity	High	

Affected tests:

- client-proxy/test-https-proxy-fetch
- client-proxy/test-use-env-proxy-cli-https
- client-proxy/test-https-proxy-request-empty-response
- client-proxy/test-https-proxy-request-malformed-response
- client-proxy/test-https-proxy-request-proxy-failure-404
- client-proxy/test-https-proxy-request-proxy-failure-500
- client-proxy/test-https-proxy-request-proxy-failure-502
- client-proxy/test-https-proxy-request-proxy-failure-hang-up

64.1 What Is The Issue

Current fetch-through-HTTPS-proxy failures now stop in Undici before the proxy tunnel logic:

```
TypeError: fetch failed
[cause]: ReferenceError: WebAssembly is not defined
```

That is shared with the HTTP fetch/proxy issue. The request-specific HTTPS proxy tests still exercise the tunnel code and expose the error-shape issues below.

HTTPS fetch through a proxy fails with:

```
ERR_SSL_PACKET_LENGTH_TOO_LONG
```

That usually means TLS is being attempted against a plain proxy socket before a successful CONNECT tunnel is established.

Failure-path proxy request tests do surface `ERR_PROXY_TUNNEL`, but the error message/body is incomplete. Some test code then tries to inspect a null match and throws `TypeError: cannot read property 'length' of null`; others assert that the string should include `Connection to establish proxy tunnel ended unexpectedly`.

64.2 2026-05-15 Native Error Stack Update

The six request-specific tunnel failure tests now pass without changing `lib/`. The root cause was vendored QuickJS eagerly calling `Error.prepareStackTrace` inside the `Error` constructor, before `NodeError` subclasses initialized public class fields such as `code = 'ERR_PROXY_TUNNEL'`. Stack and inspect output therefore showed `Error [undefined]: ...`

QuickJS now captures call-site data at construction but defers `prepareStackTrace(error, frames)` until `error.stack` is first read. The lazy getter then caches the prepared stack as a normal writable/configurable own property. This preserves Node's unchanged `lib/internal/errors.js` behavior and produces stack output such as:

Error [ERR_PROXY_TUNNEL]: Connection to establish proxy tunnel ended unexpectedly

Targeted verification passed:

```
build-edge-quickjs-cli/edge test/client-proxy/test-https-proxy-request-empty-response.mjs
build-edge-quickjs-cli/edge test/client-proxy/test-https-proxy-request-malformed-response.mjs
build-edge-quickjs-cli/edge test/client-proxy/test-https-proxy-request-proxy-failure-404.mjs
build-edge-quickjs-cli/edge test/client-proxy/test-https-proxy-request-proxy-failure-500.mjs
build-edge-quickjs-cli/edge test/client-proxy/test-https-proxy-request-proxy-failure-502.mjs
build-edge-quickjs-cli/edge test/client-proxy/test-https-proxy-request-proxy-failure-hang-up.mjs
```

64.3 Remaining Work

The request-specific tunnel error formatting path is fixed. The remaining proxy entries in this note are fetch/global proxy coverage, which currently stop in Undici because WebAssembly is unavailable in this QuickJS test configuration. When Wasm is back in scope, verify the full HTTPS proxy state flow:

1. open TCP/TLS connection to the proxy as configured;
2. send `CONNECT target-host:target-port HTTP/1.1;`
3. parse proxy response status and headers;
4. only wrap the socket in target TLS after a 2xx tunnel response;
5. construct stable `ERR_PROXY_TUNNEL` errors for empty, malformed, 4xx, 5xx, and hang-up responses.

Remaining verification:

```
build-edge-quickjs-cli/edge test/client-proxy/test-https-proxy-fetch.mjs
build-edge-quickjs-cli/edge test/client-proxy/test-use-env-proxy-cli-https.mjs
```


Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/development/troubleshooting/node-test/009_http_timers_and_header_limits.md

65 Node Test: HTTP timers and header limits

		Remarks
Status	[open]	Planned investigation. Affects HTTP edge-case tests around timers, warnings, and CLI option propagation.
Severity	Medium	

Affected tests:

- parallel/test-http-keep-alive-timeout-race-condition
- client-proxy/test-http-proxy-request-invalid-char-in-url
- parallel/test-http-timeout-client-warning
- parallel/test-set-http-max-http-headers

65.1 What Is The Issue

The log shows a mix of HTTP timing and validation failures:

- test-http-timeout-client-warning emits a TimeoutOverflowWarning, but the warning object does not satisfy the test assertion.
- test-set-http-max-http-headers reports ERR_INVALID_ARG_VALUE through the test runner instead of completing its child-process checks.
- The keep-alive timeout race and invalid-character proxy request tests do not complete normally in the log.

These failures likely share event-loop/timer and option propagation surfaces rather than parser bugs alone.

65.2 How Should We Fix It

Investigate in this order:

1. Reproduce test-http-timeout-client-warning alone and inspect the emitted warning object's name, code, message, and stack. Normalize timer overflow warning construction to Node's shape.
2. Reproduce test-set-http-max-http-headers with child process stdio captured. Verify --max-http-header-size and NODE_OPTIONS propagation through src/edge_cli.cc into the HTTP parser configuration.
3. Reproduce the timeout/hang cases with EDGE_TRACE_NET=1 and timer tracing. Confirm server close, socket timeout, keep-alive cleanup, and proxy request rejection all schedule and drain their callbacks.

Targeted verification:

```
build-edge-quickjs-cli/edge test/parallel/test-http-timeout-client-warning.js
build-edge-quickjs-cli/edge --test-reporter=test/common/test-error-reporter.js --test-reporter-d
build-edge-quickjs-cli/edge test/parallel/test-http-keep-alive-timeout-race-condition.js
build-edge-quickjs-cli/edge test/client-proxy/test-http-proxy-request-invalid-char-in-url.mjs
```

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/development/troubleshooting/node-test/010_os_constants_and_userinfo.md

66 Node Test: OS constants and userInfo errors

		Remarks
Status	[open]	Planned investigation. OS API parity issues are narrow but visible in the Node suite.
Severity	Low	

Affected tests:

- parallel/test-os-constants-signals
- parallel/test-os-userinfo-handles-getter-errors

66.1 What Is The Issue

test-os-constants-signals expects assigning to `os.constants.signals.FOOBAR` in strict mode to throw a plain `TypeError`. The QuickJS run reports `ERR_INVALID_ARG_VALUE`, which suggests the constants object shape or property descriptor behavior differs from Node.

test-os-userinfo-handles-getter-errors expects child-process behavior proving that `os.userInfo()` handles getter errors safely. The log shows the parent assertion `userInfo` crashes, meaning the expected child outcome was not observed.

66.2 How Should We Fix It

For constants, compare descriptors on `os.constants.signals` between V8 Edge and QuickJS Edge. The fix should make the object immutable/frozen in the Node-compatible way, so strict assignment fails with the expected `TypeError` rather than a custom argument error.

For `userInfo()`, run the test directly and inspect the child `stderr/stdout`. Then audit the native `os.userInfo()` binding and JS wrapper for property access that can invoke user-controlled getters during error formatting or option handling. Match Node's behavior by catching and preserving the intended error instead of crashing or converting it into an unrelated assertion path.

Targeted verification:

```
build-edge-quickjs-cli/edge --test-reporter=test/common/test-error-reporter.js --test-reporter-d
build-edge-quickjs-cli/edge test/parallel/test-os-userinfo-handles-getter-errors.js
```

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/development/troubleshooting/node-test/011_fastutf8stream_sync_wait.md

67 Node Test: FastUtf8Stream synchronous wait

		Remarks
Status	[open]	Planned investigation. Breaks synchronous stream flush/retry paths used by stdio-like streams.
Severity	Medium	

Affected tests:

- parallel/test-fastutf8stream-flush-sync
- parallel/test-fastutf8stream-retry

67.1 What Is The Issue

Both failures throw:

`TypeError: cannot block in this thread`

The stack passes through `wait`, `sleep`, and `FastUtf8Stream` private methods such as `#flushSyncUtf8()` and `#release()`. QuickJS is rejecting a blocking wait on the main thread where Node's implementation expects the synchronous stream path to be allowed or implemented differently.

67.2 How Should We Fix It

Inspect the `FastUtf8Stream` implementation and its native wait primitive. Decide whether QuickJS should:

- provide a Node-compatible blocking wait primitive for this exact sync I/O path;
- avoid the blocking path for QuickJS by draining pending writes synchronously through the platform loop; or
- gate the sync wait behind a runtime capability check and use a safe fallback.

Do not globally allow arbitrary `Atomics.wait()` on the main thread unless that matches the existing EdgeJS runtime policy. Keep the unblock narrowly tied to `FastUtf8Stream` sync flush/retry semantics.

Targeted verification:

```
build-edge-quickjs-cli/edge test/parallel/test-fastutf8stream-flush-sync.js
build-edge-quickjs-cli/edge test/parallel/test-fastutf8stream-retry.js
```

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/development/troubleshooting/node-test/012_explicit_resource_management_syntax.md

68 Node Test: explicit resource management syntax

		Remarks
Status	[open]	Planned investigation. Any test or user code using modern using / await using syntax fails at parse time.
Severity	High	

Affected tests:

- parallel/test-stream-duplex-destroy
- parallel/test-stream-readable-dispose
- parallel/test-stream-transform-destroy
- parallel/test-stream-writable-destroy

68.1 What Is The Issue

The stream destroy/dispose tests fail before execution:

SyntaxError: expecting ';'

The failing source files use modern explicit resource management syntax. The current QuickJS parser does not understand using / await using, so the CommonJS loader cannot compile the tests.

68.2 How Should We Fix It

This is a language-front-end gap. There are three viable paths:

- update the vendored QuickJS source to a version/fork that supports explicit resource management;
- add a narrowly scoped source transform in the test/runtime loader for using syntax; or
- skip these tests until the engine supports the syntax.

The preferred runtime-compatible fix is engine support. A loader transform is riskier because disposal semantics are subtle and should preserve Symbol.dispose, Symbol.asyncDispose, exception suppression, and async ordering exactly enough for Node streams.

Targeted verification after engine or loader support:

```
build-edge-quickjs-cli/edge test/parallel/test-stream-duplex-destroy.js
build-edge-quickjs-cli/edge test/parallel/test-stream-readable-dispose.js
build-edge-quickjs-cli/edge test/parallel/test-stream-transform-destroy.js
build-edge-quickjs-cli/edge test/parallel/test-stream-writable-destroy.js
```

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/development/troubleshooting/node-test/013_stream_missing_builtins_and_async_iterators.md

69 Node Test: stream missing builtins and async iterators

		Remarks
Status	[open]	Planned investigation. Blocks newer stream APIs and web-stream interop tests.
Severity	Medium	

Affected tests:

- parallel/test-stream-readable-async-iterators
- parallel/test-stream-some-find-every
- parallel/test-whatwg-readablestream

69.1 What Is The Issue

The stream suite exposes three gaps:

- test-stream-some-find-every.mjs cannot load timers/promises.
- test-whatwg-readablestream.mjs cannot load stream/web.
- test-stream-readable-async-iterators.js reaches execution but fails an assertion, so readable async iterator semantics differ from Node.

These are probably not one code bug, but they belong to the same compatibility surface: modern stream APIs expect builtin modules and async iterator behavior that QuickJS Edge does not fully provide yet.

69.2 How Should We Fix It

Implement or expose the missing builtins first:

1. Add timers/promises as a public builtin backed by the existing timers promise implementation already referenced from lib/timers.js.
2. Add stream/web as a public builtin that re-exports the available WHATWG streams implementation or explicitly documents missing constructors.

After imports work, rerun the assertion failure and compare Readable.prototype[Symbol.asyncIterator], iterator cancellation, error propagation, and microtask ordering against Node.

Targeted verification:

```
build-edge-quickjs-cli/edge test/parallel/test-stream-some-find-every.mjs
build-edge-quickjs-cli/edge test/parallel/test-whatwg-readablestream.mjs
build-edge-quickjs-cli/edge test/parallel/test-stream-readable-async-iterators.js
```

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/development/troubleshooting/node-test/014_string_decoder_utf8_boundaries.md

70 Node Test: StringDecoder UTF-8 boundaries

		Remarks
Status	[done]	Fixed in native StringDecoder binding.
Severity	Medium	Incorrect replacement-character behavior can corrupt streaming UTF-8 decoding.

Affected tests:

- parallel/test-string-decoder
- parallel/test-string-decoder-end

70.1 What Is The Issue

Invalid or incomplete UTF-8 sequences produce too many replacement characters. The log shows examples such as:

```
'  a' !== ' a'
Expected "\ufffd\u41", but got "\ufffd\ufffd\u41"
input: f ,b8,41
```

Node's StringDecoder coalesces certain invalid multi-byte prefix sequences into one replacement character across chunk/end boundaries. The current native decoder state in QuickJS Edge emits one replacement for the bad prefix and another for the following byte.

70.2 2026-05-15 Native Decoder Update

The failing UTF-8 boundary cases now pass without changing lib/. The native binding had a UTF-8 decoder with the right streaming behavior, but MakeStringFromBytes(...) tried Buffer.from(...).toString('utf8') first. That raw Buffer fallback produced too many replacement characters for cases such as f ,b8,41.

The native StringDecoder binding now bypasses the Buffer fallback for UTF-8 and uses its streaming-aware decoder directly. QuickJS Buffer UTF-8 slicing gets matching replacement behavior from the QuickJS N-API implementation of napi_create_string_utf8(), so the shared Buffer binding does not need a QuickJS-specific decoder and V8 keeps its native string behavior.

Smoke check:

```
new StringDecoder('utf8').write(Buffer.from([ f ,  b8,  x41])) === '\uFFFFDA'
```

Targeted verification passed:

```
build-edge-quickjs-cli/edge test/parallel/test-string-decoder.js
build-edge-quickjs-cli/edge test/parallel/test-string-decoder-end.js
build-edge-quickjs-cli/edge test/parallel/test-string-decoder-fuzz.js
```

70.3 Implementation Notes

Keep future streaming-state fixes in the native StringDecoder binding. Keep raw Buffer UTF-8 byte-to-string parity in the N-API backend so engine differences do not leak into the shared Buffer binding. The JS wrapper should remain unchanged.

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/development/troubleshooting/node-test/015_url_and_data_url_validation.md

71 Node Test: URL and data URL validation

		Remarks
Status	[open]	Planned investigation. Several URL tests fail through a shared validation/error path.
Severity	Medium	

Affected tests:

- parallel/test-data-url
- parallel/test-url-fileurltopath
- parallel/test-url-domain-ascii-unicode
- parallel/test-url-format
- parallel/test-url-format-whatwg
- parallel/test-url-format-invalid-input
- parallel/test-url-parse-format

71.1 What Is The Issue

These tests all report a compact test-runner failure shaped like:

```
TypeError [undefined]:  
  code: 'ERR_INVALID_ARG_VALUE'
```

The log does not include the specific subtest assertion, but the affected files all exercise URL/data URL parsing, formatting, file URL conversion, IDNA/domain conversion, or invalid-input handling.

This points at a shared mismatch in `internal/url`, `internal/data_url`, or the error constructors used by those modules under QuickJS.

71.2 How Should We Fix It

Rerun each test with a reporter that prints subtest names and with focused instrumentation around `ERR_INVALID_ARG_VALUE`. Then split any discovered failures into parsing, formatting, and error-message subtasks if needed.

Likely fix areas:

- ensure `URL.canParse`, `URL.parse`, `url.format()`, and `fileURLToPath()` match Node's coercion and invalid-input rules;
- verify IDNA/domain conversions use the same ICU/Intl or fallback behavior expected by the tests;
- make `ERR_INVALID_ARG_VALUE` construction include the expected argument name, value, and reason instead of producing an empty message.

Targeted verification:

```
build-edge-quickjs-cli/edge --expose-internals --test-reporter=test/common/test-error-reporter.js  
build-edge-quickjs-cli/edge --test-reporter=test/common/test-error-reporter.js --test-reporter-de  
build-edge-quickjs-cli/edge --test-reporter=test/common/test-error-reporter.js --test-reporter-de
```

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/development/troubleshooting/node-test/016_whatwg_url_inspect_and_searchparams.md

72 Node Test: WHATWG URL inspect and search params

		Remarks
Status	[done]	Fixed with native QuickJS/private-symbol compatibility changes. WHATWG URL behavior differs in visible formatting, encoding, and error messages.
Severity	Medium	

Affected tests:

- parallel/test-whatwg-url-custom-inspect
- parallel/test-whatwg-url-custom-parsing
- parallel/test-whatwg-url-custom-searchparams-append
- parallel/test-whatwg-url-custom-searchparams-constructor
- parallel/test-whatwg-url-custom-searchparams-delete
- parallel/test-whatwg-url-custom-searchparams-get
- parallel/test-whatwg-url-custom-searchparams-getall
- parallel/test-whatwg-url-custom-searchparams-has
- parallel/test-whatwg-url-custom-searchparams-set
- parallel/test-whatwg-url-invalidthis

72.1 What Is The Issue

The failures originally exposed three concrete mismatches:

- `util.inspect({ a: new URL(...) })` prints `{ a: [URL] }` instead of `{ a: URL {} }`.
- A custom URL parsing case preserves/encodes a lone surrogate as `%ED%A0%BD`, while Node replaces it with `%EF%BF%BD`.
- Passing a Symbol into `URLSearchParams` methods throws `TypeError: cannot convert symbol to string`; Node expects `TypeError: Cannot convert a Symbol value to a string`.

72.2 2026-05-15 QuickJS Error Text And UTF-8 Update

Two of the URL/SearchParams failures are fixed in vendored QuickJS without changing `lib/internal/url.js`:

- Implicit Symbol-to-string coercion now throws `TypeError: Cannot convert a Symbol value to a string`.
- Normal UTF-8 export replaces lone UTF-16 surrogates with `U+FFFD`, so URL parsing percent-encodes `%EF%BF%BD` instead of raw surrogate bytes.

Targeted verification passed:

```
build-edge-quickjs-cli/edge test/parallel/test-whatwg-url-custom-parsing.js
build-edge-quickjs-cli/edge test/parallel/test-whatwg-url-custom-searchparams-append.js
build-edge-quickjs-cli/edge test/parallel/test-whatwg-url-custom-searchparams-constructor.js
build-edge-quickjs-cli/edge test/parallel/test-whatwg-url-custom-searchparams-delete.js
build-edge-quickjs-cli/edge test/parallel/test-whatwg-url-custom-searchparams-get.js
build-edge-quickjs-cli/edge test/parallel/test-whatwg-url-custom-searchparams-getall.js
build-edge-quickjs-cli/edge test/parallel/test-whatwg-url-custom-searchparams-has.js
build-edge-quickjs-cli/edge test/parallel/test-whatwg-url-custom-searchparams-set.js
```


72.3 2026-05-15 URL Inspect And Brand Update

The remaining URL inspect and invalid-this failures now pass without changing `lib/internal/url.js` or `lib/internal/util/inspect.js`.

- `unofficial_napi_create_private_symbol()` now creates real QuickJS private symbols, so Node internal slots such as `node:transfer_mode` no longer appear in `Object.getOwnPropertySymbols(new URL(...))`.
- With that internal symbol hidden, unchanged `util.inspect` can format nested URL instances as `{ a: URL {} }`.
- QuickJS private brand-check failures now use `Receiver must be an instance of class`, while ordinary missing private member errors still use the `Cannot read private member ... text`.

Targeted verification passed:

```
build-edge-quickjs-cli/edge test/parallel/test-whatwg-url-custom-inspect.js
build-edge-quickjs-cli/edge test/parallel/test-whatwg-url-invalidthis.js
```

72.4 Implementation Notes

Keep URL compatibility fixes on the native/private-symbol side unless the Node-shipped `lib/` sources themselves change upstream.

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/development/troubleshooting/node-test/017_http2_native_lifecycle_crashes.md

73 Node Test: HTTP/2 native lifecycle crashes

		Remarks
Status	[open]	Original QuickJS N-API object/external crash fixed; reopened for new QuickJS macOS HTTP/2 crashes. Signal 10/11 crashes affected a large HTTP/2 test cluster and could terminate the QuickJS CLI.
Severity	High	

73.1 Symptoms

QuickJS crashed with SIGBUS/SIGSEGV in HTTP/2 tests such as:

```
test/parallel/test-http2-socket-proxy-handler-for-has.js
test/sequential/test-http2-timeout-large-write.js
test/sequential/test-http2-timeout-large-write-file.js
```

LLDB showed the crash under WriteChunksToParent(...) / FlushSessionOutput(...), after HTTP/2 attempted to write pending session bytes through the parent stream.

73.2 Root Cause

SessionConsume() stored the result of EdgeStreamBaseFromValue(socket._handle) as the HTTP/2 parent stream. QuickJS N-API constructed native class instances with the same QuickJS class used for napi_create_external(...), so napi_typeof() reported wrapped objects such as TLSWrap as napi_external. The generic stream extraction path then accepted the wrapped native object pointer directly.

For TLS sockets that meant HTTP/2 sometimes stored a TLSWrap* as if it were an EdgeStreamBase*. Later writes read the stream ops table from the wrong offset and jumped through heap data.

73.3 Fix

The fix stays outside lib/ and keeps the EdgeJS shared native stream code unchanged:

- QuickJS N-API constructor calls now create ordinary prototype-backed objects, not objects of the external class.
- napi_typeof() reports napi_external only for actual values created by napi_create_external(...).
- napi_get_value_external() rejects non-external values instead of returning a wrapped object's native pointer.

73.4 Verification

Targeted QuickJS checks passed after rebuilding build-edge-quickjs-cli/edge:

```
build-edge-quickjs-cli/edge test/parallel/test-http2-socket-proxy-handler-for-has.js
build-edge-quickjs-cli/edge test/sequential/test-http2-timeout-large-write.js
build-edge-quickjs-cli/edge test/sequential/test-http2-timeout-large-write-file.js
build-edge-quickjs-cli/edge test/parallel/test-http2-too-many-settings.js
build-edge-quickjs-cli/edge test/parallel/test-http2-multiplex.js
build-edge-quickjs-cli/edge test/parallel/test-http2-forget-closed-streams.js
```

Shared native regression checks passed after rebuilding build-edge/edge:

```
build-edge/edge test/parallel/test-http2-socket-proxy-handler-for-has.js
build-edge/edge test/sequential/test-http2-timeout-large-write.js
build-edge/edge test/parallel/test-buffer-constants.js
```

73.5 V8 JSStream Destroy Follow-up

The May 16, 2026 V8 CI log still failed test/parallel/test-http2-client-jsstream-destroy.js without crashing. The failure was an unhandled ERR_STREAM_DESTROYED from the JavaScript stream wrapped by internal/js_stream_socket, reached through:

```
JSStreamSocket.doWrite()
JSStream.onwrite()
process.processImmediate()
```

The source-side fix stays outside lib/: src/edge_js_stream.cc now treats a pending ERR_STREAM_DESTROYED from JSStream.onwrite as a write completion with UV_EPIPE, matching the existing JS wrapper behavior for async write errors. Other pending exceptions are rethrown, so unrelated userland callback errors still surface.

Verification:

```
cmake --build build-edge --target edge -j4
build-edge/edge test/parallel/test-http2-client-jsstream-destroy.js
```

The HTTP/2 test binds a local socket, so sandboxed local runs may need to be rerun outside the filesystem sandbox when they fail with listen EPERM.

73.6 2026-05-16 QuickJS macOS CI Follow-up

The test and build / quickjs-macos GitHub Actions log test-failures-qjs-2.log shows four HTTP/2 failures in the QuickJS runtime lane:

```
test/parallel/test-http2-client-set-priority.js      -> Signal 11
test/parallel/test-http2-compat-serverresponse-writehead.js -> Signal 11
test/parallel/test-http2-response-splitting.js      -> response splitting assertion
test/parallel/test-http2-reset-flood.js             -> timeout
```

These are not the same three tests that validated the previous TLSWrap*/EdgeStreamBase* object classification fix. Treat this as an open HTTP/2 follow-up and reproduce the two crashing tests under LLDB before editing native code. Useful source areas to inspect first are the QuickJS session output path and stream reset/priority handling in src/internal_binding/binding_http2.cc.

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/development/troubleshooting/node-test/018_tls_securecontext_sni_lifetime.md

74 Node Test: TLS SecureContext, SNI, and setKeyCert lifetime

		Remarks
Status	[done]	Fixed with QuickJS N-API own-wrap lookup and native TLS SecureContext retention.
Severity	High	Remaining TLS failures included one assertion failure and one native crash.

74.1 Current Reproduction

After the QuickJS N-API object/external classification fix, the old broad TLS/SNI cluster is much smaller. The current focused SNI/SecureContext sample shows these QuickJS results:

test/parallel/test-tls-add-context.js	pass
test/parallel/test-tls-empty-sni-context.js	pass
test/parallel/test-tls-sni-option.js	pass
test/parallel/test-tls-sni-servername.js	pass
test/parallel/test-tls-sni-server-client.js	pass
test/parallel/test-tls-snicallback-error.js	pass
test/parallel/test-tls-socket-snicallback-without-server.js	pass
test/parallel/test-tls-connect-secure-context.js	pass
test/parallel/test-tls-secure-context-usage-order.js	pass
test/parallel/test-tls-set-secure-context.js	pass
test/parallel/test-tls-server-setkeycert.js	crash

The full TLS category run:

```
NODE_TEST_RUNNER=build-edge-quickjs-cli/edge ./test/nodejs_test_harness --category=node:tls
```

reported two failures before the fix:

```
test/parallel/test-tls-external-accessor.js
test/parallel/test-tls-server-setkeycert.js
```

The same focused checks pass under build-edge/edge.

74.2 Root Cause: inherited wrap metadata

test-tls-external-accessor.js creates an object whose prototype is a real SecureContext native wrapper:

```
const pctx = tls.createSecureContext().context;
const cctx = { __proto__: pctx };
assert.throws(() => cctx._external, TypeError);
```

V8 napi_unwrap(...) only succeeds for the exact object that was wrapped. QuickJS currently lets napi_external__::get_wrap_record(...) read __napi_wrap__ through JS_GetPropertyStr(...), which walks the prototype chain. That makes napi_unwrap(env, cctx, ...) incorrectly unwrap pctx.

This is a QuickJS N-API semantics bug, not a TLS-specific behavior. The fix is to make wrap-record lookup own-property-only after checking the object's direct opaque slot.

74.3 Root Cause: switched SecureContext lifetime

`test-tls-server-setkeycert.js` calls `this.setKeyCert(altKeyCertVal)` from `ALPNCallback`. When `altKeyCertVal` is a plain key/cert options object, unchanged `lib/internal/tls/wrap.js` creates a temporary `SecureContext` inside `TLSSocket.prototype.setKeyCert(...)` and passes only `secureContext.context` to native code.

Native `TlsWrapSetKeyCert(...)` extracts the `SecureContextHolder*` and calls `SetSecureContextOnSsl(...)`, which switches OpenSSL to the new `SSL_CTX` and stores:

```
wrap->secure_context = holder;
```

but it does not retain a `napi_ref` to the JavaScript `SecureContext.context` object. QuickJS releases callback-local values promptly, so the temporary native `SecureContext` can be finalized while the `SSL*` is still using the new `SSL_CTX` and its `SSL_CTX_get_app_data(...)` pointer.

LLDB shows the crash later in the session-ticket callback:

```
_platform_memset
drbg_ctr_generate
ossl_prov_drbg_generate
EVP RAND_generate
edge::crypto::TicketCompatibilityCallback(...)
tls_construct_new_session_ticket
SSL_read
ReadCleartext
ParentStreamOnRead
```

The important detail is that `TicketCompatibilityCallback(...)` recovers the holder from `SSL_CTX_get_app_data(SSL_`. If the holder was deleted after the temporary JS `SecureContext` finalized, the callback reads a freed `std::vector<unsigned char>` for ticket keys.

V8 passes because its lifetime/GC timing keeps the temporary wrapper alive long enough in this test. That is not an ownership guarantee and should not be relied on.

74.4 Fix

1. QuickJS N-API unwrap semantics:
 - `napi_external__::get_wrap_record(...)` now reads `__napi_wrap__` only as an own property with `JS_GetOwnProperty(...)`.
 - The direct opaque-record path is unchanged.
 - No EdgeJS-local N-API regression test is added for this; that coverage belongs in the N-API repo.
2. TLS context ownership on native context switches:
 - `src/edge_tls_wrap.cc` now replaces `wrap->context_ref` with a strong reference to the selected `SecureContext.context` object only after `SetSecureContextOnSsl(...)` succeeds.
 - `TlsWrapSetKeyCert(...)` uses the retaining switch helper.
 - The `SNI TlsWrapCertCbDone(...)` path uses the same ownership update when a selected SNI context is installed.
 - The initial `TlsWrapWrap(...)` reference behavior is unchanged.
3. Defensive cleanup around `SecureContextHolder`:
 - Before freeing a holder-owned `SSL_CTX`, native code clears `SSL_CTX` app-data and the ticket callback.
 - This guards future forgotten ownership paths; the primary fix remains the strong `napi_ref` from active `TlsWrap` to selected context.

No `lib/` file was changed. The JavaScript lifetime shape is Node's contract; native now retains what OpenSSL continues to use.

74.5 Verification

Rebuild:

```
cmake --build build-edge-quickjs-cli --target edge -j4
cmake --build build-edge --target edge -j4
```

Targeted checks:

```
build-edge-quickjs-cli/edge test/parallel/test-tls-external-accessor.js
build-edge-quickjs-cli/edge test/parallel/test-tls-server-setkeycert.js
build-edge-quickjs-cli/edge test/parallel/test-tls-add-context.js
build-edge-quickjs-cli/edge test/parallel/test-tls-sni-option.js
build-edge-quickjs-cli/edge test/parallel/test-tls-sni-servername.js
build-edge-quickjs-cli/edge test/parallel/test-tls-sni-server-client.js
build-edge-quickjs-cli/edge test/parallel/test-tls-snicallback-error.js
build-edge-quickjs-cli/edge test/parallel/test-tls-socket-snicallback-without-server.js
build-edge-quickjs-cli/edge test/parallel/test-tls-connect-secure-context.js
build-edge-quickjs-cli/edge test/parallel/test-tls-secure-context-usage-order.js
build-edge-quickjs-cli/edge test/parallel/test-tls-set-secure-context.js
```

These targeted checks passed. The broader TLS category also passed:

```
NODE_TEST_RUNNER=build-edge-quickjs-cli/edge ./test/nodejs_test_harness --category=node:tls
```

Result:

195/195 passed

V8 regression checks:

```
build-edge/edge test/parallel/test-tls-external-accessor.js
build-edge/edge test/parallel/test-tls-server-setkeycert.js
build-edge/edge test/parallel/test-tls-add-context.js
```

These passed after rebuilding build-edge/edge.

75 Astro SSR

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/development/troubleshooting/astro-ssr/001_es_module_lexer_webassembly.md

76 Astro SSR: es-module-lexer WebAssembly Import

		Remarks
Status	[open]	Planned runtime compatibility investigation.
Severity	High	The Astro SSR entry cannot start if the WebAssembly-dependent lexer path is selected.

76.1 Issue

Running the Astro SSR native ESM entry directly with the QuickJS Edge CLI fails:

```
~/src/dev/edgejs/build-edge-quickjs-cli/edge ./dist/server/entry.mjs
```

Observed error:

```
ReferenceError: WebAssembly is not defined
    at <anonymous> (.../node_modules/es-module-lexer/dist/lexer.js:2:356)
```

Node can run the same entry:

```
node ./dist/server/entry.mjs
```

76.2 Diagnosis

Astro's server output imports es-module-lexer.

QuickJS currently resolves the bare package import to:

```
node_modules/es-module-lexer/dist/lexer.js
```

That is the package's default ESM/WASM build. It expects globalThis.WebAssembly, which the QuickJS runtime does not currently expose.

The existing bundled CJS path already avoids this by aliasing:

```
'es-module-lexer': 'es-module-lexer/js'
```

The es-module-lexer/js export resolves to the pure JS build and has already been verified to import successfully under the QuickJS Edge CLI.

76.3 Status Notes

Implement a QuickJS-only resolver compatibility alias in:

```
napi/quickjs/src/unofficial_napi.cc
```

When resolving the bare specifier:

```
es-module-lexer
```

resolve it as:

```
es-module-lexer/js
```

This should route through the package exports [".js"] target and load:

```
node_modules/es-module-lexer/dist/lexer.asm.js
```

76.4 Constraints

- Only modify files under `napi/quickjs/`.
- Do not modify the Astro app, `node_modules`, or non-QuickJS Edge runtime code.
- Do not implement a fake `WebAssembly` global for this issue.
- Keep the alias narrow and package-specific.

76.5 Validation

Rebuild:

```
cmake --build build-edge-quickjs-cli --target edge -j4
```

Smoke test resolver behavior:

```
~/src/dev/edgejs/build-edge-quickjs-cli/edge \  
-e "import('es-module-lexer').then(m=>console.log(typeof m.parse))"
```

Expected output:

```
function
```

Then rerun the Astro SSR entry:

```
~/src/dev/edgejs/build-edge-quickjs-cli/edge ./dist/server/entry.mjs
```

If a new failure appears, capture it as the next issue-specific plan in this directory before making further changes.

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/development/troubleshooting/astro-ssr/002_depd_callsite_methods.md

77 Astro SSR: depd CallSite Method Compatibility

		Remarks
Status	[done]	Fixed in the vendored QuickJS CallSite and stack trace implementation.
Severity	High	Astro SSR stopped during startup when depd could not use Node-style CallSite methods.

77.1 Issue

After aliasing es-module-lexer to its pure-JS export, the Astro SSR native ESM entry advances to a new failure under the QuickJS Edge CLI:

```
~/src/dev/edgejs/build-edge-quickjs-cli/edge \
-e "import('./dist/server/entry.mjs')"
```

Observed error:

```
TypeError: not a function
    at callSiteLocation (<input>:270:23)
    at depd (<input>:111:31)
```

The failure was first reproduced from:

```
~/src/dev/christoph/astro-app
```

It was later reproduced from:

```
~/src/dev/stackmachine.com
```

77.2 Diagnosis

The failing dependency is depd.

Its callSiteLocation(...) helper expects V8/Node-style structured stack frame objects with methods such as:

```
callSite.getFileName()
callSite.getLineNumber()
callSite.getColumnNumber()
callSite.isEval()
callSite.getEvalOrigin()
callSite.getFunctionName()
```

The QuickJS-backed runtime had two compatibility gaps:

- Node bootstrap replaces Error.prepareStackTrace with a writable data property, but QuickJS stack construction still read only its hidden ctx->error_prepare_stack slot. Userland assignments such as depd's temporary Error.prepareStackTrace = prepareObjectStackTrace were ignored, so Error.captureStackTrace(obj) produced a string instead of CallSite[].
- The native QuickJS CallSite prototype exposed only a small method subset. depd also expects methods such as isEval(), getEvalOrigin(), getThis(), getTypeName(), and getMethodName().

This is separate from the earlier `es-module-lexer` WebAssembly issue, which no longer appears after the resolver alias.

The existing bundled CJS Astro path has already avoided this dependency behavior by using `edge-depd-stub.cjs`, but the native ESM SSR entry currently imports the real `depd` package.

77.3 Current Status

Updated `quickjs/quickjs.c` so `build_backtrace(...)` reads the public `Error.prepareStackTrace` property at stack-build time, falling back to the hidden QuickJS slot only when the public property is not callable.

Also extended the native QuickJS `CallSite` prototype with conservative Node/V8-compatible methods:

```
isEval
isConstructor
isToplevel
isAsync
isPromiseAll
getMethodName
getTypeName
getThis
getEvalOrigin
getScriptNameOrSourceURL
getPromiseIndex
toString
```

Where QuickJS does not currently track the exact V8 metadata, these methods return stable conservative values (`false`, `null`, or `undefined`) instead of throwing.

77.4 Constraints

- Do not modify the Astro app, `node_modules`, or generated dist files.
- Keep any resolver compatibility aliases narrow and package-specific.
- Compare with the V8 backend or existing bundled adapter behavior before implementing.
- Fix one behavior at a time and rerun the targeted Astro SSR import before moving to any next failure.

77.5 Validation

Rebuild:

```
cmake --build build-edge-quickjs-cli --target edge -j4
```

Focused `depd` check:

```
cd ~/src/dev/stackmachine.com
~/src/dev/edgejs/build-edge-quickjs-cli/edge \
  -e "try { require('depd')('x'); console.log('depd ok') } catch(e) { console.error(e && (e.stack || '')) }"
```

Observed result after the fix:

```
depd ok
```

Rerun the focused import:

```
cd ~/src/dev/stackmachine.com
~/src/dev/edgejs/build-edge-quickjs-cli/edge \
  ./dist/server/entry.mjs
```

Observed result after the fix:

```
ReferenceError: Intl is not defined
    at ~/src/dev/stackmachine.com/dist/server/chunks/_@astrojs-ssr-adapter_BqW-NUXY.mjs:734:28
```

This is a later, different Astro SSR failure captured in 004_missing_intl.md.

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/development/troubleshooting/astro-ssr/003_cjs_reexport_named_exports.md

78 Astro SSR: CommonJS Re-Export Named Exports

		Remarks
Status	[caveated]	Historical fix superseded by removing the QuickJS C++ CommonJS facade/module-loader hack.
Severity	High	React named exports failed to link without this compatibility behavior.

78.1 Issue

The Astro standalone SSR entry for stackmachine.com starts under native Node and native V8 EdgeJS, but fails under native QuickJS EdgeJS:

```
~/src/dev/edgejs/build-edge-quickjs-cli/edge ./dist/server/entry.mjs
```

Focused reproduction:

```
cd ~/src/dev/stackmachine.com
~/src/dev/edgejs/build-edge-quickjs-cli/edge \
  -e "import('./dist/server/entry.mjs').catch(e=>{ console.error(e && e.message); process.exitCo
```

Observed error:

```
Could not find export 'createElement' in module
'~/src/dev/stackmachine.com/node_modules/react/index'
```

78.2 Diagnosis

The failing Astro renderer imports React as:

```
import React__default, { createElement } from 'react';
```

Node and the V8-backed Edge runtime expose CommonJS named exports on the ESM namespace for react. QuickJS EdgeJS creates a synthetic CommonJS module facade because QuickJS links ESM imports before a CommonJS file can execute and produce module.exports.

The original QuickJS facade statically scanned only the immediate wrapper file. React's public index.js delegates to another CommonJS file:

```
module.exports = require('./cjs/react.development.js');
```

The real exports.createElement = ... assignment is in the delegated file, so QuickJS declares only default and module.exports before module linking. LLDB confirmed the failure was thrown before the synthetic CommonJS module initializer ran, so this had to be fixed in export-name declaration rather than later evaluation.

78.3 Why this compatibility layer exists

This is not because V8 has CommonJS and QuickJS does not. V8, like QuickJS, is a JavaScript engine; Node implements CommonJS, ESM loading, and the CJS/ESM bridge around the engine.

The difference is that Node's V8 path already has mature native ModuleWrap, loader, and translator integration. Earlier QuickJS work tried to approximate that with a C++ host module loader, resolver, and CommonJS facade. That is the compatibility layer described by this historical issue.

We later solved the missing engine primitive at the QuickJS source level: commit 577fb31caf2e973b111431b6cb009f7595cc adds QuickJS APIs for enumerating module requests, binding host-supplied module records, linking, evaluating, querying module state, initializing import meta, and dispatching dynamic imports. The current `napi_module_wrap__` layer can therefore map Node/V8-shaped `unofficial_napi_module_wrap_*` calls onto QuickJS itself, without restoring the deleted package resolver or CommonJS export scanner.

QuickJS still requires declared ESM export names before link time. CommonJS modules only know their true export object after evaluation, so CJS/ESM bridge policy remains a Node loader/runtime concern rather than an engine concern.

The facade/export-name logic is therefore a bridge for Node compatibility:

- resolve package and file specifiers in the QuickJS host module loader;
- decide whether a `.js` file should be compiled as ESM or evaluated through the CommonJS loader;
- predeclare conservative named exports for CommonJS facades so ESM imports can link;
- after `require(...)` evaluates the CommonJS file, copy the actual properties onto the declared QuickJS module exports.

This behavior was needed while QuickJS lacked host-linkable module primitives. With 577fb31, the valid long-term direction is clearer: keep the engine hooks in QuickJS and keep package, CommonJS, and translator policy in Node's JavaScript loaders/translators or another EdgeJS-owned runtime path.

78.4 Current Status

CommonJS named export discovery was previously moved out of `unofficial_napi.cc` into a dedicated scanner. That scanner preserved the direct export patterns and added recursive literal re-export discovery for patterns such as:

```
module.exports = require('./target.js')
Object.assign(exports, require('./target.js'))
```

The resulting names were used both when declaring QuickJS C module exports and when setting those exports after `require(...)` evaluated.

Later cleanup removed the remaining QuickJS C++ CommonJS facade/module-loader hack. The deleted files were `unofficial_module_loader.{h,cc}` and `quickjs_cjs_exports.{h,cc}`. The current QuickJS N-API `module_wrap` public surface is implemented through `napi/quickjs/src/internal/napi_module_wrap.*` and depends on the QuickJS module APIs introduced by 577fb31, rather than on the deleted compatibility parser/resolver.

78.5 Constraints

- Keep the fix in the QuickJS-backed N-API/runtime path.
- Do not modify the Astro app, `node_modules`, or generated `dist` files.
- Keep the scanner conservative; do not execute CommonJS while computing link names.
- Preserve default and `module.exports` behavior if CJS support is rebuilt in the JavaScript loader path rather than in QuickJS C++ host-loader shims.

78.6 Validation

Rebuild:

```
cmake --build build-edge-quickjs-cli --target edge -j4
```

Check the focused namespace behavior:

```
cd ~/src/dev/stackmachine.com
~/src/dev/edgejs/build-edge-quickjs-cli/edge \
-e "import('react').then(m=>console.log(Object.prototype.hasOwnProperty.call(m,'createElement'))"
```

Then rerun the Astro SSR entry:

```
~/src/dev/edgejs/build-edge-quickjs-cli/edge ./dist/server/entry.mjs
```

Observed current state:

- react named exports such as createElement link successfully.
- Later Astro SSR failures were captured in subsequent issue-specific notes (004 through 014) rather than folded into this issue.
- The full Astro standalone path has since reached a Wasmer-served 200 text/html response after the later resolver, symlink, stack, and deploy packaging fixes.

Current design status: QuickJS N-API has a real source-text/synthetic ModuleWrap bridge over local QuickJS engine APIs. CommonJS support and CJS/ESM named-export policy should come through Node's JavaScript loaders/translators or another EdgeJS-owned runtime path, not through local QuickJS C++ host-loader parsers.

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/development/troubleshooting/astro-ssr/004_missing_intl.md

79 Astro SSR: Missing Intl Global

		Remarks
Status	[caveated]	Minimal runtime compatibility fallback implemented.
Severity	Medium	The fallback enables Astro startup but is intentionally incomplete compared with ECMA-402 Intl.

79.1 Issue

After the depd CallSite compatibility fix, the Astro standalone SSR entry for stackmachine.com advances to a new QuickJS runtime failure:

```
cd ~/src/dev/stackmachine.com
~/src/dev/edgejs/build-edge-quickjs-cli/edge ./dist/server/entry.mjs
```

Observed error:

```
ReferenceError: Intl is not defined
    at ~/src/dev/stackmachine.com/dist/server/chunks/_@astrojs-ssr-adapter_BqW-NUXY.mjs:734:28
```

The generated Astro chunk creates a logger timestamp formatter at module load:

```
const dateTimeFormat = new Intl.DateTimeFormat([], {
  hour: "2-digit",
  minute: "2-digit",
  second: "2-digit",
  hour12: false
});
```

79.2 Diagnosis

The native QuickJS Edge runtime currently exposes no `globalThis.Intl`. Astro's SSR adapter expects at least `Intl.DateTimeFormat` to exist during module evaluation.

This is separate from the depd stack compatibility issue: a focused `require('depd')('x')` check now succeeds, and the Astro entry reaches this later import-time failure.

79.3 Current Status

Implemented the narrow runtime-level fallback option in `src/edge_runtime.cc`. During early runtime bootstrap, before the process object is installed, EdgeJS now checks for `globalThis.Intl.DateTimeFormat`. If a real implementation already exists, it is left untouched. If it is missing, EdgeJS installs a small JS fallback with:

- a callable/constructible `Intl.DateTimeFormat`;
- `format(value)` for local-time HH:mm:ss / h:mm:ss AM|PM output;
- `resolvedOptions()`;
- `supportedLocalesOf()`;
- `Symbol.toStringTag`.

This is intentionally not a full ECMA-402 implementation. It does not perform real locale negotiation, ICU-backed calendar selection, numbering-system formatting, or time-zone conversion. It exists to satisfy framework bootstrap paths, including Astro's SSR logger timestamp formatter, without modifying app code or generated output.

Choosing this minimal fallback keeps native QuickJS and WASIX QuickJS moving while leaving the door open for a future real Intl provider.

79.4 Constraints

- Do not modify the Astro app, node_modules, or generated dist files.
- Fix one behavior at a time and rerun the targeted Astro SSR entry before moving to any next failure.
- Avoid pretending to implement full ECMA-402 Intl unless the implementation is actually backed by a real formatter.

79.5 Validation

Rebuild:

```
cmake --build build-edge-quickjs-cli --target edge -j4
```

Focused Intl check:

```
~/src/dev/edgejs/build-edge-quickjs-cli/edge \  
-e "const f = new Intl.DateTimeFormat([], { hour: '2-digit', minute: '2-digit', second: '2-digit'})"
```

Observed result:

string

Then rerun the Astro SSR entry:

```
cd ~/src/dev/stackmachine.com  
~/src/dev/edgejs/build-edge-quickjs-cli/edge ./dist/server/entry.mjs
```

Expected result for this issue: the Intl is not defined failure disappears.

Observed result after the fix: Astro reaches a later runtime/server issue and prints timestamped logger output, proving the fallback is being used:

```
13:03:14 [ERROR] [@astrojs/node] Unhandled rejection while rendering undefined  
Error: listen EPERM: operation not permitted ::1:4321
```

The later listen failure is captured in 005_listen_eperm.md.

80 Astro SSR: Listen EPERM On Localhost

		Remarks
Status	[caveated]	Resolved as sandbox bind policy, not a QuickJS runtime bug.
Severity	Low	The server works when localhost binding is allowed; this is a local verification constraint.

80.1 Issue

After the minimal Intl.DateTimeFormat fallback, the Astro standalone SSR entry for stackmachine.com advances past module evaluation and reaches server startup:

```
cd ~/src/dev/stackmachine.com
~/src/dev/edgejs/build-edge-quickjs-cli/edge ./dist/server/entry.mjs
```

Observed output:

```
(node:91514) [DEP0093] DeprecationWarning: The crypto.fips is deprecated. Please use crypto.getFips() instead.
(node:91514) [DEP0176] DeprecationWarning: fs.F_OK is deprecated, use fs.constants.F_OK instead
13:03:14 [ERROR] [@astrojs/node] Unhandled rejection while rendering undefined
Error: listen EPERM: operation not permitted ::1:4321
    at UVExceptionWithHostPort (<input>:704:5)
    at setupListenHandle (<input>:1920:25)
    at listenInCluster (<input>:1999:21)
    at <input>:2208:23
    at onlookupall (<input>:136:17) {
  code: 'EPERM',
  errno: -1,
  syscall: 'listen',
  address: '::1',
  port: 4321
}
```

The process exited with status 0 in this sandboxed run despite logging the unhandled rejection.

80.2 Diagnosis

This is separate from the Intl issue. The logger timestamp is formatted, so the Astro adapter has advanced to the server listen path. The failing address is IPv6 localhost (::1) on port 4321.

Focused node:net bind checks show this is caused by the current sandbox disallowing local socket binds, not by QuickJS TCP/listen behavior or Astro's default host/port selection.

Under normal sandboxed execution, native QuickJS Edge fails to listen on all tested TCP server addresses:

```
127.0.0.1 ephemeral port -> error EPERM -1 listen 127.0.0.1 undefined
::1 ephemeral port       -> error EPERM -1 listen ::1 undefined
unspecified ephemeral    -> error EPERM -1 listen 0.0.0.0 undefined
```

With the same QuickJS Edge binary allowed to run outside the sandbox, the same checks succeed:

```
127.0.0.1 ephemeral port -> listening 127.0.0.1 IPv4 <port>; closed
::1 ephemeral port      -> listening ::1 IPv6 <port>; closed
unspecified ephemeral   -> listening :: IPv6 <port>; closed
```

The available V8-backed Edge binary shows the same pattern: sandboxed 127.0.0.1 and ::1 binds fail with EPERM, while both succeed outside the sandbox. That comparison rules out a QuickJS-specific TCP bind regression.

80.3 Status Notes

Investigate with the narrowest server-listen repro before changing runtime code:

- ☒ run a tiny `node:net` server under native QuickJS Edge on 127.0.0.1, ::1, and an ephemeral port;
- ☒ compare the same repro under native V8 EdgeJS if available;
- ☒ rerun the Astro entry outside the sandbox to verify it reaches listen;
- ☒ only change Edge TCP/listen behavior if the focused repro shows a runtime bug rather than sandbox policy or application configuration.

No runtime code change is needed for this issue.

80.4 Constraints

- Do not modify the Astro app, `node_modules`, or generated dist files.
- Keep the investigation focused on listen/bind behavior, not warnings from `crypto.fips` or `fs.F_OK`.
- If a command fails due to sandboxed network permissions, rerun the important repro with explicit escalation instead of inferring a runtime bug.

80.5 Validation

Focused local listen check:

```
~/src/dev/edgejs/build-edge-quickjs-cli/edge \
-e "const net=require('node:net'); const s=net.createServer(); s.listen(0, '127.0.0.1', () => +
```

Then rerun the Astro SSR entry:

```
cd ~/src/dev/stackmachine.com
~/src/dev/edgejs/build-edge-quickjs-cli/edge ./dist/server/entry.mjs
```

Expected result for this issue: the Astro standalone server can bind or the failure is clearly attributed to sandbox permissions/app configuration rather than QuickJS runtime behavior.

Observed validation:

sandboxed Astro entry:

```
Error: listen EPERM: operation not permitted ::1:4321
```

escalated Astro entry:

```
[@astrojs/node] Server listening on http://localhost:4321
```

Decision: treat `listen EPERM` as sandbox policy in the current Codex execution environment. Continue Astro SSR compatibility work from the server-listening state by running the entry with local bind permission when server startup is the behavior under test.

81 Astro SSR: Floating UI Utils DOM Subpath

		Remarks
Status	[done]	Fixed in QuickJS package exports subpath resolution.
Severity	High	The Astro route could not render while this dependency subpath failed to load.

81.1 Issue

After the Astro standalone SSR server starts successfully outside the Codex local-bind sandbox, opening the site in Firefox triggers a route render failure:

```
cd ~/src/dev/stackmachine.com
~/src/dev/edgejs/build-edge-quickjs-cli/edge ./dist/server/entry.mjs
```

Observed server output:

```
(node:15190) [DEP0093] DeprecationWarning: The crypto.fips is deprecated. Please use crypto.getFips() instead.
(node:15190) [DEP0176] DeprecationWarning: fs.F_OK is deprecated, use fs.constants.F_OK instead
13:10:51 [@astrojs/node] Server listening on http://localhost:4321
13:11:02 [ERROR] [router] Error while trying to render the route /
13:11:02 [ERROR] ReferenceError: could not load module '@floating-ui/utils/dom'
    at runMicrotasks (native)
    at processTicksAndRejections (<input>:105:5)
```

81.2 Diagnosis

This is separate from the listen issue. The server starts and the failure appears only when a request renders /.

The failing specifier is the package subpath:

@floating-ui/utils/dom

QuickJS throws ReferenceError: could not load module ... from its module load callback when it cannot resolve/load a module.

The package metadata is valid Node-style package exports:

```
"./dom": {
  "import": {
    "types": "./dist/floating-ui.utils.dom.d.mts",
    "default": "./dist/floating-ui.utils.dom.mjs"
  },
  "types": "./dist/floating-ui.utils.dom.d.ts",
  "module": "./dist/floating-ui.utils.dom.esm.js",
  "default": "./dist/floating-ui.utils.dom.umd.js"
}
```

The QuickJS resolver's string scanner found the import condition but then picked the nested key name types as though it were a target. It tried to resolve a file named types and stopped before trying the nested default runtime target.

81.3 Current Status

Updated the former QuickJS C++ package resolver so `TryResolvePackageSubpath(...)` did not stop after the first condition string if that candidate did not resolve to a runtime file. That implementation tried runtime condition targets in order, returning the first target that resolved.

Later cleanup removed the remaining QuickJS C++ CommonJS facade/module-loader support, so this note is historical context rather than a pointer to live code.

This keeps the fix narrow: it does not add a full JSON parser or a complete Node package exports implementation, but it handles the nested condition shape that exposed this issue.

81.4 Constraints

- Do not modify the Astro app, `node_modules`, or generated dist files.
- Keep the fix in EdgeJS/QuickJS runtime code if a runtime fix is required.
- Fix one behavior at a time and rerun the focused import before rerendering the Astro route.

81.5 Validation

Focused import check:

```
cd ~/src/dev/stackmachine.com
~/src/dev/edgejs/build-edge-quickjs-cli/edge \
  -e "import('@floating-ui/utils/dom').then(m=>console.log('loaded', Object.keys(m).length)).catch(e=>console.log('error', e))"
```

Observed result after the fix:

```
loaded 20
```

Then rerun the server and request /:

```
cd ~/src/dev/stackmachine.com
~/src/dev/edgejs/build-edge-quickjs-cli/edge ./dist/server/entry.mjs
```

Expected result for this issue: the `@floating-ui/utils/dom` module loads, or the failure is narrowed to a different concrete module/runtime behavior and captured in a follow-up note.

Observed route result after the fix: the Floating UI error disappears and route rendering reaches a later module resolution failure:

```
ReferenceError: could not load module 'react-remove-scroll-bar/constants'
```

The later failure is captured in `007_react_remove_scroll_bar_constants.md`.

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/development/troubleshooting/astro-ssr/007_react_remove_scroll_bar_constants.md

82 Astro SSR: React Remove Scroll Bar Constants Subpath

		Remarks
Status	[done]	Fixed in QuickJS package subpath directory entry resolution.
Severity	High	The Astro route could not render until this package subpath resolved correctly.

82.1 Issue

After the @floating-ui/utils/dom package exports subpath fix, the Astro SSR server on stackmachine.com starts and the / route advances to a new module load failure:

```
13:16:26 [ERROR] [router] Error while trying to render the route /
13:16:26 [ERROR] ReferenceError: could not load module 'react-remove-scroll-bar/constants'
    at runMicrotasks (native)
    at processTicksAndRejections (<input>:105:5)
```

Focused route validation was run against a temporary server on port 4322 because port 4321 was already occupied by another server instance.

82.2 Diagnosis

This is separate from the Floating UI issue: the focused @floating-ui/utils/dom import now loads successfully, and the route reaches a later package subpath.

The failing specifier is:

react-remove-scroll-bar/constants

The package publishes constants as a subdirectory with its own package.json:

```
{
  "main": "../dist/es5/constants.js",
  "module": "../dist/es2015/constants.js"
}
```

Native CommonJS resolution accepts this shape:

```
require.resolve('react-remove-scroll-bar/constants')
=> ../node_modules/react-remove-scroll-bar/dist/es5/constants.js
```

Native ESM rejects the same bare directory import with ERR_UNSUPPORTED_DIR_IMPORT, so this is a CommonJS-compatible package subpath case rather than strict ESM package resolution.

The QuickJS resolver already had permissive CommonJS-style package resolution for bare package entries and subpaths, but its directory handling only tried index files. It did not inspect a package subpath directory's own package.json, so it missed this published entrypoint.

82.3 Current Status

Updated the former QuickJS C++ package resolver so TryResolvePackageSubpath(...) tried a subpath directory's own package entry metadata when parent package exports did not resolve the subpath. This kept the compatibility fallback narrow:

- parent package exports still win first;
- only actual directory subpaths get the nested package entry fallback;
- normal file and index fallback behavior remains unchanged.

Later cleanup removed the remaining QuickJS C++ CommonJS facade/module-loader support, so this note is historical context rather than a pointer to live code.

82.4 Status Notes

Investigated with narrow import checks before changing runtime code:

- inspected react-remove-scroll-bar package metadata and files in ~/src/dev/stackmachine.com/node_modules
- ran a focused QuickJS Edge import check for react-remove-scroll-bar/constants;
- compared against native Node CommonJS and ESM resolution;
- updated the QuickJS resolver for the CommonJS-compatible subpath directory entrypoint case.

82.5 Constraints

- Do not modify the Astro app, node_modules, or generated dist files.
- Keep any fix in EdgeJS/QuickJS runtime code if a runtime fix is required.
- Fix one behavior at a time and rerun the focused import before rerendering the Astro route.

82.6 Validation

Focused import check:

```
cd ~/src/dev/stackmachine.com
~/src/dev/edgejs/build-edge-quickjs-cli/edge \
  -e "import('react-remove-scroll-bar/constants').then(m=>console.log('loaded', Object.keys(m).length))"
```

Then rerun the server and request /:

```
cd ~/src/dev/stackmachine.com
PORT=4322 ~/src/dev/edgejs/build-edge-quickjs-cli/edge ./dist/server/entry.mjs
curl -i http://localhost:4322/
```

Expected result for this issue: the react-remove-scroll-bar/constants module loads, or the failure is narrowed to a different concrete module/runtime behavior and captured in a follow-up note.

Observed focused import result after the fix:

```
loaded 4
string string
```

With module tracing enabled, the resolver selected:

```
~/src/dev/stackmachine.com/node_modules/react-remove-scroll-bar/dist/es2015/constants.js
```

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/development/troubleshooting/astro-ssr/008_zustand_ind_create_export.md

83 Astro SSR: Zustand Ind Create Export

		Remarks
Status	[done]	Fixed in QuickJS package exports condition and wildcard resolution.
Severity	High	The Astro route could not link because Zustand resolved to a module without the required named export.

83.1 Issue

After the react-remove-scroll-bar/constants package subpath directory fix, the Astro SSR server on stackmachine.com starts and the / route advances to a new module linking failure:

```
13:20:43 [ERROR] [router] Error while trying to render the route /
13:20:43 [ERROR] SyntaxError: Could not find export 'create' in module '~/src/dev/stackmachine.com'
    at runMicrotasks (native)
    at processTicksAndRejections (<input>:105:5)
```

Focused route validation was run against a temporary server on port 4322 because port 4321 was already occupied by another server instance.

83.2 Diagnosis

This is separate from the scroll-bar constants issue: the focused react-remove-scroll-bar/constants import now loads successfully, and the route reaches a later module export mismatch.

The failing resolved module path is:

~/src/dev/stackmachine.com/node_modules/zustand/ind

Zustand's package metadata exposes the ESM entry via nested package exports conditions:

```
".": {
  "import": {
    "types": "./esm/index.d.mts",
    "default": "./esm/index.mjs"
  }
},
"./*": {
  "import": {
    "types": "./esm/*.d.mts",
    "default": "./esm/*.mjs"
  }
}
```

Native ESM resolves import('zustand') to:

file:///~/src/dev/stackmachine.com/node_modules/zustand/esm/index.mjs

Before this fix, QuickJS resolved the same import to:

~/src/dev/stackmachine.com/node_modules/zustand/index.js

That CommonJS file re-exports values dynamically through `Object.keys(...).forEach(...)`, a pattern the synthetic named export scanner does not currently declare. The missing create export was therefore caused by resolving the package to its CommonJS fallback instead of the ESM import target.

83.3 Current Status

Updated the former QuickJS C++ package exports scanner so it handled the Zustand export shape:

- when a condition value is an object, choose its nested default runtime target instead of treating nested metadata keys like types as targets;
- resolve the root `"."` package export before falling back to `main`;
- support the simple wildcard export key `"./"` and substitute the requested subpath into targets such as `"./esm/*.mjs"`.

This keeps the fix focused on published package exports metadata rather than adding synthetic named export support for broader dynamic CommonJS patterns.

Later cleanup removed the remaining QuickJS C++ CommonJS facade/module-loader support, so this note is historical context rather than a pointer to live code.

The considered causes were:

- truncated or extensionless module resolution;
- CommonJS synthetic named export discovery missing a re-export pattern;
- ESM facade generation for a CommonJS entrypoint;
- or a genuinely invalid import emitted by the app build.

The actual cause was package export condition/wildcard resolution selecting the CommonJS fallback.

83.4 Status Notes

Investigated with the narrowest checks before changing runtime code:

- inspected Zustand package metadata and files in `~/src/dev/stackmachine.com/node_modules`;
- reproduced the issue with focused `import('zustand')` and `import('zustand/react')` probes;
- compared against native Node ESM resolution for the same specifier and import shape;
- updated the QuickJS resolver only because the package exposes a valid Node-compatible path/export shape that QuickJS misses.

83.5 Constraints

- Do not modify the Astro app, `node_modules`, or generated dist files.
- Keep any fix in EdgeJS/QuickJS runtime code if a runtime fix is required.
- Fix one behavior at a time and rerun the focused import before rerendering the Astro route.

83.6 Validation

Focused import check:

```
cd ~/src/dev/stackmachine.com
EDGE_TRACE_QUICKJS_MODULES=1 \
  ~/src/dev/edgejs/build-edge-quickjs-cli/edge \
  -e "import('zustand').then(m=>{ console.log('keys', Object.keys(m).join(',')); console.log('cr
```

Observed result after the fix:

```
quickjs-module normalize ... spec=zustand -> ~/src/dev/stackmachine.com/node_modules/zustand/esm
quickjs-module normalize ... spec=zustand/vanilla -> ~/src/dev/stackmachine.com/node_modules/zust
quickjs-module normalize ... spec=zustand/react -> ~/src/dev/stackmachine.com/node_modules/zustan
```



```
keys create,createStore,useStore  
create function
```

Regression checks for the two previous module resolution fixes still pass:

```
@floating-ui/utils/dom -> .../node_modules/@floating-ui/utils/dist/floating-ui.utils.dom.mjs  
react-remove-scroll-bar/constants -> .../node_modules/react-remove-scroll-bar/dist/es2015/constants.mjs
```

Then rerun the server and request /:

```
cd ~/src/dev/stackmachine.com  
PORT=4322 ~/src/dev/edgejs/build-edge-quickjs-cli/edge ./dist/server/entry.mjs  
curl -i http://localhost:4322/
```

Expected result for this issue: the module exposes create, or the failure is narrowed to a different concrete module/runtime behavior and captured in a follow-up note.

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/development/troubleshooting/astro-ssr/009_zustand_esm_default_export.md

84 Astro SSR: Zustand ESM Default Export

		Remarks
Status	[done]	Fixed in QuickJS module path canonicalization for pnpm symlinks.
Severity	High	The Astro route linked the wrong package instance until symlinked module paths were canonicalized.

84.1 Issue

After the Zustand package exports condition/wildcard fix, the Astro SSR server on stackmachine.com starts and the / route advances to a new module linking failure:

```
13:24:35 [ERROR] [router] Error while trying to render the route /
13:24:35 [ERROR] SyntaxError: Could not find export 'default' in module '~/src/dev/stackmachine.com'
    at runMicrotasks (native)
    at processTicksAndRejections (<input>:105:5)
```

Focused route validation was run against a temporary server on port 4322 because port 4321 was already occupied by another server instance.

84.2 Diagnosis

This is separate from the earlier missing create export: after the 008 fix, focused import('zustand') resolves to the ESM entry and exposes create.

The new failing resolved module path is:

```
~/src/dev/stackmachine.com/node_modules/zustand/esm
```

The import site that exposed the failure is in @react-three/fiber:

```
import create from 'zustand';
```

The app uses a pnpm-style dependency layout where node_modules/@react-three/fiber is a symlink into .pnpm/.... Native Node resolves the module filename through that symlink before resolving nested dependencies, so @react-three/fiber finds its compatible Zustand version:

```
.../node_modules/.pnpm/zustand@3.7.2_react@18.3.1/node_modules/zustand/esm/index.mjs
```

Before this fix, QuickJS preserved the symlinked module path:

```
.../node_modules/@react-three/fiber/dist/react-three-fiber.esm.js
```

Its nested import 'zustand' then climbed the wrong node_modules tree and selected top-level Zustand 5:

```
.../node_modules/zustand/esm/index.mjs
```

Zustand 5 has named exports such as create but no default export, so the default import from @react-three/fiber failed during module linking.

84.3 Current Status

Updated the former QuickJS C++ module path helpers so resolved module filenames were canonicalized with `std::filesystem::weakly_canonical(...)` before they were returned to QuickJS. That made later relative and package resolution use the real pnpm package path, matching Node's dependency lookup behavior for symlinked packages.

Later cleanup removed the remaining QuickJS C++ CommonJS facade/module-loader support, so this note is historical context rather than a pointer to live code.

The considered causes were:

- an invalid generated import that expects a default export where none exists;
- directory resolution selecting `esm/index.mjs` for a specifier Node rejects;
- or an import-facade/export declaration mismatch in QuickJS.

The actual cause was non-canonical symlink paths causing package resolution to choose the wrong installed dependency version.

84.4 Status Notes

Investigated with narrow checks before changing runtime code:

- found the dependency import site that requests default from zustand;
- inspected top-level Zustand 5 and the compatible dependency-scoped Zustand 3 package;
- compared native Node and QuickJS behavior for `import('@react-three/fiber')`;
- updated runtime code because the app is using a valid Node-compatible resolution/export shape that QuickJS mishandles.

84.5 Constraints

- Do not modify the Astro app, `node_modules`, or generated dist files.
- Keep any fix in EdgeJS/QuickJS runtime code if a runtime fix is required.
- Fix one behavior at a time and rerun the focused import before rerendering the Astro route.

84.6 Validation

Focused import check:

```
cd ~/src/dev/stackmachine.com
EDGE_TRACE_QUICKJS_MODULES=1 \
  ~/src/dev/edgejs/build-edge-quickjs-cli/edge \
  -e "import('@react-three/fiber').then(m=>console.log('fiber loaded', Object.keys(m).length))"
```

Observed result after the fix:

```
quickjs-module normalize ... spec=@react-three/fiber -> .../node_modules/.pnpm/@react-three+fiber
quickjs-module normalize ... spec=zustand -> .../node_modules/.pnpm/zustand@3.7.2_react@18.3.1/n
fiber loaded 31
```

Then rerun the server and request /:

```
cd ~/src/dev/stackmachine.com
PORT=4322 ~/src/dev/edgejs/build-edge-quickjs-cli/edge ./dist/server/entry.mjs
curl -i http://localhost:4322/
```

Expected result for this issue: the default export mismatch is explained or fixed, or the failure is narrowed to a different concrete module/runtime behavior and captured in a follow-up note.

85 Astro SSR: Use Gesture Controller Export

		Remarks
Status	[done]	Fixed in QuickJS package exports condition preference.
Severity	High	The Astro route could not link until the ESM module target was preferred over the CommonJS default.

85.1 Issue

After the pnpm symlink canonicalization fix, the Astro SSR server on stackmachine.com starts and the / route advances to a new module linking failure:

```
13:28:38 [ERROR] [router] Error while trying to render the route /
13:28:38 [ERROR] SyntaxError: Could not find export 'Controller' in module '~/src/dev/stackmachine.com'
    at runMicrotasks (native)
    at processTicksAndRejections (<input>:105:5)
```

Focused route validation was run against a temporary server on port 4322 because port 4321 was already occupied by another server instance.

85.2 Diagnosis

This is separate from the Zustand default export issue: focused @react-three/fiber imports now resolve through pnpm real paths and load.

The displayed module path was truncated at:

```
~/src/dev/stackmachine.com/node_modules/.pnpm/@use-
```

The full package behind the path is @use-gesture/core. The import site that exposed the failure is in @use-gesture/react:

```
import { Controller, parseMergedHandlers } from '@use-gesture/core';
```

@use-gesture/core publishes package exports with both module and default conditions:

```
": {
  "module": "./dist/use-gesture-core.esm.js",
  "default": "./dist/use-gesture-core.cjs.js"
}
```

Before this fix, QuickJS preferred default before module, resolving the ESM import to the CommonJS wrapper:

```
.../node_modules/@use-gesture/core/dist/use-gesture-core.cjs.js
```

That CommonJS wrapper then required environment-specific files and did not provide the ESM export declarations expected by the importing ESM module. Native Node also resolves this package to the CJS default because the package does not declare an import condition, but this Astro/Vite SSR bundle uses the package's legacy module condition as the ESM target.

85.3 Current Status

Updated the former QuickJS C++ package exports scanner so it preferred runtime targets in this order:

`import`, `module`, `default`

The same order is used for legacy package entry candidates. This keeps the runtime compatible with bundler-oriented packages that expose ESM through `module` without an explicit `import` condition.

Later cleanup removed the remaining QuickJS C++ CommonJS facade/module-loader support, so this note is historical context rather than a pointer to live code.

85.4 Status Notes

Investigated with narrow checks before changing runtime code:

- searched the pnpm dependency tree for modules exporting or importing `Controller`;
- identified `@use-gesture/core` behind the truncated `@use-...` path;
- compared native Node and QuickJS import behavior for the exact package;
- updated runtime package condition preference for the bundler-compatible module export shape.

85.5 Constraints

- Do not modify the Astro app, `node_modules`, or generated dist files.
- Keep any fix in EdgeJS/QuickJS runtime code if a runtime fix is required.
- Fix one behavior at a time and rerun the focused import before rerendering the Astro route.

85.6 Validation

Focused import check:

```
cd ~/src/dev/stackmachine.com
EDGE_TRACE_QUICKJS_MODULES=1 \
  ~/src/dev/edgejs/build-edge-quickjs-cli/edge \
  -e "import('@use-gesture/core').then(m=>{ console.log('core', Object.keys(m).join(',')); console.log('module', m.module); console.log('default', m.default); })"
```

Observed result after the fix:

```
quickjs-module normalize ... spec=@use-gesture/core -> .../node_modules/@use-gesture/core/dist/use-gesture-core
core Controller,parseMergedHandlers
Controller function
```

`import('@use-gesture/react')` also loads and resolves `@use-gesture/core/actions`, `@use-gesture/core/utils`, and `@use-gesture/core/types` to their ESM module targets.

Then rerun the server and request `/`:

```
cd ~/src/dev/stackmachine.com
PORT=4322 ~/src/dev/edgejs/build-edge-quickjs-cli/edge ./dist/server/entry.mjs
curl -i http://localhost:4322/
```

Expected result for this issue: the `Controller` export mismatch is explained or fixed, or the failure is narrowed to a different concrete module/runtime behavior and captured in a follow-up note.

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/development/troubleshooting/astro-ssr/011_route_stack_overflow.md

86 Astro SSR: Route Stack Overflow

		Remarks
Status	[done]	Fixed with a larger QuickJS runtime stack guard.
Severity	High	The Astro route streamed a partial response and failed without the larger stack guard.

86.1 Issue

After the @use-gesture/core package export condition fix, the Astro SSR server on stackmachine.com starts and the / route advances to a runtime failure:

```
13:31:35 [ERROR] [router] Error while trying to render the route /
13:31:35 [ERROR] Maximum call stack size exceeded
```

Focused route validation was run against temporary servers on ports 4322 and 4323 because port 4321 was already occupied by another server instance.

86.2 Diagnosis

This is separate from the previous module linking failures: route rendering got past the missing named/default exports and reached an actual runtime stack overflow.

The first focused reproduction showed that importing the page module failed before rendering:

```
cd ~/src/dev/stackmachine.com
~/src/dev/edgejs/build-edge-quickjs-cli/edge -e "import('./dist/server/pages/index.astro.mjs')"
```

The failure narrowed to the page's stackmachine dependency. That package loads relay-runtime, whose large CommonJS dependency graph exceeded QuickJS's default 1 MiB stack guard during module evaluation.

Raising the guard to 2 MiB fixed relay-runtime, stackmachine, and the page module import, but the full Astro render still overflowed in the middle of the HTML stream. Raising the Edge-created QuickJS runtime guard to 4 MiB allowed the full route render to complete.

This is not an infinite recursion in the app or in package resolution. It is a compatibility difference between the default QuickJS stack guard and the depth of modern framework/package evaluation plus React SSR rendering.

86.3 Status Notes

The runtime fix is intentionally narrow:

- keep the stack guard enabled;
- increase the default stack size only for Edge-created QuickJS environments;
- do not modify the Astro app, generated dist, or node_modules;
- keep the package-resolution fixes from previous notes unchanged.

Implemented in:

napi/quickjs/src/unofficial_napi.cc

The default Edge QuickJS stack guard is now $4 * 1024 * 1024$ bytes.

86.4 Constraints

- Do not modify the Astro app, node_modules, or generated dist files.
- Keep any fix in EdgeJS/QuickJS runtime code if a runtime fix is required.
- Fix one behavior at a time and rerun the focused reproduction before rerendering the full Astro route.

86.5 Validation

Focused checks after the fix:

```
cd ~/src/dev/stackmachine.com
```

```
~/src/dev/edgejs/build-edge-quickjs-cli/edge -e "const r=require('relay-runtime'); console.log('r',r)"
```

Result:

```
require relay 93
```

```
~/src/dev/edgejs/build-edge-quickjs-cli/edge -e "import('relay-runtime').then(m=>console.log('imp',m))"
```

Result:

```
import relay 104
```

```
~/src/dev/edgejs/build-edge-quickjs-cli/edge -e "import('stackmachine').then(m=>console.log('sta',m))"
```

Result:

```
stackmachine StackMachine,createZip
```

```
~/src/dev/edgejs/build-edge-quickjs-cli/edge -e "import('./dist/server/pages/index.astro.mjs').then(m=>console.log('ind',m))"
```

Result:

```
page page,renderers
```

Then rerun the server and request /:

```
cd ~/src/dev/stackmachine.com
```

```
PORT=4323 ~/src/dev/edgejs/build-edge-quickjs-cli/edge ./dist/server/entry.mjs
```

```
curl -i http://localhost:4323/
```

Result:

```
HTTP/1.1 200 OK
```

```
content-type: text/html
```

```
...
```

```
<!DOCTYPE html><html lang="en">
```

The response completed successfully and the server emitted no follow-up render error.

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/development/troubleshooting/astro-ssr/012_wasix_pnpm_symlink_resolution.md

87 Astro SSR: WASIX pnpm Symlink Resolution

		Remarks
Status	[done]	Fixed by shared QuickJS module path resolution and fs stat symlink fallback.
Severity	High	The Astro standalone server cannot start in Wasmer when react cannot resolve from /app/node_modules.

87.1 Issue

Running the stackmachine.com Astro standalone server through the Wasmer package fails before the server starts:

```
cd ~/src/dev/stackmachine.com
wasmer run --net .
```

Observed failure:

ReferenceError: could not load module 'react'

The failing import is in the generated Astro renderer:

```
import React__default, { createElement } from 'react';
```

87.2 Diagnosis

Native QuickJS can import react from the app:

```
cd ~/src/dev/stackmachine.com
~/src/dev/edgejs/build-edge-quickjs-cli/edge \
-e "import('react').then(m=>console.log(Object.keys(m).length))"
```

The same app mounted at /app under Wasmer can see the pnpm symlink through the JavaScript fs API:

```
/app/node_modules/react lstat true false false
link .pnpm/react@18.3.1/node_modules/react
/app/node_modules/react/package.json lstat false false true
```

But the QuickJS C++ module normalizer misses the bare package:

```
quickjs-module normalize base=file:///app/dist/server/entry.mjs spec=./renderers.mjs -> /app/dist
quickjs-module normalize-miss base=/app/dist/server/renderers.mjs spec=react
```

This points at the C++ resolver's std::filesystem checks across pnpm symlink path components inside a Wasmer-mounted directory. The runtime should resolve symlink components before checking candidate package files, so /app/node_modules/react/index.js is tested through its real pnpm store path.

87.3 Status Notes

- Keep the fix in the QuickJS runtime module resolver.
- Add a small path helper that follows symlink components lexically before TryResolveAsFile(...) checks candidates.

- Preserve native behavior and the existing package exports condition order.
- Do not modify the Astro app, node_modules, or generated dist files.

87.4 Validation

Focused Wasmer checks:

```
cd /private/tmp/edge-wasmer-probe
wasmer run . -- -e "import('/app/dist/server/renderers.mjs').then(()=>console.log('renderers ok'))"
wasmer run --net . -- /app/dist/server/entry.mjs
```

Observed after the fix:

```
renderers ok
/app/node_modules/.pnpm/react@18.3.1/node_modules/react/index.js
[@astrojs/node] Server listening on http://127.0.0.1:3311
```

The targeted CJS peer dependency probe now resolves react from the real pnpm package path when the parent module is under react-dom, and the generated dist/server/renderers.mjs import succeeds under Wasmer. The standalone server advances past this issue and reaches the next route-rendering failure captured in 013_lucide_react_chevrondown_export.md.

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/development/troubleshooting/astro-ssr/013_lucide_react_chevrondown_export.md

88 Astro SSR: Lucide React ChevronDown Export

		Remarks
Status	[done]	Fixed by precise package "type": "module" detection.
Severity	High	The Astro standalone server starts, but rendering / fails before a response can be produced.

88.1 Issue

After the WASIX pnpm symlink resolution fix, the stackmachine.com Astro standalone server starts under Wasmer:

```
cd ~/src/dev/stackmachine.com
wasmer run --net .
```

Observed route-rendering failure:

```
[@astrojs/node] Server listening on http://localhost:4321
[ERROR] [router] Error while trying to render the route /
[ERROR] SyntaxError: Could not find export 'ChevronDown' in module
'/app/node_modules/.pnpm/lucide-react@0.462.0_react@18.3.1/node_'
```

88.2 Diagnosis

The generated Astro route imports named icons from lucide-react, including ChevronDown. The package exposes that name from its CommonJS entry:

```
lucide-react/dist/cjs/lucide-react.js
exports.ChevronDown = ChevronDown;
```

Focused checks showed that the package itself can be required through createRequire(...) and exposes thousands of named properties at runtime.

The failing path was:

```
/app/node_modules/.pnpm/lucide-react@0.462.0_react@18.3.1/node_modules/lucide-react/dist/cjs/lucide-react.js
```

QuickJS incorrectly classified that .js file as ESM because FileLooksCommonJs(...) used a substring check against the nearest package.json: if the file contained both "type" and "module" anywhere, it treated the package as ESM. lucide-react/package.json has "repository": { "type": "git" } and a top-level "module" entry, but it does not have top-level "type": "module".

That made QuickJS compile the CommonJS bundle as ESM. The static linker then could not see exports.ChevronDown = ..., so route linking failed before the CommonJS runtime body could execute.

88.3 Status Notes

- Reproduced the failure with a focused import of the generated layout chunk.
- Confirmed the resolver targets lucide-react/dist/cjs/lucide-react.js.
- Confirmed the former CommonJS export scanner handled exports.ChevronDown = ...; the bad behavior was earlier CJS/ESM classification.

- Moved package type detection to the shared parsed package metadata helper so only top-level "type": "module" affects .js classification.
- Kept the fix in the QuickJS runtime; no app, node_modules, or generated Astro output changes were needed.

88.4 Validation

Focused checks:

```
cd /private/tmp/edge-wasmer-probe
wasmer run . -- -e "import('/app/dist/server/chunks/Layout_BoMatPJA.mjs').then(()=>console.log(''
```

```
cd ~/src/dev/stackmachine.com
wasmer run --net .
```

Expected result: lucide-react named exports link successfully, and requesting / advances past the Chevron-Down missing export.

Observed after the fix:

```
object
layout ok
200 text/html
```

Validation commands run:

```
cmake --build build-edge-quickjs-cli --target edge -j4
cd ~/src/dev/edgejs/quickjs-wasm/ && ./build.sh
wasmer run . -- -e "import { ChevronDown } from '/app/node_modules/.pnpm/lucide-react@0.462.0_rea
wasmer run . -- -e "import('/app/dist/server/chunks/Layout_BoMatPJA.mjs').then(()=>console.log('
wasmer run --net --env PORT=3311 --env HOST=127.0.0.1 .
```

The final app-level request to http://127.0.0.1:3311/ returned 200 text/html.

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/development/troubleshooting/astro-ssr/014_pnpm_deploy_externalized_runtime_links.md

89 Astro SSR: pnpm Deploy Externalized Runtime Links

		Remarks
Status	[done]	Fixed in the app deploy preparation step; later Wasmer deploy packaging work materialized the runtime package graph into a symlink-free .deploy artifact. The full app works from the source tree, but the pruned .deploy artifact failed at startup without addressable runtime packages.
Severity	Medium	

89.1 Issue

The stackmachine.com Astro SSR app can run under Wasmer from the full project tree, but the smaller .deploy directory prepared with `pnpm deploy --prod` failed immediately:

```
cd ~/src/dev/stackmachine.com
npm run edge:prepare-deploy
cd .deploy
wasmer run --net .
```

Observed failure:

```
file:///app/dist/server/entry.mjs:1
import { renderers } from './renderers.mjs';
```

ReferenceError: could not load module 'piccolore'

89.2 Diagnosis

The missing `piccolore` package exists inside the pnpm virtual store in the deploy artifact:

```
.deploy/node_modules/.pnpm/piccolore@0.1.3/node_modules/piccolore
```

However, `pnpm deploy --prod --legacy` only materializes top-level `node_modules` links for packages that are reachable as deployed package dependencies. Astro's generated server chunks still contain bare runtime imports for packages that were externalized into `dist/server`, including `piccolore`.

That means the runtime starts resolving from `/app/dist/server/...`, walks up to `/app/node_modules`, and expects a top-level link:

```
/app/node_modules/piccolore
```

The package was present in the virtual store but not addressable from the standard Node-style package lookup path. This differs from the earlier WASIX pnpm symlink issue: QuickJS symlink and package resolution were already working once a top-level package entry existed.

The first generated top-level `piccolore` link still failed because it pointed at Astro's dependency symlink:

```
.pnpm/astro@.../node_modules/piccolore
```

Rebuilding the link against the resolved package directory fixed that part:

```
.pnpm/piccolore@0.1.3/node_modules/piccolore
```

After the server started, the first route render exposed a second packaging issue: `graphql-ws` imported `graphql` at runtime, but `graphql` was listed only under `app devDependencies`. Since `pnpm deploy --prod --legacy` correctly omits dev dependencies, the deploy artifact did not include `graphql`.

89.3 Status Notes

- Keep the standard `pnpm deploy` flow so the artifact remains much smaller than the full project tree.
- Copy the `Astro dist`, `wasmer.toml`, optional `app.yaml`, and deployed production `node_modules` into `.deploy`.
- Scan `.deploy/dist/server` for bare `import`, `dynamic import(...)`, and `require(...)` specifiers.
- For each bare server-runtime package that is absent from `.deploy/node_modules`, find the matching package under the `pnpm` virtual store and make it addressable from the deploy root `node_modules`.
- Move packages needed by runtime peers into production dependencies so `pnpm deploy --prod` includes them normally.
- Treat this as app deploy packaging glue rather than an Edge QuickJS runtime change.

89.4 Current Status

`~/src/dev/stackmachine.com/scripts/prepare-edge-deploy.cjs` first fixed this local `.deploy` failure by building the `Astro` app, running `pnpm deploy --prod --legacy`, assembling `.deploy`, and making externalized runtime packages from the generated server bundle addressable at the deploy root `node_modules`.

`graphql` was moved from `devDependencies` to `dependencies` in `~/src/dev/stackmachine.com/package.json`, because it is imported at runtime through `stackmachine / graphql-ws`.

The first fixed `prepare` run added access to:

```
@oslojs/encoding, cookie, cssesc, destr, devalue, es-module-lexer,  
html-escaper, mrmime, piccolore, send, server-destroy, sharp
```

That symlink-based local fix was later superseded by `../wasmer-deploy/001_pnpm_directory_symlinks_webc.md`, after `wasmer deploy` showed that the cloud package path can lose `pnpm` directory symlink contents. The current deploy preparation step builds a runtime package closure, prunes unimported packages, materializes package links as real directories, removes `.pnpm`, rewrites virtual-store source imports, and asserts the final artifact contains no symlinks or nested module stores.

The current deploy directory is:

```
347M    ~/src/dev/stackmachine.com/.deploy
```

89.5 Validation

Prepare command:

```
cd ~/src/dev/stackmachine.com  
npm run edge:prepare-deploy
```

Runtime check:

```
cd ~/src/dev/stackmachine.com/.deploy  
wasmer run --net --env PORT=3311 --env HOST=127.0.0.1 .  
curl -i http://127.0.0.1:3311/
```

Observed result:

```
[@astrojs/node] Server listening on http://127.0.0.1:3311  
HTTP/1.1 200 OK  
content-type: text/html
```

The generated HTML rendered the `StackMachine` landing page from the `.deploy` artifact.

The later flattened artifact also passed the Wasmer deploy packaging invariants: no symlinks, only one root `node_modules`, no `.pnpm` directory, and no source imports targeting `node_modules/.pnpm/.../node_modules/...`.

90 Vite App

91 Vite App Standalone Build Findings

		Remarks
Status	[caveated]	Findings captured for the plain Vite standalone build shape.
Severity	Low	This is an architecture note; the current custom static server path remains viable.

91.1 Query

Can a plain Vite app use a standalone build shape like the Astro app, so the app does not provide its own `server/router.cjs` and the server entry is produced as part of the standard build?

91.2 Summary

Not in the same way as the Astro app by default, but `vite-plugin-standalone` is a relevant candidate if the Vite app has, or is given, a server entrypoint.

The Astro app can use this shape because it is configured for Astro server output with the Node adapter in standalone mode:

```
export default defineConfig({
  output: 'server',
  adapter: node({
    mode: 'standalone',
  }),
});
```

That build emits a server entry under `dist/server/entry.mjs`. The local EdgeJS compatibility step then bundles that emitted server entry into `dist/server/entry.cjs`, which Wasmer can execute through Edge QuickJS.

The Vite app is a plain SPA build. Its standard build is only:

```
vite build
```

That emits static client files under `dist`; it does not emit an HTTP server entrypoint equivalent to Astro's standalone `dist/server/entry.mjs`.

There is a third-party plugin, `vite-plugin-standalone`, whose README describes it as a Vite plugin that uses `@vercel/nft` and `esbuild` to create a standalone server build so deployment can copy only the build folder rather than `node_modules`. The plugin supports an explicit entry option, including multiple named entries:

```
standalone({
  entry: {
    index: './server/index.ts',
    worker: './server/worker.js',
  },
});
```

The plugin source applies only to SSR builds (`env.isSsrBuild`), then finds the Rollup bundle entries, rebundles them with `esbuild` for Node, and traces/copies native or external dependencies with `@vercel/nft`. In other words, it packages a server entry; it does not remove the need for server semantics to exist somewhere.

Its examples also point in that direction:

- the Rakkas example uses `standalone()` alongside the Rakkas Vite plugin;
- the Vike example uses a server entry at `./src/server/index.ts` and `viteNode({ entry: './src/server/index.ts', standalone: true })`;
- that Vike server entry creates an Express app and listens on `PORT`.

For this reason, if the Vite app remains a plain SPA, its `server/router.cjs` is currently real runtime plumbing, not incidental project code. It provides the HTTP/static adapter that Vite's SPA build does not generate:

- serve files from `/dist`;
- support GET and HEAD;
- reject path traversal;
- map directories to `index.html`;
- fall back to `/dist/index.html` for client-side routes;
- send explicit content types and cache headers.

91.3 Options For Next Work

1. Keep an app-owned `server/router.cjs`. This is the simplest shape and matches the current Vite static SPA adapter notes for Edge QuickJS WASIX.
2. Generate or bundle `server/router.cjs` during the app build. This can remove hand-maintained router code from the app, but it would still be an EdgeJS adapter/template rather than standard Vite output.
3. Try `vite-plugin-standalone` with an explicit minimal server entry. This may let the Vite app produce a build-contained server artifact, but it still needs compatibility testing under Edge QuickJS WASIX. The default plugin output is ESM and Node-targeted, and examples use Express/framework server code, so it may need an esbuild format/config adjustment or a small CJS wrapper for the current Edge QuickJS execution path.
4. Move the app to a framework/runtime that emits a standalone server adapter. Astro SSR, Rakkas, or Vike are examples. This is a larger migration and changes the app architecture.
5. Add a reusable EdgeJS static SPA adapter package or runtime entry. This is likely the cleanest longer-term direction: Vite apps could point at a shared adapter instead of carrying their own `server/router.cjs`.

91.4 Conclusion

It is possible to remove app-owned `server/router.cjs`, but not by flipping a Vite `standalone` option. Plain Vite does not currently provide an Astro-style standalone HTTP server entry as part of `vite build`.

`vite-plugin-standalone` is worth a focused experiment. The experiment should answer whether a minimal static SPA HTTP entry can be bundled into a build-owned artifact that Edge QuickJS WASIX can execute. If that works, the app can avoid carrying a bespoke router file. If it does not, the best next story is probably a reusable EdgeJS static SPA adapter so Vite apps can keep the standard `vite build` output while avoiding per-app router copies.

92 Next.js

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/development/troubleshooting/next-app/001_standalone_v8_serdes.md

93 Next App Standalone: require("v8") / Serdes Findings

		Remarks
Status	[done]	QuickJS serdes constructors implemented and verified.
Severity	High	The Next standalone server cannot start until the v8 serdes surface is handled.

93.1 Context

The next-app project was switched to the standard Next.js standalone output:

- next.config.ts uses output: "standalone".
- The standalone build emits .next/standalone/server.js.
- Node can run the standalone server directly.
- The old custom server/router scripts are no longer part of the intended path.

The failing Edge QuickJS repro copied the standalone output into a test folder:

```
cp -rf ./next/standalone ./testing/app/
cd ./testing
~/src/dev/edgejs/build-edge-quickjs-cli/edge ./app/server.js
```

The failure looked like:

```
undefined[Error: Failed to execute builtin 'internal/main/run_main_module':
  at normalizeRequirableId (<input>:302:35)
  at <anonymous> (<input>:1367:43)
  ...
]
```

Node.js v24.13.2

93.2 Reduction

The standalone server failure reduces to loading Next's normal startup module:

```
require("next/dist/server/lib/start-server")
```

Tracing module resolution showed the failing dependency was the public builtin:

```
require("v8")
```

The smaller reproducer is:

```
~/src/dev/edgejs/build-edge-quickjs-cli/edge /private/tmp/edge-repro-v8.js
```

with /private/tmp/edge-repro-v8.js containing:

```
require("v8");
```

93.3 LLDB Findings

Breaking at `TakePendingExceptionInfo` only showed Edge's final wrapped top-level exception:

Error: Failed to execute builtin 'internal/main/run_main_module'

Breaking earlier in `DescribeAndClearPendingException(...)`, before the native main-builtin wrapper rewrites the error, revealed the original exception:

`TypeError: cannot read property 'prototype' of undefined`

The source location corresponds to `lib/v8.js`:

```
Serializer.prototype._getDataCloneError = Error;
```

So `require("v8")` is not failing because the v8 builtin is missing. It is failing because:

- `lib/v8.js` loads `internalBinding("serdes")`.
- It destructures `{ Serializer, Deserializer }` from that binding.
- On QuickJS, `Serializer` is currently undefined.
- The first access to `Serializer.prototype` throws.

93.4 Code Evidence

The V8 backend creates real `serializer/deserializer` constructors in:

`napi/v8/src/unofficial_napi.cc`

around `unofficial_napi_create_serdes_binding(...)`, where it exports:

- `Serializer`
- `Deserializer`

The QuickJS backend currently returns an empty object:

`napi/quickjs/src/unofficial_napi.cc`

```
napi_status NAPI_CDECL unofficial_napi_create_serdes_binding(napi_env env,
                                                             napi_value *result_out)
{
    if (!CheckEnv(env) || result_out == nullptr)
        return napi_invalid_arg;
    return WrapOwned(env, JS_NewObject(Ctx(env)), result_out);
}
```

That empty binding explains the exact LLDB exception.

93.5 Impact On Standalone Next

Next's standalone server imports v8 from:

`.next/standalone/node_modules/next/dist/server/lib/start-server.js`

The observed usage is heap telemetry:

```
v8.getHeapStatistics()
```

So the app-level standalone build is valid and works under Node. The blocker is Edge QuickJS compatibility with Node's v8 builtin, specifically the incomplete QuickJS `serdes` internal binding.

93.6 Likely Runtime Fix

Implement a minimal QuickJS-backed `internalBinding("serdes")` that exports stable `Serializer` and `Deserializer` constructors.

For the immediate Next standalone path, it may be enough that `require("v8")` loads cleanly and `getHeapStatistics()` continues to work. Full `v8.serialize()` / `v8.deserialize()` behavior can be added using the existing QuickJS structured clone helpers based on `JS_WriteObject` / `JS_ReadObject`.

Candidate implementation area:

`napi/quickjs/src/unofficial_napi.cc`

Relevant existing helpers:

- `unofficial_napi_serialize_value(...)`
- `unofficial_napi_deserialize_value(...)`
- `JS_WriteObject(...)`
- `JS_ReadObject(...)`

93.7 Current Conclusion

The next-app standalone setup itself is correct. The failure under Edge QuickJS is caused by `require("v8")` loading `lib/v8.js`, which assumes `internalBinding("serdes").Serializer` exists. QuickJS currently returns an empty `serdes` binding, so the builtin throws before Next's standalone server can start.

93.8 Current Status

`napi/quickjs/src/unofficial_napi.cc` now exports QuickJS-backed `Serializer` and `Deserializer` constructors from `internalBinding("serdes")`. Native and WASIX smoke tests verified that `require("v8")` loads and that `v8.serialize()` / `v8.deserialize()` can round-trip a plain object.

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/development/troubleshooting/next-app/002_standalone_inspector_stub.md

94 Next App Standalone: require("inspector") Stub

		Remarks
Status	[done]	Unavailable-inspector stub implemented and verified.
Severity	High	The Next standalone server cannot start while require("inspector") throws during module load.

94.1 Context

After implementing a QuickJS-backed internalBinding("serdes"), the private-poker Next.js standalone server advanced past the previous require("v8") failure and reached a new startup error:

```
Failed to execute builtin 'internal/main/run_main_module':
undefinedError: Inspector is not available
    at NodeError (<input>:447:11)
    at anonymous (<input>:27:13)
```

The app is mounted through:

```
~/src/ImperfectProtocol/private-poker/wasmer.toml
```

which runs:

```
/app/server.js
```

from .next/standalone.

94.2 Reduction

Next 16 standalone output imports the public inspector builtin from startup logging and profiling helpers:

```
.next/standalone/node_modules/next/dist/server/lib/app-info-log.js
.next/standalone/node_modules/next/dist/server/lib/cpu-profile.js
```

The immediate startup path only needs inspector.url() to be callable and return undefined when no inspector is active.

94.3 Historical Action Plan

1. Keep internalBinding("config").hasInspector false so the runtime does not claim real inspector support.
2. Keep lib/inspector.js donor-clean and add the unavailable-inspector surface below the JavaScript library layer.
3. Make passive APIs such as url(), close(), and Network hooks no-op.
4. Keep active inspector APIs such as open(), waitForDebugger(), and Session.connect() throwing inspector-specific errors.
5. Rebuild native QuickJS and WASIX, then rerun private-poker until it reaches the next runtime blocker or starts serving.

94.4 Current Status

`src/edge_module_loader.cc` now intercepts the public `inspector` builtin in the native builtins compile hook, so `require("inspector")` and `require("node:inspector")` return the native `unavailable-inspector` fallback from `internalBinding("inspector")`, implemented in `src/internal_binding/binding_inspector.cc`, without executing donor-clean `lib/inspector.js`. Passive consumers can import the builtin and call `url()`, while active debugging APIs still report `inspector` unavailability or disconnected state.

Native smoke tests verified that `require("inspector").url()` and `require("inspector/promises").url()` return `undefined`.

May 14, 2026 local rerun with:

`PORT=3100 /Users/syrusakbary/Development/edgejs/build-edge-quickjs-cli/edge npm run start`
from `/Users/syrusakbary/Development/private-poker` advanced past `ERR_INSPECTOR_NOT_AVAILABLE` and reached a later `next.config.ts` transform failure:

```
Error: Failed to get Buffer pointer and length
  at transform (null) {
    code: 'InvalidArg'
  }
```

Earlier WASIX verification had also advanced past startup and reached the request-time stack exhaustion captured in [003_route_stack_exhausted.md](#).

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/development/troubleshooting/next-app/003_route_stack_exhausted.md

95 Next App Standalone: Route Request Stack Exhaustion

		Remarks
Status	[open]	Planned investigation after startup compatibility fixes.
Severity	High	The Next standalone server starts, but the first HTTP request crashes the Wasmer process.

95.1 Context

After fixing the QuickJS serdes binding and adding an unavailable-inspector stub, private-poker starts under Wasmer:

```
▲ Next.js 16.1.6
- Local:      http://localhost:3000
- Network:    http://0.0.0.0:3000
```

```
✓ Starting...
✓ Ready in 1303ms
```

A request to the root route then fails:

```
curl -i --max-time 10 http://127.0.0.1:3000/
```

with the client receiving:

```
curl: (52) Empty reply from server
```

and Wasmer reporting:

```
RuntimeError: call stack exhausted
```

95.2 Impact

This is past module-load compatibility and server startup. The remaining blocker is now request-time rendering or request dispatch under the Next standalone runtime.

95.3 Action Plan

1. Reproduce with the smallest route possible, starting with /, then known static asset paths, then any simple route if available.
2. Enable focused tracing if useful to find whether the exhaustion occurs in HTTP dispatch, module loading, React/Next rendering, or user app code.
3. Compare native QuickJS CLI behavior against WASIX to determine whether this is QuickJS JS stack, native/wasm stack, or request lifecycle recursion.
4. If it is a known stack-depth issue, adjust the narrow WASIX/QuickJS stack configuration rather than changing broad module semantics.
5. Rerun private-poker under Wasmer and verify at least one HTML route returns an HTTP response.

95.4 Native Versus Wasmer Samples

Two macOS sample captures from private-poker show different runtime shapes:

- ~/src/dev/edgejs/next-server.txt: native Edge QuickJS.
- ~/src/dev/edgejs/wasmer-sample.txt: wasmer run of the same app package.

The native sample is quiescent in the expected way. Its main thread is parked in `RunEventLoopUntilQuiescent(...) -> uv_run(...) -> uv__io_poll(...) -> kevent`, and libuv worker threads are blocked in `uv_cond_wait(...)`. This matches a server that has no pending CPU work while waiting for socket activity.

The Wasmer sample is not just the same libuv loop behind a WASIX boundary. It shows Wasmer/WASIX task machinery on the hot path:

- TokioTaskManager Thread Pool_thread_1
- stack_call_trampoline
- corosensei::coroutine::on_stack::wrapper
- wasmer_wasix::syscalls::wasix::sched_yield
- wasmer_wasix::syscalls::wasix::thread_sleep_internal
- wasmer_wasix::syscalls::handle_rewind_ext
- a very deep repeated sequence of unknown generated wasm frames

Other TokioTaskManager pool threads are blocked in:

```
stack_call_trampoline
corosensei::coroutine::on_stack::wrapper
wasmer_wasix::syscalls::wasix::futex_wait
wasmer_wasix::syscalls::__asyncify_with_deep_sleep
virtual_mio::waker::block_on
parking::Inner::park
_pthread_cond_wait
```

This points away from QuickJS JavaScript promise continuations as the direct stack-growth mechanism. In our Edge QuickJS layer, `async/await` is represented as QuickJS promise jobs drained by `JS_ExecutePendingJob(...)` through `unofficial_napi_process_microtasks(...)`. The N-API backend also installs QuickJS promise hooks that capture, enter, and leave async context frames around promise reaction jobs.

The coroutine names in the Wasmer sample are from Wasmer/WASIX `asyncify` and task scheduling, not from our QuickJS promise-job implementation. The suspicious pattern is that a WASIX syscall path that should suspend/yield through `asyncify` appears to keep re-entering generated wasm frames until the host reports `RuntimeError: call stack exhausted`.

95.5 Stack Size Probe

Increasing Wasmer's runtime stack changes the failure mode but does not resolve the route:

- default stack: first request returns an empty reply and Wasmer reports `RuntimeError: call stack exhausted`;
- `--stack-size 4194304`: same `call stack exhausted` on the first request;
- `--stack-size 6291456`: same `call stack exhausted` on the first request;
- `--stack-size 8388608`: the server reaches Ready, but HTTP requests hang without returning bytes.

This suggests stack size is only exposing the next symptom. The likely blocker is a WASIX coroutine/asyncify re-entry or wakeup issue around a syscall used by the Next request path, rather than a simple need for a larger QuickJS JS stack.

95.6 Updated Investigation Plan

1. Capture a focused Wasmer sample during the first hanging or overflowing HTTP request and map the generated wasm addresses if possible.

2. Reproduce outside Next with a small Edge QuickJS WASIX script that performs the likely syscall pattern: HTTP accept, read, response write, and any setImmediate / Promise continuation used by the request path.
3. Add tracing around uv_run callbacks and QuickJS microtask checkpoints to verify whether JavaScript continuations are re-entering normally or whether execution is stuck before returning from a WASIX syscall suspension.
4. If the small repro shows the same deep corosensei / handle_rewind_ext stack, treat this as a Wasmer/WASIX asyncify task scheduling issue rather than a QuickJS promise-hook issue.

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/development/troubleshooting/next-app/004_next_config_swc_options_buffer.md

96 Next config SWC options Buffer rejected by N-API

		Remarks
Status	[done]	Fixed by deriving QuickJS N-API Buffer detection from the JS Buffer prototype.
Severity	High	Blocks next start before the app can finish loading next.config.ts.

96.1 Symptom

Running the private Next.js app with the native QuickJS-backed Edge CLI now gets past `require("inspector")`, but fails while loading `next.config.ts`:

```
× Failed to load next.config.ts, see more info here https://nextjs.org/docs/messages/next-config-Error: Failed to get Buffer pointer and length
    at transform (null) {
      code: 'InvalidArg'
    }
Unhandled Rejection: undefined
```

Reproduction:

```
cd /Users/syrusakbary/Development/private-poker
PORT=3100 /Users/syrusakbary/Development/edgejs/build-edge-quickjs-cli/edge npm run start
```

A narrower reproduction is:

```
cd /Users/syrusakbary/Development/private-poker
/Users/syrusakbary/Development/edgejs/build-edge-quickjs-cli/edge -e '
(async () => {
  const { loadBindings } = require("./node_modules/next/dist/build/swc");
  const bindings = await loadBindings();
  const opts = {
    jsc: { parser: { syntax: "typescript" }, paths: { "@/*": ["./*"] }, baseUrl: process.cwd() },
    module: { type: "commonjs" },
    isModule: "unknown",
    env: { targets: { node: process.versions.node || "20.19.0" } }
  };
  await bindings.transform(require("fs").readFileSync("next.config.ts", "utf8"), opts);
})();
'
```

This fails with the same `InvalidArg / Failed to get Buffer pointer and length` message. The same wrapped call succeeds under Node.

96.2 Current Finding

The failing call is not in the app's config logic. It is in Next's SWC wrapper:

```
// node_modules/next/dist/build/swc/index.js
function toBuffer(t) {
```

```
    return Buffer.from(JSON.stringify(t));
}
```

```
return bindings.transform(isModule ? JSON.stringify(src) : src, isModule, toBuffer(options));
```

The error string exists in `node_modules/@next/swc-darwin-arm64/next-swc.darwin-arm64.node`, so the native addon is calling `napi_get_buffer_info()` on the third argument, which is the JS `Buffer.from(JSON.stringify(op`

QuickJS originally had two Buffer identities:

- JavaScript sees `Buffer.from(...)` as a Buffer.
- The QuickJS N-API layer only accepted native-created values recorded by the N-API backend.

That leaves JS-created Buffers, including Next's `toBuffer(options)`, looking valid to JS but invalid to `napi_get_buffer_info()`.

96.3 Fix

Implemented QuickJS Buffer identity inside the N-API backend without a Buffer-specific unofficial API:

- QuickJS lazily reads the canonical `globalThis.Buffer.prototype` once it is available during normal bootstrap and caches that prototype in the env.
- `napi_is_buffer()` reports a value as a Buffer only when it is a typed-array view whose prototype chain reaches the cached Buffer prototype, or when it is a native-created pre-bootstrap buffer still tracked by the env.
- N-API-created QuickJS buffers adopt the cached Buffer prototype once it is available; buffers created before bootstrap continue to work through the env tracking path until their prototype can be updated.
- `internalBinding("buffer").setBufferPrototype(Buffer.prototype)` remains only an Edge bootstrap/lifetime helper and no longer drives QuickJS N-API Buffer identity.

The fix intentionally does not make every `Uint8Array` a Buffer.

Owner files:

- `napi/quickjs/CMakeLists.txt`
- `napi/quickjs/src/internal/napi_buffer.h`
- `napi/quickjs/src/internal/napi_buffer.cc`
- `napi/quickjs/src/internal/napi_external.h`
- `napi/quickjs/src/internal/napi_external.cc`
- `napi/quickjs/src/internal/napi_env.h`
- `napi/quickjs/src/internal/napi_env.cc`
- `napi/quickjs/src/js_native_api_quickjs.cc`
- `napi/tests/runners/test_21_general.cc`
- `src/edge_buffer.cc`

96.4 Plan

1. Keep the env-owned Buffer prototype identity in QuickJS instead of restoring marker properties on JS objects or adding a Buffer-specific unofficial API.
2. Cache the first real `globalThis.Buffer.prototype` seen after bootstrap and keep that identity stable even if user code later replaces `globalThis.Buffer`.
3. Keep `napi_get_buffer_info()` backed by QuickJS typed-array APIs, with `napi_invalid_arg` preserved for plain objects, plain `ArrayBuffer`, and plain `Uint8Array`.
4. Keep regression coverage for JS-created `FastBuffer` values, sliced buffers, zero-length buffers, plain typed-array negatives, global Buffer replacement, and native-created buffers before and after the global Buffer prototype is available.

96.5 Risks

- Treating all typed arrays as Buffers would be too broad and could break addons that rely on Node's Buffer versus Uint8Array distinction.
- Observing the Buffer prototype too late would leave native-created buffers without Buffer methods. The QuickJS env therefore tracks pre-bootstrap native buffers and updates their prototype when the global Buffer prototype is first seen.
- The later code 139 observed after one server run may be a separate unhandled rejection or teardown issue. It should be rechecked after the Buffer compatibility fix, but should not be folded into this change unless it still reproduces.

96.6 Verification

Passed:

```
cmake --build build-edge-quickjs-cli-napi --target napi_quickjs_test_21_general -j4
ctest --test-dir build-edge-quickjs-cli-napi --output-on-failure -R 'napi_quickjs_test_21_general'
cmake --build build-edge-quickjs-cli --target edge -j4
make test-napi-quickjs
```

make test-napi-quickjs result: 52/52 QuickJS N-API tests passed.

Passed focused SWC config transform reproduction:

```
cd /Users/syrusakbary/Development/private-poker
/Users/syrusakbary/Development/edgejs/build-edge-quickjs-cli/edge -e '/* loadBindings().transform
```

Result:

swc-transform-ok

Passed Next startup/root-route smoke with the rebuilt CLI:

```
cd /Users/syrusakbary/Development/private-poker
PORT=3100 /Users/syrusakbary/Development/edgejs/build-edge-quickjs-cli/edge npm run start
curl -s -o /dev/null -w '/ %{{http_code}}\n' http://127.0.0.1:3100/
```

Result:

/ 200

Known separate failure still observed on a dynamic route:

/tables/abc 500

Invariant: Cannot access "entryCSSFiles" without a work store.

Passed next.config.ts transpile path:

```
cd /Users/syrusakbary/Development/private-poker
/Users/syrusakbary/Development/edgejs/build-edge-quickjs-cli/edge -e '/* transpileConfig(...) */
```

Passed full next start smoke; the server reached Ready:

```
cd /Users/syrusakbary/Development/private-poker
PORT=3100 /Users/syrusakbary/Development/edgejs/build-edge-quickjs-cli/edge npm run start
```

Passed focused and full QuickJS N-API suites from a test-configured build:

```
cmake -S . -B build-edge-quickjs-cli-napi -DCMAKE_BUILD_TYPE=Debug -DEDGE_NAPI_PROVIDER=quickjs -
cmake --build build-edge-quickjs-cli-napi --target napi_quickjs_test_21_general -j4
ctest --test-dir build-edge-quickjs-cli-napi --output-on-failure -R 'napi_quickjs\.napi_quickjs_t
cmake --build build-edge-quickjs-cli-napi -j4
ctest --test-dir build-edge-quickjs-cli-napi --output-on-failure -R '^napi_quickjs\.'
```

Result: 49/49 QuickJS N-API tests passed.

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/development/troubleshooting/next-app/005_entry_css_work_store_async_context.md

97 Next App: entryCSSFiles work store async context

		Remarks
Status	[done]	Fixed by preserving QuickJS async-function continuation frames through await.
Severity	High	Dynamic Next routes failed with a 500 unless request-scoped work store survived promise/microtask boundaries.

97.1 Symptom

Running the private Next.js app with the native QuickJS-backed Edge CLI reaches `next start`, but request rendering repeatedly fails:

```
Error [InvariantError]: Invariant: Cannot access "entryCSSFiles" without a work store.
    at new b (.next/server/chunks/ssr/7211d_next_dist_d6caa501._.js:3:2526)
    at get (.next/server/chunks/ssr/src_ImperfectProtocol_private-poker_223478e5._.js:3:14063)
    at apply (null)
    at runMicrotasks (null)
```

Reproduction:

```
cd /Users/sadhbh/src/ImperfectProtocol/private-poker
/Users/sadhbh/src/dev/edgejs/build-edge-quickjs-cli/edge npm run start
```

The earlier SWC Buffer issue is separate and remains fixed. This failure points at Next's request AsyncLocalStorage work store being unavailable during a promise/microtask continuation.

97.2 Initial Finding

The expected fix belongs in QuickJS promise and async-context propagation, not in app code or Next-specific shims. Existing promise-hook work moved hook, rejection, microtask, and continuation-preserved embedder data behavior into `napi_promises__`, but this app is a stronger integration test than the current native suite.

Likely risk areas:

- promise hooks are attached to the env but not applied exactly when Next's async hooks setup expects them;
- continuation-preserved embedder data is captured on the wrong promise identity or released too early for chained reactions;
- QuickJS job draining enters the callback through `runMicrotasks` without the request frame restored;
- Node's JavaScript AsyncLocalStorage implementation may rely on a V8-shaped promise hook detail not covered by the current N-API promise tests.

97.3 Root Cause

`next start` exposes the loss through `workAsyncStorage.getStore()` returning undefined while Next accesses the proxied `entryCSSFiles` manifest. A focused reproduction showed that the work store survived `Promise.resolve().then(...)`, `setImmediate(...)`, and `process.nextTick(...)`, but was lost after `await`:

```
/Users/sadhbh/src/dev/edgejs/build-edge-quickjs-cli/edge --async-context-frame -e "require('next,
```

The vendored QuickJS promise hook support already had `js_promise_function_promise(...)` and `js_promise_reaction_promise(...)` to recover the promise identity associated with resolving functions and async function resume objects. The remaining gap was in `promise_reaction_job(...)`: `async/await` can place the `JS_CLASS_ASYNC_FUNCTION_RESOLVE` continuation in the reaction handler, while both resolving function slots are intentionally undefined because QuickJS avoids creating the throwaway capability.

The fix teaches `promise_reaction_job(...)` to also inspect handler with `js_promise_reaction_promise(...)` before emitting `JS_PROMISE_HOOK_BEFORE`. The N-API promise subsystem now keeps captured continuation frames until the promise resolves, releasing rejected-promise frames from the rejection tracker.

97.4 Implemented Fix

- Kept runtime promise hook wiring inside `napi_promises__`.
- Added a QuickJS `promise_reaction_job(...)` fallback from resolving funcs to the `async-function` handler.
- Deferred continuation-frame cleanup from `AFTER` to `settle/rejection` paths so multi-`await` async functions keep their request frame across each resume.
- Added native regressions for ordinary promise reactions and two consecutive `await` resumptions preserving `continuation_preserved_embedder_data`.

97.5 Verification

```
ctest --test-dir /Users/sadhbh/src/dev/edgejs/build-napi-quickjs --output-on-failure -R 'napi-qu
```

Result: 5/5 passing.

```
cd /Users/sadhbh/src/dev/edgejs/napi
make test-native-quickjs
```

Result: 67/67 passing.

```
cd /Users/sadhbh/src/ImperfectProtocol/private-poker
PORT=3113 /Users/sadhbh/src/dev/edgejs/build-edge-quickjs-cli/edge npm run start
curl -i http://127.0.0.1:3113/lobby/1
```

Result: HTTP/1.1 200 OK; the previous entryCSSFiles work-store invariant no longer appears. The server still logs unrelated remote fetch failures ending in `ReferenceError: WebAssembly is not defined`, but those do not cause this route to return 500.

97.6 Debug Teardown Follow-Up

After the promise fix, a Debug edge `-e "console.log('hi')"` run still tripped QuickJS `JS_FreeRuntime(...)` teardown. Enabling the QuickJS object leak dump showed one remaining object:

```
Object { importModuleDynamically: [AsyncFunction ...], callbackReferrer: [NapiExternal ...] }
```

LLDB showed this came from the JS ESM `moduleRegistries WeakMap` entry created for `ContextifyScript` dynamic import callbacks. The fix stays native-side:

- `napi_module_wrap__` now unregisters temporary script dynamic-import referrer symbols after `napi_contextify__::run_script(...)` finishes.
- Edge's `ContextifyScript` wrapper now stores the host-defined option symbol in a native record, restores it onto the JS wrapper only while a script is running, then clears the JS properties after the run. This breaks the registry $\rightarrow$ `callbackReferrer` $\rightarrow$ `host_defined_option_symbol` $\rightarrow$ registry retention cycle before env teardown without changing `edgejs/lib`.
- Env cleanup detaches tracked `ContextifyScript` native records and drops their `napi_refs` before the QuickJS runtime is freed.

Verification:

```
cmake --build /Users/sadhbh/src/dev/edgejs/build-edge-quickjs-cli-debug --target edge -j4
/Users/sadhbh/src/dev/edgejs/build-edge-quickjs-cli-debug/edge -e "console.log('hi')"
/Users/sadhbh/src/dev/edgejs/build-edge-quickjs-cli-debug/edge -e "import('node:fs').then(m => co
/Users/sadhbh/src/dev/edgejs/build-edge-quickjs-cli-debug/edge --experimental-vm-modules -e "cons
```

Results: the Debug teardown assertion no longer reproduces; the dynamic import smokes print function and 2.

```
cd /Users/sadhbh/src/dev/edgejs/napi
make test-native-quickjs
```

Result: 67/67 passing.

Release route verification after the native-only leak fix:

```
cd /Users/sadhbh/src/ImperfectProtocol/private-poker
PORT=3020 HOSTNAME=127.0.0.1 /Users/sadhbh/src/dev/edgejs/build-edge-quickjs-cli/edge npm run sta
node -e "fetch('http://127.0.0.1:3020/lobby/1').then(async r=>{const t=await r.text(); console.l
```

Result: 200 text/html; charset=utf-8.

97.7 Original Action Plan

1. Add or strengthen a local regression that uses AsyncLocalStorage across an ordinary promise reaction and verifies the store is visible inside the reaction after the outer store changes.
2. Compare current napi_promises__ behavior with unofficial_napi.ref.cc, keeping the refactored ownership model but importing any missing promise context behavior.
3. Rebuild the native QuickJS Edge CLI and rerun the focused AsyncLocalStorage smoke:

```
./build-edge-quickjs-cli/edge --async-context-frame -e "const { AsyncLocalStorage } = require
```

4. Rerun the private Next app startup/request smoke:

```
cd /Users/sadhbh/src/ImperfectProtocol/private-poker
PORT=3100 /Users/sadhbh/src/dev/edgejs/build-edge-quickjs-cli/edge npm run start
```

5. Update this issue with the exact route(s), status codes, and any remaining caveats after verification.

98 Node Compatibility

99 Node Compatibility Troubleshooting

		Remarks
Status	[open]	Node compatibility known-issue registry.
Severity	Low	Documentation registry only; individual issue pages carry runtime severity.

This directory is the known-issue registry for Node compatibility in the QuickJS WASIX work. It does not describe a C++ compatibility layer. Each page records the current incompatibility, status, and ownership boundary.

Some issues are intentionally accepted for now because the QuickJS N-API backend is not trying to be a full Node runtime. Others are active work items for EdgeJS bootstrap/runtime code, deployment tooling, or focused QuickJS N-API internals.

99.1 Areas

- [N-API known issues](#): QuickJS N-API behavior, intentional non-Node behavior, and focused internal subsystems under `napi/quickjs/src/internal`.
- [EdgeJS runtime](#): work that belongs in EdgeJS runtime source or JavaScript bootstrap code under `src/` and `lib/`.
- [Deploy and packaging](#): build, packaging, npm graph, and deployment issues.

99.2 Current Status

The cleanup direction is to keep QuickJS N-API small and explicit, keep real Node-runtime behavior in EdgeJS itself when it is required, and route module loading through Node's JavaScript loaders/translators instead of rebuilding Node's loader policy in C++.

The C++ CJS/module-loader hack has been removed from `napi/quickjs/src`. The removed files were `unofficial_module_loader.{h,cc}` and `quickjs_cjs_exports.{h,cc}`. This means current QuickJS N-API behavior is intentionally without that parser/resolver.

The missing engine-side module primitives were added to the vendored QuickJS submodule in `577fb31caf2e973b111431b6c` and the current `napi_module_wrap__` implementation adapts the V8-shaped `module_wrap` surface onto those APIs. This keeps module mechanics in QuickJS while leaving package resolution, CommonJS wrapping, package conditions, and builtin policy in EdgeJS/Node loader code.

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/development/troubleshooting/node-compat/deploy/016_napi_extern_wasix_linkage.md

100 Known Issue: NAPI_EXTERN= WASIX linkage rule

		Remarks
Status	[caveated]	Linkage rule is known and enforced in current embedded-provider builds.
Severity	Medium	Correct but easy to forget on future targets.

100.1 Current State

This is a build and deployment issue, not a QuickJS N-API compatibility file.

100.2 Source Notes

- plans/quickjs-wasm/development/008_runtime_change_containment_rollback.md
- plans/quickjs-wasm/development/troubleshooting/wasmer-deploy/002_quickjs_wasix_napi_import_m

100.3 Known Incompatibility

Targets that include N-API headers before linking napi_quickjs must compile with NAPI_EXTERN= so wasm objects agree that napi_* symbols are provided by the embedded provider rather than imported from napi or env.

100.4 Risk

The rule is real, but it is tribal knowledge unless every consumer inherits it centrally. A future target can silently compile with the wrong import/export mode and fail at wasm link time.

100.5 Current Status

Move embedded-provider declaration mode into a single CMake interface target or generated config header. Make all N-API consumers inherit from it. Keep the post-link no-imported-napi_* check and make failures identify the offending target.

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/development/troubleshooting/node-compat/deploy/017_framework_static_server_adapters.md

101 Known Issue: Framework static and ad hoc server adapters

		Remarks
Status	[caveated]	Useful bring-up adapters exist, but framework support boundaries remain explicit.
Severity	Low	Useful for bring-up, but not a general framework integration model.

101.1 Current State

This is a framework deployment issue, not a QuickJS N-API compatibility file.

101.2 Source Notes

- plans/quickjs-wasm/development/006_framework_app_adapters.md
- plans/quickjs-wasm/development/007_framework_standalone_builds.md
- plans/quickjs-wasm/development/troubleshooting/vite-app/001_standalone_build.md

101.3 Known Incompatibility

Astro and Vite validation used small static or ad hoc server adapters to get framework output running under Edge QuickJS.

101.4 Risk

These adapters prove runtime capability, but they can bypass the actual server semantics users expect from each framework: routing, headers, streaming, error pages, assets, and environment handling.

101.5 Current Status

Define supported deployment modes per framework: static assets, standalone Node server, generated dynamic shell, or unsupported. Generate adapters as tested build artifacts instead of hand-maintained one-offs. Add fixtures for routing, assets, headers, streaming, error pages, and env vars under native QuickJS and WASIX.

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/development/troubleshooting/node-compat/deploy/018_pnpm_deploy_graph_materialization.md

102 Known Issue: pnpm deploy graph materialization

		Remarks
Status	[caveated]	Deploy graph materialization exists, with drift risk against pnpm and bundlers.
Severity	Medium	Custom package graph rewriting can drift from pnpm and bundler behavior.

102.1 Current State

This is a deployment artifact issue, not a QuickJS N-API compatibility file.

102.2 Source Notes

- plans/quickjs-wasm/development/troubleshooting/astro-ssr/014_pnpm_deploy_externalized_runtime
- plans/quickjs-wasm/development/troubleshooting/wasmer-deploy/001_pnpm_directory_symlinks_web

102.3 Known Incompatibility

Deploy preparation scans runtime imports, materializes pnpm package links, removes .pnpm, rewrites virtual-store imports, and validates a symlink-free artifact.

102.4 Risk

It is pragmatic, but it is a custom package graph transformer. It can go stale as pnpm, framework bundlers, and package export patterns change.

102.5 Current Status

Make deploy preparation a tested graph tool with explicit inputs, outputs, and invariants. Prefer framework or bundler output that already contains a closed runtime graph. Validate every bare runtime import inside the final artifact with the same resolver QuickJS uses at runtime.

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/development/troubleshooting/node-compat/deploy/README.md

103 Deploy And Packaging Node Compatibility

		Remarks
Status	[open]	Registry for deployment and package-layout compatibility issues.
Severity	Low	Documentation registry only; individual issue pages carry runtime severity.

These notes track known issues that are primarily about builds, package layout, deployment, and framework packaging. They are not N-API compatibility files; they describe deploy artifact and package graph status.

- [NAPI_EXTERN WASIX linkage](#)
- [Framework static server adapters](#)
- [pnpm deploy graph materialization](#)

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/development/troubleshooting/node-compat/edgejs/003_minimal_intl_fallback.md

104 Known Issue: Minimal Intl.DateTimeFormat fallback

		Remarks
Status	[caveated]	Minimal runtime fallback exists with known ECMA-402 limits.
Severity	Medium	Unblocks frameworks but can misrepresent ECMA-402 support.

104.1 Current State

This issue belongs to EdgeJS runtime/bootstrap code, not QuickJS N-API.

104.2 Source Notes

- plans/quickjs-wasm/development/troubleshooting/astro-ssr/004_missing_intl.md
- plans/quickjs-wasm/development/dev_001_pr_cleanup_containment/003_intl_fallback_module.md

104.3 Known Incompatibility

When QuickJS has no real `globalThis.Intl`, Edge installs a deliberately tiny `Intl.DateTimeFormat` fallback. It covers enough timestamp formatting for Astro bootstrap paths.

104.4 Risk

It looks like `Intl`, but it is not full ECMA-402. It does not do real locale negotiation, calendars, numbering systems, time-zone rules, or ICU-backed formatting. Code can easily infer more support than exists.

104.5 Current Status

Either link a real ECMA-402/ICU provider or expose `Intl` as an explicit unsupported capability. If a fallback remains, make its subset explicit in a support matrix and add tests proving unsupported options do not silently pretend to work.

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/development/troubleshooting/node-compat/edgejs/004_native_inspector_stub.md

105 Known Issue: Native unavailable inspector stub

		Remarks
Status	[caveated]	Native unavailable-inspector module exists with known shape limits.
Severity	Medium	Makes imports work while the runtime still has no real inspector.

105.1 Current State

This issue belongs to EdgeJS runtime/bootstrap code, not QuickJS N-API.

105.2 Source Notes

- plans/quickjs-wasm/development/troubleshooting/next-app/002_standalone_inspector_stub.md
- plans/quickjs-wasm/development/dev_001_pr_cleanup_containment/002_native_inspector_fallback.

105.3 Known Incompatibility

`require("inspector")` and `require("node:inspector")` return a native fallback object even though `internalBinding("config").hasInspector` and `process.features.inspector` remain false. Passive APIs no-op; active APIs throw `inspector-unavailable` errors.

105.4 Risk

The module is publicly loadable while builtin metadata can still say it cannot be required. The fallback `Session` is smaller than Node's real object and does not fully behave like an `EventEmitter`.

105.5 Current Status

Define the contract explicitly: absent inspector, or a documented unavailable inspector module. If the module is present, make metadata agree with `require()` and match Node's public shape closely enough for passive consumers, including `Session` inheritance and stable no-op methods.

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/development/troubleshooting/node-compat/edgejs/010_stream_wrapper_unwrap_fallback.md

106 Known Issue: Stream wrapper-specific unwrap fallback

		Remarks
Status	[done]	Runtime stream handling works, and the underlying QuickJS N-API object/external ambiguity is fixed. Previously worked around a deeper N-API object identity problem.
Severity	High	

106.1 Current State

This issue crosses EdgeJS stream code and QuickJS N-API object typing. The observable runtime issue is tracked here because it surfaced in EdgeJS stream conversion.

106.2 Source Notes

- plans/quickjs-wasm/development/005_wasix_wasmer_http.md
- AGENTS.md

106.3 Historical Incompatibility

QuickJS class instances could look like `napi_external`, so stream conversion treated a wrapped TCP object as a raw external pointer. The fix tries TCP/Pipe/TTY wrapper-specific `napi_unwrap(...)` paths before raw external fallback.

The root ambiguity is now fixed in QuickJS N-API: constructed class instances are ordinary prototype-backed objects, while public externals and internal wrap records use separate QuickJS classes.

106.4 Risk

The stream fix is still a useful defensive conversion path, but wrapped objects should no longer be ambiguous with raw external values.

106.5 Current Status

QuickJS N-API type tagging now keeps wrapped objects and raw externals distinct. Future cleanup can simplify stream-base conversion where the wrapper-specific fallback is no longer needed for compatibility.

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/development/troubleshooting/node-compat/edgejs/011_url_parse_deprecation_node_modules.md

107 url.parse() deprecation gating and node_modules callers

		Remarks
Status	[done]	Fixed in native isInsideNodeModules(...) without editing lib/.
Severity	Medium	Emits user-visible deprecation warnings that Node suppresses for dependency code.

107.1 Failure

Running the V8-backed Edge CLI in /Users/syrusakbary/Development/private-poker:

```
/Users/syrusakbary/Development/edgejs/build-edge/edge pnpm run start
```

then visiting:

```
http://localhost:3000/main-lobby
```

prints:

```
(node:...) [DEP0169] DeprecationWarning: `url.parse()` behavior is not standardized and prone to
```

Native Node does not print that warning for the same app and route.

107.2 Diagnosis

The route enters Next.js server routing. With NODE_OPTIONS=--trace-deprecation, Edge shows the call originates in dependency code:

```
at Object.urlParse [as parse] (node:url:136:13)
```

```
at resolveRoutes (.../private-poker/node_modules/next/dist/server/lib/router-utils/resolve-routes
```

The user explicitly requested that the fix must not touch any code under lib/. The compatibility gap must therefore be fixed in the native internalBinding('util').isInsideNodeModules(...) implementation used by the existing JavaScript runtime files.

There are two relevant compatibility gaps:

1. lib/url.js calls isInsideNodeModules(2), while Node v24.15.0 calls isInsideNodeModules(4). Because lib/ cannot be changed in this task, the native helper must classify correctly for Edge's existing one-argument callers.
2. internalBinding('util').isInsideNodeModules(...) returns false for a module loaded from node_modules. The current V8 call-site path mostly sees loader frames, so dependency calls are not recognized as dependency calls.

107.3 Fix

1. Do not edit lib/.
2. Fix isInsideNodeModules(frameLimit) in src/edge_util.cc so it follows Node v24.15.0:
 - captures the current stack without skipping the JavaScript caller,
 - ignores only node: built-in frames,
 - classifies the first non-node: frame,
 - returns true only when that frame is under node_modules.

3. Add focused tests for:
 - first-party file `url.parse()` with default flags: one DEP0169,
 - first-party eval/stdin `url.parse()` with default flags: Node-compatible default handling,
 - node_modules `url.parse()` with default flags: no DEP0169,
 - node_modules `url.parse()` with `--pending-deprecation`: no DEP0169.
4. Rebuild build-edge/edge.
5. Verify the real private-poker route no longer prints DEP0169 when visiting /main-lobby.

107.4 Verification

Node oracle:

```
source ~/.nvm/nvm.sh && nvm use 24
node -v
node -p "require('url').parse.toString().slice(0, 420)"
```

Observed Node v24.15.0 uses `isInsideNodeModules(4)` for `url.parse()`.

Focused behavior checks passed under both Node v24.15.0 and the rebuilt Edge binary:

- first-party file `url.parse()` emits one DEP0169,
- `-e` and stdin `url.parse()` emit one DEP0169,
- node_modules `url.parse()` emits no DEP0169,
- node_modules `url.parse()` with `--pending-deprecation` emits no DEP0169.

Regression test:

```
/Users/syrusakbary/Development/edgejs/build-edge/edge test/parallel/test-url-parse-node-modules-0
source ~/.nvm/nvm.sh && nvm use 24 >/dev/null && node test/parallel/test-url-parse-node-modules-0
```

Both passed.

Real app verification:

```
cd /Users/syrusakbary/Development/private-poker
PORT=3010 /Users/syrusakbary/Development/edgejs/build-edge/edge pnpm run start
curl -sS -o /tmp/private-poker-main-lobby-edge.html -w '%{http_code}\n' http://localhost:3010/main-lobby
```

The route returned 200, and the Edge server emitted no DEP0169 warning.

107.5 QuickJS Follow-up

The QuickJS-backed Edge CLI later reproduced the same warning for dependency code even after the shared native helper was fixed. The remaining difference was not in `src/edge_util.cc`: QuickJS exposed a source-less native host callback frame through `unofficial_napi_get_current_stack_trace(...)`, so `isInsideNodeModules(2)` saw only the native frame and `node:url` before the frame limit was exhausted. V8/Node do not count that native binding frame.

The follow-up fix lives in `napi/quickjs/src/internal/napi_callsite.cc`. The N-API callsite adapter now compacts source-less native callback frames before returning V8-shaped stack data to Edge's shared native helpers.

QuickJS verification passed:

```
/Users/syrusakbary/Development/edgejs/build-edge-quickjs-cli/edge test/parallel/test-url-parse-node-modules-0
```

The real private-poker /main-lobby route returned 200 under build-edge-quickjs-cli/edge on port 3011 and emitted no DEP0169 warning.

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/development/troubleshooting/node-compat/edgejs/012_v8_ctest_environment_attach.md

108 V8 CTest Runtime Fixture Environment Attachment

		Remarks
Status	[caveated]	The fixture environment attachment is fixed; broader V8 CTest coverage still has unrelated failures.
Severity	Medium	The V8 CLI smoke path works, but the generated runtime CTest suite fails before fixture scripts run.

108.1 Observation

After merging main into quickjs, the V8 build completed and the V8 CLI smoke tests passed. Running:

```
ctest --test-dir build-edge --output-on-failure
```

failed immediately in Test0RuntimePhase01 fixtures because EdgeInitializeTimersHost(env) returned napi_generic_failure. The first failures reported Failed to initialize timers host before any fixture script could run.

108.2 Diagnosis

The generated CTest fixture creates a raw N-API environment with unofficial_napi_create_env(...), installs platform hooks, and initializes the timers host. The runtime code now expects an attached edge::Environment behind the N-API environment before timer handles are initialized. The normal CLI path attaches that runtime environment through EdgeAttachEnvironmentForRuntime(...).

The test fixture was therefore exercising a stale setup path: it asked the timers host to initialize before the env had the same Edge runtime attachment the CLI uses.

108.3 Action Plan

1. Update tests/runners/test_env.h to attach the Edge runtime environment immediately after creating the N-API env.
2. Keep the existing platform hook and timer initialization checks in place.
3. Rebuild the V8 target and rerun the focused failing test first.
4. Rerun the broader V8 CTest suite if the focused test passes.
5. Confirm the QuickJS CTest suite remains green.

108.4 Result

tests/runners/test_env.h now attaches the Edge runtime environment through EdgeAttachEnvironmentForRuntime(...) immediately after creating the raw N-API environment. The focused CTest regression:

```
ctest --test-dir build-edge -R ValidFixtureScriptReturnsZero --output-on-failure
```

passes after the change. ctest --test-dir build-edge-quickjs-cli --output-on-failure also passes 66/66.

The full V8 CTest registry advances past the original timers-host initialization failure, then exposes other existing broad-suite failures, including a CJS/ESM fixture timeout, a zlib/debuglog binding mismatch, a REPL version-string expectation mismatch, and QuickJS-specific promise-details coverage being run against the V8 binary. Those are outside this fixture-attachment fix.

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/development/troubleshooting/node-compat/edgejs/README.md

109 EdgeJS Runtime Node Compatibility

		Remarks
Status	[open]	Registry for Node compatibility issues owned by EdgeJS runtime/bootstrap code. Documentation registry only; individual issue pages carry runtime severity.
Severity	Low	

These notes track Node compatibility work that belongs in EdgeJS runtime source or JavaScript bootstrap code. They are separate from QuickJS N-API internals: when a behavior is really part of the Node runtime surface, it should be solved in EdgeJS itself rather than hidden behind removed N-API compatibility files.

- [Minimal Intl fallback](#)
- [Native inspector stub](#)
- [Stream wrapper unwrap fallback](#)

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/development/troubleshooting/node-compat/napi/001_buffer.md

110 Known Issue: Buffer

		Remarks
Status	[caveated]	Accepted incompatibility after removing the N-API Buffer repair code.
Severity	Medium	Node-facing code frequently treats binary data as Buffer, not only as generic typed arrays.

110.1 Current State

There is no QuickJS N-API Buffer repair layer now. Values created as QuickJS typed arrays keep QuickJS typed-array behavior unless a real EdgeJS/Node Buffer implementation creates them.

110.2 Known Incompatibility

Node embedders and package code often rely on Buffer methods, `Buffer.isBuffer(...)`, and string conversion semantics even when binary data originates from native N-API allocation. QuickJS does not have Node's built-in Buffer object model, so plain typed arrays can be too weak for some Node programs.

110.3 Current Status

If Node Buffer compatibility becomes required again, the work should be a real Buffer implementation in EdgeJS bootstrap/runtime code, with the N-API allocation path constructing Buffer-backed values deliberately. It should not be a prototype repair pass after allocation.

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/development/troubleshooting/node-compat/napi/002_console.md

111 Known Issue: Console bootstrap binding

		Remarks
Status	[caveated]	N-API console repair was removed; remaining work belongs in EdgeJS bootstrap if needed.
Severity	Medium	Incorrect console bindings break diagnostics, tests, and bootstrap visibility.

111.1 Current State

`RepairBootstrapConsoleBindings(...)` is no longer part of QuickJS N-API startup. QuickJS N-API environment creation should not patch the EdgeJS console object after bootstrap.

111.2 Known Incompatibility

Node programs expect `console.log`, `console.error`, and related methods to work early in process startup and from native-backed contexts. If EdgeJS bootstrap assembles the global object and CommonJS-visible console module in the wrong order, logging can appear present while being disconnected from the actual `stdout/stderr`-backed console implementation.

111.3 Current Status

If this regresses, fix EdgeJS JavaScript bootstrap so it creates the final console object in the correct order, with methods bound once. A bootstrap assertion is fine; a late N-API repair pass should not return.

112 Known Issue: Contextify diagnostics

		Remarks
Status	[done]	Implemented as napi_contextify__ under napi/quickjs/src/internal. Contextify is central to script compilation, source metadata, and framework bootstraps.
Severity	High	

112.1 Current State

Contextify helpers and state live in:

- napi/quickjs/src/internal/napi_contextify.h
- napi/quickjs/src/internal/napi_contextify.cc

napi_env__ owns napi_contextify__. Source-map settings and callback state are kept with that class instead of in removed compatibility files.

112.2 Known Incompatibility

Node's contextify APIs sit under vm, internal loaders, and framework bootstraps. QuickJS can compile and evaluate scripts, but its diagnostics and context model do not naturally match V8 objects. Compile-time failures can be annotated while QuickJS still exposes useful metadata. A caught Error returned from JavaScript does not currently provide the same reliable V8-style message object for preserved source-map output.

112.3 Current Status

A fuller design should keep contextify as a small QuickJS subsystem with explicit compiled-script objects, cache-data policy, source-map state, and structured diagnostics. The documentation and tests should clearly separate supported QuickJS diagnostics from V8-only caught-error formatting.

113 Known Issue: Environment lifecycle

		Remarks
Status	[caveated]	JS_FreeRuntime(...) is enabled again; remaining lifecycle risk is finalizer/order correctness during teardown.
Severity	High	Environment lifecycle state affects cleanup, teardown, stack limits, and runtime stability.

113.1 Current State

Environment state is no longer carried by a separate compatibility directory. The current direction is direct ownership on `napi_env__` plus focused internal classes such as `napi_promises__`, `napi_contextify__`, and `napi_serdes__`. `napi_env__` now owns allocator-backed scope, reference, cleanup-hook, deferred, and external backing-store storage. Runtime release calls `JS_FreeContext(...)` and `JS_FreeRuntime(...)`; the env object is kept alive until after QuickJS runtime finalizers can run, then instance data is finalized and the env is deleted.

113.2 Known Incompatibility

Node's N-API assumes a rich environment lifecycle with deterministic cleanup hooks and V8-backed runtime services. QuickJS exposes different primitives, so cleanup, task draining, stack limits, and release still need careful ownership. The previous disabled `JS_FreeRuntime(...)` workaround is historical. The current teardown issue is narrower: QuickJS GC/finalizer callbacks can still re-enter N-API during context/runtime release, so env-owned data and external/wrap finalizer state must remain valid until those callbacks finish.

113.3 Current Status

Keep `JS_FreeRuntime(...)` enabled. Stack sizing, cleanup queues, task queues, allocator-backed handle storage, and internal subsystem ownership should remain explicit environment configuration or RAIL state, not side tables. Treat new teardown crashes as ordering/finalizer lifetime bugs rather than as permission to disable runtime release again.

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/development/troubleshooting/node-compat/napi/005_global_shims.md

114 Known Issue: Global shims

		Remarks
Status	[caveated]	Removed from QuickJS N-API; remaining global support belongs in EdgeJS bootstrap.
Severity	Medium	Node and framework code probe for modern globals during bootstrap.

114.1 Current State

QuickJS N-API no longer installs broad Node-facing globals. Missing globals are either accepted incompatibilities or EdgeJS runtime/bootstrap work.

114.2 Known Incompatibility

Modern Node packages often probe for globals before choosing code paths. QuickJS may not expose the same objects, or may expose them with different behavior, which can push libraries into failing branches during startup.

114.3 Current Status

Provide real engine-backed or EdgeJS-backed implementations where feasible. Unsupported globals should fail in a way that works with Node-style capability detection. Do not add silent N-API-level globals just to satisfy probes.

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/development/troubleshooting/node-compat/napi/006_microtasks.md

115 Known Issue: Promise hooks and microtasks

		Remarks
Status	[done]	Implemented as napi_promises__ under napi/quickjs/src/internal; native suites pass. Promise hooks, async context, and job draining affect nearly every async workload.
Severity	High	

115.1 Current State

Promise hook, rejection tracking, and microtask job logic live in:

- napi/quickjs/src/internal/napi_promises.h
- napi/quickjs/src/internal/napi_promises.cc

napi_env__ owns napi_promises__ directly. The old separate microtask compatibility files are gone.

115.2 Known Incompatibility

Node/V8 has a well-defined microtask queue integration with promise hooks, async resources, and embedder task scheduling. QuickJS exposes pending jobs differently, so draining must be coordinated from the N-API layer to keep promises, cleanup callbacks, and framework startup tasks progressing.

115.3 Current Status

Keep this owned by the QuickJS N-API environment with clear rules for when jobs are enqueued, drained, and associated with async context. If promise behavior regresses, fix unofficial_napi_enqueue_microtask, unofficial_napi_process_microtasks, and unofficial_napi_set_enqueue_foreground_task_callback in the N-API layer rather than adding drain loops in EdgeJS runtime code.

116 Known Issue: Module loading

		Remarks
Status	[caveated]	QuickJS-backed ModuleWrap is implemented and focused native module tests pass; remaining public VM failures are Edge V8-baseline gaps and do not block QuickJS module-loading parity. Package resolution and CommonJS/ESM interop decide whether real Node apps can bootstrap.
Severity	High	

116.1 Current State

The QuickJS N-API backend should not carry a parallel C++ approximation of Node's package resolver, CommonJS wrapper, or ESM translator policy. The C++ CJS/module-loader hack has been removed: `unofficial_module_loader` and `quickjs_cjs_exports` no longer exist under `napi/quickjs/src`.

The QuickJS `module_wrap/unofficial` public surface now forwards to `napi/quickjs/src/internal/napi_module_wrap.{h,c}`. Source-text modules, explicit host linking, `instantiate/evaluate`, synthetic modules, `import-meta`, dynamic import, required module facades, and synchronous `require(esm)` all use QuickJS module records instead of the previous `napi_generic_failure` stubs.

Focused native loader and module-wrap checks pass, including `test-internal-module-wrap`, VM synthetic modules, `import-meta`, dynamic import, `top-level-await`, `async-graph`, `moduleRequests`, link tests, and a manual CommonJS `require(esm)` smoke. The prior `node:test/Symbol.description` startup failure was caused by missing QuickJS `Symbol.dispose` and `Symbol.asyncDispose`; the QuickJS env initialization shim now restores those symbols.

Two broader VM tests are explicitly excluded from the current QuickJS module-loading gate because the rebuilt Edge V8 backend does not pass them:

- `test/parallel/test-vm-module-basic.js`: Edge V8 hangs on `SourceTextModule.evaluate({ timeout: 500 })`; QuickJS reaches the inspection assertion but its context object still exposes QuickJS-specific markers.
- `test/parallel/test-vm-module-linkmodulerequests.js`: Edge V8 passes 2/4, rejects `import source Foo from "foo"` as a syntax error, and reports only `Module is not linked` for the `unlinked-child` diagnostic.

These tests should be treated as diagnostics, not release gates, until Edge's V8 backend passes them or the test expectations are made Edge-specific.

116.2 Known Incompatibility

QuickJS asks the embedder to normalize and load modules, while Node packages expect Node's resolver, CommonJS wrapper behavior, package conditions, and builtin module names. Framework bundles also expose `pnpm` symlink layouts and mixed CJS/ESM graphs that fail if resolution is only browser-like or `filesystem-local`.

Classic CommonJS loading should stay in Node's JavaScript CJS loader path under `lib/internal/modules/cjs/loader.js` with CJS compilation through `internalBinding('contextify').compileFunctionForCJSLoader`. The

missing parity surface is the engine-backed `internalBinding('module_wrap')` implementation that V8 provides and that the Node ESM loader/translators depend on.

116.3 Design Decision

Do not resurrect the removed C++ CommonJS resolver/facade. Instead:

1. Keep `lib/internal/modules/cjs/loader.js`, `lib/internal/modules/esm/loader.js`, and `lib/internal/modules/esm/translators.js` as the source of truth for package resolution, package exports, `node: builtins`, format detection, CJS wrapping, JSON modules, CJS named export preparsing, and CJS/ESM cache interaction.
2. Implement the QuickJS backend's `unofficial_napi_module_wrap_*` APIs with a real QuickJS module record bridge.
3. Add only narrow QuickJS engine APIs needed to expose parsed module requests, link/evaluate separately, report module status/TLA, initialize `import.meta`, and route dynamic `import()` to the JS loader callback.
4. Keep native QuickJS loader callbacks as a prelinked-record lookup layer. They must map requests already resolved by Node's JS loader to QuickJS `JSMODULEDEF*`; they must not resolve packages themselves.

116.4 Target Architecture

The implementation should add `napi/quickjs/src/internal/napi_module_wrap.h` and `napi/quickjs/src/internal/` following the current internal helper direction. `napi/quickjs/src/unofficial_napi.cc` should become thin forwarding glue for the `unofficial_napi_module_wrap_*` symbols, matching how `contextify` and `serdes` have moved into internal classes.

Each `napi_module_wrap__` record should own:

- the `napi_env`;
- the JS wrapper reference;
- the module URL/name;
- the QuickJS module value and `JSMODULEDEF*`;
- parsed module requests, including import attributes and phase;
- linked dependency record pointers supplied by `JSModuleWrap.link(...)`;
- source text or synthetic export metadata as needed for diagnostics;
- status and error mirrors for the Node `ModuleWrap` status constants;
- synthetic evaluation callback and pending export values for synthetic modules.

The bridge should use RAI for QuickJS `JSValue` ownership and should never store raw N-API values without a reference or a duplicated QuickJS value.

116.5 Implementation Phases

116.5.1 1. QuickJS Module Introspection and Split Lifecycle

Add a minimal vendored QuickJS API surface in `napi/quickjs/src/quickjs/quickjs.h` and `quickjs.c`:

- expose source module request count and request data, including import attributes;
- expose module status in a backend-neutral enum that can be mapped to `kUninstantiated`, `kInstantiating`, `kInstantiated`, `kEvaluating`, `kEvaluated`, and `kErrored`;
- expose `has_tla` and an async-graph query;
- expose a link-only operation around QuickJS's internal module linking;
- expose an evaluate-only operation that returns QuickJS's evaluation promise or namespace result as appropriate;
- expose the saved module evaluation exception.

This is the highest-risk part because QuickJS's public API currently combines link and evaluate through `JS_EvalFunction(...)`. Node's `ModuleWrap` needs `instantiate()` to validate and link the graph before evaluation.

116.5.2 2. Source Text ModuleWrap

Implement source text module creation by compiling with `JS_EVAL_TYPE_MODULE | JS_EVAL_FLAG_COMPILE_ONLY`, recording the `JSMODULEDEF*`, and extracting the parsed requests for `getModuleRequests()`.

`ModuleWrap.link(linkedModules)` should store JS-loader-selected dependencies by request index. During `instantiate()` and `evaluate()`, install a QuickJS module normalizer/loader that maps (parent module URL, request specifier, attributes) to the already linked child record. This callback is a lookup adapter only; all real resolution has already happened in JavaScript.

`getNamespace()`, `getStatus()`, `getError()`, `hasTopLevelAwait`, and `hasAsyncGraph` should reflect QuickJS module state and Node's error expectations closely enough for `lib/internal/modules/esm/module_job.js`.

116.5.3 3. Synthetic Modules for CJS, Builtins, JSON, and WASM Facades

Implement `unofficial_napi_module_wrap_create_synthetic(...)` using `JS_NewCModule(...)`, `JS_AddModuleExport(...)`, and `JS_SetModuleExport(...)`. The JS synthetic evaluation callback must run during module evaluation with this bound to the JS ModuleWrap wrapper so existing translator code can call `this.setExport(...)`.

This unlocks the important Node translator paths:

- CommonJS modules imported from ESM;
- CommonJS modules required from imported CJS;
- JSON modules;
- builtin ESM facades;
- CJS named export facades generated by `lib/internal/modules/esm/translators.js`.

116.5.4 4. require(esm) and Required Module Facades

Implement `evaluateSync(...)` so `require(esm)` can link and evaluate a synchronous ESM graph and return the namespace object. If QuickJS reports TLA or an async graph, throw the same `ERR_REQUIRE_ASYNC_MODULE` path used by `lib/internal/modules/esm/module_job.js`.

Implement `createRequiredModuleFacade(...)` by compiling the same kind of source-text facade V8 uses:

```
export * from 'original';
export { default } from 'original';
export const __esModule = true;
```

The temporary facade resolver should map the synthetic 'original' request to the original module record. Returning a plain object is not enough for full V8 parity because the facade is expected to behave like a module namespace with live bindings.

116.5.5 5. Dynamic import() and import.meta

Wire `setImportModuleDynamicallyCallback(...)` and `setInitializeImportMetaObjectCallback(...)` into QuickJS instead of keeping them as no-op stubs.

QuickJS currently handles dynamic import internally through its module loader. For Node parity, patch or wrap that path so `import(specifier, attributes)` calls the registered JS callback from `lib/internal/modules/esm/utils.js` and returns its promise. The native side should preserve QuickJS's string conversion and attribute validation, but resolution and loading must be delegated to the Node JS loader.

For `import.meta`, call the registered initializer the first time QuickJS creates the module's meta object, passing the meta object and the owning `ModuleWrap`.

116.5.6 6. Contextify and Format Detection Tightening

Keep CommonJS compilation in `compileFunctionForCJSLoader(...)`, but replace the current string-search `containsModuleSyntax(...)` fallback with a parser-backed check where possible. False positives or false negatives here can send ambiguous `.js` entry points to the wrong loader.

This phase should stay separate from `ModuleWrap` unless tests prove it blocks the main CJS parity path.

116.5.7 7. Verification

Use staged verification:

1. New QuickJS N-API tests for source text modules:
 - static import/export;
 - duplicate request linkage;
 - missing named export error;
 - namespace access after instantiate;
 - top-level await reporting.
2. Synthetic module tests:
 - `new ModuleWrap(url, undefined, ['default'], callback);`
 - `setExport(...);`
 - CJS translator facade with named exports, default, and `module.exports`.
3. Loader tests:
 - `require('./cjs.cjs');`
 - ESM imports CJS and reads named/default exports;
 - CJS `require()` of synchronous ESM;
 - JSON import and CJS cache sharing;
 - node: builtin ESM facade.
4. Dynamic import and import-meta tests.
5. Existing native gates:

```
make build-napi-quickjs
make test-napi-quickjs
make test-napi-quickjs-only
make build-edge-quickjs-cli JOBS=4
cmake --build build-edge-quickjs-cli --target edge -j4
```

WASIX is excluded from the current native module-loading acceptance pass. When WASIX verification is re-enabled, changes under `src/`, `lib/`, or `napi/quickjs/` should also run:

```
cd /Users/syrusakbary/Development/edgejs/quickjs-wasm/ && ./build.sh
```

116.6 Development Task Notes

Implementation planning is split under:

`plans/quickjs-wasm/development/dev_003_quickjs_module_loading/`

The task notes define dependency order and write ownership for:

- QuickJS module API and core record implementation;
- Node loader interop and synthetic modules;
- dynamic import, `import.meta`, and required module facades;
- native tests, deferred WASIX rebuilds, and framework smoke checks.

116.7 Non-Goals

- No C++ package resolver.
- No native CommonJS wrapper or export scanner outside Node's JS loader.
- No app-specific patches in framework output or `node_modules`.

- No broad QuickJS fork changes beyond the small APIs needed to support Node-compatible ModuleWrap behavior.

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/development/troubleshooting/node-compat/napi/008_properties.md

117 Known Issue: Property setting semantics

		Remarks
Status	[done]	Implemented as focused QuickJS N-API internal property-setting logic.
Severity	Medium	Property assignment differences can surface as surprising N-API behavior.

117.1 Current State

Property setting logic lives in:

- napi/quickjs/src/internal/napi_set_property.h
- napi/quickjs/src/internal/napi_set_property.cc

js_native_api_quickjs.cc delegates through this focused internal helper rather than carrying local property semantics.

117.2 Known Incompatibility

N-API callers expect property operations to follow Node/V8 behavior, not raw QuickJS descriptor semantics in every edge case. Libraries that attach state to objects during initialization can trip over inherited readonly or accessor-only properties if the backend simply forwards to QuickJS.

117.3 Current Status

Keep public N-API property setters routed through this tested property semantics layer. The layer should document when QuickJS behavior is preserved and when Node/V8 observable behavior is intentionally matched.

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/development/troubleshooting/node-compat/napi/009_quickjs_utilities.md

118 Known Issue: QuickJS utility ownership

		Remarks
Status	[done]	Shared utilities moved to napi_util__ under napi/quickjs/src/internal. Shared helpers are not a feature by themselves, but mistakes here spread widely.
Severity	Low	

118.1 Current State

Shared QuickJS helpers live in:

- napi/quickjs/src/internal/napi_util.h
- napi/quickjs/src/internal/napi_util.cc

The helpers were renamed to lower_case style, redundant code was removed, and call sites now use the internal utility class instead of removed compatibility files.

118.2 Known Incompatibility

This is not a user-visible Node compatibility issue by itself. The risk is that path handling, value conversion, atom handling, or source loading can drift if each subsystem invents its own QuickJS boilerplate.

118.3 Current Status

Keep napi_util__ deliberately boring. General QuickJS mechanics can live there, while Node policy should stay in EdgeJS runtime/bootstrap code or in the focused subsystem that owns the behavior. Avoid turning utilities into a policy dumping ground.

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/development/troubleshooting/node-compat/napi/010_serdes.md

119 Known Issue: Serialization and deserialization

		Remarks
Status	[done]	Implemented as napi_serdes__ under napi/quickjs/src/internal.
Severity	Medium	Node internals and frameworks may import V8 serialization bindings even outside V8.

119.1 Current State

Serialization state and callbacks live in:

- napi/quickjs/src/internal/napi_serdes.h
- napi/quickjs/src/internal/napi_serdes.cc

napi_serdes__ owns serializer, deserializer, and serialized-payload state. unofficial_napi.cc delegates into that class.

119.2 Known Incompatibility

Node exposes V8 serialization through internal and public-facing paths, and some framework code imports those paths even when it only needs a subset of behavior. QuickJS has its own serialization format rather than V8's wire format.

119.3 Current Status

Maintain a support matrix that clearly separates Node-compatible wire semantics, QuickJS-only serialization, and unsupported V8 behavior. If exact V8 wire compatibility is required, return explicit failures instead of silently producing incompatible bytes.

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/development/troubleshooting/node-compat/napi/011_quickjs_wasix_atomics_patch.md

120 Known Issue: QuickJS WASIX atomics guard patch

		Remarks
Status	[open]	Open engine-maintenance issue. Local engine patch must be preserved across QuickJS updates.
Severity	Medium	

120.1 Current State

This note tracks a QuickJS engine-level change that supports the QuickJS runtime on WASIX. It is not part of a removed N-API compatibility directory.

120.2 Source Notes

- plans/quickjs-wasm/development/005_wasix_wasmer_http.md
- plans/quickjs-wasm/development/006_framework_app_adapters.md
- AGENTS.md

120.3 Known Incompatibility

Vendored QuickJS was patched so WASIX builds expose `Atomics` and `SharedArrayBuffer` when `__wasm_atomics__` is defined, instead of excluding all `__wasi__` targets.

120.4 Risk

It may be correct, but it is still a local engine fork delta. It can be lost or misapplied when updating QuickJS.

120.5 Current Status

Turn the change into an explicit patch file with rationale and upstream context, or move to a QuickJS/QuickJS-NG version that supports WASIX atomics cleanly. Add build-time and runtime smoke tests for `Atomics`, `SharedArrayBuffer`, and blocking-wait limitations.

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/development/troubleshooting/node-compat/napi/013_v8_shaped_callsite_methods.md

121 Known Issue: V8-shaped CallSite methods

		Remarks
Status	[caveated]	Implemented in the vendored QuickJS CallSite and raw stack trace APIs, with conservative metadata gaps.
Severity	Medium	Provides Node/V8 stack APIs over incomplete QuickJS metadata.

121.1 Current State

This note tracks a QuickJS engine-level structured-stack issue that supports Node-facing diagnostics. It is not part of a removed N-API compatibility directory.

We implemented the missing QuickJS source-level support in 41b00d4cc34cf79188cd9255f050e95ea1a2e9d6. That commit adds `JS_GetCurrentStackTrace(...)`, raw stack frame objects with V8-shaped fields, and tests for public `Error.prepareStackTrace` data-property behavior.

121.2 Source Notes

- plans/quickjs-wasm/development/troubleshooting/astro-ssr/002_depd_callsite_methods.md
- plans/quickjs-wasm/development/002_native_bootstrap_contextify.md

121.3 Known Incompatibility

QuickJS stack construction honors public `Error.prepareStackTrace`, and native `CallSite` objects expose Node/V8-style methods needed by packages such as `depd`.

121.4 Risk

V8 `CallSite` is not a JavaScript standard. QuickJS does not naturally track all the same metadata, so some methods are approximations or conservative defaults.

121.5 Current Status

The vendored QuickJS `CallSite` prototype now exposes the Node/V8-shaped methods needed by `depd` and Edge diagnostics. The QuickJS N-API helper `napi_callsite__` maps `unofficial_napi_get_call_sites`, `unofficial_napi_get_current_stack_trace`, and caller-location helpers onto `JS_GetCurrentStackTrace(...)`.

The remaining caveat is metadata fidelity. QuickJS does not naturally track all V8 fields, so methods such as `eval` origin, constructor flags, `async` frames, and `promise-all` metadata return conservative defaults when the engine lacks the data. Treat regressions here as diagnostics fidelity issues, not as a reason to restore a removed compatibility directory.

122 Known Issue: QuickJS N-API lifetime tracing

		Remarks
Status	[caveated]	napi_allocator__ owns N-API handle/helper storage and feeds napi_lifetime_tracker__; native event scopes are in place, function metadata uses raw QuickJS externals, and weak/external opaque lifetimes are now delegated to QuickJS instead of env identity maps.
Severity	Medium	Growth can destabilize long-running Edge QuickJS servers, but this note tracks diagnostics rather than a confirmed blocker.

122.1 Current State

The current source of truth is:

- JS_FreeRuntime(...) is enabled in napi/quickjs/src/unofficial_napi.cc. Env release keeps napi_env__ alive until after QuickJS context/runtime finalizers can run, then finalizes instance data and deletes the env.
- napi_allocator__ is the storage mechanism for QuickJS N-API scopes, refs, cleanup hooks, deferred promises, external backing-store hints, and scope-owned values. The current allocator uses fixed-size aligned blocks and stable pointer-shaped public handles.
- napi_lifetime_tracker__ receives allocator create/release hooks. The tracker is compile-time gated and can report slot totals, active counts, scope levels, tags, string/symbol values, object prototype buckets, and external backing hint counts.
- napi_function__::trampoline(...) opens a callback-local handle scope around JS-to-native callbacks, duplicates callback return values before closing that scope, and releases the temporary local handle.
- napi_env__::wrap_external_data(...) creates a raw QuickJS external JSValue for native pointers. That raw value is not itself a napi_value and is not inserted into any handle scope until a public N-API call explicitly wraps it with env->wrap_value_in_current_scope(...).
- The remaining server.js growth diagnosis is not that the allocator leaks by itself. The tracker shows native EdgeJS event paths creating request-local N-API values while env->current_scope() is still the env root scope. Those values then survive until environment teardown.

EdgeJS runtime code under src/ still creates many legitimate long-lived refs for binding singletons, wrapper objects, stream and filesystem requests, async hooks, timers, and process state. Many paths delete refs explicitly, but others rely on object finalizers or environment teardown. The lifetime tracker is used to separate expected persistent ownership from request-local handle retention.

122.2 Action Plan

1. Map allocation and free paths for napi_value__, napi_ref__, napi_env__, handle scopes, escapable handle scopes, callback info, externals, functions, deferred promises, and cleanup hooks.
2. Inspect representative EdgeJS src/ bindings for explicit napi_delete_reference(...), napi_wrap(...) finalizers, and handle-scope discipline.

3. Keep `napi_lifetime_tracker__` wired through allocator hooks so normal builds stay silent but diagnostic builds can report live handle/helper counts.
4. Use the tracker to distinguish root-scope value retention and env-owned refs. Native opaque payloads attached to QuickJS values should be tracked by QuickJS finalizer/weak-record paths, not by env identity side maps.
5. Continue with narrow native event-entry handle scopes around paths that construct N-API arguments before invoking JavaScript.

122.3 Initial Investigation Notes

- `plans/quickjs-wasm/development/001_merge_analysis.md` already identified the key QuickJS N-API runtime types and the preference for internal RAII-style ownership over broad public structs.
- `004_environment.md` records the current teardown caveat around `JS_FreeRuntime(...)`: runtime release is enabled, and finalizer/order bugs should be fixed with env lifetime and allocator ownership, not by disabling runtime release again.
- The first instrumentation pass should remain diagnostic-only and avoid changing normal runtime semantics.

122.4 Instrumentation Added

Added `napi_lifetime_tracker__` under `napi/quickjs/src/internal`. The whole tracker is compile-time gated behind `NAPI_QUICKJS_ENABLE_LIFETIME_TRACKER`, which defaults to OFF. When compiled in, it records created, destroyed, live, and peak counts for:

- `napi_env__`
- `napi_scope__`
- `napi_handle_scope__`
- `napi_escapable_handle_scope__`
- `napi_value__`
- `napi_ref__`
- `napi_callback_info__`
- `napi_external_backing_store_hint__`
- `napi_deferred__`
- `napi_env_cleanup_hook__`

Enable the tracker at configure time, then enable event logging at runtime with:

```
cmake -S . -B build-edge-quickjs-cli -DNAPI_QUICKJS_ENABLE_LIFETIME_TRACKER=ON
cmake --build build-edge-quickjs-cli --target edge -j4
EDGE_TRACE_NAPI_LIFETIME=1 ./build-edge-quickjs-cli/edge server.js
```

When the compile-time flag is off, `napi_lifetime_tracker.cc` is not compiled into `napi_quickjs`, tracker headers are not included by the owner classes, and the `record_create(...)` / `record_destroy(...)` call sites are excluded by the preprocessor.

The tracker dumps live counts at `napi_env__` teardown. When compiled in, it also exposes:

```
napi_quickjs_lifetime_dump("lldb checkpoint")
```

so LLDB can request a snapshot while the process is paused.

Set `EDGE_TRACE_NAPI_LIFETIME_DUMP_EVERY=<N>` to print periodic summary dumps every N creations of a given type.

122.5 Lifetime Map

- `napi_env__` is allocated in `unofficial_napi_create_env_from_context(...)` and destroyed through `DestroyEnvInstance(...)` / `ReleaseEnvScope(...)`.
- `napi_env__` creates a root `napi_scope__`; later allocator-backed scope work made root-scope teardown explicit so root-owned values/refs are released during env destruction.

- Most public N-API value-producing calls wrap JSValues into `env->current_scope()->wrap_value(...)`, which allocates `napi_value__`.
- Explicit handle scopes are allocated by `napi_open_handle_scope(...)` / `napi_open_escapable_handle_scope(...)` and destroyed only by the matching close APIs.
- `napi_ref__` is allocated by `napi_create_reference(...)` and freed by `napi_delete_reference(...)`; weak refs are also tracked on `napi_env__`.
- `napi_external_backing_store_hint__` backs externals, wraps, finalizers, and external array buffers; it is attached to QuickJS as opaque data and is destroyed by QuickJS class or array-buffer finalizers, or by `napi_remove_wrap(...)`.
- Internal QuickJS function metadata must not create `napi_value` wrappers just to smuggle native callback/data pointers into `JS_NewCFunctionData(...)`. `napi_env__::wrap_external_data(...)` creates the raw QuickJS external JSValue; public `napi_create_external(...)` then wraps that JSValue into the current handle scope only when returning a public `napi_value`.
- `napi_callback_info__` is stack-allocated in the QuickJS C-function trampoline for each N-API callback.

122.5.1 Scope and ownership picture

The QuickJS backend has three different lifetime containers that must not be blurred together:

```
napi_env__
|
+-- root scope, level 0
|   owns root-local napi_value slots
|
+-- current scope pointer
|   points at root unless a handle scope is open
|
+-- env-owned ref allocator
|   owns napi_ref slots independently of handle scopes
|
+-- env-owned helper allocators
    cleanup hooks, deferreds, external backing-store hints,
    callback helpers, scope records
```

Opening a handle scope pushes a short-lived local allocation layer:

before callback:

```
current_scope
|
v
[level 0 root scope]
```

inside native event or callback:

```
current_scope
|
v
[level 1 local scope] <-- new napi_value handles live here
|
v
[level 0 root scope]
```

inside helper that must return one value:

```
current_scope
|
v
```

```

[level 2 escapable scope]  -- Escape(value) --> value moves to level 1
    |
    v
[level 1 local scope]
    |
    v
[level 0 root scope]

```

Facts to keep straight:

- A JSValue is the QuickJS engine value.
- A napi_value is a public handle slot around a JSValue.
- env->wrap_value_in_current_scope(js_value, owned) creates the napi_value in whatever env->current_scope() points to at that exact moment.
- If no local handle scope is open, the current scope is the root scope, so temporary napi_value handles become root-owned and survive until env teardown.
- napi_ref is not a local handle. It is persistent env-owned storage that keeps or observes a JSValue across scopes. napi_get_reference_value(...) creates a fresh napi_value in the current scope when code asks for the referenced object again.
- napi_external_backing_store_hint__ is native pointer/finalizer state handed to QuickJS as opaque data. It is allocated from napi_env__::external_backing_stores_, then attached to a QuickJS object with JS_SetOpaque(...) or to an ArrayBuffer as the free-function opaque pointer. It is not a napi_value, does not live in a handle scope, and is not tracked through an env identity map.

The simplest mental model is:

QuickJS object/value

```

|
+--- optional public napi_value handle
|     lives in current handle scope
|
+--- optional napi_ref
|     lives in env ref allocator
|
+--- optional opaque external/wrap hint
|     lives in env external backing-store allocator and is reached through
|     JS_SetOpaque or ArrayBuffer opaque data

```

122.5.2 External data flow

External native pointer data now has two deliberately separate paths.

Internal QuickJS metadata can wrap a native pointer as a raw JSValue only:

```
JSValue callback_data = env->wrap_external_data(reinterpret_cast<void *>(cb));
```

That allocates an external backing-store hint and creates a QuickJS external object with the hint as its opaque payload. It does not allocate a napi_value slot in root or in the current handle scope.

Public napi_create_external(...) uses the same raw primitive, then creates a public handle in the current scope because the N-API contract returns a napi_value:

```

napi_status NAPI_CDECL napi_create_external(napi_env env,
                                           void *data,
                                           napi_finalize finalize_cb,
                                           void *finalize_hint,
                                           napi_value *result)
{
    if (!napi_util__::check_env(env) || result == nullptr)
        return napi_invalid_arg;

```

```

JSValue obj = env->wrap_external_data(data, finalize_cb, finalize_hint);
if (JS_IsException(obj))
{
    return napi_util__::return_pending_if_caught(env,
                                                "Failed to create external object");
}

*result = env->wrap_value_in_current_scope(obj, true);
return (*result == nullptr) ? napi_generic_failure : napi_ok;
}

```

So the public path is:

```

native pointer
-> napi_env__::wrap_external_data(...)
    -> QuickJS external JSValue + env-owned backing hint
-> env->wrap_value_in_current_scope(...)
    -> public napi_value in current scope

```

The internal path is shorter:

```

native pointer
-> napi_env__::wrap_external_data(...)
    -> QuickJS external JSValue + env-owned backing hint
    -> no napi_value handle

```

The QuickJS external object's finalizer invokes the stored finalizer, clears any weak refs for the finalizer target, and releases the backing hint through the env allocator. `napi_remove_wrap(...)` is the explicit early-release path for native objects attached with `napi_wrap(...)`: it extracts the external data, detaches the opaque/wrap record from the JS object, destroys the backing hint, and returns the native pointer to the caller.

122.5.3 Function and class metadata flow

`napi_function__::create(...)` is the core function factory for `napi_create_function(...)`, methods, getters/setters, and constructors created by `napi_define_class(...)`.

The metadata passed into QuickJS C functions should stay raw:

```

JSValue data_values[2];
data_values[0] = env->wrap_external_data(reinterpret_cast<void *>(cb));
data_values[1] = env->wrap_external_data(data);

JSValue fn = JS_NewCFunctionData(env->context(), trampoline, 0, magic, 2,
                                data_values);
JS_FreeValue(env->context(), data_values[0]);
JS_FreeValue(env->context(), data_values[1]);

*result = env->wrap_value_in_current_scope(fn, true);

```

The important ownership detail is that `data_values[]` are QuickJS values used to initialize `JS_NewCFunctionData(...)`. They are freed after QuickJS has captured them. No temporary `napi_value` is created for the callback pointer or the callback data pointer.

The resulting function itself is a public N-API value because callers need a `napi_value`:

```

napi_function__::create(...)
|
+-- raw external JSValue for napi_callback
+-- raw external JSValue for callback data
|
+-- JS_NewCFunctionData(...)

```

stores those values inside the QuickJS function object

```
|
+-- wrap_value_in_current_scope(function)
    returns public napi_value for the function
```

When JavaScript calls the function, `napi_function__::trampoline(...)` opens a callback-local handle scope before building callback arguments and `napi_callback_info__`. Those callback argument `napi_value` handles are local to that callback scope. If the native callback returns a value, the trampoline duplicates the returned `JSValue` before the callback-local scope closes.

Classes are layered on top of functions:

```
napi_define_class(...)
|
+-- napi_function__::create(constructor)
    returns constructor napi_value in current scope
|
+-- JS_NewObject(ctx) prototype
|
+-- for each method/getter/setter:
    napi_function__::create(property callback)
    JS_DefineProperty... on constructor or prototype
|
+-- result = constructor napi_value
```

Constructing an instance uses the constructor's `JSValue` and wraps the created instance into the current scope:

```
napi_new_instance(...)
|
+-- prepare_call_args(env, argc, argv)
    converts input napi_value handles to JSValue array
|
+-- JS_CallConstructor(...)
|
+-- wrap_value_in_current_scope(instance)
    returns instance napi_value in current scope
```

Native instance ownership is separate from constructor/class creation. A later `napi_wrap(...)` attaches a native pointer/finalizer to an object using an external backing-store hint. If QuickJS accepts the object's opaque slot, the hint is attached there. If the object already has incompatible opaque storage, the hint is stored in a separate QuickJS external object under the internal `__napi_wrap__` property. In both cases, `napi_unwrap(...)` reads the hint and returns its native pointer, while `napi_remove_wrap(...)` detaches and destroys the hint early.

122.5.4 QuickJS WeakRef intrinsic and N-API weak refs

`JS_AddIntrinsicWeakRef(ctx)` is QuickJS's installer for the ECMAScript `WeakRef` and `FinalizationRegistry` constructors. It registers QuickJS internal classes, creates the global constructors/prototypes, and relies on internal `JSWeakRefRecord` lists hanging off `JSObject / JSAtomStruct first_weak_ref` fields. During object or symbol teardown, QuickJS calls `reset_weak_ref(...)` to clear `WeakRef` targets, remove `WeakMap/WeakSet` records, and enqueue `FinalizationRegistry` cleanup jobs when allowed.

Because this tree vendors QuickJS, the backend now exposes only the native parts it needs instead of double-tracking target identity in `napi_env__`:

- QuickJS has a new native weak record kind, `JS_WEAK_REF_KIND_NATIVE`, plus a shared native weak anchor/link API. `JS_GetNativeWeakRef(...)` returns the one native weak anchor for a live target `JSValue`, `JS_AddNativeWeakRefLink(...)` links each weak `napi_ref__` into that shared anchor, and `JS_DeleteNativeWeakRefLink(...)` unlinks it.
- Multiple weak `napi_ref__` slots pointing at the same `JSValue` therefore share one QuickJS-owned native weak anchor. Each `napi_ref__` embeds one `JSNativeWeakRefLink weak_link_` by value. When

QuickJS resets the target's internal weak-ref list, it walks those embedded links, each linked `napi_ref__` clears itself to `{ JS_UNDEFINED, nullptr }`, and QuickJS frees the shared native weak anchor as part of target teardown.

- QuickJS has `JS_GetArrayBufferFreeInfo(...)`, which exposes the `ArrayBuffer` `free_func` and opaque pair already stored in `JSArrayBuffer`. During `napi_detach_arraybuffer(...)`, the backend asks QuickJS for that pair and recognizes `napi_external__::free_external_array_buffer_data` directly.
- QuickJS has `JS_GetRefCount(...)` for diagnostics. That count is the engine's total `JSValue` ownership pressure for the pointed-to payload, not the Node-API logical reference count returned by `napi_reference_ref(...)` and `napi_reference_unref(...)`.
- `napi_env__` no longer owns `weak_refs_` or `external_array_buffer_hints_`. There is no env-side identity mirror for those lifetimes.
- `napi_ref__` no longer caches `can_be_weak_`; weak state is the fact that QuickJS linked its embedded `JSNativeWeakRefLink`. It still keeps the N-API logical `ref_count_`, matching the V8 backend, because that value is API state rather than duplicate lifetime tracking.

The resulting ownership rule is deliberately simple:

`JSValue` → `napi_value`
stored in current handle scope

`JSValue` → `napi_ref`
stored in N-API persistent/root ref storage
`napi_ref` keeps the public N-API ref/unref count

native pointer → QuickJS opaque `JSValue` / `ArrayBuffer` opaque
not stored in `napi_env__`
QuickJS owns the death notification/finalizer path

same `JSValue` → multiple weak `napi_refs`
QuickJS target weak list contains one `JSNativeWeakRef` anchor
each `napi_ref` embeds one `JSNativeWeakRefLink` in that anchor's list
target GC walks the embedded links
each subscribed `napi_ref` clears to `JS_UNDEFINED + nullptr`
QuickJS frees the shared native weak anchor

After collection, cloning or otherwise copying from an existing weak `napi_ref` state must preserve emptiness:

```
a = JSValue { someObject }  
b = napi_ref { a, weak_link_ linked }  
c = napi_ref { a, weak_link_ linked }
```

GC collects a

```
b = napi_ref { JS_UNDEFINED, nullptr }  
c = napi_ref { JS_UNDEFINED, nullptr }
```

```
d = clone/copy state from c  
d = napi_ref { JS_UNDEFINED, nullptr }
```

There is no route back from c to the old a; `napi_get_reference_value(c)` returns `nullptr`, and `napi_reference_ref(c)` returns 0.

122.5.5 2026-05-13 external data split

Development plan for this pass:

1. Add `napi_env__::wrap_external_data(void *data)` as the internal raw QuickJS external constructor.

2. Keep a finalizer-aware overload for public `napi_create_external(...)` so Node-API finalizer semantics stay intact.
3. Change `napi_create_external(...)` to call `wrap_external_data(...)` and only then wrap the returned `JSValue` into the current scope.
4. Change `napi_function_::__create(...)` to pass raw external `JSValues` into `JS_NewCFunctionData(...)`, with no `napi_create_external(...)` and no `JS_DupValue(napi_quickjs_value_inner(...))` bridge.
5. Verify with the full QuickJS N-API suite and the Edge CTest suite.

Implementation result:

- `napi/quickjs/src/internal/napi_env.{h,cc}` defines `wrap_external_data(...)`.
- `napi/quickjs/src/js_native_api_quickjs.cc::napi_create_external(...)` now uses the raw helper plus `wrap_value_in_current_scope(...)`.
- `napi/quickjs/src/internal/napi_function.cc::napi_function_::__create(...)` now stores callback/data metadata as raw QuickJS external values.
- The exact root-scope pattern with `napi_create_external(...)` followed by `JS_DupValue(napi_quickjs_value_inner(...))` is gone from `napi_function.cc`.

Verification:

```
make test-napi-quickjs JOBS=4
ctest --test-dir build-edge-quickjs-cli --output-on-failure
```

Results: 45/45 QuickJS N-API tests passed, and 46/46 Edge CTest tests passed.

122.6 EdgeJS Embedder Findings

Initial inspection found that `src/` did not call `napi_open_handle_scope(...)` / `napi_close_handle_scope(...)` around native event callbacks. After the 2026-05-12 scope pass, the Node-mapped callback boundaries listed below use `edge::HandleScope` or `edge::EscapableHandleScope`. Runtime bindings still rely on explicit `napi_delete_reference(...)`, object finalizers from `napi_wrap(...)`, and environment-slot destructors for persistent state.

Representative paths that do close refs explicitly:

- binding singleton state such as task queue, timers, stream symbols, tcp/pipe constructor refs, and process binding refs deletes old refs before replacing them or in state destructors;
- stream write/shutdown/connect request wrappers keep request and buffer refs until completion/finalizer cleanup;
- filesystem deferred completions destroy created refs on failure and later completion cleanup.

The original missing embedder handle-scope discipline meant temporary values created during callbacks accumulated in the QuickJS root scope unless the backend explicitly deleted them. The remaining lifetime work is now to find unscoped native callback paths outside the current Node-mapped set and to separate truly persistent `napi_ref` ownership from callback-local `napi_value` churn.

122.7 Verification

Built the native CLI:

```
cmake --build build-edge-quickjs-cli --target edge -j4
```

Result: succeeded. The build emitted existing dependency deprecation warnings from OpenSSL/c-ares paths.

Trace smoke:

```
EDGE_TRACE_NAPI_LIFETIME=1 ./build-edge-quickjs-cli/edge -e "console.log('life smoke')" \
  > /private/tmp/edge-lifetime-eval.log 2>&1
```

Result: succeeded and produced lifetime events. Teardown summary showed:

```
napi_value__ live=10088 peak=10088 created=10280 destroyed=192
napi_ref__ live=3 peak=147 created=148 destroyed=145
napi_callback_info__ live=0 peak=2 created=192 destroyed=192
napi_external_backing_store_hint__ live=2982 peak=2982 created=2982 destroyed=0
```

The existing root `server.js` and `tests/js/webserver.js` failed inside the default sandbox with `listen EPERM`. Rerunning the loopback server with approval allowed:

```
PORT=3311 EDGE_TRACE_NAPI_LIFETIME=1 ./build-edge-quickjs-cli/edge tests/js/webserver.js \
> /private/tmp/edge-lifetime-webserver.log 2>&1
curl -sS http://127.0.0.1:3311/
curl -sS http://127.0.0.1:3311/again
```

Both requests returned `hello`. Around `listen`, `napi_value__` live count was about 11755; after two requests it had reached 12367, while `napi_callback_info__` repeatedly returned to zero.

122.8 Current Leak Suspects

1. Root-scope retention: `napi_env__::~~napi_env__` does not destroy `root_scope__`, and `src/` does not open per-callback handle scopes, so temporary `napi_value__` wrappers are retained for the life of the env.
2. External hint finalization: `napi_external_backing_store_hint__` live count stayed high through short `eval/server` runs. This may be expected until QuickJS GC/finalizers run, but it should be checked with explicit `JS_RunGC(...)` and root-scope cleanup experiments.
3. Request churn: two HTTP requests increased live `napi_value__` counts by hundreds. The next step is to dump between requests under LLDB and correlate the growth with specific callbacks or property/value creation paths.

122.9 Vector-Backed Handle Plan

The next implementation step is to remove per-wrapper allocation for `napi_value__` and `napi_ref__`.

Planned shape:

1. Add an internal `napi_allocator__` that owns opaque handle encoding, decoding, slot construction, and slot release for values and refs.
2. Store actual `napi_value__` entries in `napi_scope__::values_`; the later env-owned reference allocator stores actual `napi_ref__` entries in `napi_env__::refs_`. Public `napi_value` and `napi_ref` handles are encoded allocator handles rather than separate heap-allocated wrapper objects.
3. Reuse freed vector entries by keeping free indexes in each scope instead of deleting wrapper allocations.
4. Keep value handles local to the current scope, with parent-scope lookup for outer handles used from nested scopes.
5. Keep refs persistent by allocating ref slots from `napi_env__::refs_`, because EdgeJS stores refs across callbacks and async completions. They are root-lifetime handles conceptually, but their allocator owner is the env.
6. Destroy the root scope during `napi_env__` teardown so root-owned values release their duplicated JS-Values, then close the env-owned ref allocator during env teardown.
7. Preserve `EDGE_TRACE_NAPI_LIFETIME=1` counters while changing the allocation backend so before/after server traces remain comparable.

122.10 Historical Vector-Backed Handle Implementation

This section records the earlier vector-backed allocator shape. It was later superseded by the fixed-block pointer-handle allocator.

That pass implemented `napi_allocator__<T>` for `napi_value__` and `napi_ref__` slots. The public handles were encoded slot indexes, not addresses of heap-allocated wrappers. Slot index zero was encoded as pointer value one so `nullptr` remained the invalid handle.

At that point, `napi_scope__` owned:

```
napi_allocator__<napi_value__> values_;
napi_allocator__<napi_ref__> refs_;
```

`napi_value__` and `napi_ref__` were reusable slot payloads with custom move operations plus `initialize(...)`, `release()`, and `is_active()` methods. Later allocator work removed that payload protocol in favor of non-copyable, non-movable payloads constructed in-place and destroyed explicitly.

Value handles were allocated from `env->current_scope()`. Ref handles were allocated from `env->root_scope()` because EdgeJS stores refs across callbacks, async completions, finalizers, and binding singleton state. Accessors routed through:

```
napi_quickjs_value_inner(env, value)
napi_quickjs_value_slot(env, value)
napi_quickjs_ref_slot(env, ref)
```

so encoded handles were resolved before reading the slot payload.

`napi_env__::~~napi_env__` destroyed the root scope. This released all root-owned value/ref slots and ran their QuickJS value frees before env teardown completed.

122.11 2026-05-12 Fixed-Block Allocator Plan

Sadhbh noted that the vector-backed allocator can become inefficient for scope, value, and ref churn because vector growth can reallocate storage and requires the reserved-prefix scheme for nested scope handle lookup. The current allocator keeps stable pointer-shaped handles without keeping cached block index vectors or list nodes.

Implementation:

- `napi_allocator__<T, Owner>` allocates fixed-size `block__` instances and exposes only `T *` payload pointers.
- Blocks are owned by circular intrusive `napi_intrinsic_link__` lists. `first_free_` is the sentinel for completely empty blocks, `first_partial_` is the sentinel for blocks with both free and live slots, and `first_used_` is the sentinel for full blocks. Each block has exactly one allocator-list `link_`, and that link is present in exactly one of those three lists except while `destroy(T *)` has intentionally unlinked the block before invoking the payload destructor. `napi_intrinsic_link__` lives header-only in `napi/lib/src/napi_intrinsic_link.h`, stores only `next_link_` and `prev_link_`, owns circular-list operations such as `link(...)`, `unlink()`, `first()`, and `contains(...)`, and recovers embedded owners with `unsafe_get<&T::link_>()`.
- `block_layout__` exists only to compute the aligned block size. `block__` stores the actual fixed `std::array<slot__, N>` storage, intrusive links, and block-local allocation/free-list behavior directly, so pointer-to-member link offsets refer to `block__` itself.
- Each `block__` is aligned to the nearest power of two that can hold the block layout.
- Each `slot__` stores raw aligned payload storage plus `free_link_` and `used_link_`; there is no per-slot active flag. A slot is active when it is linked through the block's `first_used_slot_` list.
- The allocator recovers the slot from a payload pointer by subtracting `offsetof(slot__, storage_)`, then masks the slot pointer down to the aligned power-of-two block boundary to recover the enclosing block.
- `static_assert(sizeof(block__) == block_alignment__)` keeps the mask-down assumption honest if the block layout changes.
- `allocate(...)` uses the first block from `first_partial_`, then the first block from `first_free_`. If neither list has a block, the allocator creates one. After allocation, the block is relinked to `first_partial_` or `first_used_` according to its final slot state.
- `destroy(T *)` unlinks the slot from the block used-slot list and, if the block was linked at entry, puts the block in vacuum before recording release accounting and destroying the payload. It then links the slot to the block free list and relinks the block to `first_free_`, `first_partial_`, or `first_used_` only if this call put the block in vacuum. Reentrant sibling destruction that enters an already-vacuum block leaves relinking to the outer destroy.
- `take_used()` / `take_next_used()` marks one live slot free and returns its still-constructed payload pointer to the caller for external teardown work.

- Nested scope lookup no longer needs reserved prefix slots. A child allocator rejects a parent-owned pointer handle, and `napi_scope__::value_from_handle` walks to the parent scope as before.
- `napi_scope__` stores a `debug_level_` instead of an allocator index: root scope is level 0, and each child scope is `parent.level() + 1`.
- `napi_env__::next_scope_index_` was removed; scope level is derived from the handle-scope stack shape, not from allocation order.
- `napi_value__` and `napi_ref__` no longer store copied scope levels either; their owning scope is implicit in the allocator that contains the slot.

Complexity and performance notes:

- The allocation hot path is $O(1)$: use the front free block or create one, pop one slot from that block's internal free list, construct the payload in place, and move the block to `first_used_` only when it becomes full.
- The release hot path is $O(1)$: recover the slot from the pointer, mask down to the aligned block base, unlink the slot from `first_used_slot_`, call the payload destructor, and push the slot back through `first_free_slot_`.
- Block scheduling uses only circular `napi_intrinsic_link__` relinks. There is no full-block scan and no cached block-vector erase.
- Handles are stable real pointers. Allocating or relinking a block cannot relocate existing slots, so `napi_value`, `napi_ref`, and scope handles do not depend on vector capacity or integer index encoding.
- Block-local slot arrays keep ordinary allocation/free churn cache-friendly within each block, while avoiding the large copy/move behavior of vector growth.
- `unsafe_owner(T *)` is the raw pointer-to-owner calculation. Ownership checks are separate and use `owns(T *)` or higher-level `env/scope/ref` validation helpers.
- Diagnostic operations such as `count_active()` and `begin() / end()` remain intentionally $O(\text{number of blocks or slots})$, because they are aggregate queries used by tracing, teardown, and tests rather than the request-time allocation hot path.

Cycle and memory-use notes:

- The old vector-backed allocator could turn one allocation into a capacity growth event that moves many existing entries. That is bad for CPU cycles and fundamentally incompatible with pointer-shaped handles unless handles are encoded indexes. The fixed-block allocator removes that relocation class entirely.
- Normal allocation now touches one available-block pointer, one slot pointer, and the payload initialization. Normal release touches the payload release plus a few fields in the owning block. That is the shape we want for request churn: small, predictable work per N-API local.
- The main memory cost is bounded slack inside the last partially used block: with block size N , each allocator can retain up to $N - 1$ unused slots in a non-empty tail block. That trades a little reserved memory for stable handles and $O(1)$ slot reuse.
- Empty blocks live on `first_free_`, partially used blocks live on `first_partial_`, and full blocks live on `first_used_`. `close()` destroys live slots and releases every block from all three mutually exclusive chains.
- Compared with vector capacity growth, this model should reduce allocator cycle spikes under bursty request load and make memory behavior easier to reason about: memory grows in fixed block increments, slots are reused immediately, and no active handle is invalidated by growth.

Verification:

```
cmake --build build-edge-quickjs-cli --target \
  napi_quickjs_test_36_handle_scope \
  napi_quickjs_test_15_function \
  napi_quickjs_test_16_reference \
  napi_quickjs_test_41_instance_data -j4
./build-edge-quickjs-cli/napi-quickjs/tests/napi_quickjs_test_36_handle_scope --gtest_filter=Test
./build-edge-quickjs-cli/napi-quickjs/tests/napi_quickjs_test_15_function --gtest_filter=Test15F
./build-edge-quickjs-cli/napi-quickjs/tests/napi_quickjs_test_16_reference --gtest_filter=Test16F
./build-edge-quickjs-cli/napi-quickjs/tests/napi_quickjs_test_41_instance_data --gtest_filter=Test
make test-napi-quickjs-only
cmake --build build-edge-quickjs-cli --target edge -j4
```

Results:

- Focused allocator-sensitive tests passed.
- `make test-napi-quickjs-only` passed 45/45.
- Edge CLI relink passed.
- After switching block alignment from fixed 64 KiB to computed nearest power-of-two, the focused handle-scope/function/reference tests were rebuilt and passed again.

122.12 2026-05-12 Extending Fixed-Block Allocation To Other N-API Helpers

Action plan before code changes:

1. Reuse `napi_allocator__` for helper objects that currently use QuickJS `js_mallocz(...)` manually.
2. Classify ownership by semantic lifetime rather than by convenience: `napi_callback_info__` is already a good stack object because it is callback-frame data and cannot escape the callback; `napi_env_cleanup_hook__` is environment data and must be owned by `napi_env__`.
3. Keep `napi_deferred__` environment-owned. The public `napi_deferred` handle is intentionally resolved/rejected later, often after the handle scope that created the promise has closed.
4. Keep `napi_external_backing_store_hint__` environment-owned because QuickJS stores raw hint pointers in object opaques and array-buffer finalizer opaques. Ownership separation means QuickJS owns the death notification path, not that the backing-store record must use `new/delete`.
5. Do not add an allocator for `napi_external__` itself because it is a static QuickJS class helper, not an allocated per-instance wrapper. Its allocated payload is `napi_external_backing_store_hint__`.
6. Preserve existing public `create/destroy` entry points where useful, but route them through the owning allocator.

Implementation:

- Left `napi_callback_info__` as a stack object in `napi_function__::trampoline(...)`. It is callback-frame metadata, cannot outlive the native callback, and stack storage is cheaper and clearer than an allocator slot.
- Added env-owned `napi_allocator__` instances for `napi_env_cleanup_hook__`, `napi_deferred__`, and `napi_external_backing_store_hint__`.
- Routed `napi_env_cleanup_hook__::create/destroy(...)` and `napi_deferred__::create/destroy(...)` through `napi_env__`. `napi_external_backing_store_hint__::create/destroy(...)` also uses `napi_env__::external_backing_stores_`; QuickJS finalizer/remove paths still decide when the record dies.
- Closed deferred and cleanup-hook allocators during `napi_env__` teardown while the QuickJS context is still alive. External backing hints are not proactively closed during `prepare_teardown()` because QuickJS object and ArrayBuffer finalizers can still hold raw hint pointers until the context/runtime finalizer phase. The env-owned allocator is finally closed by normal `napi_env__` member destruction after that phase.
- Removed the old helper-local `js_mallocz(...)` / `js_free(...)` allocation path for those env-owned helper objects.

Verification:

```
cmake --build build-edge-quickjs-cli --target \
  napi_quickjs_test_35_promise \
  napi_quickjs_test_38_finalizer \
  napi_quickjs_test_41_instance_data \
  napi_quickjs_test_15_function -j4
./build-edge-quickjs-cli/napi-quickjs/tests/napi_quickjs_test_35_promise --gtest_filter='Test35P
./build-edge-quickjs-cli/napi-quickjs/tests/napi_quickjs_test_38_finalizer --gtest_filter=Test38
./build-edge-quickjs-cli/napi-quickjs/tests/napi_quickjs_test_41_instance_data --gtest_filter=Tes
./build-edge-quickjs-cli/napi-quickjs/tests/napi_quickjs_test_15_function --gtest_filter=Test15Fu
make test-napi-quickjs-only
cmake --build build-edge-quickjs-cli --target edge -j4
```

Results:

- Focused promise, finalizer, instance-data, and function tests passed.
- `make test-napi-quickjs-only` passed 45/45.
- Edge CLI relink passed.

122.13 2026-05-12 Object Prototype Lifetime Buckets

Action plan before code changes:

1. Extend the existing heavier lifetime value dump path so object prototype buckets are captured only on the slower content-dump cadence, alongside repeated string/symbol value counts.
2. Scan active `napi_value__` and `napi_ref__` slots for `JS_TAG_OBJECT`.
3. For each object, derive a debug type label from the object's prototype: `prototype.constructor.name` where possible.
4. Avoid letting diagnostics perturb runtime state: clear and ignore QuickJS exceptions raised while reading prototype or constructor metadata.
5. Collapse object labels into per-scope/per-owner counts and print repeated buckets while reporting how many object labels appeared only once.

Implementation:

- Added object prototype bucket collection to the existing `NAPI_QUICKJS_ENABLE_LIFETIME_STRING_SYMBOL_DUMP` path. This keeps object type inspection on the slower content-dump cadence instead of the normal two-second tag-stat cadence.
- For each active object-valued `napi_value__` or `napi_ref__`, the tracker reads the object's prototype, then reads `prototype.constructor.name`.
- If the prototype is null, the bucket is `<null-prototype>`.
- If prototype/constructor/name lookup throws or produces no usable name, the tracker clears the diagnostic exception and falls back to the QuickJS class name, then finally to `<object>`.
- Output lines use:

```
[napi-lifetime-objects] scope_level=0 napi_value prototype="Object" count=62
[napi-lifetime-objects] scope_level=0 napi_ref prototype="Pipe" count=2
[napi-lifetime-objects] scope_level=0 napi_value singular_object_type_count=1
```

Verification:

```
cmake --build build-edge-quickjs-cli --target napi_quickjs -j4
EDGE_TRACE_NAPI_LIFETIME_STATS=1 ./build-edge-quickjs-cli/edge -e "class Foo{}; const xs=[]; for
make test-napi-quickjs
```

Results:

- `napi_quickjs` rebuilt with the lifetime tracker changes.
- The smoke command printed object prototype buckets, including `Object`, `<null-prototype>`, `Function`, `Array`, `Pipe`, and typed-array buckets.
- The smoke then hit the existing `JS_FreeRuntime` GC-list assertion after `napi_env__ teardown end`; the lifetime dumps had already been emitted.
- `make test-napi-quickjs` rebuilt the test-enabled cache and passed 45/45.

122.14 2026-05-11 Periodic Allocator Stats

Action plan:

1. Preserve the existing `EDGE_TRACE_NAPI_LIFETIME` event/dump behavior when the tracker is compiled in.
2. Add compile-time-off-by-default flags for the tracker and periodic slot stats so normal builds keep the tracker source, includes, and call sites compiled out.
3. Track value/ref allocator slot deltas at allocation, release, prefix reserve, and scope close time instead of scanning all scopes globally.
4. Print at most once every two seconds from allocator activity using a monotonic clock check, with no background thread.

5. Rebuild the native QuickJS Edge target both with the default flag-off build and with the flag enabled long enough to verify output.

122.15 2026-05-11 Server Stats Growth Investigation

Action plan:

1. Treat the then-current vector-backed allocator, teardown fixes, and periodic stats feature as the baseline; inspect their accounting before changing behavior.
2. Confirm whether `build-edge-quickjs-cli` was configured with `NAPI_QUICKJS_ENABLE_LIFETIME_PERIODIC_STATS`; reconfigure only if needed for reproduction and record the change.
3. Reproduce `napi-lifetime-stats` growth with `./build-edge-quickjs-cli/edge server.js` or a minimal local HTTP server, driving a small request batch with `ab` or a simple local client.
4. Use LLDB breakpoints on value wrapping, scope open/close, callback trampolines, and `ref create/delete` to identify whether per-request values are allocated into the root scope, a temporary handle scope, or persistent `ref` storage.
5. Separate true retention from allocator bookkeeping: check whether `slots_total` growth is expected capacity/freelist growth while active reflects still-open scopes or root-scope values.
6. If the evidence shows N-API callback entry is missing a temporary handle scope, implement the narrowest RAII/internal scope around the QuickJS callback trampoline and rerun targeted handle-scope/function/reference tests.

Implementation:

- Added CMake option `NAPI_QUICKJS_ENABLE_LIFETIME_TRACKER`, default OFF. This controls compilation of `napi_lifetime_tracker.cc`, guarded includes of `internal/napi_lifetime_tracker.h`, and all calls to `napi_lifetime_tracker__`. Owner files use the lightweight `NAPI_QUICKJS_LIFETIME_RECORD(...)` / `NAPI_QUICKJS_LIFETIME_DUMP(...)` macros from `internal/napi_lifetime_macros.h`, so the call sites stay compact and compile to no-ops when the tracker flag is off.
- Added CMake option `NAPI_QUICKJS_ENABLE_LIFETIME_PERIODIC_STATS`, default OFF. Enabling it also enables `NAPI_QUICKJS_ENABLE_LIFETIME_TRACKER`, because periodic stats are implemented inside the tracker.
- When enabled, `napi_allocator__<napi_value__>` and `napi_allocator__<napi_ref__>` report total slot and active slot deltas to `napi_lifetime_tracker__`.
- The tracker emits a single-line summary to `stderr` no more than once every two seconds when allocator activity crosses the interval and `EDGE_TRACE_NAPI_LIFETIME_STATS=1` or `EDGE_TRACE_NAPI_LIFETIME=1` is set:

```
[napi-lifetime-stats] napi_value slots_total=10676 active=10676 napi_ref slots_total=176 active=176
```

Enable for a CMake build directory with:

```
cmake -S . -B build-edge-quickjs-cli -DNAPI_QUICKJS_ENABLE_LIFETIME_PERIODIC_STATS=ON
cmake --build build-edge-quickjs-cli --target edge -j4
```

Run with `EDGE_TRACE_NAPI_LIFETIME_STATS=1` for stats-only output, or the existing `EDGE_TRACE_NAPI_LIFETIME=1` for full lifetime event tracing plus stats. Set the CMake option back to OFF and rebuild to return to the default silent build.

Slot-count caveat: `slots_total` is the aggregate size of the backing vectors for currently live allocators. For nested handle scopes, value allocators reserve inactive prefix slots so encoded handles from parent scopes can remain resolvable by falling back to the parent. Those reserved prefix entries are included in `napi_value slots_total` but not in `active`.

Verification:

```
cmake --build build-edge-quickjs-cli --target edge -j4
make test-napi-quickjs-only
cmake -S . -B build-edge-quickjs-cli -DNAPI_QUICKJS_ENABLE_LIFETIME_TRACKER=ON
cmake --build build-edge-quickjs-cli --target edge -j4
cmake -S . -B build-edge-quickjs-cli -DNAPI_QUICKJS_ENABLE_LIFETIME_PERIODIC_STATS=ON
```



```
cmake --build build-edge-quickjs-cli --target edge -j4
./build-edge-quickjs-cli/edge -e "let n=0; const id=setInterval(=>{ console.log('tick', ++n);
cmake -S . -B build-edge-quickjs-cli -DNAPI_QUICKJS_ENABLE_LIFETIME_TRACKER=OFF -DNAPI_QUICKJS_EN
cmake --build build-edge-quickjs-cli --target edge -j4
```

Results:

- Default flag-off rebuild passed.
- make test-napi-quickjs-only passed, 45/45.
- Flag-on rebuild passed and printed the example [napi-lifetime-stats] ... line above after the interval script crossed two seconds.
- The CLI interval and eval smokes still abort after printing user output with QuickJS debug assertion Assertion failed: (list_empty(&rt->gc_obj_list)) in JS_FreeRuntime(...). That assertion was reproduced with the flag off as well, so it is recorded as current teardown state rather than caused by the periodic stats flag.

122.16 Verification After Vector-Backed Handles

Rebuilt native Edge QuickJS:

```
cmake --build build-edge-quickjs-cli --target edge -j4
```

Result: passed.

Ran eval smoke:

```
./build-edge-quickjs-cli/edge -e "const http=require('http'); console.log('ok', typeof http.create"
```

Result:

ok function

Ran lifetime teardown smoke:

```
EDGE_TRACE_NAPI_LIFETIME=1 ./build-edge-quickjs-cli/edge -e "console.log('allocator smoke')"
```

Result: passed. The final teardown summary now reports:

```
napi_value__ live=0 peak=10621 created=10829 destroyed=10829
napi_ref__ live=0 peak=171 created=172 destroyed=172
napi_callback_info__ live=0 peak=3 created=208 destroyed=208
napi_external_backing_store_hint__ live=2916 peak=3202 created=3202 destroyed=286
```

Ran the QuickJS N-API suite:

```
make test-napi-quickjs-only
```

Result: 45/45 tests passed.

122.17 2026-05-15 allocator used/free block refactor

The shared allocator in napi/lib/src/napi_allocator.h is now napi_allocator__<T, Owner> and only exposes typed payload pointers. The old handle-parameter slab allocator surface, cached full/available vectors, and std::list block storage are gone.

Current shape:

- Blocks are owned by circular intrusive napi_intrinsic_link__ lists. first_free_ owns completely empty blocks; first_partial_ owns blocks with both free and live slots; first_used_ owns full blocks. Each block is on exactly one of those lists through its single link_, except during the explicit vacuum window in destroy(T *). The link implementation is header-only in napi/lib/src/napi_intrinsic_link.h; it stores only next_link_ and prev_link_, with unsafe_get<T::link_>() doing owner recovery from an embedded link pointer.

- Slots contain `storage_`, `free_link_`, and `used_link_`. There is no slot active flag and no per-block active counter. A block is full when its block-local `first_free_slot_list` is empty; a block is empty when its block-local `first_used_slot_list` is empty.
- `allocate(...)` is $O(1)$: allocate from the first block in `first_partial_`, then `first_free_`, creating a block only when both lists are empty. Relink the block to `first_partial_` or `first_used_` after the slot is taken.
- `destroy(T *)` remembers whether the block was linked at entry, unlinks the slot from the block used list, unlinks a linked block before running the payload destructor, and links the slot to the block free list afterward. If the block was already in vacuum at entry, it remains in vacuum; otherwise, the call relinks the block to `first_free_`, `first_partial_`, or `first_used_`.
- `take_used()` / `take_next_used()` removes one live slot from a full block first, or from a partial block if no full blocks exist, marks that slot free, records allocator release accounting, and returns the still-constructed payload pointer to the caller for external teardown work.
- `begin()` / `end()` iterate over active slots by walking `first_used_` and then `first_partial_`, following each block's `first_used_slot_chain` through slot `used_link_` links. `count_active()` walks the same full and partial lists.
- `owns(T *)` is implemented from `unsafe_owner(T *)`, which derives the block from the slot address and checks the block owner.

Verification:

```
cmake --build build-edge-quickjs-cli --target edge -j4
make test-napi-quickjs-only
```

Result: edge target built, and the QuickJS N-API suite passed 48/48.

122.18 2026-05-13 Async-Wrap Destroy Hook Scope Fix

The remaining Function/Object `napi_value` growth under `server.js` was tracked to `async-wrap` destroy-hook draining. A live LLDB run against `./build-edge-quickjs-cli/edge server.js` showed repeated root-scope Function handle creation from this stack:

```
napi_env__::wrap_value_in_current_scope(...)
-> napi_get_named_property(..., utf8name="destroy", ...)
-> internal_binding::EmitDestroyHookForAsyncId(...)
-> internal_binding::DrainQueuedDestroyHooks(...)
-> EdgeRuntimePlatformDrainImmediateTasks(...)
-> edge::Environment::OnImmediateCheck(...)
-> uv_run(..., UV_RUN_DEFAULT)
-> RunEventLoopUntilQuiescent(...)
```

`EmitDestroyHookForAsyncId(...)` is invoked from the native immediate/check drain, not from a JS callback trampoline, so it did not inherit the temporary `quickjs_callback_handle_scope__`. The "destroy" property lookup was therefore wrapping the hook function into the root scope on each drain. The adjacent `IsDestroyHookAlreadyHandled(...)` helper also rehydrates the destroyed object reference and reads the "destroyed" flag, so it needs the same local-scope protection.

Implementation:

- Added `edge::HandleScope` to `src/internal_binding/binding_async_wrap.cc::EmitDestroyHookForAsyncId(...)`
- Added `edge::HandleScope` to `src/internal_binding/binding_async_wrap.cc::IsDestroyHookAlreadyHandled(...)` after the existing `env/reference` null guard.

Verification used a Debug edge build with all lifetime diagnostics enabled:

```
CMAKE_BUILD_TYPE=Debug \
EXTRA_CMAKE_ARGS='-DNAPI_QUICKJS_ENABLE_LIFETIME_TRACKER=ON -DNAPI_QUICKJS_ENABLE_LIFETIME_PERIODIC_CHECKS=ON' \
make build-edge-quickjs-cli
```

```
CMAKE_BUILD_TYPE=Debug \
```

```
EXTRA_CMAKE_ARGS='-DNAPI_QUICKJS_ENABLE_LIFETIME_TRACKER=ON -DNAPI_QUICKJS_ENABLE_LIFETIME_PERIODIC_DUMP=ON'
make test-napi-quickjs
```

```
cd /Users/sadhbh/src/dev/edgejs/napi
CMAKE_BUILD_TYPE=Debug make test-native-quickjs
```

Both test suites passed 45/45.

Runtime profile used the patched server on an alternate port so the existing 8080 run stayed untouched:

```
PORT=3316 EDGE_TRACE_NAPI_LIFETIME_STATS=1 ./build-edge-quickjs-cli/edge server.js
ab -n 5000 -c 10 http://127.0.0.1:3316/
```

After 5,000 requests, the periodic full dump showed stable active value counts instead of the previous 1,500-per-period growth:

```
napi_value created=1331955 released=1331512 x[i-1]=443 speed=0
napi_ref created=81000 released=80802 x[i-1]=198 speed=0
```

```
[napi-lifetime-objects]
Function values:x=109 speed=0 refs:x=64 speed=0
Object values:x=58 speed=0 refs:x=93 speed=0
```

Conclusion: `binding_async_wrap.cc` had the same missing native callback scope shape as the earlier `stream/parser` paths. The `destroy-hook` property lookup now lands in the short-lived local scope instead of the root scope, and the observed Function/Object active counts stay flat under HTTP load.

122.18.1 2026-05-13 Internal Binding Async Scope Audit

Follow-up review request: scan `src/internal_binding` for the same class of leak as `binding_async_wrap.cc`, especially async workloads where a native/libuv callback starts JS-facing N-API work without an explicit local scope. The current truth from the scan is that more candidates exist.

Definite missing local scopes:

- `src/internal_binding/binding_fs_event_wrap.cc::OnEvent(...)` is registered through `uv_fs_event_start(...)`. It rehydrates the wrapper, reads the "onChange" property, builds status/event/filename values, and calls JS via `EdgeMakeCallback(...)`. It has no `edge::HandleScope`.
- `src/internal_binding/binding_fs.cc::OnStatWatcherChange(...)` is registered through `uv_fs_poll_start(...)`. It rehydrates the wrapper, reads "onChange", creates the status value and stat array, then calls JS. The other async fs completions in this file already have scopes, but this `StatWatcher` callback does not.
- `src/internal_binding/binding_messaging.cc::OnMessagePortAsync(...)` is a `uv_async_t` callback and calls `ProcessQueuedMessages(...)`. That path deserializes payloads, restores transferred ports, creates event objects and arrays, and calls JS through `EmitMessageToPort(...)`. It has no `edge::HandleScope`.
- `src/internal_binding/binding_worker.cc::OnWorkerCompletionAsync(...)` is a parent-thread `uv_async_t` callback. It rehydrates task callback refs, builds result/error objects and strings, and calls `ondone`. It has no `edge::HandleScope`.

Potential/brittle path:

- `src/internal_binding/binding_performance.cc::PerformanceEmitEntry(...)` rehydrates an observer callback and calls JS. Current observed callers appear to be covered by other scopes, but the helper itself does not document or enforce that requirement. It should either open a scope itself or have an explicit "caller must already be scoped" contract.

Covered paths found during the same scan:

- `src/internal_binding/binding_zlib.cc` async completion runs through `src/node_api.cc::UvAfterWork(...)` which opens `edge::HandleScope` before invoking the N-API async-work completion callback.

- Handle close notifications are covered by `src/edge_handle_wrap.cc::EdgeHandleWrapMaybeCallOnClose(...)`, which opens a scope before rehydrating and invoking JS close hooks.
- Stream listener callbacks are intended to be covered by `src/edge_stream_listener.cc`, whose scoped dispatch helpers open `edge::HandleScope` around listener calls when an env is present.
- `src/edge_environment.cc` drains interrupt and threadsafe-immediate tasks with `edge::HandleScope`, so worker parent-completion foreground tasks such as `CallWorkerOnExit(...)` look covered when reached through that scheduler.

Next continuation point: patch the four definite callbacks first, then rebuild with lifetime stats and stress the matching workloads (`fs.watch`, `fs.watchFile`, `MessagePort`, and worker operation callbacks). Recheck whether `PerformanceEmitEntry(...)` should become self-scoped after inspecting all current and expected callers.

2026-05-14 resolution pass:

- Added `edge::HandleScope` to `src/internal_binding/binding_fs_event_wrap.cc::OnEvent(...)`.
- Added `edge::HandleScope` to `src/internal_binding/binding_fs.cc::OnStatWatcherChange(...)`.
- Added `edge::HandleScope` to `src/internal_binding/binding_messaging.cc::OnMessagePortAsync(...)`.
- Added `edge::HandleScope` to `src/internal_binding/binding_worker.cc::OnWorkerCompletionAsync(...)`.

The patch keeps the scope at the native async-entry boundary, immediately after the existing env guard. If the scope cannot open, the callback returns before creating or rehydrating any public `napi_value`.

Verification:

```
make build-edge-quickjs-cli JOBS=4
```

Result: passed. This is the edge-side compile/link check for the touched `src/internal_binding` files. The build emitted existing vendored/OpenSSL deprecation warnings, but no errors from the scope patch.

```
cd /Users/sadhbh/src/dev/edgejs/napi
CMAKE_BUILD_TYPE=Debug make test-native-quickjs
```

Result: passed, 45/45 QuickJS N-API tests.

2026-05-14 one-minute server.js fs-watch stress:

`server.js` was expanded for local profiling so each HTTP request asynchronously reads `server.js` and returns a randomly selected, fixed-width source line. It also creates a temporary watched file under `/private/tmp`, installs both `fs.watch(...)` and `fs.watchFile(...)`, and periodically writes to that file during request handling. This exercises:

- async fs read completions,
- `binding_fs_event_wrap.cc::OnEvent(...)` through `fs.watch(...)`,
- `binding_fs.cc::OnStatWatcherChange(...)` through `fs.watchFile(...)`,
- the normal HTTP parser/stream callback scopes under load.

Build:

```
CMAKE_BUILD_TYPE=Debug \
EXTRA_CMAKE_ARGS='-DNAPI_QUICKJS_ENABLE_LIFETIME_TRACKER=ON -DNAPI_QUICKJS_ENABLE_LIFETIME_PERIODIC_STATS=ON' \
make build-edge-quickjs-cli JOBS=4
```

Run:

```
PORT=3318 EDGE_TRACE_NAPI_LIFETIME_STATS=1 ./build-edge-quickjs-cli/edge server.js
ab -t 60 -c 10 http://127.0.0.1:3318/
```

ab result:

```
Complete requests:      21477
Failed requests:        0
Requests per second:    357.93 [#/sec] (mean)
Time per request:       27.938 [ms] (mean)
Document Length:       155 bytes
```

Last under-load full dump:

```
napi_value created=11976548 released=11976091 x[i-1]=447 peak=2854 speed=-9
napi_ref created=781992 released=781739 x[i-1]=260 peak=285 speed=-1
external_backing_store_hint created=225484 released=222087 x[i-1]=3397 peak=3408 speed=0
```

```
Function values:x=110 speed=0 refs:x=72 speed=1
Object values:x=58 speed=0 refs:x=96 speed=0
FSEvent refs:x=2 speed=0
StatWatcher refs:x=2 speed=0
```

Final dump after stopping the server:

```
napi_value created=11982061 released=11981618 x[i-1]=443 speed=-14
napi_ref created=782279 released=782065 x[i-1]=214 speed=-39
external_backing_store_hint created=225565 released=222178 x[i-1]=3387 speed=-10
```

Release ratios from the final dump:

```
napi_value: 99.9963% released, 443 active
napi_ref: 99.9726% released, 214 active
external_backing_store_hint: 98.4984% released, 3387 active
```

Conclusion: the fs watcher workload did not show unbounded `napi_value` or `napi_ref` growth. `FSEvent` and `StatWatcher` refs remained stable at their expected watcher refs while the callback-generated values were released through the new local scopes.

122.18.2 2026-05-13 `napi_wrap(...)` result references must be weak

LLDB investigation of the server lifetime run showed `napi_ref__::make_weak()` was never called. That ruled out the QuickJS native weak-ref callback path as an active participant in the observed server growth.

The mismatch was in the QuickJS implementation of `napi_wrap(...)`:

```
return napi_create_reference(env, js_object, 1, result);
```

The V8 backend creates the optional `napi_wrap(..., result)` reference with initial refcount 0. That makes the returned wrapper reference weak by default. Embedder code can then explicitly pin it with `napi_reference_ref(...)` while a native handle/request is active and release it back to weak with `napi_reference_unref(...)`.

QuickJS creating this reference with initial refcount 1 made every wrapper ref strong immediately. For handle wrappers such as `TCP`, `WriteWrap`, and `ShutdownWrap`, that means:

```
napi_wrap(..., &wrapper_ref)
-> strong napi_ref
-> wrapped JS object stays alive
-> QuickJS finalizer does not run
-> external backing-store hint also stays alive
```

The fix is to match V8:

```
return napi_create_reference(env, js_object, 0, result);
```

After this change, wrapper references enter `napi_ref__::make_weak()` at creation time. Active native resources still pin themselves explicitly through their normal `napi_reference_ref(...)` / `napi_reference_unref(...)` lifecycle, but idle wrapper refs no longer start life as unintended strong roots.

122.19 2026-05-13 Allocator Handle Decoding Checks

Sadhbh caught a real invariant mismatch in the fixed-block allocator lookup: `napi_value` handles do not have to belong to the current scope. A value can be owned by the current scope, any parent scope, and later possibly a detached scope. Therefore, decoding a public handle into a candidate slot/block must be separate from proving that the current allocator owns that slot.

Current allocator contract:

- The allocator public API is pointer-shaped: `allocate(...)`, `destroy(T *)`, `take_used()` / `take_next_used()`, `owns(T *)`, and `unsafe_owner(T *)`.
- Release builds trust that incoming public handles came from this N-API backend and treat the opaque public handle as the typed payload pointer at the N-API boundary.
- Debug builds derive the owner from the slot/block and assert the expected ownership fact: scope handles must belong to the env scope allocator, value handles must point at a scope owned by the env, and ref handles must belong to the env-owned ref allocator.
- Relationship operations validate the narrower relationship at the point of use instead of adding broad null checks everywhere. `napi_scope__::parent()` stops at the root parent handle. `napi_scope__::delete_value(...)` releases only if the value is owned by that exact scope, so a callback returning a parent-scope value does not destroy the parent handle. `escape_value(...)` remains an exact-scope operation.
- `napi_ref` slots are persistent env-owned slots in `napi_env__::refs_`. They are root-lifetime handles conceptually, but the allocator owner checked by `ref_from_handle(...)` is the env, which also preserves the existing `napi_lifetime__<napi_ref__>` tracking.

Verification after applying the caller-side ownership fixes:

```
cmake --build build-edge-quickjs-cli --target edge -j4
make test-napi-quickjs-only
ctest --test-dir build-edge-quickjs-cli --output-on-failure
cmake --build build-edge-quickjs-cli-debug --target \
  napi_quickjs_test_36_handle_scope \
  napi_quickjs_test_15_function \
  napi_quickjs_test_16_reference \
  napi_quickjs_test_37_reference_double_free -j4
```

Results:

- Release Edge build passed.
- Release QuickJS N-API suite passed 45/45.
- Release full Edge CTest passed 46/46.
- Debug focused handle-scope/function/reference/double-free executables built and passed individually. The Debug CTest registry still contains many `_NOT_BUILT` placeholder entries, so the focused Debug executables are the meaningful Debug verification for this build tree.

122.20 2026-05-13 Allocator Constructor/Destructor Payloads

The allocator payload protocol now matches the `napi_scope__` pattern: allocation constructs the payload directly in its slot, and release destroys the object directly.

Current facts:

- `napi_allocator__` stores raw `alignas(T) std::byte storage[sizeof(T)]` in each slot rather than a live `T` data member.
- `allocate(args...)` calls placement construction:

```
new (address) T(args...);
```

- `release(handle)` and `allocator close()` call the payload's explicit destructor before marking the slot reusable:

```
value->~T();
```

- Debug builds zero the raw slot storage after the destructor returns. Release builds intentionally leave the old bytes alone.
- The allocator no longer requires payloads to implement `initialize(...)` or `release()`.
- Allocator-owned payload types are non-copyable and non-movable: `napi_scope__`, `napi_value__`, `napi_ref__`, `napi_deferred__`, and `napi_env_cleanup_hook__`. `napi_external_backing_store_hint__`

follows the same constructor/destructor and non-copyable/non-movable pattern once routed through `napi_env__::external_backing_stores_`.

Verification after this pass:

```
cmake --build build-edge-quickjs-cli --target edge -j4
make test-napi-quickjs-only
ctest --test-dir build-edge-quickjs-cli --output-on-failure
cmake --build build-edge-quickjs-cli-debug --target \
  napi_quickjs_test_36_handle_scope \
  napi_quickjs_test_15_function \
  napi_quickjs_test_16_reference \
  napi_quickjs_test_37_reference_double_free -j4
./build-edge-quickjs-cli-debug/napi-quickjs/tests/napi_quickjs_test_36_handle_scope
./build-edge-quickjs-cli-debug/napi-quickjs/tests/napi_quickjs_test_15_function
./build-edge-quickjs-cli-debug/napi-quickjs/tests/napi_quickjs_test_16_reference
./build-edge-quickjs-cli-debug/napi-quickjs/tests/napi_quickjs_test_37_reference_double_free
```

Results:

- Release Edge build passed.
- Release QuickJS N-API suite passed 45/45.
- Release full Edge CTest passed 46/46.
- Debug focused handle-scope, function, reference, and reference double-free executables passed.

122.21 2026-05-13 External Backing Store Allocator

`napi_external_backing_store_hint__` is allocator-backed again. The important ownership distinction is:

```
napi_env__::external_backing_stores_
  owns fast/stable native storage for napi_external_backing_store_hint__
```

QuickJS object opaque / ArrayBuffer free opaque

stores the raw pointer and decides when that record should die

`napi_external_backing_store_hint__::create(...)` delegates to `napi_env__::create_external_backing_store(...)` which allocates from:

```
napi_allocator__<napi_external_backing_store_hint,
  napi_external_backing_store_hint__,
  napi_env__> external_backing_stores_;
```

`napi_external_backing_store_hint__::destroy_with_runtime(...)` reads the stored env from the hint and returns the slot through `napi_env__::destroy_external_backing_store(...)`. That keeps allocator-backed lifetime tracking for external hints without reintroducing env identity maps.

The allocator is not closed in `prepare_teardown()`, because QuickJS can still hold raw hint pointers in object and ArrayBuffer finalizers until the context/runtime finalizer phase. The allocator closes as an env member after that phase.

Verification:

```
cmake --build build-edge-quickjs-cli --target edge -j4
make build-napi-quickjs
make test-napi-quickjs-only
ctest --test-dir build-edge-quickjs-cli --output-on-failure
git diff --check
git -C napi diff --check
```

Results:

- Release Edge build passed.

- N-API-enabled rebuild passed.
- QuickJS N-API suite passed 45/45.
- Edge CTest passed 46/46.
- Diff whitespace checks passed.

122.22 2026-05-12 Compact Detailed Lifetime Tables

The compact lifetime dump keeps the rolling three-sample columns:

- $x[i-1]$: the centered current value;
- speed: $x[i] - x[i-2]$;
- accel: $x[i] - 2*x[i-1] + x[i-2]$.

Detailed attribution was added back in the same format:

```
[napi-lifetime-scopes]
  level                x[i-1]          speed          accel

[napi-lifetime-strings]
  string                x[i-1]          speed          accel

[napi-lifetime-objects]
  type                values:x          speed          accel          refs:x          speed          accel
```

Scope rows are aggregated by scope level, so multiple active scopes at level 2 produce one level-2 row with the total active napi_value count. String and object detail rows only print entries whose count is greater than one; singleton strings/object types are collapsed into a single count == 1 summary row. The object table merges napi_value object prototypes and napi_ref object prototypes side by side.

While validating NAPI_QUICKJS_ENABLE_LIFETIME_STRING_SYMBOL_DUMP=1, napi_quickjs_test_16_reference exposed a crash in the old detailed capture path: the tracker attempted to stringify QuickJS symbols by treating the symbol payload as a JSAtom. The detailed dump no longer stringifies symbols; symbols remain visible through the tag table only.

Verification:

```
NAPI_QUICKJS_BUILD_TESTS=1 \
NAPI_QUICKJS_ENABLE_LIFETIME_TRACKER=1 \
NAPI_QUICKJS_ENABLE_LIFETIME_PERIODIC_STATS=1 \
NAPI_QUICKJS_ENABLE_LIFETIME_TAG_STATS=1 \
NAPI_QUICKJS_ENABLE_LIFETIME_STRING_SYMBOL_DUMP=1 \
cmake -S . -B build-edge-quickjs-cli -DEDGE_BUILD_NAPI_TESTS=ON
cmake --build build-edge-quickjs-cli --target edge \
  napi_quickjs_test_16_reference \
  napi_quickjs_test_36_handle_scope \
  napi_quickjs_test_37_reference_double_free \
  napi_quickjs_test_38_finalizer -j4
ctest --test-dir build-edge-quickjs-cli --output-on-failure \
  -R 'napi_quickjs_test_16_reference|napi_quickjs_test_36_handle_scope|napi_quickjs_test_37_refer
EDGE_TRACE_NAPI_LIFETIME_STATS=1 \
  ./build-edge-quickjs-cli/edge -e 'console.log("tracker details smoke")'
```

The focused CTest run passed 4/4. The detailed smoke printed the scopes, strings, and objects tables.

122.23 2026-05-12 HTTP Load Detailed Snapshot

Sadhbh captured the following detailed lifetime dump while running the local HTTP server under load:

```
NAPI LIFETIME TRACKER
=====
```


[napi-lifetime-slots]						
metric	x[i-1]	speed	accel			
napi_value.slots_total	195328	13312	0			
napi_value.active	195225	13418	-6			
napi_value.tracked_active	195225	13418	-6			
napi_ref.slots_total	6144	512	0			
napi_ref.active	6003	400	0			
napi_ref.tracked_active	6003	400	0			
napi_scope.slots_total	256	0	0			
napi_scope.active	1	0	0			
napi_scope.escape_value.calls	0	0	0			
napi_scope.escape_value.succeeded	0	0	0			
napi_scope.escape_value.failed	0	0	0			
[napi-lifetime-scopes]						
level	x[i-1]	speed	accel			
0	195225	13418	-6			
[napi-lifetime-types]						
type	created	released	x[i-1]	peak	speed	
napi_value	404368	202437	195225	201931	13418	
napi_ref	24204	18001	6003	6211	400	
napi_env_cleanup_hook	0	0	0	0	0	
napi_deferred	0	0	0	0	0	
napi_external_backing_store_hint	7937	73	7714	7864	300	
[napi-lifetime-tags] owner=napi_value						
tag	x[i-1]	speed	accel			
symbol	1494	100	0			
string	10278	700	0			
object	129587	8909	-3			
int	13115	903	-1			
bool	4385	300	0			
null	1485	103	-1			
undefined	30485	2100	0			
float64	4396	303	-1			
[napi-lifetime-tags] owner=napi_ref						
tag	x[i-1]	speed	accel			
symbol	5	0	0			
object	5995	400	0			
undefined	3	0	0			
[napi-lifetime-strings]						
string	x[i-1]	speed	accel			
*/	1250	500	0			
/	1250	500	0			
127.0.0.1:8080	1250	500	0			
Accept	1250	500	0			
ApacheBench/2.3	1250	500	0			
Host	1250	500	0			
User-Agent	1250	500	0			
primordials	11	0	0			
internalBinding	7	0	0			
process	7	0	0			
require	6	0	0			
./build-edge-quickjs-cli/edge	3	0	0			
deps/undici/undici.js	3	0	0			
exports	3	0	0			
perIsolateSymbols	3	0	0			
privateSymbols	3	0	0			

10		2	0	0		
count == 1		80	0	0		
[napi-lifetime-objects]						
type	values:x	speed	accel	refs:x	speed	accel
ArrayBuffer	26252	10500	0	-	-	-
Function	20169	8013	1	1314	500	-
Float64Array	18750	7500	0	3	0	-
TCP	8751	3500	0	1252	500	-
Array	8765	3500	0	3	0	-
Object	7585	3013	1	95	0	-
Uint32Array	7502	3000	0	5	0	-
HTTPParser	6250	2500	0	2	0	-
process	3791	1513	1	-	-	-
ShutdownWrap	2500	1000	0	1250	500	-
WriteWrap	-	-	-	1251	500	-
Socket	1250	500	0	-	-	-
<null-prototype>	171	0	0	4	0	-
Signal	33	0	0	2	0	-
Int32Array	-	-	-	4	0	-
TTY	-	-	-	4	0	-
Uint8Array	-	-	-	3	0	-
count == 1	0	0	0	3	0	-

Interpretation:

- The decisive line is [napi-lifetime-scopes] level 0 = 195225, with napi_scope.active = 1. Every active napi_value in this snapshot is in the root scope. No request/callback-local scope is active when these values are created.
- Growth is linear and stable: slot/value speeds stay around 13418 per two-sample window and acceleration is near zero. That points to per-request retention, not compounding retention.
- The repeated strings are exactly request-local HTTP parse data: Host, 127.0.0.1:8080, User-Agent, ApacheBench/2.3, Accept, */*, and /. These should be temporary N-API local values, but they remain in level 0 because the native HTTP/parser path enters N-API without first preparing a scoped lifetime boundary.
- The object table has the same shape: ArrayBuffer, Function, Float64Array, TCP, Array, Uint32Array, HTTPParser, Socket, and ShutdownWrap grow at request-correlated rates. Their value handles are local wrappers allocated into the current scope, and the current scope is the root scope in these native-entry paths.
- napi_ref growth is smaller and separately meaningful. Refs are persistent by design, but Function, TCP, WriteWrap, and ShutdownWrap refs growing by roughly 500 over the same window indicates per-request or per-connection persistent ownership that must be released by completion/finalizer paths.
- napi_external_backing_store_hint growth (x[i-1] = 7714, speed 300) is another separate lifetime issue. It is likely tied to ArrayBuffer/external backing-store finalization rather than local handle scopes alone.

Working conclusion:

The root problem for napi_value growth is not the allocator. The allocator is now correctly showing where live handles are owned. The issue is that EdgeJS native event paths call N-API while env->current_scope() is still the env root scope. In Node/V8, a handle scope is normally prepared around such native/API entry boundaries. In the QuickJS backend, if no scope is prepared before native HTTP/parser/stream callbacks create local values, those local values are owned by root scope and survive until environment teardown.

The next fix should add a small RAIL scope around native event-entry paths that call N-API and do not return a napi_value to their caller. The likely first targets remain the HTTP parser callback path, stream read/event callbacks, and TCP/Pipe connection callbacks. Refs and external backing-store hints should be tracked separately after local value scope containment is fixed.

122.24 2026-05-12 Allocator Hook Lifetime Refactor

The lifetime tracker was moved away from scattered `NAPI_QUICKJS_LIFETIME_MAYBE_DUMP(...)` call sites. `napi_allocator__` now calls `napi_lifetime__<T>::record_create(...)` after slot initialization and `record_release(...)` before slot release, including `allocator_close()`.

The allocator is now owner-aware:

```
template <napi_allocator_payload__ T, napi_allocator_owner__ Owner_, size_t N = 256>
class napi_allocator__
```

The allocator stores `Owner_ *owner_` and passes it to lifetime hooks. Env-owned allocators use `napi_env__` as owner, while `napi_scope__::values_` uses `napi_scope__` as owner so value counting can attribute through the scope.

The generic `napi_lifetime__<T>` is a no-op so untracked allocator payloads can still use `napi_allocator__`. Real specializations are compile-time gated in `napi_lifetime_tracker.h` for:

- `napi_value__`
- `napi_ref__`
- `napi_env_cleanup_hook__`
- `napi_deferred__`
- `napi_external_backing_store_hint__`

The implementation in `napi_lifetime_tracker.cc` keeps one process-static counter/snapshot state. Value and ref records capture tag, string/symbol text, and object prototype names at create time, then remove those snapshots at release time. Periodic stats are triggered from allocator lifetime hooks.

`napi_scope__::escape_value(...)` now has a separate semantic tracker hook that counts every escape attempt and splits successful versus failed escapes. This is reported in the compact stats table.

Lifetime dumps now use compact readable tables. Each metric keeps a three-sample rolling window. Once three samples exist, the displayed point is centered on $x[i-1]$; speed is $x[i] - x[i-2]$, and accel is $x[i] - 2*x[i-1] + x[i-2]$. String/symbol and object details collapse entries with count < 2 into one row, and print one row per repeated value/type.

Verification:

```
cmake --build build-edge-quickjs-cli --target edge -j4
cmake --build build-edge-quickjs-cli --target \
  napi_quickjs_test_16_reference \
  napi_quickjs_test_36_handle_scope \
  napi_quickjs_test_37_reference_double_free \
  napi_quickjs_test_38_finalizer -j4
ctest --test-dir build-edge-quickjs-cli/napi-quickjs/tests \
  --output-on-failure \
  -R 'napi_quickjs_test_16_reference|napi_quickjs_test_36_handle_scope|napi_quickjs_test_37_refer
EDGE_TRACE_NAPI_LIFETIME_STATS=1 \
  ./build-edge-quickjs-cli/edge -e "console.log('tracker smoke')"
```

Build and focused tests passed. The tracker smoke emitted create/release totals and showed `tracked_active` matching active value/ref slots at teardown begin and returning to zero after env value/ref/scope close.

122.25 2026-05-11 Root test_6 Debug Check

Sadhbh reported that `make test-native-quickjs` from `napi/` passed, while a clean root rebuild with `make test-napi-quickjs` failed `napi_quickjs_test_6.Test6.PortedCoreFlow`.

Current action plan:

1. Reproduce from the root build directory, not the standalone `napi/` build.
2. Run `napi_quickjs_test_6` directly, through CTest, and under LLDB.

3. Check both Release and Debug root build shapes, because the root Makefile defaults to Release unless `CMAKE_BUILD_TYPE=Debug` is supplied.
4. If the failure reproduces, inspect the post-test teardown path and callback scope/result ownership before changing code.

Findings:

- The current root-built `napi_quickjs_test_6 --gtest_filter=Test6.PortedCoreFlow` passes directly.
- `ctest --test-dir build-edge-quickjs-cli/napi-quickjs/tests -R napi_quickjs_test_6 --output-on-failure -V` passes the discovered `Test6.PortedCoreFlow` case.
- A clean root Release run via `make test-napi-quickjs` passes 45/45 tests.
- A forced root Debug rebuild via `CMAKE_BUILD_TYPE=Debug make build-napi-quickjs`, followed by `CMAKE_BUILD_TYPE=Debug make test-napi-quickjs-only`, also passes 45/45 tests.
- LLDB initially stopped a stale Release binary in `malloc` while GoogleTest was printing the test result, but after the Debug rebuild LLDB ran the same `Test6.PortedCoreFlow` filter to process exit status 0. No stable `test_6` failure is currently reproducible.

Interpretation:

The `test_6` symptom appears to have been stale-build or configuration-state dependent rather than a remaining object-wrap semantic failure in the current source. The callback trampoline now opens a child handle scope, duplicates the callback result into a QuickJS return value before closing that scope, and deletes the local `napi_value` handle from the current scope. That is the important ownership boundary for object-wrap constructor and method callbacks.

Follow-up implementation:

- Tightened `napi_scope__::delete_value(...)` so deleting a callback return handle only releases a value owned by that exact scope. Lookup still walks parents, but deletion should not; otherwise a callback that returns a parent-scope handle could accidentally invalidate the outer handle while the trampoline is only trying to drop its local return handle.
- Added `test_function` coverage where a callback returns a module-init parent-scope value twice. The second return verifies the trampoline did not destroy the outer handle during the first callback return cleanup.
- Fixed reentrant slot release during teardown: `napi_ref__::release()` and `napi_value__::release()` now mark the slot inactive before calling `JS_FreeValue(...)`. This matters for object-wrap refs because freeing the strong JS value can synchronously run the wrapped object's finalizer, and that finalizer may call `napi_delete_reference(...)` on the same ref.
- Moved instance-data finalization out of `prepare_teardown()` for owned env release. `test_6` object finalizers call `napi_get_instance_data(...)`, and QuickJS can run those finalizers during `JS_FreeContext(...)` and the final `JS_FreeRuntime(...)` GC; the instance data must remain alive until after runtime finalizers have run.

122.26 2026-05-11 Reentrancy Audit

Action plan before code changes:

1. Scan QuickJS N-API paths that call user callbacks or QuickJS operations that can synchronously run finalizers: `JS_FreeValue(...)`, `JS_RunGC(...)`, `JS_FreeContext(...)`, `JS_FreeRuntime(...)`, `JS_DetachArrayBuffer(...)`, N-API finalizers, and env cleanup hooks.
2. Look for state that is cleared only after one of those calls, because a finalizer can re-enter N-API on the same thread before the outer operation finishes.
3. Fix only high-confidence cases with direct user callback/finalizer exposure.
4. Add focused regression coverage where the current N-API suite can observe the issue without depending on timing or multiple threads.

Initial findings:

- `napi_env__::prepare_teardown()` runs cleanup hooks while iterating the `env_cleanup_hooks_` vector. A cleanup hook can call `napi_remove_env_cleanup_hook(...)`, which can erase and destroy the

current or a later entry while teardown still owns an iterator/entry pointer.

- `napi_env__::set_instance_data(...)` and `finalize_instance_data()` invoke the old instance-data finalizer while the old data/finalizer fields are still set. If that finalizer re-enters instance-data APIs, it can observe stale data or trigger duplicate finalization.
- `napi_ref__::rem_ref()` transitions a strong ref to weak by calling `JS_FreeValue(...)`. That can synchronously run a finalizer that deletes the same ref, so the outer unref path must not read or mutate the slot after the free call.

Implementation:

- Env cleanup teardown now pops a hook entry out of the vector before running the user hook. Reentrant removal of that same hook no longer finds it, and removal of a later hook cannot invalidate the teardown iterator.
- Instance-data replacement/finalization now snapshots and clears the old fields before invoking the user finalizer.
- `napi_ref__::rem_ref()` now snapshots the env/value/ref-count state and returns the local count after `JS_FreeValue(...)`, avoiding post-finalizer slot access.
- Added `test_instance_data` coverage where one cleanup hook removes another cleanup hook during env teardown. The removed hook aborts if it runs.

Verification:

```
cmake --build build-edge-quickjs-cli --target \
  napi_quickjs_test_41_instance_data \
  napi_quickjs_test_6 \
  napi_quickjs_test_16_reference \
  napi_quickjs_test_37_reference_double_free -j4
./build-edge-quickjs-cli/napi-quickjs/tests/napi_quickjs_test_41_instance_data --gtest_filter=Test41
./build-edge-quickjs-cli/napi-quickjs/tests/napi_quickjs_test_6 --gtest_filter=Test6.PortedCoreF
./build-edge-quickjs-cli/napi-quickjs/tests/napi_quickjs_test_16_reference --gtest_filter=Test16R
./build-edge-quickjs-cli/napi-quickjs/tests/napi_quickjs_test_37_reference_double_free --gtest_f
make test-napi-quickjs-only
```

Results:

- The focused build passed.
- All four focused test binaries passed.
- `make test-napi-quickjs-only` passed 45/45.

122.27 2026-05-11 String And Symbol Value Dump Plan

Action plan before code changes:

1. Keep the existing slot and tag counters as aggregate diagnostics.
2. Add a separate compile-time flag for the heavier value-content dump, default off.
3. Use a separate 10-second content-dump cadence and the runtime `EDGE_TRACE_NAPI_LIFETIME_STATS=1` gate, while aggregate stats stay on their two-second cadence.
4. Capture only active `JS_TAG_STRING`, `JS_TAG_STRING_ROPE`, and `JS_TAG_SYMBOL` values from `napi_value__` and `napi_ref__`.
5. Split output by scope level and by owner kind, then collapse duplicates into `count=N value="..."` lines so repeated per-request values are visible.
6. Convert strings with `JS_ToCStringLen(...)`; convert symbols with `JS_ValueToAtom(...)` and `JS_AtomToCStringLen(...)`, because direct string conversion throws for symbols.
7. Escape control bytes and cap stored display text to keep periodic output usable.
8. Rebuild the existing `build-edge-quickjs-cli` cache with the new diagnostic flag on, and leave that cache in the diagnostic configuration.

Implementation notes:

- Added `NAPI_QUICKJS_ENABLE_LIFETIME_STRING_SYMBOL_DUMP`, default off.

- The new flag enables tag stats if needed; tag stats already enable periodic stats, and periodic stats enable the lifetime tracker.
- Active string/symbol values are counted in the lifetime tracker by (scope_level, napi_value/napi_ref, tag, escaped value).
- Output lines use:

```
[napi-lifetime-values] scope_level=0 napi_value tag=string count=1000 value="Host"
[napi-lifetime-values] scope_level=0 napi_value tag=symbol count=1002 value="handle_onclose"
```

Verification:

```
cmake -S . -B build-edge-quickjs-cli \
  -DNAPI_QUICKJS_ENABLE_LIFETIME_PERIODIC_STATS=ON \
  -DNAPI_QUICKJS_ENABLE_LIFETIME_TRACKER=ON \
  -DNAPI_QUICKJS_ENABLE_LIFETIME_TAG_STATS=ON \
  -DNAPI_QUICKJS_ENABLE_LIFETIME_STRING_SYMBOL_DUMP=ON
cmake --build build-edge-quickjs-cli --target edge -j4
EDGE_TRACE_NAPI_LIFETIME_STATS=1 ./build-edge-quickjs-cli/edge server.js
for i in {1..20}; do ab -n 50 -c 10 http://127.0.0.1:8080/ >/dev/null; sleep 1; done
```

The build succeeded. During the local server run the 10-second dumps showed string/symbol content counts increasing with request churn:

```
[napi-lifetime-stats] napi_value slots_total=67788 active=67788 napi_ref slots_total=2214 active=
[napi-lifetime-tags] scope_level=0 napi_value symbol=545 string=3628 object=45003 int=4522 bool=
[napi-lifetime-values] scope_level=0 napi_value tag=symbol count=502 value="handle_onclose"
[napi-lifetime-values] scope_level=0 napi_value tag=string count=500 value="Host"
[napi-lifetime-values] scope_level=0 napi_value tag=string count=500 value="127.0.0.1:8080"
[napi-lifetime-values] scope_level=0 napi_value tag=string count=500 value="User-Agent"
[napi-lifetime-values] scope_level=0 napi_value tag=string count=500 value="ApacheBench/2.3"
[napi-lifetime-values] scope_level=0 napi_value tag=string count=500 value="Accept"
[napi-lifetime-values] scope_level=0 napi_value tag=string count=500 value="*/*"
[napi-lifetime-values] scope_level=0 napi_value tag=string count=500 value="/"
```

```
[napi-lifetime-stats] napi_value slots_total=134840 active=134840 napi_ref slots_total=4214 active=
[napi-lifetime-tags] scope_level=0 napi_value symbol=1045 string=7128 object=89529 int=9032 bool=
[napi-lifetime-values] scope_level=0 napi_value tag=symbol count=1002 value="handle_onclose"
[napi-lifetime-values] scope_level=0 napi_value tag=string count=1000 value="Host"
[napi-lifetime-values] scope_level=0 napi_value tag=string count=1000 value="127.0.0.1:8080"
[napi-lifetime-values] scope_level=0 napi_value tag=string count=1000 value="User-Agent"
[napi-lifetime-values] scope_level=0 napi_value tag=string count=1000 value="ApacheBench/2.3"
[napi-lifetime-values] scope_level=0 napi_value tag=string count=1000 value="Accept"
[napi-lifetime-values] scope_level=0 napi_value tag=string count=1000 value="*/*"
[napi-lifetime-values] scope_level=0 napi_value tag=string count=1000 value="/"
```

The cache was intentionally left with the diagnostic flags on.

122.28 2026-05-11 Tracker-Owned Value Extraction Correction

Action plan before code changes:

1. Remove string/symbol extraction helpers from napi_value__ and napi_ref__.
2. Remove direct tag/string diagnostic macros from value/ref call sites.
3. Keep call-site instrumentation to lifetime record macros only: NAPI_QUICKJS_LIFETIME_RECORD(create|destroy|update, this, env_).
4. Add typed overloads on napi_lifetime_tracker__ for napi_value__ and napi_ref__, so C++ overload resolution routes value/ref slots through the diagnostic-aware tracker path.
5. Let the tracker read the slot's env and JSValue, with scope labels supplied by the active-scope scan, extract tag and string/symbol content internally, and maintain active per-slot snapshots so ref value changes can

decrement the old value and increment the new value.

6. Restore periodic aggregate stats to the intended 2-second cadence.
7. Keep string/symbol value dumps on a separate 10-second cadence.
8. Rebuild the existing diagnostic cache and rerun a short local server/request sample.

Implementation notes:

- Removed create/destroy/update lifetime recording for value/ref slots. Because `napi_value__` and `napi_ref__` entries are owned by `napi_allocator__` vectors inside `napi_scope__`, the env scan is the source of truth.
- Removed the global active-value snapshot registry, per-scope tag/string counters, mutexes, atomics, and slot-delta accounting from `napi_lifetime_tracker.cc`.
- Added read-only allocator/scope/env iteration helpers so the tracker scans `napi_env__` → `scopes_` → `values_/refs_` whenever it dumps.
- The only periodic trigger is `NAPI_QUICKJS_LIFETIME_MAYBE_DUMP(env)` at scope/env ownership boundaries such as `wrap_value(...)`, `delete_value(...)`, `wrap_ref(...)`, `delete_ref(...)`, `close()`, and env scope create/destroy.
- Aggregate slot stats and tag breakdowns are computed fresh from active scopes at most every two seconds. String content lines are computed fresh from active slots at most every ten seconds.
- Symbol names/atoms are intentionally not materialized in the value-content dump; symbols remain visible only as tag counts.
- The build cache was intentionally left in diagnostic mode with `NAPI_QUICKJS_ENABLE_LIFETIME_TRACKER=ON`, `NAPI_QUICKJS_ENABLE_LIFETIME_PERIODIC_STATS=ON`, and `NAPI_QUICKJS_ENABLE_LIFETIME_STRING_SYMBOL`.

122.29 2026-05-11 Native Callback Handle-Scope Investigation

Action plan before code changes:

1. Preserve the then-current vector-backed allocator, root-scope teardown, periodic stats, and string/value dump diagnostics as the baseline.
2. Confirm which callback direction is already scoped. In the current worktree, `napi_function__::trampoline` opens a temporary handle scope before invoking the underlying `napi_callback`, so JS-to-native callbacks are already covered.
3. Reproduce or sample the request path around `napi_scope__::wrap_value`, `napi_create_string_utf8`, `napi_get_named_property`, `napi_get_reference_value`, and HTTP/TCP native event callbacks.
4. Determine whether the repeated header/path strings are created before `napi_call_function(...)`. If so, a scope opened inside `napi_call_function` is too late to own those argument handles.
5. Identify the native-to-JS event boundary that should own temporary argument handles: TCP connection, stream read, HTTP parser callbacks, timers, and other `EdgeMakeCallback` / `EdgeAsyncWrapMakeCallback` entry paths.
6. Prefer one shared callback-scope guard at the Edge callback-dispatch layer if it can cover argument construction safely. If that is not possible, document the app/runtime-specific event guards required before changing broad N-API call semantics.
7. Treat return values separately from temporary arguments. Any result that must survive past the callback boundary must be consumed before the scope closes or explicitly escaped/duplicated into the parent scope.
8. Classify `napi_ref` growth separately. Persistent references live in the env root scope and should be deleted by their owners or finalizers; they are not automatically freed by local handle-scope closure.

Initial findings:

- The current `napi_function__::trampoline` already creates `quickjs_callback_handle_scope__` before invoking the native `napi_callback`. It also duplicates a non-null callback return value into a raw QuickJS result and deletes the temporary local handle before returning to QuickJS.
- `src/edge_http_parser.cc` creates the repeated request strings before entering JS: `BuildHeadersArray(...)` creates header field/value strings such as `Host`, `127.0.0.1:8080`, `User-Agent`, `ApacheBench/2.3`, `Accept`, and `*/*`; `ParserOnHeadersComplete(...)` creates the URL string such as `/`.

- Those strings are then passed as arguments to `EdgeMakeCallback(...)` / `EdgeMakeCallbackWithFlags(...)` or, in propagate-exception mode, `napi_call_function(...)`. Because the values are already wrapped by then, an automatic scope opened only inside `napi_call_function(...)` would not own them.
- The intended lifetime model for those request strings is a native callback local scope that opens before the parser/TCP/timer/stream event constructs N-API arguments and closes after the JS callback path has returned and any required task-queue checkpoint has run. Closing that scope should release the local `napi_value` handles for the header/path strings; JavaScript objects that stored the actual strings keep their own QuickJS references independently of the N-API wrapper handles.

Verification:

```
cmake --build build-edge-quickjs-cli --target edge -j4
EDGE_TRACE_NAPI_LIFETIME_STATS=1 ./build-edge-quickjs-cli/edge server.js
for i in {1..12}; do ab -n 50 -c 10 http://127.0.0.1:8080/ >/dev/null; sleep 1; done
```

Result: build passed. The server run showed two-second scanner-derived stats/tag output and a separate ten-second string-only value dump. Representative lines:

```
[napi-lifetime-stats] napi_value slots_total=67661 active=67661 napi_ref slots_total=2207 active=
[napi-lifetime-tags] scope_level=0 napi_value symbol=544 string=3628 object=44905 int=4512 bool=
[napi-lifetime-tags] scope_level=0 napi_ref symbol=5 object=2191 undefined=3

[napi-lifetime-values] scope_level=0 napi_value tag=string count=500 value="Host"
[napi-lifetime-values] scope_level=0 napi_value tag=string count=500 value="127.0.0.1:8080"
[napi-lifetime-values] scope_level=0 napi_value tag=string count=500 value="User-Agent"
[napi-lifetime-values] scope_level=0 napi_value tag=string count=500 value="ApacheBench/2.3"
[napi-lifetime-values] scope_level=0 napi_value tag=string count=500 value="Accept"
[napi-lifetime-values] scope_level=0 napi_value tag=string count=500 value="*/*"
[napi-lifetime-values] scope_level=0 napi_value tag=string count=500 value="/"
[napi-lifetime-values] scope_level=0 napi_value singular_string_count=80
[napi-lifetime-values] scope_level=0 napi_ref singular_string_count=0
```

The direct helper names, old direct diagnostic macros, global snapshot state, and mutex/atomic includes are absent from the tracker:

```
rg -n "NAPI_QUICKJS_LIFETIME_RECORD|record_create|record_destroy|record_update|record_allocator_s
```

122.30 2026-05-11 Handle Representation And Trampoline RAI Update

Action plan before code changes:

1. Remove the temporary internal `napi_handle_scope__` and `napi_escapable_handle_scope__` wrapper classes.
2. Stop using the private `napi_scope_handle__ = void * alias`.
3. Use the public opaque N-API handle types, `napi_handle_scope` and `napi_escapable_handle_scope`, as encoded index-plus-one handles into `napi_env::scopes_`.
4. Keep `napi_scope__` as the single internal scope payload; all QuickJS N-API scopes can support escaping.
5. Expose `napi_scope__ *scope_from_handle(napi_handle_scope scope) const` as the internal decode point instead of exposing current/root raw `napi_scope__ * values`.
6. Make `quickjs_callback_handle_scope__` in `napi_function.cc` the RAI owner for JS-to-native callback execution: create a child scope in the constructor, make it current, restore the parent in the destructor, and destroy the child scope.

Implementation notes:

- Deleted `napi_handle_scope.h/.cc` and `napi_escapable_handle_scope.h/.cc` from `napi/quickjs/src/inter` and removed those source files from `napi/quickjs/CMakeLists.txt`.
- `napi_env__` now returns `napi_handle_scope` from `root_scope()`, `current_scope()`, and `create_scope(...)`; scope payload access goes through `scope_from_handle(...)`.

- `napi_open_handle_scope(...)`, `napi_close_handle_scope(...)`, `napi_open_escapable_handle_scope(...)`, `napi_close_escapable_handle_scope(...)`, and `napi_escape_handle(...)` now operate on encoded scope handles. Escapable handles are represented by casting the same encoded `napi_handle_scope` value; the internal `napi_scope__` tracks whether it has already escaped.
- `quickjs_callback_handle_scope__` no longer depends on the deleted wrapper class. It creates a child scope from the current parent scope and closes it with `env_>destroy_scope(scope_)` on unwind.

LLDB evidence:

```
breakpoint set --name 'napi_function__::trampoline(JSContext*, JSValue, int, JSValue*, int, JSValue*)'
run --gtest_filter=Test3.Ported
bt
```

showed JS calling a native N-API function through the QuickJS C function data path:

```
frame #0: napi_function__::trampoline(JSContext*, JSValue, int, JSValue*, int, JSValue*)
frame #1: js_call_c_function_data
frame #2: JS_CallInternal
frame #8: napi_run_script
frame #12: Test3_Ported_Test::TestBody()
```

Ignoring the env-constructor root-scope allocation and breaking on scope create:

```
breakpoint set --name 'napi_env__::create_scope(napi_handle_scope__*)'
breakpoint modify 1 --ignore-count 1
run --gtest_filter=Test3.Ported
bt
```

showed the callback frame opening inside the trampoline:

```
frame #0: napi_env__::create_scope(napi_handle_scope__*)
frame #1: napi_function__::trampoline(JSContext*, JSValue, int, JSValue*, int, JSValue*) + 140
frame #2: js_call_c_function_data
```

Breaking on scope destroy showed the matching close on the trampoline unwind path:

```
frame #0: napi_env__::destroy_scope(napi_handle_scope__*)
frame #1: napi_function__::trampoline(JSContext*, JSValue, int, JSValue*, int, JSValue*) + 768
frame #2: js_call_c_function_data
```

Verification:

```
cmake --build build-edge-quickjs-cli --target edge -j4
make build-napi-quickjs
make test-napi-quickjs-only
```

Results:

- Edge CLI build passed.
- Test-enabled N-API QuickJS build passed.
- `make test-napi-quickjs-only` passed, 45/45.

Request-load validation:

```
EDGE_TRACE_NAPI_LIFETIME_STATS=1 ./build-edge-quickjs-cli/edge server.js
watch -n 1 ab -n 50 -c 10 http://127.0.0.1:8080/
```

The sandboxed listen failed with EPERM; rerunning the server and watch load with localhost/network approval succeeded. After several `ab -n 50 -c 10` batches, root-scope request strings still grew:

```
[napi-lifetime-values] scope_level=0 napi_value tag=string count=500 value="Host"
[napi-lifetime-values] scope_level=0 napi_value tag=string count=500 value="127.0.0.1:8080"
[napi-lifetime-values] scope_level=0 napi_value tag=string count=500 value="User-Agent"
[napi-lifetime-values] scope_level=0 napi_value tag=string count=500 value="ApacheBench/2.3"
[napi-lifetime-values] scope_level=0 napi_value tag=string count=500 value="Accept"
```

```
[napi-lifetime-values] scope_level=0 napi_value tag=string count=500 value="*/*"
[napi-lifetime-values] scope_level=0 napi_value tag=string count=500 value="/"
```

Conclusion: the trampoline RAI frame is now implemented and verified for JS-to-native N-API function calls, but it is not the lifetime boundary that owns HTTP request header/path argument handles. Those strings are created in native HTTP parser/server code before `napi_call_function(...)` enters JavaScript, so they need a native event callback/request scope opened before argument construction and closed after callback dispatch. `napi_ref` growth remains separate; these root refs are persistent handles and must be classified by owner/finalizer or explicit `napi_delete_reference(...)`, not by local handle-scope closure.

122.31 2026-05-11 String And Symbol Value Dump Plan

Action plan:

1. Keep the periodic slot and tag counters separate from content dumping.
2. Add a new compile-time flag for string/symbol value content dumps so the potentially noisy string capture code is compiled out by default.
3. Reuse active value/ref slot lifecycle hooks, and only record content for `JS_TAG_STRING`, `JS_TAG_STRING_ROPE`, and `JS_TAG_SYMBOL`.
4. Split dump output by scope level and owner kind (`napi_value` versus `napi_ref`) so root-scope retention remains easy to identify.
5. Use QuickJS C-string conversion only while the underlying `JSValue` is known to be alive; store a bounded escaped copy in the tracker.
6. Print the captured string/symbol values on a separate 10-second cadence; aggregate stats remain on the two-second cadence.
7. Rebuild with the stats, tag-stats, and content-dump flags enabled and leave the build cache in that diagnostic state.

122.32 2026-05-11 QuickJS Tag Breakdown Plan

Action plan:

1. Keep the existing periodic slot stats gated behind `NAPI_QUICKJS_ENABLE_LIFETIME_PERIODIC_STATS`.
2. Add a separate compile-time flag for QuickJS value/ref tag breakdowns so the additional counters are compiled out unless explicitly requested.
3. Count active `napi_value__` slots and active `napi_ref__` slots by `JS_VALUE_GET_TAG(...)`, using the QuickJS tag names from `quickjs.h`.
4. Update counts when a value/ref slot initializes, releases, or changes the retained `JSValue` to `JS_UNDEFINED` during weak/empty ref cleanup.
5. Print the tag breakdown alongside periodic stats. The current corrected implementation uses the two-second aggregate stats cadence.
6. Rebuild with stats and tag stats enabled, run a small smoke command, and run focused value/reference/handle-scope tests.

Implementation notes:

- Added `NAPI_QUICKJS_ENABLE_LIFETIME_TAG_STATS`, default OFF. Enabling it adds per-active-slot `JS_VALUE_GET_NORM_TAG(...)` accounting for `napi_value__` and `napi_ref__`.
- This first implementation printed periodic stats every 10 seconds; the later tracker-owned extraction correction restored aggregate stats and tag breakdowns to the intended two-second cadence.
- Tag stats are split by the lifetime tracker's scope levels, so output can distinguish root-scope retention from child-scope retention without storing an index inside `napi_scope__`:

```
[napi-lifetime-tags] scope_level=0 napi_value symbol=794 string=5378 object=67179 int=6781 bool=2
[napi-lifetime-tags] scope_level=0 napi_ref symbol=5 object=3191 undefined=3
```

Sample run:

```
cmake -S . -B build-edge-quickjs-cli \
  -DEDGE_NAPI_PROVIDER=quickjs \
```

```

-DEDGE_BUILD_NAPI_TESTS=ON \
-DNAPI_QUICKJS_ENABLE_LIFETIME_TRACKER=ON \
-DNAPI_QUICKJS_ENABLE_LIFETIME_PERIODIC_STATS=ON \
-DNAPI_QUICKJS_ENABLE_LIFETIME_TAG_STATS=ON
cmake --build build-edge-quickjs-cli --target edge -j4
EDGE_TRACE_NAPI_LIFETIME_STATS=1 ./build-edge-quickjs-cli/edge server.js
ab -n 50 -c 10 http://127.0.0.1:8080/

```

Observed after driving repeated small ab batches for about 30 seconds:

```

[napi-lifetime-stats] napi_value slots_total=34136 active=34136 napi_ref slots_total=1207 active=
[napi-lifetime-tags] scope_level=0 napi_value symbol=294 string=1878 object=22643 int=2269 bool=
[napi-lifetime-tags] scope_level=0 napi_ref symbol=5 object=1191 undefined=3

```

```

[napi-lifetime-stats] napi_value slots_total=67678 active=67678 napi_ref slots_total=2207 active=
[napi-lifetime-tags] scope_level=0 napi_value symbol=544 string=3628 object=44914 int=4526 bool=
[napi-lifetime-tags] scope_level=0 napi_ref symbol=5 object=2191 undefined=3

```

```

[napi-lifetime-stats] napi_value slots_total=101208 active=101208 napi_ref slots_total=3207 active=
[napi-lifetime-tags] scope_level=0 napi_value symbol=794 string=5378 object=67179 int=6781 bool=
[napi-lifetime-tags] scope_level=0 napi_ref symbol=5 object=3191 undefined=3

```

Conclusion from this sample: retained values/refs are all in `scope_level=0`, the root scope. The dominant retained value tag is object, followed by undefined, int, string, and float64; refs are almost entirely object.

Ran loopback webserver smoke with elevated localhost bind permission:

```

PORT=3313 ./build-edge-quickjs-cli/edge tests/js/webserver.js
curl -sS http://127.0.0.1:3313/
curl -sS http://127.0.0.1:3313/again

```

Result: server printed webserver listening on port 3313; both requests returned hello.

Remaining leak suspect after this change: `napi_external_backing_store_hint__` still has live objects at teardown. That is no longer explained by `napi_value__` or `napi_ref__` wrapper retention and should be investigated as a separate QuickJS finalizer/external backing-store lifetime issue.

122.33 2026-05-11 Standalone N-API Segfault Regression Plan

User reported that running `make test-napi-quickjs` from `/Users/sadhbh/src/dev/edgejs/napi` now produces 32 segfault failures, while the root QuickJS CLI build/test path previously passed. This investigation worked with the then-current vector-backed `napi_value__` / `napi_ref__` allocator state and avoided reverting concurrent edits. The current source has since moved to fixed-block `napi_allocator__` storage.

Action plan:

1. Reproduce from the `napi` subdirectory, then reduce to the first crashing binary/test: `napi_quickjs_test_2 --gtest_filter=Test2.Portcd`.
2. Use LLDB on that exact test to prove where an encoded `napi_value` or `napi_ref` handle was still being treated as a direct pointer, or whether the then-current allocator slot lookup collided across scopes/build harnesses.
3. Patch only the focused QuickJS N-API implementation path that bypasses the allocator decode layer, preserving scope-owned value slots, root-scope ref slots, free-list reuse, and root-scope teardown.
4. Verify the targeted test first, then run the `napi` subdirectory `make test-napi-quickjs`; run the root `make test-napi-quickjs-only` if time allows.

Findings and fix:

- The standalone 32-crash pattern was not a handle-storage collision. LLDB on `napi_quickjs_test_2 --gtest_filter=Test2.Portcd` stopped in QuickJS runtime teardown: `JS_FreeRuntime(...)` finalized

GC objects after DestroyEnvInstance had deleted napi_env__. External/wrap finalizers still carried napi_env pointers, so the env must stay alive until after JS_FreeRuntime(...).

- After rebuilding the root harness with CMAKE_BUILD_TYPE=Debug, the remaining root crashes reduced to Test6 and Test33. Test6 stopped in napi_delete_reference(...) -> napi_scope__::delete_ref(...) from MyObject::~~MyObject, called by a QuickJS finalizer after root_scope_ had already been destroyed. The root scope now closes allocator slots first, runs a GC pass while the root-scope owner still exists, then frees the scope storage; a late napi_delete_reference(...) after root teardown is treated as an idempotent cleanup.
- Test33 stopped in js_free_rt(rt=0x12004, ptr=hint) from napi_external_backing_store_hint__::destroy. The bad runtime pointer was stale hint memory: QuickJS calls an ArrayBuffer free callback on JS_DetachArrayBuffer(...), then can call it again when the detached ArrayBuffer object finalizes. External ArrayBuffer hints now keep their hint object alive across detach, invoke the user finalizer only once, and destroy the hint on the final object free path using the JSRuntime* passed by QuickJS.
- Escapable handle ownership was reviewed while checking the allocator model: escape_value(...) wraps the inner JSValue into the parent scope with owned=false, and napi_value__::initialize(..., owned=false) performs the JS_DupValue(...). The child scope later releases its own allocator slot, so the escaped parent slot does not borrow freed storage. A future allocator move_to(...) helper could reduce the temporary duplicate, but the current duplicate-and-release behavior is logically safe.

Verification:

```
./build-edge-quickjs-cli/napi-quickjs/tests/napi_quickjs_test_6 --gtest_filter=Test6.PortedException
./build-edge-quickjs-cli/napi-quickjs/tests/napi_quickjs_test_33_typedarray --gtest_filter=Test33
CMAKE_BUILD_TYPE=Debug make test-napi-quickjs
(cd /Users/sadhbh/src/dev/edgejs/napi && CMAKE_BUILD_TYPE=Debug make test-napi-quickjs)
```

Both full suites reported 45/45 passing.

122.34 LLDB Breakpoints And Calls

Useful breakpoints:

```
breakpoint set -n napi_quickjs_lifetime_dump
breakpoint set -n napi_scope__::wrap_value
breakpoint set -n napi_value__::create
breakpoint set -n napi_ref__::create
breakpoint set -n napi_external_backing_store_hint__::create
breakpoint set -n napi_env__::~~napi_env__
```

Useful LLDB expression while stopped:

```
expr napi_quickjs_lifetime_dump("after request")
```

122.35 Node/V8 Lifetime Comparison

Node's V8 backend does not allocate a C++ napi_value__ wrapper per returned value. node/src/js_native_api_v8.h asserts that v8::Local<v8::Value> and napi_value are one pointer wide, then converts with reinterpret_cast<napi_value>(*local). Rehydration copies that pointer back into a stack v8::Local<v8::Value>.

For older/indirect V8 locals, that pointer is the local handle slot. In that mode, v8::LocalBase<T>::New(...) calls v8::HandleScope::CreateHandle(...); the internal implementation takes isolate->handle_scope_data()->next, extends the handle block if next == limit, advances next, writes the tagged value into the slot, and returns the slot address. Closing a V8 handle scope restores the previous next/limit, so all local slots created inside that scope become invalid as a group.

Modern V8 also has a direct-handle path. Many API implementations in node/deps/v8/src/api/api.cc allocate JS heap objects through the factory as DirectHandles, for example v8::Object::New(...), v8::Array::New(...), v8::Number::New(...), and String::NewFrom*. They then return public Local<T> values through Utils::ToLocal(...) / Utils::Convert(...). With V8_ENABLE_DIRECT_HANDLE,

`Utils::Convert(...)` returns `Local<T>::FromAddress(obj.address())`, so no local handle slot is allocated for that conversion. Without direct handles, the same conversion uses `Local<T>::FromSlot(indirect_handle(obj))` which routes through the handle-scope slot machinery above. Node's bundled V8 defaults direct handles to the conservative-stack-scanning setting; `node/common.gypi` sets `v8_enable_conservative_stack_scanning` to 0 by default, so exact behavior is build-configuration dependent.

Node-owned allocations are therefore concentrated in the surrounding lifetime objects:

- `napi_handle_scope` and `napi_escapable_handle_scope` are new-allocated wrappers around stack-like V8 handle scopes, then deleted on close.
- `napi_ref` is a real Reference object that owns a V8 persistent handle and links into the env reference lists; `napi_get_reference_value(...)` creates a fresh local handle slot from that persistent.
- Function callback arguments, `this`, `new.target`, return values from V8 factory/property/call APIs, and escaped handles are all surfaced as borrowed local handle slots rather than per-value heap wrappers.

This is the main design gap for QuickJS tracing: `napi_value__` creation in the QuickJS backend is closer to allocating per-value handles. If those handles live in the env root scope, the backend behaves more like an always-open V8 handle scope whose cursor never rewinds.

122.36 QuickJS Scope Model

The QuickJS backend has a scope stack, but it is implemented differently from V8's local handle blocks. `napi_env__` creates one `root_scope__` during env construction and initializes `current_scope__` to that root. Opening a handle scope allocates a `napi_handle_scope__` with `parent = current_scope__`, then makes it current. Closing requires that the supplied scope is exactly the current scope, restores `current_scope__` to `scope->parent()`, and destroys the scope.

Each scope owns fixed-block `napi_allocator__` storage for local values:

```
napi_allocator__<napi_value__, napi_scope__> values_;
```

Public `napi_value` handles are stable pointers to allocator payload slots. `napi_scope__::wrap_value(...)` initializes an active value slot in the current scope. Releasing a handle marks the slot inactive and returns it to the block's free list. Closing a scope releases active slots in reverse order and drops the allocator blocks.

Persistent `napi_ref` handles are env-owned rather than scope-owned:

```
napi_allocator__<napi_ref__, napi_env__> refs_;
```

That matches their semantic lifetime: refs can outlive the local handle scope that created them and are released by explicit `napi_delete_reference(...)`, object/finalizer cleanup, or env teardown.

Escapable scopes duplicate the underlying QuickJS value into the parent scope: `escape_value(...)` calls `parent->wrap_value(value->get_inner(), false)`. That creates a separate parent-scope `napi_value__` with its own `JS_DupValue(...)`; closing the child then frees the child wrapper without invalidating the escaped parent wrapper.

Callback invocation now opens an automatic temporary N-API handle scope in `napi_function__::trampoline(...)`. The trampoline stack-allocates `napi_callback_info__`, invokes the native callback, duplicates a non-null callback return value into a raw QuickJS result, and deletes the temporary local return handle before the child scope closes.

That trampoline scope does not cover earlier native libuv/event-entry work such as `OnConnection(...)` or HTTP parser callbacks that create N-API argument values before entering JavaScript. Those paths still need explicit Edge-side handle scopes or an equivalent backend-owned entry scope.

The root scope is destroyed during environment teardown. That releases root-owned value slots before env teardown completes; remaining teardown failures should be treated as QuickJS GC/finalizer or external backing-store lifetime issues, not as allocator storage leaks.

122.37 2026-05-11 Server Stats Growth Findings

Reproduced the reported shape with the native QuickJS CLI and the local server.js:

```
cmake -S . -B build-edge-quickjs-cli \
  -DNAPI_QUICKJS_ENABLE_LIFETIME_TRACKER=ON \
  -DNAPI_QUICKJS_ENABLE_LIFETIME_PERIODIC_STATS=ON
cmake --build build-edge-quickjs-cli --target edge -j4
PORT=3312 EDGE_TRACE_NAPI_LIFETIME_STATS=1 ./build-edge-quickjs-cli/edge server.js
ab -n 5000 -c 10 http://127.0.0.1:3312/
```

Before adding any callback scope experiment, periodic stats climbed during request traffic from roughly:

```
napi_value slots_total=11733 active=11733 napi_ref slots_total=196 active=196
napi_value slots_total=487814 active=487814 napi_ref slots_total=8211 active=8203
napi_value slots_total=1677931 active=1677931 napi_ref slots_total=28211 active=28203
```

The primary value growth is temporary N-API handle creation in native event entry paths while env->current_scope() is still the env root scope. LLDB stack samples from a single request showed:

```
OnConnection
  EdgeStreamBaseGetWrapper
  napi_get_reference_value
  napi_scope__::wrap_value

OnConnection
  napi_get_named_property(..., "onconnection")
  napi_scope__::wrap_value

OnConnection
  EdgeStreamBaseMakeInt32
  napi_create_int32
  napi_scope__::wrap_value
```

At the wrap_value(...) breakpoint in these stacks:

```
this == env->root_scope()
this->parent() == nullptr
env->current_scope() == env->root_scope()
```

This means the large napi_value active == slots_total growth is not mainly QuickJS heap values leaking from JavaScript function calls. It is QuickJS N-API handle wrappers for normal native operations such as reference lookup, property access, and small integer/value creation being allocated into the always-open root scope.

An internal temporary scope around napi_function__::trampoline(...) does open and close around JS-to-native callback bodies. LLDB confirmed the scope is created and destroyed for the TcpCtor callback during napi_new_instance(...). However, that does not cover the earlier native libuv entry work in OnConnection(...), so it is insufficient as a complete server-growth fix.

Ref growth is a separate but related signal. LLDB samples showed persistent refs created for per-connection wrappers and active-resource tracking:

```
napi_wrap
  TcpCtor
  napi_create_reference

Environment::RegisterActiveHandle("TCPSocketWrap")
  EdgeStreamBaseSetWrapperRef
  napi_create_reference
```

Deletes were also observed on request cleanup paths:

```
FreeWriteReq
  EdgeUnregisterActiveRequestToken
  Environment::UnregisterActiveRequest
  napi_delete_reference
```

So some `napi_ref` churn is expected live socket/request ownership, but the value-slot growth is root-scope temporary handle retention.

The old vector-backed allocator's reserved-prefix scheme was also checked while experimenting with automatic callback scopes. Materializing parent prefixes in child scopes made each callback scope expensive once the root had many slots. The current fixed-block pointer-handle allocator removed that reserved-prefix class of overhead.

Current conclusion: the right fix is to open short-lived handle scopes around native event-entry / libuv callback processing that calls N-API before entering JS, or to add an equivalent backend-owned entry scope for those EdgeJS runtime boundaries. A trampoline-only scope is useful but does not address `OnConnection(...)` and similar native paths.

122.38 2026-05-12 Native Event Scope Plan

The active reproduction server is the repository-local `/Users/sadhbh/src/dev/edgejs/server.js`, a minimal `node:http` server that writes a plain text response. The repeated strings in the root-scope dump map directly to `BuildHeadersArray(...)` and URL-string creation in `src/edge_http_parser.cc`, reached from the consumed stream listener while `llhttp_execute(...)` calls the parser callbacks.

Completed action plan for the first native-event scope pass:

1. Add a tiny Edge-side RAI helper around `napi_open_handle_scope(...)` / `napi_close_handle_scope(...)`.
2. Use it only around native libuv/event callbacks that call N-API and do not return a `napi_value` to their caller.
3. Start with the server hot path: TCP/Pipe `OnConnection(...)`, consumed HTTP parser reads, and stream read callbacks that create `ArrayBuffer` handles before calling JavaScript.
4. Leave normal N-API callback functions and helpers that return `napi_value` untouched unless they use an escapable handle scope.

122.38.1 Node/V8 native event scope comparison

The direct leak-shaped call in the consumed HTTP parser path is:

```
napi_value ret = ParserExecuteCommon(p, data, len);
```

inside `src/edge_http_parser.cc::DispatchConsumedParserRead(...)`. That return value is only used as the `kOnExecute` callback argument. In the current QuickJS backend, because no local handle scope has been opened for the native stream-read entry, `ParserExecuteCommon(...)` wraps the return value in `env->current_scope()`, which is still the env root scope. The same root-scope ownership also applies to `parser_obj`, `onexecute`, and request-local values created while `llhttp_execute(...)` runs parser callbacks.

Node's V8 implementation opens the native-event handle scope explicitly before this kind of work:

- `node/src/node_http_parser.cc::Parser::OnStreamRead(...)` opens `HandleScope scope(env()->isolate())` before calling `Execute(...)`, reading `kOnExecute`, and calling JavaScript.
- `Parser::Execute(...)` opens an `EscapableHandleScope`, builds the parser result/error value, then escapes the return value back to the surrounding `OnStreamRead(...)` scope.
- `node/src/connection_wrap.cc::ConnectionWrap::OnConnection(...)` opens `HandleScope handle_scope(env->isolate())` before instantiating the accepted socket object, building `argv`, and calling `onconnection`.
- `node/src/stream_base.cc` listener implementations that materialize JS values also open scopes themselves: `EmitToJSSStreamListener::OnStreamRead(...)`, `CustomBufferJSLListener::OnStreamRead(...)`, and `ReportWritesToJSSStreamListener::OnStreamAfterReqFinished(...)`.

The native Node HTTP read flow is:

```

HTTP bytes arrive from socket
-> libuv read callback
-> StreamBase::EmitRead(...)
-> DebugSealHandleScope
-> listener_>OnStreamRead(nread, buf)

    HTTPParser::OnStreamRead(...) {
        HandleScope scope;                                // level 1 opens

        ret = Execute(buf.base, nread);

        Execute(...) {
            EscapableHandleScope scope;                    // level 2 opens

            current_buffer = buf;
            llhttp_execute(...)

            -> parser callback: on_headers_complete / on_body / Flush
                may open its own HandleScope when calling JS

            nread_obj = Integer::New(...)
            return scope.Escape(nread_obj);                // escape to level 1
        }                                                  // level 2 closes

        cb = object()->Get(kOnExecute)
        current_buffer = buf;
        MakeCallback(cb, [ret])
        current_buffer = null
    }                                                        // level 1 closes

```

The generic Node stream dispatch layer does not open the allocation scope for listeners. `node/src/stream_base-inl.h::StreamResource::EmitRead(...)` wraps the listener call in `DebugSealHandleScope`, which is a debug barrier that rejects handle creation unless the listener opens an inner `HandleScope`. This matches V8's handle discipline: `v8::HandleScope::CreateHandle(...)` checks that there is an active non-sealed handle scope before creating a local handle.

The analogous EdgeJS paths now provide explicit `edge::HandleScope` / `edge::EscapableHandleScope` coverage at the matching native-event boundaries where EdgeJS has implemented the corresponding Node subsystem. The current scope audit table is:

location	Node	Edge
HTTP parser stream read: node/src/node_http_parser.cc::Parser::OnStreamRead(...) / src/edge_http_parser.cc::ParserConsumedListenerOnRead(...) via EdgeStreamEmitRead(...)	yes	yes
HTTP parser execute return: Parser::Execute(...) / ParserExecuteCommon(...)	yes, EscapableHandleScope	yes, edge::EscapableHandleScope
Stream alloc/read entry: node/src/stream_wrap.cc::LibuvStreamWrap::OnUvAlloc(...), OnUvRead(...) / EdgeStream- BaseOnUvAlloc(...), EdgeStreamBaseOnUvRead(...)	yes	yes

location	Node	Edge
Stream JS listener implementations: node/src/stream_base.cc listener classes / src/edge_stream_listener.cc dispatch	yes	yes
Stream write/shutdown completion: AfterUvWrite(...), AfterUvShutdown(...) / OnWriteDone(...), OnShutdownDone(...)	yes	yes
TCP/Pipe connect completion: Connection- Wrap::AfterConnect(...) / edge_tcp_wrap.cc::OnConnectDone(...), edge_pipe_wrap.cc::OnConnectDone(...)	yes	yes
TCP/Pipe accepted connection: Connection- Wrap::OnConnection(...) / edge_tcp_wrap.cc::OnConnection(...), edge_pipe_wrap.cc::OnConnection(...)	yes	yes
Handle close callback: node/src/handle_wrap.cc::HandleWrap::OnClose(...) / EdgeHandleWrapMaybeCallOn- Close(...), MaybeCallHandleOnClose(...)	yes	yes
DNS query completion: node/src/cares_wrap.h::ParseError(...), CallOnComplete(...), node/src/cares_wrap.cc parse methods / edge_cares_wrap.cc::CompleteQuery(...)	yes	yes
DNS getaddrinfo: AfterGetAddrInfo(...) / OnGetAddrInfo(...)	yes	yes
DNS getnameinfo: AfterGetNameInfo(...) / OnGetNameInfo(...)	yes	yes
UDP receive/send completion: UDPWrap::OnRecv(...), UDPWrap::OnSendDone(...) / UdpWrap::OnRecv(...), UdpWrap::OnSendDone(...)	yes	yes
Signal callback: node/src/signal_wrap.cc signal lambda / edge_signal_wrap.cc::OnSignal(...)	yes	yes
Child process exit: node/src/process_wrap.cc::OnExit(...) / edge_process_wrap.cc::EmitOnExit(...)	yes	yes

location	Node	Edge
N-API async work completion: node/src/node_api.cc::AsyncWorker::AfterThreadPoolWork(...) / src/node_api.cc::UvAfterWork(...)	yes	yes
Async fs completion: node/src/node_file.h::FSReqAfterScope used by AfterNoArgs(...), AfterStat(...), AfterInteger(...), AfterScanDir(...) / bind- ing_fs.cc::AfterAsyncFsReq(...)	yes	yes
Async file-handle close/read completion: node/src/node_file-inl.h / bind- ing_fs.cc::AfterFileHandleClose(...), AfterFileHandleRead(...)	yes	yes
Async fs dir completion: Node FS after-scope pattern / bind- ing_fs_dir.cc::AfterOpenDir(...), AfterReadDir(...), AfterCloseDir(...)	yes	yes
Timers/immediates callback dispatch: Node timer callback dispatch runs inside V8 handle scope / edge_timers_host.cc::CallTimersCallback(...), CallImmediateCallback(...)	yes	yes
Threadsafe immediate/interrupt task callbacks: Node N-API/threadsafe callback paths open a V8 handle scope / edge_environment.cc::DrainInterrupts(...), DrainThreadsafeImmedi- ates(...)	yes	yes
HTTP/2 callback helpers: node/src/node_http2.cc callback dispatch sites / bind- ing_http2.cc::CallCallbackRef(...), CallCallbackRefWithRe- source(...), CallNamedIntMethod(...)	yes	yes
TLS/OpenSSL callbacks: node/src/crypto/crypto_tls.cc callback paths / edge_tls_wrap.cc handshake, keylog, session, OCSP, ALPN, PSK, info, error, and parent-method callbacks	yes	yes
Crypto async completion: Node crypto job completion callback dispatch / bind- ing_crypto.cc::RunCryptoOnDoneTask(...)	yes	yes

`src/edge_runtime.cc::EdgeMakeCallbackWithFlags(...)` still intentionally does not try to retroactively scope handles created before the callback call. The fix is at the native event boundary, matching Node's ownership model.

122.38.2 Native event scope implementation

Added `src/edge_handle_scope.h` with two non-copyable, non-movable RAII helpers:

- `edge::HandleScope`, which opens with `napi_open_handle_scope(...)` and closes with `napi_close_handle_scope(...)`.
- `edge::EscapableHandleScope`, which opens with `napi_open_escapable_handle_scope(...)`, closes with `napi_close_escapable_handle_scope(...)`, and exposes `Escape(...)` through `napi_escape_handle(...)`.

`EdgeStreamListener` and `EdgeStreamListenerState` now carry `napi_env` so the generic listener dispatch layer can open a scope without knowing each concrete listener payload type. The env is set directly for the default/user stream listeners in `EdgeStreamBaseInit(...)`, for the consumed parser listener in `ParserConnector(...)`, and propagated in `EdgePushStreamListener(...)`. When a specific listener has no env, dispatch scans the remaining listener chain or uses the caller/state env as a fallback. If no env is available, dispatch does not invoke the listener, because executing a JS-facing listener without a local handle scope would silently allocate into the root scope.

The following stream-listener callback paths now invoke listener functions under `edge::HandleScope` when an env is available:

- `EdgeStreamEmitAlloc(...)`
- `EdgeStreamEmitRead(...)`
- `EdgeStreamEmitAfterWrite(...)`
- `EdgeStreamEmitAfterShutdown(...)`
- `EdgeStreamEmitWantsWrite(...)`
- `EdgeStreamPassAfterWrite(...)`
- `EdgeStreamPassAfterShutdown(...)`
- `EdgeStreamPassWantsWrite(...)`
- `EdgeStreamNotifyClosed(...)`

`ParserExecuteCommon(...)` now mirrors Node's inner `Parser::Execute(...)` shape: it opens `edge::EscapableHandleScope` before `llhttp_execute(...)` / `llhttp_finish(...)`, then escapes only the intended return value (nread, undefined, or parse error) back to the caller's outer listener scope. The RAII scope helpers treat a null `napi_env` as a fatal internal misuse and abort with a FATAL ERROR message instead of permitting unscoped execution.

The second scope pass added `edge::HandleScope` to the broader Node-mapped native callback set:

- `src/edge_stream_base.cc`: libuv stream alloc/read, write completion, shutdown completion, and stream handle-close callback dispatch.
- `src/edge_tcp_wrap.cc` and `src/edge_pipe_wrap.cc`: connect completion and accepted connection callbacks.
- `src/edge_cares_wrap.cc`: c-ares query completion plus `uv_getaddrinfo(...)` and `uv_getnameinfo(...)` completion callbacks.
- `src/edge_udp_wrap.cc`: UDP receive and send completion callbacks.
- `src/edge_signal_wrap.cc`: signal delivery callback.
- `src/edge_process_wrap.cc`: child process exit callback.
- `src/edge_handle_wrap.cc`: generic handle close callback dispatch.
- `src/node_api.cc`: N-API async work completion callback.
- `src/internal_binding/binding_fs.cc`: async fs completion and async file-handle close/read completion.
- `src/internal_binding/binding_fs_dir.cc`: async directory open/read/close completion.
- `src/edge_timers_host.cc` and `src/edge_environment.cc`: timers, immediates, interrupt tasks, and threadsafe immediate task dispatch.
- `src/internal_binding/binding_http2.cc`: HTTP/2 callback reference and named integer method dispatch.

- `src/edge_tls_wrap.cc`: OpenSSL/TLS handshake, keylog, session, OCSP, ALPN, PSK, info, error, and parent-method callback dispatch.
- `src/internal_binding/binding_crypto.cc`: crypto async ondone task dispatch.

Verification:

```
cmake --build build-edge-quickjs-cli --target edge -j4
./build-edge-quickjs-cli/napi-quickjs/tests/napi_quickjs_test_36_handle_scope \
  --gtest_filter=Test36HandleScope.PortedException
./build-edge-quickjs-cli/edge -e "const http=require('http'); console.log('http', typeof http.cre
PORT=3315 EDGE_TRACE_NAPI_LIFETIME_STATS=1 ./build-edge-quickjs-cli/edge server.js
curl -sS http://127.0.0.1:3315/
curl -sS http://127.0.0.1:3315/again
```

Results:

- Native QuickJS Edge build passed.
- Focused `napi_quickjs_test_36_handle_scope` passed.
- HTTP eval printed `http` function, then hit the pre-existing `JS_FreeRuntime GC-list teardown` assertion.
- The sandboxed server run still failed with the expected `listen EPERM`; the approved localhost run served requests successfully.
- Server lifetime stats after requests showed root-scope `napi_value.active` bounded in the hundreds with `napi_scope.escape_value.calls` incrementing, rather than the previous request-correlated root-scope growth into tens or hundreds of thousands.
- After the broader Node-mapped callback scope pass, including HTTP/2, TLS, and crypto callback helpers, `cmake --build build-edge-quickjs-cli --target edge -j4` passed again. The build still emits pre-existing c-ares and OpenSSL deprecation warnings.

122.38.3 Periodic lifetime stats verbosity split

`napi/quickjs/src/internal/napi_lifetime_tracker.cc` now separates the periodic stats tick from the heavy diagnostic dump:

- `env->should_dump_lifetime_stats(now)` emits a single `[napi-lifetime-stats]` line with `napi_value`, `napi_ref`, and `napi_scope` slot totals plus active counts.
- `env->should_dump_lifetime_string_symbol_values(now)` emits the full `NAPI_LIFETIME_TRACKER` dump, including the detailed slot/type/scope/tag tables and, when `NAPI_QUICKJS_ENABLE_LIFETIME_STRING_SYMBOL` is compiled in, the strings and object-type tables.

This keeps normal request-load tracing readable while still allowing the heavier string/object classification snapshots at the slower interval.

122.39 2026-05-12 Env-Owned Reference Allocator Plan

`napi_ref` handles are persistent references rather than local handles. They should therefore be owned directly by `napi_env__`, not by the root `napi_scope__`.

122.40 2026-05-16 Platform Task And Contextify Teardown Pass

Follow-up from the Next entry `CSSFiles` investigation:

- Added `edge::HandleScope` to `EdgeRunTaskQueueTickCallback(...)`, `DrainProcessTickCallback(...)`, direct loop microtask checkpoints, callback scope checkpoints, and the Edge runtime platform foreground/immediate task callbacks. These paths can enter JS or run promise jobs without passing through the normal N-API callback trampoline.
- Investigated the remaining `Debug JS_FreeRuntime(...)` assertion with LLDB. The promise-frame maps were empty; the leaked object was the ESM dynamic import registry value `{ importModuleDynamically, callbackReferrer }`.

- Fixed the contextify/module-wrap lifetime natively. Script dynamic-import referrer symbols are unregistered after `napi_contextify__::run_script(...)`, and Edge's ContextifyScript wrapper now keeps its host-defined option in a native record, restoring it onto JS only while the script is running and clearing the JS properties afterward. Env cleanup detaches these records before QuickJS runtime teardown.

Verification:

```
cmake --build /Users/sadhbh/src/dev/edgejs/build-edge-quickjs-cli-debug --target edge -j4
/Users/sadhbh/src/dev/edgejs/build-edge-quickjs-cli-debug/edge -e "console.log('hi')"
/Users/sadhbh/src/dev/edgejs/build-edge-quickjs-cli-debug/edge -e "import('node:fs').then(m => co
/Users/sadhbh/src/dev/edgejs/build-edge-quickjs-cli-debug/edge --experimental-vm-modules -e "cons
cd /Users/sadhbh/src/dev/edgejs/napi && make test-native-quickjs
```

Results: Debug teardown assertion no longer reproduces; dynamic import smokes print function and 2; native QuickJS tests pass 67/67.

Action plan before code changes:

1. Move `napi_allocator__<napi_ref__>` from `napi_scope__` into `napi_env__`.
2. Keep public reference creation/deletion routed through env helpers.
3. Leave `napi_value__` allocation in `napi_scope__`, because values remain local to the current handle scope.
4. Update lifetime scanning so `napi_ref` totals and tag dumps come from the env-level ref allocator rather than scope-level ref allocators.
5. Historical decision, superseded on 2026-05-13: the first pass replaced the linear weak-ref vector with an object-identity keyed multimap. The later QuickJS native weak-ref API removed `weak_refs__` entirely.
6. Historical decision, superseded on 2026-05-13: the first pass replaced the linear external ArrayBuffer hint vector with an object-identity keyed multimap. The later `JS_GetArrayBufferFreeInfo(...)` API removed `external_array_buffer_hints__` entirely.

Verification after the investigation:

```
cmake --build build-edge-quickjs-cli --target edge -j4
./build-edge-quickjs-cli/napi-quickjs/tests/napi_quickjs_test_36_handle_scope
./build-edge-quickjs-cli/napi-quickjs/tests/napi_quickjs_test_15_function
./build-edge-quickjs-cli/napi-quickjs/tests/napi_quickjs_test_16_reference
./build-edge-quickjs-cli/napi-quickjs/tests/napi_quickjs_test_37_reference_double_free
env -u CPPFLAGS -u LDFLAGS \
./build-edge-quickjs-cli/napi-quickjs/tests/napi_quickjs_test_6 \
--gtest_filter=Test6.PortedExceptionFlow
```

All of the direct targeted tests passed. `make test-napi-quickjs-only` was run twice and reported 44/45 passing both times, with `napi_quickjs_test_6.Test6.PortedExceptionFlow` marked failed by CTest (SEGFault once, SIGTRAP once) even though the test's own output showed PASSED and direct reruns of that test passed. Treat that as a separate intermittent CTest/process-exit issue unless it reproduces under direct LLDB.

122.41 2026-05-13 Env-Owned Ref Teardown Assertion

After the env-owned `napi_ref` allocator and constructor/destructor allocator refactor, `make test-native-quickjs` in Debug reproduced a hard assertion in the allocator's pointer release path:

```
napi_quickjs.napi_quickjs_test_6.Test6.PortedExceptionFlow
Assertion failed: (owns_block(block)), function release, file napi_allocator.h
```

LLDB showed the assert was not allocator address math. The call stack was:

```
JS_FreeRuntime(...)
-> JS_RunGC(...)
-> napi_external__::finalizer(...)
-> napi_external_backing_store_hint__::invoke_finalizer(...)
-> MyObject::Destructor(...)
```

```

-> MyObject::~MyObject()
-> napi_delete_reference(env_, wrapper_)
-> napi_env_::delete_ref_from_root_scope(...)
-> refs_.release(wrapper_)

```

The stale-handle bug was teardown ordering. `napi_env_::prepare_teardown()` closed the env-owned `refs_` allocator before QuickJS runtime finalization. QuickJS later finalized a wrapped object during `JS_FreeRuntime(...)`; that object's native destructor called `napi_delete_reference(...)` on its wrapper ref, but the allocator blocks had already been destroyed.

The fix keeps allocator slots alive through QuickJS finalization, while still dropping JS ownership before the context/runtime teardown:

- `napi_ref_::clear_for_teardown()` clears the weak link, releases any strong JSValue while the context is still valid, then sets the ref to { `env=nullptr`, `value=JS_UNDEFINED`, `ref_count=0` }.
- `napi_env_::prepare_teardown()` now clears active refs instead of closing the ref allocator. The allocator itself closes later with `napi_env_` destruction, after `JS_FreeRuntime(...)` has had a chance to run object finalizers.
- Late finalizer-driven `napi_delete_reference(...)` now releases an active but already-cleared ref slot, so the operation is idempotent and does not touch freed allocator storage or a freed QuickJS context.

Verification:

```

cd /Users/sadhbh/src/dev/edgejs/napi
CMAKE_BUILD_TYPE=Debug make build-native-quickjs
/Users/sadhbh/src/dev/edgejs/build-napi-quickjs/quickjs/tests/napi_quickjs_test_6 --gtest_filter=
CMAKE_BUILD_TYPE=Debug make test-native-quickjs
make test-native-quickjs

```

The focused Debug test passed. The full Debug native QuickJS suite passed 45/45. The exact user command `make test-native-quickjs` also passed 45/45 in the default Release configuration.

122.42 2026-05-15 Reentrant Allocator Destroy Assertion

With the shared slab allocator and lifetime tracker enabled, the native QuickJS Edge CLI could serve `server.js` initially, then abort under concurrent HTTP load:

Assertion failed during allocator ``destroy(T *)`` block-list membership checking.

LLDB showed the assertion was caused by reentrant destruction, not by wrong owner decoding or bad block alignment. The outer destroy was releasing one `napi_ref__` slot. Its payload destructor freed a QuickJS value, which ran an external finalizer for a stream wrapper. That finalizer called `napi_delete_reference(...)` for another `napi_ref__` in the same allocator block:

```

napi_allocator__<napi_ref__, napi_env__>::destroy(ref A)
-> napi_ref_::~~napi_ref__()
-> napi_ref_::clear_for_teardown()
-> JS_FreeValue(...)
-> napi_external_::finalizer(...)
-> EdgeStreamBaseFinalize(...)
-> DeleteOnReadRefs(...)
-> napi_delete_reference(ref B)
-> napi_allocator__<napi_ref__, napi_env__>::destroy(ref B)

```

The allocator had already unlinked the block from its owning list before running `slot->destroy()`. During the nested `destroy(ref B)`, the same block was temporarily in no owning list, so the membership assertion failed even though the pointer was a valid live slot.

The allocator invariant is now stricter and simpler:

- `destroy(T *)` first unlinks the slot from the block's `first_used_slot_list`.

- If the block was linked when `destroy(T *)` began, it unlinks the block's single allocator-list `link_` before running the payload destructor.
- It records release, runs `slot->destroy()`, links the slot through `first_free_slot_`, then relinks the block only if this call put it in vacuum. A nested destroy that enters while the block is already in vacuum leaves it in vacuum; the outer destroy owns the final relink.
- `close()` follows the same slot release rule, then unlinks and deletes each block after `block->close()` returns.

Verification:

```
env CMAKE_BUILD_TYPE=Debug \
  NAPI_ENABLE_LIFETIME_TRACKER=ON \
  NAPI_ENABLE_LIFETIME_PERIODIC_STATS=ON \
  NAPI_ENABLE_LIFETIME_TAG_STATS=ON \
  NAPI_ENABLE_LIFETIME_STRING_SYMBOL_DUMP=ON \
  EDGE_TRACE_NAPI_LIFETIME=1 \
  make build-edge-quickjs-cli

lldb -- ./build-edge-quickjs-cli/edge ./server.js
# target env included the same lifetime flags plus PORT=8081
ab -n 500 -c 10 http://127.0.0.1:8081/
ab -n 5000 -c 10 http://127.0.0.1:8081/
```

```
env CMAKE_BUILD_TYPE=Debug \
  NAPI_ENABLE_LIFETIME_TRACKER=ON \
  NAPI_ENABLE_LIFETIME_PERIODIC_STATS=ON \
  NAPI_ENABLE_LIFETIME_TAG_STATS=ON \
  NAPI_ENABLE_LIFETIME_STRING_SYMBOL_DUMP=ON \
  EDGE_TRACE_NAPI_LIFETIME=1 \
  make test-native-quickjs
```

The patched server handled 500 and then 5,000 ApacheBench requests at concurrency 10 with zero failed requests while LLDB stayed running and did not hit `__assert_rtn`. The native QuickJS N-API suite passed 48/48 with the lifetime tracker options enabled.

One separate cleanup-path bug was exposed while port 8080 was still occupied: the bind failure path produced `EADDRINUSE`, then an `FSEvent` close callback called `napi_open_handle_scope(...)` with a stale poisoned `napi_env` pointer during environment cleanup. That is not the allocator membership bug; track it as an EdgeJS handle-wrapper/env cleanup ordering issue if it reproduces after the listener conflict is removed.

122.43 2026-05-13 Root-Scope Function Value Growth

After the reference leak was reduced, lifetime stats still showed request-rate growth in root-scope `napi_value` object buckets, especially Function and Object, while `napi_ref` stayed flat. That points at temporary public `napi_value` handles being created with `env->current_scope() == root_scope_`, not persistent reference retention.

LLDB attached to a live `server.js` process and set a conditional breakpoint on:

```
napi_env__::wrap_value_in_current_scope(JSValue, bool)
condition: this->current_scope_ == this->root_scope_ && JS_IsFunction(this->context_, value)
```

The repeated Function root-scope stack was:

```
napi_env__::wrap_value_in_current_scope(...)
-> napi_get_named_property(..., utf8name="destroy", ...)
-> internal_binding::EmitDestroyHookForAsyncId(...)
-> internal_binding::DrainQueuedDestroyHooks(...)
-> EdgeRuntimePlatformDrainImmediateTasks(...)
-> edge::Environment::OnImmediateCheck(...)
```

```

-> uv__run_check(...)
-> uv_run(..., UV_RUN_DEFAULT)
-> RunEventLoopUntilQuiescent(...)

```

So at least one live Function handle leak is not an escaped child scope. It is the async-wrap destroy-hook drain running as a native immediate/check callback without an Edge/N-API handle scope around the JS-facing work. Each `napi_get_named_property(..., "destroy", ...)` wraps the hook function into a new public `napi_value` in the root scope.

Resolution: `binding_async_wrap.cc::EmitDestroyHookForAsyncId(...)` and the adjacent `IsDestroyHookAlreadyHandled(...)` helper now open `edge::HandleScope` around their JS-facing N-API work. The follow-up profile is recorded in 2026-05-13 Async-Wrap Destroy Hook Scope Fix; after a patched 5,000-request HTTP run, Function and Object value counts stayed flat with speed 0.

122.44 2026-05-11 Scope Handle Allocator Plan

Action plan:

1. Keep the then-existing vector-backed value/ref allocator behavior intact.
2. Add an internal opaque `napi_scope_handle__` for env-owned scope identity, backed by `napi_allocator__<napi_scope__>` rather than raw scope pointers.
3. Store `root_scope__` and `current_scope__` in `napi_env__` as scope handles, and resolve them through env helpers when N-API code needs the underlying scope object.
4. Preserve public `napi_handle_scope` and `napi_escapable_handle_scope` semantics; these may continue to be the concrete public handles returned to callers while env bookkeeping uses the allocator-backed internal handle.
5. Extend periodic stats so scopes report total created and active live counts alongside value/ref slot totals.
6. Rebuild and rerun focused handle-scope/function/reference tests, plus the already-built QuickJS N-API suite if practical.

Implementation notes:

- `napi_scope_handle__` is now an internal opaque handle type.
- `napi_env__::root_scope()` and `napi_env__::current_scope()` return `napi_scope_handle__`; code that needs the underlying scope slot resolves it through `root_scope_value()`, `current_scope_value()`, or `scope_from_handle(...)`.
- `napi_env__` owns a `napi_allocator__<napi_scope__>` for all base scope slots. The root scope and current scope are allocator handles, not raw `napi_scope__` * fields.
- Public `napi_handle_scope__` and `napi_escapable_handle_scope__` remain stable public handle objects, but each owns an internal allocator-backed scope handle.
- `napi_allocator__::close()` now resets its logical base index so reused scope slots do not inherit stale child-scope prefix offsets.
- Periodic stats now print scope slot totals alongside value/ref slot totals:

```
[napi-lifetime-stats] napi_value slots_total=595 active=595 napi_ref slots_total=196 active=196 r
```

Server smoke with allocator-backed scopes:

```
PORT=3314 EDGE_TRACE_NAPI_LIFETIME_STATS=1 ./build-edge-quickjs-cli/edge server.js
ab -n 5000 -c 10 http://127.0.0.1:3314/
```

Observed scope slots stayed bounded during the run:

```
napi_scope slots_total=3 active=1
napi_scope slots_total=3 active=1
napi_scope slots_total=3 active=1
```

Verification:

```
cmake -S . -B build-edge-quickjs-cli \
-DNAPI_QUICKJS_ENABLE_LIFETIME_TRACKER=ON \
-DNAPI_QUICKJS_ENABLE_LIFETIME_PERIODIC_STATS=ON
```



```
cmake --build build-edge-quickjs-cli --target edge -j4
./build-edge-quickjs-cli/napi-quickjs/tests/napi_quickjs_test_36_handle_scope
./build-edge-quickjs-cli/napi-quickjs/tests/napi_quickjs_test_15_function
./build-edge-quickjs-cli/napi-quickjs/tests/napi_quickjs_test_16_reference
./build-edge-quickjs-cli/napi-quickjs/tests/napi_quickjs_test_37_reference_double_free
make test-napi-quickjs-only
```

Focused tests passed, and the full already-built QuickJS N-API suite reported 45/45 passing.

The build cache was then restored to the default lifetime flags:

```
cmake -S . -B build-edge-quickjs-cli \
  -DNAPI_QUICKJS_ENABLE_LIFETIME_TRACKER=OFF \
  -DNAPI_QUICKJS_ENABLE_LIFETIME_PERIODIC_STATS=OFF
cmake --build build-edge-quickjs-cli --target edge -j4
```

The default-off build succeeded. The focused handle-scope, function, reference, and reference double-free tests also passed against that build. A subsequent default-off full rerun passed as well:

```
make test-napi-quickjs-only
```

Result: 45/45 tests passed.

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/development/troubleshooting/node-compat/napi/015_v8_structured_clone_handle_scope.md

123 V8 structured clone handle scope

		Remarks
Status	[done]	V8-backed structuredClone() no longer aborts on ordinary objects or pnpm startup.
Severity	High	Blocks build-edge/edge pnpm run start before pnpm can launch Astro.

123.1 Symptom

Running the V8-backed Edge binary in stackmachine.com fails before Astro starts:

```
cd /Users/syrusakbary/Development/stackmachine.com
/Users/syrusakbary/Development/edgejs/build-edge/edge pnpm run start
```

The process prints a V8 stack trace and can remain stopped or hot instead of opening the dev server. A minimal reproduction does not need pnpm or Astro:

```
/Users/syrusakbary/Development/edgejs/build-edge/edge -e "structuredClone({start:'astro dev'})"
```

123.2 Diagnosis

pnpm calls Node's global structuredClone() while reading and normalizing package.json. The native backtrace for the minimal reproduction reaches:

```
StructuredCloneCallback
CloneMessageValueWithTransfers
RestoreTransferableDataAfterStructuredClone
napi_has_named_property
v8::Object::Has
v8::internal::LookupIterator::GetRootForNonJSReceiver
v8::internal::Isolate::PushStackTraceAndDie
v8::base::OS::Abort
```

The likely cause is a V8 local handle lifetime violation in napi/v8/src/unofficial_napi.cc. StructuredCloneImpl(...) opens a nested v8::HandleScope, deserializes a cloned v8::Value, wraps it as napi_value, and returns it to src/internal_binding/binding_messaging.cc. Since the V8 backend now represents napi_value as a direct local-handle slot, the returned value becomes invalid when that nested scope closes. The next property probe in RestoreTransferableDataAfterStructuredClone(...) then calls into V8 with a stale handle and aborts.

This matches the existing audit in plans/quickjs-wasm/development/dev_004_v8_napi_lifetime_refactor/004_unofficial N-API helpers must escape returned V8 locals from nested scopes or avoid opening the nested scope.

123.3 Action Plan

1. Keep the fix narrow to the structured-clone result path, not the broader unofficial N-API audit list.
2. Convert StructuredCloneImpl(...) to use v8::EscapableHandleScope.
3. Have DeserializeTransferredClone(...) return a V8 local result to its caller, then escape that local in StructuredCloneImpl(...) before wrapping it as napi_value.

4. Apply the same escaped-return pattern to `unofficial_napi_deserialize_value` because it has the same serialize/deserialize result shape.
5. Rebuild `build-edge/edge`.
6. Verify:

```
/Users/syrusakbary/Development/edgejs/build-edge/edge -e "const x = structuredClone({start:'astro'
/Users/syrusakbary/Development/edgejs/build-edge/edge -e "const ab = new ArrayBuffer(4); const c
cd /Users/syrusakbary/Development/stackmachine.com && /Users/syrusakbary/Development/edgejs/build
```

The `pnpm run start` check should be bounded and cleaned up after observing that startup advances past `pnpm`'s manifest normalization and either opens an Astro listener or reports the next runtime issue.

123.4 Fix

Implemented in `napi/v8/src/unofficial_napi.cc`:

- Changed `DeserializeTransferredClone(...)` to return a `v8::Local<v8::Value>` to its caller instead of directly wrapping a local as `napi_value`.
- Changed `StructuredCloneImpl(...)` from `v8::HandleScope` to `v8::EscapableHandleScope`, then escaped the deserialized local before converting it to `napi_value`.
- Applied the same escaped-return pattern to `unofficial_napi_deserialize_value(...)`, which uses the same serializer/deserializer result shape.

This keeps the returned direct local handle alive in the caller's parent handle scope, matching the direct local-handle `napi_value` design.

123.5 Verification

Rebuilt:

```
cmake --build /Users/syrusakbary/Development/edgejs/build-edge --target edge -j4
```

Passed:

```
/Users/syrusakbary/Development/edgejs/build-edge/edge -e "const x = structuredClone({start:'astro'
# astro dev true
```

```
/Users/syrusakbary/Development/edgejs/build-edge/edge -e "const ab = new ArrayBuffer(4); const c
# 0 4
```

```
/Users/syrusakbary/Development/edgejs/build-edge/edge -e "const v8 = require('v8'); const x = v8.
# 1 2
```

```
/Users/syrusakbary/Development/edgejs/build-edge/edge /Users/syrusakbary/Development/edgejs/test/
```

Bounded `stackmachine.com` startup now advances through `pnpm` and starts Astro:

```
astro v5.17.2 ready in 977 ms
Local http://localhost:4321/
```

The verification process tree was killed after the bounded run.

123.6 Residual Issues

These are separate structured-clone semantic gaps, not the V8 handle-scope abort:

- `test/parallel/test-structuredClone-global.js` still fails when cloning a transferable `ReadableStream`.
- `test/parallel/test-structuredClone-domexception.js` still misses an expected `DataCloneError` for `DOMException` transfer.

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/development/troubleshooting/node-compat/napi/016_contextify_module_syntax_detection.md

124 QuickJS Contextify Module Syntax Detection

		Remarks
Status	[done]	Fixed with an internal QuickJS compile_cjs_function(...) helper and parser-backed syntax detection.
Severity	High	Incorrect ambiguous .js format detection sends CommonJS packages through the ESM linker and blocks Astro/Vite startup.

124.1 Failure

Running the native QuickJS Edge CLI in stackmachine.com currently fails:

```
/Users/syrusakbary/Development/edgejs/build-edge-quickjs-cli/edge pnpm run start
```

Observed failure:

```
SyntaxError: Could not find export 'default' in module  
'file:///Users/syrusakbary/Development/stackmachine.com/node_modules/.pnpm/esbuild@0.25.12/node_modules/esbuild/lib/main.js'
```

The reduced static import repro is:

```
cd /Users/syrusakbary/Development/stackmachine.com  
/Users/syrusakbary/Development/edgejs/build-edge-quickjs-cli/edge --input-type=module \  
-e "import esbuild from 'file:///Users/syrusakbary/Development/stackmachine.com/node_modules/.pnpm/esbuild@0.25.12/node_modules/esbuild/lib/main.js'"
```

V8 Edge and native Node load the same file successfully and expose the CommonJS namespace facade with default, module.exports, and named exports.

124.2 Diagnosis

esbuild@0.25.12/lib/main.js is CommonJS:

- package.json has no "type": "module".
- package.json points "main" at lib/main.js.
- lib/main.js assigns module.exports = __toCommonJS(node_exports).

The loader divergence happens before synthetic CommonJS export population:

- V8 Edge logs Translating CJSModule ../esbuild/lib/main.js.
- QuickJS Edge logs Translating StandardModule ../esbuild/lib/main.js.

For ambiguous .js files, lib/internal/modules/esm/get_format.js delegates to internalBinding('contextify').containsModuleSyntax(...). V8 implements this by parsing as a CommonJS function first and only retrying as ESM for CJS parse failures that are plausibly ESM-only syntax.

QuickJS currently implements the same API with substring checks:

```
src.find("export ") != std::string::npos ||  
src.find("import ") != std::string::npos ||  
src.find("import(") != std::string::npos
```

esbuild/lib/main.js contains no actual static ESM syntax, but it does contain:

```
var __export = (target, all) => { ... }  
// Annotate the CommonJS export names for ESM import in node:
```

That makes QuickJS return `true` for `containsModuleSyntax(...)`, so Node's JS loader treats the file as ESM. The QuickJS ESM linker then correctly rejects the Vite import because a CommonJS source file has no declared ESM default export.

124.3 Correctness Bar

The fix must not be a lexer workaround, package exception, or esbuild/Vite special case.

The desired behavior is parser-backed:

1. Try to parse the source as a CommonJS function body with the same wrapper parameter policy requested by `cjs_var_in_scope`.
2. If that parse succeeds, the source does not require ESM classification.
3. If the CommonJS parse fails, try to parse the original source as an ESM source text module.
4. Return `true` only when the ESM parse succeeds.
5. Preserve the original CommonJS parse error for `compileFunctionForCJSLoader` when detection is disabled or the ESM parse does not succeed.

This matches the important V8/Node property without depending on V8 error message strings: actual parser acceptance decides the classification.

124.4 Upstream-Quality Design

The QuickJS-facing change should be an engine/parser capability, not Node module policy. The clean target is a small public or internal QuickJS parser helper that can validate source text under a requested parse goal without executing it.

Candidate API shape:

```
typedef enum JSParseGoalEnum {  
    JS_PARSE_GOAL_SCRIPT,  
    JS_PARSE_GOAL_MODULE,  
    JS_PARSE_GOAL_FUNCTION_BODY,  
} JSParseGoalEnum;  
  
int JS_ParseSource(JSContext *ctx,  
                  const char *input,  
                  size_t input_len,  
                  const char *filename,  
                  JSParseGoalEnum goal,  
                  const char * const *param_names,  
                  int param_count);
```

Expected semantics:

- return 0 on parse success and -1 with a pending exception on parse failure;
- allocate and free parser/compiler state through the same ownership paths as existing QuickJS compile operations;
- never evaluate user code;
- avoid looking up global constructors such as `Function`;
- support function-body parsing with an explicit parameter list so embedders do not have to synthesize wrapper source strings;
- keep module parsing identical to normal QuickJS source-text module parsing.

If upstream does not want a general public API, keep the helper local to the vendored QuickJS patch but design it with the same constraints and naming style so the patch remains reviewable and minimal.

124.5 Edge Integration Plan

1. Add focused parser helper tests at the QuickJS layer. Cover CommonJS bodies, ESM declarations, `import.meta`, top-level `await`, comments containing `import/export`, identifiers such as `__export`, string literals, regex literals, and syntax that is invalid in both CJS and ESM.
2. Replace `napi_contextify__::contains_module_syntax(...)` with parser-backed classification:
 - CJS goal first, using wrapper params only when `cjs_var_in_scope` is true;
 - ESM goal second only after CJS parse failure;
 - clear discarded parser exceptions so the N-API env is left clean on a boolean success result.
3. Update `ContextifyCompileFunctionForCJSLoaderCallback(...)` and/or `napi_contextify__::compile_function` so `should_detect_module` uses the same parser-backed decision and still rethrows the original CJS compile error when the source is not valid ESM.
4. Add N-API tests under the existing `contextify` test runner:
 - `containsModuleSyntax("module.exports = 1") == false;`
 - `containsModuleSyntax("export const x = 1") == true;`
 - `containsModuleSyntax("await 1") == true` for ambiguous source;
 - `containsModuleSyntax("__export(...); // export names import in node") == false;`
 - esbuild-style CommonJS prologue/body fixture returns false;
 - invalid syntax returns false or propagates only through the compile loader path as Node expects.
5. Add loader-level regression tests:
 - ESM imports a fixture `.js` CommonJS file with esbuild-style comments and receives default, `module.exports`, and declared named exports.
 - ESM imports a true ambiguous `.js` module and gets ESM behavior.
 - V8 and QuickJS expectations match for the same fixtures.

6. Verify with real commands:

```
make build-napi-quickjs
./build-edge-quickjs-cli/napi-quickjs/tests/napi_quickjs_test_65_unofficial_contextify
make test-napi-quickjs-only
cd /Users/syrusakbary/Development/stackmachine.com
NODE_DEBUG=esm /Users/syrusakbary/Development/edgejs/build-edge-quickjs-cli/edge --input-type=module
-e "import esbuild from 'file:///Users/syrusakbary/Development/stackmachine.com/node_modules/esbuild'
/Users/syrusakbary/Development/edgejs/build-edge-quickjs-cli/edge pnpm run start
```

If implementation touches `napi/quickjs/` and the native gate passes, run the WASIX-impacting rebuild:

```
cd /Users/syrusakbary/Development/edgejs/quickjs-wasm/ && ./build.sh
```

124.6 Non-Goals

- Do not patch `stackmachine.com`, `Vite`, `Astro`, `pnpm`, or `node_modules`.
- Do not add package-name checks for `esbuild`.
- Do not replace the substring check with a hand-written JavaScript lexer.
- Do not resurrect a C++ CommonJS resolver or CommonJS export scanner.
- Do not classify dynamic `import(...)` alone as ESM syntax if it parses as a CommonJS function body.

124.7 Open Questions

- Whether the parser helper should be a public QuickJS API, a private vendored API, or an Edge-local wrapper around existing QuickJS compile internals.
- Whether QuickJS should expose function-body parsing directly, or whether Edge should temporarily use wrapped source text while an upstreamable helper is prepared.
- Exact parity for `cjs_var_in_scope == false`, used by Node when CommonJS wrapper variables should not affect syntax detection.

124.8 Implementation Result

The fix stayed inside the QuickJS contextify backend. It did not add a new `unofficial_napi` entry point.

`napi_contextify__` now has an internal `compile_cjs_function(...)` helper that matches the important V8 `CompileCjsFunction(...)` behavior for Edge's needs:

- compile a function body with an explicit parameter list;
- pass the CommonJS wrapper params only when the caller requests a CJS scope;
- return a function object without executing the user body;
- preserve parser exceptions for the compile path;
- let syntax detection discard parser exceptions after classification.

The helper compiles a parenthesized CommonJS wrapper expression with `JS_Eval2(..., JS_EVAL_TYPE_GLOBAL | JS_EVAL_FLAG_COMPILE_ONLY)` and then instantiates that wrapper expression with `JS_EvalFunction(...)`. It does not look up or invoke the mutable global `Function` constructor.

`contains_module_syntax(...)` now uses parser behavior instead of substring matching:

1. Compile as a CommonJS function body.
2. If that succeeds, return `false`.
3. If that fails, compile the original source as a source-text module.
4. Return `true` only when the module compile succeeds.

Regression coverage was added for:

- normal CommonJS source;
- static ESM syntax;
- compiling a function after `globalThis.Function` has been monkey-patched;
- esbuild-style `__export` identifiers and comments containing `export / import`;
- dynamic `import(...)` inside valid CommonJS;
- top-level `await` as an ESM-only parse case.

Verified:

```
make build-napi-quickjs
./build-edge-quickjs-cli/napi-quickjs/tests/napi_quickjs_test_65_unofficial_contextify
make test-napi-quickjs-only
```

The direct esbuild checks now match V8 behavior:

```
/Users/syrusakbary/Development/edgejs/build-edge-quickjs-cli/edge -e \
  "const src=require('fs').readFileSync('/Users/syrusakbary/Development/stackmachine.com/node_modules/...')
/Users/syrusakbary/Development/edgejs/build-edge-quickjs-cli/edge --input-type=module \
  -e "import esbuild from 'file:///Users/syrusakbary/Development/stackmachine.com/node_modules/...'"
```

The first command prints `false`. The second succeeds and imports the CommonJS default object.

The original `pnpm run start` command now advances past the esbuild default-export linker failure and stops at the separate known compatibility gap:

ReferenceError: WebAssembly is not defined

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/development/troubleshooting/node-compat/napi/017_v8_finalizer_gc_microtask_reentry.md

125 V8 finalizer GC microtask reentry

		Remarks
Status	[done]	Fixed by moving GC weak-callback finalizer drains onto the embedder foreground-task queue.
Severity	High	V8 backend can abort under HTTP load when a weak callback schedules a microtask during GC.

125.1 Failure

Running the V8-backed Edge CLI in ~/Development/stackmachine.com:

```
/Users/syrusakbary/Development/edgejs/build-edge/edge pnpm run start  
ab -n 1000 -c 10 http://localhost:4321/
```

can abort inside V8:

```
Check failed: v8_flags.separate_gc_phases && young_gc_while_full_gc_ implies current_.state == Ev  
...  
v8::internal::MicrotaskQueue::EnqueueMicrotask(...)  
napi_env__::InvokeFinalizerFromGC(napi_ref_tracker__*)  
v8::internal::GlobalHandles::InvokeFirstPassWeakCallbacks()
```

125.2 Diagnosis

The weak reference path correctly avoids running the user finalizer directly inside V8's weak callback. However, `napi_env__::EnqueueFinalizer(...)` still calls `isolate->EnqueueMicrotask(...)` from that weak callback.

That call allocates V8 objects while V8 is already running GC weak callbacks. Under load, the allocation can trigger a nested young GC while the outer full GC is not in the sweeping state V8 expects, causing the fatal check.

The same weak-callback shape also existed for native buffer records: `BufferWeakCallback(...)` removed the record from `buffer_records`, reset its weak holder, then called `isolate->EnqueueMicrotask(...)` to run the buffer finalizer. That path did not appear in the reported stack, but it had the same V8-GC reentry hazard.

125.3 Fix

The V8 backend now treats GC weak callbacks as V8-heap allocation-free handoff points:

1. `napi_env__::InvokeFinalizerFromGC(...)` queues the `napi_ref_tracker__` in `pending_finalizers`.
2. `BufferWeakCallback(...)` queues the detached `napi_buffer_record__` in `pending_buffer_finalizers`.
3. Both queues schedule a single drain through Edge's embedder foreground-task hook when present.
4. Explicit microtask checkpoints also drain the finalizer queue so standalone tests and embedders without the foreground-task hook still observe finalizers.

125.4 Verification

Built both the root V8 Edge CLI and the standalone V8 N-API test tree:


```
cmake --build build-edge --target edge -j8
make -C napi build-napi BUILD_NAPI_DIR=/Users/syrusakbary/Development/edgejs/build-napi-v8-standalone
```

Targeted tests passed:

```
/Users/syrusakbary/Development/edgejs/build-napi-v8-standalone/v8/tests/napi_v8_test_38_finalizer
/Users/syrusakbary/Development/edgejs/build-napi-v8-standalone/v8/tests/napi_v8_test_21_general
```

The V8 N-API suite passes when excluding the known contextify test that asserts QuickJS-specific global marker behavior:

```
ctest --test-dir /Users/syrusakbary/Development/edgejs/build-napi-v8-standalone \
  --output-on-failure -R '^napi_v8\.' \
  -E 'SandboxGlobalThisAndMarkerAreNotEnumerableForDeepFreeze'
```

The full V8 N-API suite still has that unrelated failure:

```
napi_v8.napi_v8_test_65_unofficial_contextify.Test65UnofficialContextify.SandboxGlobalThisAndMarkerAreNotEnumerableForDeepFreeze
```

The original load reproduction now completes and the server remains alive:

```
cd /Users/syrusakbary/Development/stackmachine.com
/Users/syrusakbary/Development/edgejs/build-edge/edge pnpm run start
ab -n 1000 -c 10 http://localhost:4321/
curl -I --max-time 5 http://localhost:4321/
ab completed all 1000 requests, and the follow-up curl -I returned HTTP/1.1 200 OK.
```

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/development/troubleshooting/node-compat/napi/README.md

126 N-API Node Compatibility Known Issues

		Remarks
Status	[open]	Registry for QuickJS N-API compatibility issues and internal subsystem notes. Documentation registry only; individual issue pages carry runtime severity.
Severity	Low	

This directory records known Node-compatibility issues for the QuickJS N-API backend. The issue pages are canonical; this index only carries status, severity, and links.

Status	Severity	Issue	Topic
[caveated]	Medium	001_buffer.md	Buffer
[caveated]	Medium	002_console.md	Console bootstrap binding
[done]	High	003_contextify.md	Contextify diagnostics
[caveated]	High	004_environment.md	Environment lifecycle
[caveated]	Medium	005_global_shims.md	Global shims
[done]	High	006_microtasks.md	Promise hooks and microtasks
[caveated]	High	007_module_loading.md	Module loading
[done]	Medium	008_properties.md	Property setting semantics
[done]	Low	009_quickjs_utilities.md	QuickJS utility ownership
[done]	Medium	010_serdes.md	Serialization and deserialization
[open]	Medium	011_quickjs_wasix_atomics_patch.md	QuickJS WASIX atomics guard patch
[caveated]	Medium	013_v8_shaped_callsite_methods.md	V8-shaped CallSite methods
[caveated]	Medium	014_lifetime_tracing.md	QuickJS N-API lifetime tracing

127 Wasmer Deploy And WASIX Packaging

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/development/troubleshooting/wasmer-deploy/001_pnpm_directory_symlinks_webc.md

128 Wasmer Deploy: pnpm Directory Symlinks In WEBC Package

		Remarks
Status	[caveated]	Deploy preparation now emits a symlink-free, flattened .deploy artifact and passes local wasmer run --net .; wasmer.io still needs a fresh deploy confirmation.
Severity	High	The app runs locally from .deploy, but the app deployed to wasmer.io exits during startup when bare react cannot resolve.

128.1 Issue

The prepared stackmachine.com deploy directory works locally:

```
cd ~/src/dev/stackmachine.com
npm run edge:prepare-deploy
cd .deploy
wasmer run --net .
```

The same app created by wasmer deploy on wasmer.io exits with:

```
file:///app/dist/server/entry.mjs:1
import { renderers } from './renderers.mjs';
```

```
ReferenceError: could not load module 'react'
Instance exited with code ExitCode::1
```

This means the cloud runtime reached the Astro server entry, then failed while resolving the bare react import from /app/dist/server/renderers.mjs.

128.2 Source Findings

Wasmer has two different deploy/package paths that matter here.

Remote autobuild ZIP creation lives in:

```
~/src/dev/wasmer/lib/sdk/src/app/deploy_remote_build.rs
```

```
create_zip_archive(...) configures the ignore walker with standard_filters(true), .ignore(true),
.git_ignore(true), .git_exclude(true), .git_global(true), .require_git(true), and .follow_links(false). It also explicitly rejects symlinks:
```

```
cannot deploy projects containing symbolic links
```

So if wasmer deploy takes the remote autobuild ZIP path for a tree containing pnpm links, it should fail before runtime. This path also enables standard and git ignore filtering, so hidden paths and ignored paths can be filtered there.

The local package publish path is used when the app manifest points at a local package and a wasmer.toml is present. That path is in:

```
~/src/dev/wasmer/lib/cli/src/commands/app/deploy.rs
~/src/dev/wasmer/lib/sdk/src/package/publish.rs
~/src/dev/wasmer/lib/package/src/package/package.rs
```

The package walker uses `standard_filters(false)`, `follow_links(false)`, `parents(true)`, and `add_custom_ignore_filename(".wasmerignore")`. Therefore this path does not skip hidden directories merely because their names begin with `.`. In the original pnpm-deployed artifact, the prepared app had no `.deploy/.wasmerignore`, `.gitignore`, or `.ignore`, and `.deploy/node_modules/.pnpm` existed locally.

The important remaining behavior is symlink handling. Package volumes are created through:

```
~/src/dev/wasmer/lib/package/src/package/volume/fs.rs
```

That calls `WEBC's Directory::from_path_with_walker(...)` with the same walker. The WEBC crate code inspected locally is:

```
/Users/sadhbh/.cargo/registry/src/index.crates.io-1949cf8c6b5b557f/webc-11.0.0/src/from_path_with
```

Because the walker has `follow_links(false)`, it sees pnpm top-level package links but does not descend into linked directories. WEBC then classifies entries from the path shape. A directory symlink such as:

```
/app/node_modules/react -> .pnpm/react@18.3.1/node_modules/react
```

can become a directory entry without the package contents that live behind the symlink.

128.3 Diagnosis

This is not the same as the earlier WASIX local filesystem symlink issue. Local `wasmer run --net .` sees the host `.deploy` tree and follows the real pnpm symlinks, so QuickJS can resolve:

```
/app/node_modules/react/package.json
```

The cloud package is serialized first. If the WEBC package contains `/app/node_modules/react` as an empty directory, a non-followed symlink entry, or otherwise not as the real React package directory, QuickJS correctly fails Node-style bare package resolution and reports:

```
ReferenceError: could not load module 'react'
```

The hidden `.pnpm` directory is less likely to be the primary issue for the local package publish path, because that path disables standard filters. The top-level `node_modules/react` directory symlink is the sharper suspect.

128.4 Action Plan

- Keep the existing `.deploy` preparation flow as the starting point.
- Copy deploy manifests that Wasmer may need at upload time, including `wasmer.toml` and optional `app.yaml`.
- Build a runtime package closure from literal imports in `.deploy/dist/server`, then recursively scan included package source files for literal `import(...)`, `static import/export ... from`, and `require(...)` specifiers.
- Prune top-level packages outside that import closure.
- Materialize the remaining pnpm package links as real directories, skipping nested `node_modules` directories so every module exists only at the root `.deploy/node_modules` level.
- Remove pnpm runtime scaffolding (`.pnpm` and `.bin`) from the final artifact.
- Rewrite source references from:

```
node_modules/.pnpm/<store-entry>/node_modules/<package>
```

to:

```
node_modules/<package>
```

- Fail deploy preparation if any of these invariants are violated: no symlinks, no nested `node_modules`, no `.pnpm` directory, and no remaining source imports targeting pnpm virtual-store paths.
- Verify the deploy artifact before upload:

```
cd ~/src/dev/stackmachine.com/.deploy
find . -type l -print
find . -type d -name node_modules -print
find . -type d -name .pnpm -print
rg -n 'node_modules/\.pnpm/\.+/node_modules/' .
```

128.5 Current Status

~/src/dev/stackmachine.com/scripts/prepare-edge-deploy.cjs now creates a flattened deploy artifact:

- builds Astro and runs `pnpm deploy --prod --legacy` into a temporary `.deploy-pnpm`;
- copies `dist`, `wasmer.toml`, optional `app.yaml`, `package.json`, and the deployed `node_modules`;
- links missing server-runtime bare imports from the pnpm virtual store;
- expands the runtime import closure by scanning included source files;
- prunes packages outside that closure;
- materializes the remaining package links as real directories;
- removes `.pnpm` and `.bin`;
- rewrites source references to materialized `node_modules/<package>` paths;
- asserts the final artifact has a single root `node_modules` tree and no symlinks.

The fresh local artifact now reports:

```
Runtime package closure: 488 package(s)
Pruned 34 unimported top-level packages
Materialized 488 package links
347M    ~/src/dev/stackmachine.com/.deploy
```

128.6 Validation

Fresh prepare:

```
cd ~/src/dev/stackmachine.com
npm run edge:prepare-deploy
```

Final artifact checks:

```
find ~/src/dev/stackmachine.com/.deploy -type l -print
find ~/src/dev/stackmachine.com/.deploy -type d -name node_modules -print
find ~/src/dev/stackmachine.com/.deploy -type d -name .pnpm -print
rg -n 'node_modules/\.pnpm/\.+/node_modules/' ~/src/dev/stackmachine.com/.deploy
```

Observed:

- no symlinks;
- only `~/src/dev/stackmachine.com/.deploy/node_modules`;
- no `.pnpm` directory;
- no pnpm virtual-store import paths;
- `~/src/dev/stackmachine.com/.deploy/app.yaml` exists when the source app has `app.yaml`, and matches the source file.

Runtime check:

```
cd ~/src/dev/stackmachine.com/.deploy
wasmer run --net .
curl -m 30 -s -o /tmp/stackmachine-deploy-check.html \
  -w '%{http_code} %{content_type} %{size_download}\n' \
  http://127.0.0.1:4321/
```

Observed:

```
[@astrojs/node] Server listening on http://localhost:4321  
200 text/html 167960
```

128.7 Open Questions

- Whether Wasmer Cloud was reached through local package publish or remote autobuild for the reported run. The runtime failure suggests the local package path, because the remote ZIP path should reject symlinks during archive creation.
- Whether Wasmer should preserve directory symlinks in WEBC packages, reject them like the remote ZIP path, or document that publish inputs must be symlink-free.

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/development/troubleshooting/wasmer-deploy/002_quickjs_wasix_napi_import_module_mismatch.md

129 Wasmer Deploy: QuickJS WASIX N-API Import Module Mismatch

		Remarks
Status	[done]	Fixed by applying the embedded QuickJS N-API declaration mode to edge_environment_core; quickjs-wasm/build.sh now completes and the final no-N-API-imports check passes. Blocks the QuickJS WASIX artifact from linking, so no deployable edgejs.wasm can be produced.
Severity	High	

129.1 Issue

~/src/dev/edgejs/quickjs-wasm/build.sh reached the final WASM link and failed with wasm-ld import module mismatches for several napi_* symbols:

```
wasm-ld: error: import module mismatch for symbol: napi_get_reference_value
>>> defined as env in libedge_runtime.a(edge_runtime.cc.obj)
>>> defined as napi in libedge_environment_core.a(edge_environment.cc.obj)
```

The same pattern appeared for napi_delete_reference, napi_create_reference, napi_is_exception_pending, napi_get_global, napi_get_named_property, napi_create_string_utf8, napi_call_function, napi_strict_equals, napi_create_array, napi_set_element, and napi_add_env_cleanup_hook.

129.2 Diagnosis

The QuickJS WASIX build uses EDGE_NAPI_PROVIDER=quickjs, so the final wasm should link against the embedded QuickJS N-API implementation and should not import napi_*, node_api_*, or unofficial_napi_* symbols. The napi_quickjs target exposes NAPI_EXTERN= to prevent the common N-API header from annotating declarations as WASM imports.

edge_runtime already linked napi_quickjs, but edge_environment_core includes N-API headers and only linked uv_a. Under __wasm__, the default NAPI_EXTERN macro annotates declarations with __import_module__("napi"). That made edge_environment_core object files disagree with other QuickJS Edge objects that referenced the same unresolved N-API calls through the default env import module. wasm-ld rejects the same symbol being imported from two different modules.

129.3 Current Status

For EDGE_NAPI_PROVIDER=quickjs, edge_environment_core now compiles with:

```
EDGE_EMBEDDED_NAPI_PROVIDER=1
NAPI_EXTERN=
```

This keeps N-API declarations consistent with the embedded QuickJS provider without changing the shared common N-API headers.

129.4 Verification

The fixed build was verified with:

```
cd ~/src/dev/edgejs/quickjs-wasm  
./build.sh
```

Result:

Built QuickJS WASIX targets at ~/src/dev/edgejs/build-quickjs-wasix/edge.wasm and ~/src/dev/edgejs/build-quickjs-wasix/edge.wasm

The script also completed its final no-N-API-imports check.

129.5 Follow-Up Rule

When adding new Edge static libraries or object targets that include N-API headers and participate in embedded QuickJS WASIX linking, make sure the target inherits or explicitly defines `NAPI_EXTERN=` for the QuickJS provider.

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/development/troubleshooting/wasmer-deploy/003_ci_safe_mode_missing_quickjs_artifact.md

130 Wasmer Deploy: CI safe-mode missing QuickJS artifact

		Remarks
Status	[caveated]	QuickJS-specific CI wiring fixed locally; full WASIX execution still needs the GitHub Actions toolchain.
Severity	High	Blocks the WASIX Linux job before the safe-mode smoke tests can start.

130.1 Issue

The build-wasix-linux workflow builds the legacy WASIX artifact through:

```
make build-wasix
```

That writes:

```
build-wasix/edgejs.wasm
```

At the time this issue was first diagnosed, the active workflow/package path had the root wasmer.toml describing the embedded QuickJS package and pointing at:

```
build-quickjs-wasix/edgejs.wasm
```

The safe-mode smoke test therefore failed before executing JavaScript:

```
Unable to read the "edge" module's file from "./build-quickjs-wasix/edgejs.wasm"
No such file or directory
```

130.2 Action Plan

1. Add a Makefile target for the current QuickJS WASIX build script so CI does not need to know the script path directly.
2. Update the build-wasix-linux workflow to build the embedded QuickJS WASIX artifact before running wasmer run ..
3. Update WASIX dist packaging so BUILD_DIR=build-quickjs-wasix is treated as a wasm distribution directory, not as a native Edge build.
4. Remove or bypass the legacy napi_wasmer smoke path from this job because it belongs to EDGE_NAPI_PROVIDER=import while this package now embeds the QuickJS provider.
5. Verify the edited Makefile targets and safe-mode test script syntax locally; full WASIX execution still requires the CI Wasmer/WASIX toolchain.

130.3 Current Status

The root wasmer.toml has since been restored to the main V8 Edge package and must stay that way. QuickJS packaging now belongs to the separate .github/workflows/test-and-build-quickjs.yml workflow and the quickjs-wasm/wasmer.toml manifest.

The Makefile now exposes:

```
make build-quickjs-wasix
```

That target runs `quickjs-wasm/build.sh`, producing the `build-quickjs-wasix/edgejs.wasm` artifact referenced by the QuickJS package manifest.

The QuickJS `build-wasix-linux` workflow now builds the QuickJS WASIX artifact before the safe-mode smoke test and packages `BUILD_DIR=build-quickjs-wasix` for the WASIX zip. The pre-packaging smoke test uses `WASIX_PACKAGE_DIR="$(pwd)/quickjs-wasm"`, and the packaged WASIX dist is rehydrated with `quickjs-wasm/wasmer.toml`, then rewrites the module source to `./bin/edgejs` and the SSL certificate mount to `./ssl-certs`, so the root V8 manifest is not used for the QuickJS artifact. The legacy `napi_wasmer` smoke path was removed from this job because it tests the host-import N-API artifact, not the embedded QuickJS package.

Native N-API, standalone N-API Cargo, and `napi_wasmer` host-import smoke workflows now belong outside EdgeJS runtime CI. EdgeJS no longer runs duplicate native N-API or standalone N-API Cargo test jobs from its root workflows. EdgeJS WASIX coverage is now the engine-specific package build lanes: `build-wasix` for V8 and `build-quickjs-wasix` for QuickJS. Publishing remains active only for push events on main, after the native matrix and WASIX build for that engine have passed.

`make dist-only` now treats both `build-wasix` and `build-quickjs-wasix` as WASIX distribution directories, so it copies `edgejs.wasm`, `wasmer.toml`, and certificates instead of looking for native edge and edgeenv binaries.

130.4 Verification

Local structural checks passed:

```
make -n build-quickjs-wasix
make -n dist-only BUILD_DIR=build-quickjs-wasix ZIP_NAME=edge-wasix.zip
python3 -m py_compile scripts/test-wasix-safe-mode.py
ruby -e "require 'yaml'; YAML.load_file('.github/workflows/test-and-build.yml')"
git diff --check
```

The full `make build-quickjs-wasix` and `make test-wasix-safe-mode` flow was not run locally because it depends on the WASIX/Wasmer CI toolchain.

130.5 Follow-Up: Native Linux Job

The native `build-linux` job in the QuickJS workflow formerly built the V8 N-API test target and default V8-backed Edge binary. It was later aligned with the QuickJS provider:

```
make build-edge-quickjs-cli CMAKE_BUILD_TYPE=Release JOBS=4
make dist-only BUILD_DIR=build-edge-quickjs-cli JOBS=4 ZIP_NAME=edge-linux-amd64.zip
make test-quickjs-only TEST_JOBS=4
```

`make build-edge-quickjs-cli` now builds both the edge and edgeenv targets so native dist packaging has the executables expected by `dist-only`.

130.5.1 May 7, 2026 Native QuickJS N-API Release Build Follow-Up

The Linux `make build-napi-quickjs CMAKE_BUILD_TYPE=Release JOBS=4` job failed under Ubuntu 24.04 / Clang 18.1.3 because `napi/quickjs/src/js_native_api_quickjs.cc` used `INT_MAX` without directly including `<climits>`. The V8 backend already includes `<climits>` for the same `INT_MAX` checks, so the QuickJS backend now includes it explicitly.

Reproducing the job in an Ubuntu 24.04 Docker container with Clang 18.1.3 also exposed a second `libstdc++/Clang` compile issue in the former QuickJS C++ module-loader helper: the recursive `JsonValue` type instantiated `std::vector<std::pair<std::string, JsonValue>>` special-member machinery while `JsonValue` was still incomplete. `JsonValue` was changed to declare its special members and `Get(...)` in the struct and default/define them out-of-line after the type was complete. Later cleanup removed that helper with the rest of the QuickJS C++ CommonJS facade/module-loader support.

Verification:

```
CC=clang CXX=clang++ make build-napi-quickjs CMAKE_BUILD_TYPE=Release JOBS=4
```

Result: passed in the Ubuntu 24.04 / Clang 18.1.3 Docker container.

130.5.2 May 7, 2026 QuickJS Workflow Split Follow-Up

The main `.github/workflows/test-and-build.yml` workflow and root `wasmer.toml` were restored to their main/V8 behavior. The QuickJS-specific copy at `.github/workflows/test-and-build-quickjs.yml` now keeps active Edge runtime build/test targets on the QuickJS provider:

```
make build-edge-quickjs-cli CMAKE_BUILD_TYPE=Release JOBS=4
make dist-only BUILD_DIR=build-edge-quickjs-cli JOBS=4 ZIP_NAME=edge-<platform>.zip
make test-quickjs-only TEST_JOBS=4
make build-quickjs-wasix
make dist-only BUILD_DIR=build-quickjs-wasix JOBS=4 ZIP_NAME=edge-quickjs-wasix.zip
```

The macOS job matches the Linux QuickJS native job. The old duplicate QuickJS-only WASIX workflow was removed; QuickJS native/WASIX coverage is now reported as `quickjs-linux`, `quickjs-macos`, and `quickjs-wasix` in `.github/workflows/test-and-build-quickjs.yml`. The QuickJS workflow also has a `publish-nightly` job gated to push on main; it uploads the QuickJS native/WASIX artifacts to the nightly release and publishes the packaged QuickJS WASIX manifest through Wasmer.

Source: /Users/sadhbh/src/dev/edgejs/plans/quickjs-wasm/development/troubleshooting/wasmer-deploy/004_wasix_safe_mode_https_exit.md

131 Wasmer Deploy: WASIX safe-mode HTTPS exits before callbacks

		Remarks
Status	[done]	Runtime fix is implemented and the CI-matching Linux/Wasmer safe-mode suite passes.
Severity	High	Blocks the root build-wasix-linux job in the main V8/imports WASIX workflow.

131.1 Issue

The root build-wasix-linux workflow builds the V8/imports WASIX artifact with:

```
make build-wasix
make test-wasix-safe-mode WASMER_BIN=wasmer
```

The safe-mode smoke suite passed the earlier microtask, Blob, and HTTP fetch cases, then failed on HTTPS fetch:

```
fetch https://example.com/ stdout mismatch
expected: 'FETCH HTTPS 200\n'
actual:   ''
stderr: [callback trampoline] error calling function: RuntimeError: WASI exited with code: ExitC
```

The callback trampoline message means a host-side N-API callback attempted to re-enter the guest module after Wasmer considered the WASI process exited. The HTTPS path exposed this because TLS/fetch completion can enqueue promise and process task work after the libuv loop briefly appears idle to the Edge runtime.

131.2 Action Plan

1. Keep the main .github/workflows/test-and-build.yml and root wasmer.toml unchanged.
2. Fix the runtime drain in src/, not the workflow.
3. Rebuild the WASIX artifact.
4. Verify the safe-mode smoke suite with Wasmer 7.1.0, including HTTPS fetch, https.get, and tls.connect.
5. Rebuild once inside a Linux Docker container with the CI wasixcc release and sysroot tag.

131.3 Current Status

RunEventLoopUntilQuiescent(...) now drains all runtime-visible JavaScript task queues during its idle grace window and after beforeExit, rather than draining only the platform queue. The combined drain runs:

```
EdgeRuntimePlatformDrainTasks
DrainProcessTickCallback
unofficial_napi_process_microtasks
```

This gives promise continuations and process ticks scheduled from late host/N-API callbacks a chance to run before the runtime emits exit.

131.4 Verification

Linux Docker smoke testing against the rebuilt WASIX artifact passed with Wasmer 7.1.0:

```
python3 ./scripts/test-wasix-safe-mode.py --wasmer-bin wasmer --package-dir /work --timeout 45
```

Result:

```
[ok] fetch https://example.com/: FETCH HTTPS 200
[ok] https.get https://example.com/: HTTPS 200
[ok] tls.connect verified example.com: TLS CONNECTED true
All WASIX safe-mode smoke tests passed.
```

The final Linux Docker rebuild and post-rebuild safe-mode rerun are still in complete.

The artifact was rebuilt in native arm64 Ubuntu Docker using wasixcc v0.4.2 and the CI sysroot tag v2026-02-16.1:

```
make build-wasix
```

That produced:

```
build-wasix/edge.wasm
build-wasix/edgejs.wasm
```

The safe-mode verification was then run under Linux amd64 Docker with Wasmer 7.1.0, matching the GitHub ubuntu-latest runner architecture:

```
python3 ./scripts/test-wasix-safe-mode.py --wasmer-bin wasmer --package-dir /work --timeout 45
```

Result:

```
[ok] queueMicrotask: A
C
B
[ok] blob.arrayBuffer: BLOB 3
[ok] fetch http://example.com/: FETCH 200
[ok] fetch https://example.com/: FETCH HTTPS 200
[ok] https.get https://example.com/: HTTPS 200
[ok] tls.connect verified example.com: TLS CONNECTED true
TLS CLOSE
TLS EXIT 0
All WASIX safe-mode smoke tests passed.
```

Native arm64 Wasmer 7.1.0 still failed before JavaScript startup with an unknown napi_extension_wasmer_v0.unofficial import. The CI-matching Linux amd64 Wasmer path succeeds.